

SECRET	ENGINEERING STANDARD	SPEC NO.
ESTABLISHED BY		ES 95486-00
Electronics Network Development Team		
REL. DATE	ENGINEERING SPECIFICATON	PAGE
2025-10-16		210
TITLE	Unified Diagnostic Services (UDS) on CAN	

INDEX

- 1. Scope
- 2. Protocol Definition
- 3. Diagnostic Service Implementation
- 4. Diagnostic and Communication Management Functional Unit
- 5. Data Transmission Functional Unit
- 6. Stored Data Transmission Functional Unit
- 7. Input/Output Control Functional Unit
- 8. Remote Activation Of Routine Functional Unit
- 9. Upload/Download Functional Unit
- 10. Non-volatile Server Memory Programming Process

In the diagnostic specification document, delete the Quiescent Current (D0, D1) session from Table in section 3.1 and also remove it from the subfunctions of DiagnosticSessionControl. Additionally, specify the Safety session only in DiagnosticSessionControl and remove it from section 3.1.

30	EESP0615	2. Add NRC24 condition to RequestDownload			2025-10-16	INJUN KO
		3. Add OEUK 13 parameters to Response format and specify that OEUK authorization requires ASK + Rollback				
		4. Revise specification so that upon receipt of functional addresses				
REV . NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV .DATE	REV . BY	
AUTOONE (STANDARD INFORMATION MAMAGEMENT SYSTEM)		REFERENCE	SORT	PREPARED	CHECKED	CHECKED
			APPR	confirm	JeongHwa	MyeongGy
			OV AL	INJUN KO	n Kim	u Jeong
				gnostic		Dongyoul
				2025-10-01	2025-10-14	2025-10-16

Ethernet physical layer diagnostics

ENGINEERING STANDARD

SPEC NO.

PAGE (SHT/SHTS)

ES 95486-00

1/1

REV . NO		EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
30		EESP0615	1. In the diagnostic specification document, delete the Quiscent Current (D0, D1) session from Table in section 3.1 and also remove it from the subfunctions of DiagnosticSessionControl. Additionally, specify the Safety session only in DiagnosticSessionControl and remove it from section 3.1. 2. Add NRC24 condition to RequestDownload 3. Add OEUK 13 parameters to Response format and specify that OEUK authorization requires ASK + Rollback 4. Revise specification so that upon receipt of functional addresses (18DB33F1 / DoIP: E000 / E400), the controller will stop forwarding diagnostic requests to other controllers 5. Add data parameters to confirm F1C0 External Bootloader state 6. Assign DIDs for Fault diagnostic functions, and assign DIDs for Ethernet physical layer diagnostics		2025-10-16	INJUN KO

REV . NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
29	EESO1767	1. 세션 컨트롤 7DF 10 01 대신 7DF 수신시 진단 요청 메시지 전송 중단(2.8절) 2. 고전압 전원 사용 제어기의 전원조건 예외/제약 추가 (3.1.1.3절) 3. RequestRoutineResult(0x03) subfunction에서 RequestRoutineStatus 허용 (Table 8-1) 4. 하나의 제어기에서 진단통신이 논리적으로 구분되어있는 경우, 엔진조건 E2E 기준 일치 (3.1.1.3절) 5. IVD의 CRC32 삭제 SHA256 이상으로 변경(Annex C, Annex K) 6. Security Access 표 4-81 No9의 Att_cnt +1 조건 삭제, DID F1C0 이름 변경 (Table 4-81, Annex C)		2024-12-12	INJUN KO
28	EESO0588	1. Security access 사양 상세 명세 (4.3절) 2. DID F1C0 추가(Annex. C) 3. SW Unit 상한 확대 : F1B1~F1BF, F1C1~F1CF (Annex. C) 4. OBD법규 제어기 대상 ClearDiagnosticInformation 서비스의 엔진조건 추가(3.1절) 5. 사양서 기타 오기/누락사항		2024-08-07	INJUN KO
27	EESN0824	1. 미슬립 검출 기능 DID 적용 : AnnexC 2. OBD on UDS 법규 반영 -1)ReadDTCInformation Service Subfunction 추가 : Table 6-13 -2)DTC format Identifier(SAE_J2012-DA_DTCFormat_04) : Table 6-14 -3) ClearDiagnosticInformation Group 변경 : Annex D Table D-1 -4)Legislative use DID 영역 반영 : Annex C 3. 사양서 오기 수정 -ReadMemory ByAddress : Default session 삭제 3.1절 - RXSWIN figure 수정 : Figure J-13, J-14 - Addressing 범위 오기 수정 2.4.1.2 절		2023-11-30	INJUN KO

REV. NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV. DATE	REV. BY
26	EESN0501	1) 8.1.4 Routine NRC 설명 추가 NRC F4 : Downgrade NRC 72 : Integrity fails 2) Annex C의 DID추가 F1C1~F1CA: Unit이 없을 때 F1C1값의 NRC31 케이스 추가 ED90~ED99 : 미 슬립 DID추가 F1A2 : Application 버전 DID 추가 3) 10.1 본 리프로그래밍 사양은 Legacy이며 ES98765-02 참조 문구 추가 4) Annex A NRC F4, NRC F5 추가 5) Annex J.4 29bit 진단 ID의 경우 Local RXSWIN 응답 예 추가		2023-07-13	INJUN KO
25	EESN0188	[A] 2.2.1 Addressing 29bit CAN ID (Ethdiag / DoIP) 분리 [B] 3.1 ReadMemoryByAddress 보안 레벨 변경 with security access [C] 4.3 Security access OEUK 추가 subfunction(13h/14h) [D] 9.1.1 Requestdownload exception 추가		2023-02-13	INJUN KO
24	EESM0697	1. Annex.C (RXSWINEOLRESULT) DID F1EE 추가 2. ReadDTCInformation 의 reportSupportedDTC(0x0A) Subfunction을 Optional에서 Mandatory로 변경		2022-11-04	INJUN KO
23	EESM0315	1) 표 9.1 IMPORTANT 추가 (Encrypting method) 2) Annex J RXSWIN 사양 상세 업데이트 (Example) 3) 2.2.1 절 29bit Addressing 추가 4) 3.1.1 엔진 Running/Stop 예시 업데이트 5) Annex C 진단기 표출 문구 통일을 위한 Notation 추가		2022-07-15	INJUN KO

REV . NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
22	EESM0169	1. External test equipment 신규 사양 추가 2.7절 2. Engine 조건 변경 (Communication control의 IGN on 삭제) 3.1절 3. Annex C. 다음의 Data들은 ASCII code로 기록(DID F18C~F195) 4. 진단기의 DID표시 순서 변경 Table C-4.1. The order of DID in the diagnostic tool and Notation		2022-03-14	INJUN KO
21	EESL0711	a) 3.1절 표3-1 과 표 3-1.1 의 불일치 수정 b) Annex C, F1C1~F1CA의 DID 이름 변경 'Hash -> IV D'		2021-11-01	INJUN KO
20	EESL0583	1. Annex. C 진단 Data identifier 변경 제어기 Hash DID추가 F1C1~F1CA DID F1A0 User optional로 변경 2. Routine control CheckProgrammingDependancy Downgrade protection을 위한 NRC 10 설명 추가 3. Engine condition 관련 3.1절 사양 추가 : 사양 상세화, Communication control 엔진 조건 추가(GN7 부터 적용)		2021-09-07	INJUN KO
19	EESL0493	1. Annex.J RXSWIN 사양 -RXSWIN 포맷 수정 : 년도 제거 -Central Gateway RXSWIN management 추가 -RXSWIN 수집 조회 분리를 위한 Routine control 추가		2021-07-20	INJUN KO
18	EESL0178	챕터 4.3 Security access lock time -요구사항 Mandatory로 변경, -Lock time 10초에서 180초로 변경 -False attempts 2에서 3회로 변경 Annex J RXSWIN		2021-04-15	INJUN KO
17	EESJ1057	1. 게이트웨이 인증 NRC 추가 (F0~F3) 2. \$22 NRC13/31 ISO 개정 사양 반영 3. \$2A 응답 메시지 ISO 개정 사양 반영		2019-11-29	PILYONG PARK
16	EESJ0249	1. 이더넷 진단기 P2CAN_Client 값 증 대 : 기존 50ms → 변경 80ms 2. 이더넷 제어기와 외부 장비 사이의 STmin 값 정의 : 10ms ~		2019-04-26	PILYONG PARK

REV . NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
15	EESI0648	1) Diagnostic CAN ID 신규 추가 제어기 및 변경사항 일괄 반영 2) 로트 추적 시스템 구축 강화 위한 Data ID(DID) 명확화 및 조건 변경 : F187(차량 제조사 Part Number) / F18B(ECU 제조 일자) / F18C(ECU Serial Number) / F1A1[신규](ECU Supplier 코드) 3) Extended Session에서 Non-Default Session으로 천이 시 Security Unlock 상태 유지 (ICU, ESU, CCU 대상)		2018-09-13	PILYONG PARK
14	EESI0112	Annex. H - 신규 제어기 진단 CAN ID 추가 및 일부 제어기 명칭 변경		2018-03-08	HOJIN JUNG
13	EESI0062	1. 신규 제어기 진단 CAN ID 신규 반영 2. 부정응답 메시지 구조 추가 3. 진단 서비스 엔진 조건 예외처리 문구 추가		2018-01-22	HOJIN JUNG
12	EESH0519	1. 신규 제어기 CAN ID 추가 2. 1, 2절 CAN-FD 관련 내용 모두 삭제 3. 사양 명확화 : 부트 SW에서의 필수 & 옵션 서비스 리스트 문구 추가 4. Routine Control (31hex) Sub-function Cvt 변경 (StartRoutine만 Mandatory) 5. Functional Addressing 사양 명확화 6. D0 Session (암전류 모드) 천이도 추가 7. Supported NRC critical normal mode activity 문구 삭제 (28h/29h/85h/86h) → 대체 문구로 수정 8. Annex. Negative Response Codes 일부 내용 수정 및 NRC 추가 9. 프로그래밍 세션 천이 시 엔진 조건 추가		2017-05-19	HOJIN JUNG
11	EESH0104	1. CAN ID update 2. ENMT 서비스 NRC 추가 3. 19 01 DTC FID description 수정 4. 10서비스 일부 sub-function 영역 삭제 5. CAN-FD 사양 추가		2017-01-16	HOJIN JUNG
10	EESG1513	1. 28/29 서비스 SPRMIB 적용 불가 문구 삭제 2. Diagnostic Service List 일부 서비스 예외 문구 추가 3. 일괄 리딩 필수 DID 추가 (F1A0) 4. 진단 CAN ID 신규 2건 추가		2016-07-11	HOJIN JUNG

REV . NO	EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
9	EESG0763	- CAN ID List 업데이트 - CAN-FD 사양 추가 - DTC 상태 정의 명확화 (Pending 유/무) - 일반 제어기 응답 동작 추가 - 리프로그래밍 시퀀스 문구 변경 - Security Access (27hex) 서브 평션 추가 (11, 41) - Communication Control, ENMT 서브 평션 업데이트 - Default Session Service 업데이트 - 19hex 서브평션 수정 (reportNumberofDTC C→M) - 일괄리딩을 위한 DID 조건 수정 (F18B, C→M) - 엔진 조건 추가 - 엔진 Off 의미 명확화		2016-04-12	HOJIN JUNG
8	EESF2824	1. CAN 진단 ID 추가 및 변경 사항 반영 2. 생기 EOL (End Of Line) 진단 모드 추가	13, 184	2015-11-09	HOJIN JUNG
7	EESF2278	1. STmin 값 관련 예외 문구 추가 - KWP on CAN 표 2-7. ECU Network Layer Parameters 위 사양 표 아래 하기 예외 문구 추가 → 멀티 프레임 전송하는 경우, ECU와 외부 진단 장비 사이에 Gateway가 있고, DNMT (Disable Normal Message Transmission) 사용 불가 시, 제어기 응답과 상관없이 진단기에서 STmin 을 10~12ms 로 연장할 수 있다. 2. 신규 제어기 2개 (4개 진단ID) 반영 → FCS (Floor Console Switch) : 0721/0729 → AAF_3 (Active Air Flap 3) : 0722/072A		2015-09-17	HOJIN JUNG
6	EESF2020	1. STmin 값 변경 2. 신규 CAN ID 추가		2015-07-07	HOJIN JUNG

REV . NO		EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
5	D	EESF1401	1. "Status Of DTC bit 표기에 따른 고장코드상태 정의 명확화" 추가 (표 D-3) 2. "DNMT 명령 후 S3server 동안 어떠한 명령도 수신하지 않을 경우, 일반 메시지 전송 재개한다."문구 추가 3. "CAN 관련 고장 진단은 통신 재개 이후에 실시한다." 문구 추가 4. "리프로그래밍 시, 아래 서비스 절차대로 수행해야 한다." 문구 추가 5. "Security Access " (0x27) Negative Response Code (NRC) 순서 변경 (0x35 → 0x36 → 0x37) 6. CAN ID list (Appendix H) 업데이트 7. "Programming → Extended, 10 03" 세션 상태 천이 그림에 추가 8. "DiagnosticSessionControl 긍정 응답 중 SessionParameterRecord ", U→M 조건 변경 9. "Request message definition #nd memory size #m", M→U 조건 변경 10. InputOutputControlByIdentifier (2F) NRC 0x31(requestOutOfRange) 추가 11. ControlDTCSetting (85) NRC 0x31 삭제 12. InputOutputControlByIdentifier (2F) Table 7-1, 7-2 문구 수정 (사양 명확화) 13. ReadDataByID (22) NRC 0x14 추가 14. Communication Control communication type condition : C →M 조건 변경, Hex value : #01h→ #XXh 변경		2015-03-22	HOJIN JUNG
4	C	EESF4296	Spec 작성 및 관리 '업무표준 주요 내용 반영 - 사양 작성일반 사항 수정 (제목 표기 방법, 날짜, 기타 형식)		2014-11-17	HYUN SOO AHN

REV . NO		EO NO.	DESCRIPTION OF CHANGE	CHANGE PAGE	REV . DATE	REV . BY
3	B	EESE0063	1. [진단 모드 추가] - D0h 세션 (Quiescent-Current Diagnostic Mode) 에서 지원하는 SID 칼럼 추가, D0h 세션에서 10h, 22h 서비스 지원하도록 명시 2. [초기 접속 절차 추가] - 외부 진단장비가 진단을 시작할 때, Functional Request 메시지를 전송하도록 문구 추가 3. [타이밍 파라미터 수정] - CGW를 통해 리프로그래밍 시, 고속 CAN 제어기 STmin값 변경 (2ms → 0.3ms) 4. [CAN ID 수정 및 추가] - CAN ID 30개 변경 (총 174개)		2014-01-21	HYUN SOO AHN
2	A	EESA0966	1. 신규 추가 - ACU Safety System Diag. Session Service - ACU Security Access Service - LKAS Read Service Data Unit - General Negative Response Code - Service Implementation rule - Diagnostic Session Control Diagram - 신규 적용 제어기 진단 ID 2. 변경 - Session Parameter Record - PeriodicReadData 속도 - ReadDTC Status - 사양서 내 오타 및 오류 수정		2010-12-01	SeongWu Mun
1	-	EES90551	Initial Release		2009-04-10	SUNG GU KIM

1. Scope

This standard specifies requirements of diagnostic services, which allow a diagnostic tester (client) to control diagnostic functions in an on-vehicle Electronic Control Unit (server) in all HMC/KMC vehicles that are not applied with CAN-FD protocol via CAN communication.

NOTE) This standard specification is described based on ISO 14229-1/2 and ISO 15765-2/3 except two (2) diagnostic services. StartDiagnostic(0x10) service and CommunicationControl(0x28) service are partially modified to be compatible with ES95485-00.

This standard does not apply to nondiagnostic message transmission, use of the communication data link between two Electronic Control Units.

This standard does not include CARB-Mode test (Emission-related diagnostic services) separately. For the CARB-Mode test, must comply with the OBD regulation document, such as ISO 15031-5 (Road vehicles – Communication between vehicle and external equipment for emissions-related diagnostics – Part 5: Emission-related diagnostic services) or SAE J1979 (E/E Diagnostic Test Modes - Equivalent to ISO/DIS 15031-5)

Service Identifier (hex)	Service Type
00 – 0F	Reserved (OBD service request)
10 – 3E	HMC/KMC service request
3F	Reserved
40 – 4F	Reserved (OBD service response)
50 – 7E	HMC/KMC service positive response
7F	Negative response service identifier
80 – BE	HMC/KMC service request
BF	Reserved
C0 - FE	HMC/KMC service positive response
FF	Reserved

Table 1-1. Service Identifier (SI) Overview

1.1 Normative References

This standard and the descriptions of diagnostic communication protocol are based on the following standards. Regarding implementation related specification which is not described in this HMC/KMC standard and other details, refer to the following standards :

1.1.1 ISO Publications

ISO 15765-2	Road vehicles – Diagnostics Communication over CAN (DoCAN) - Part 2 : Transport protocol and network layer services : 2016
ISO 15765-3	Road vehicles – Diagnostics on CAN - Part 3 : Implementation of unified diagnostic services (UDS on CAN) : 2004
ISO 14229-3	Road vehicles – Unified Diagnostic Services (UDS) - Part 3 : Unified diagnostic services on CAN implementation



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 2/211

(UDS on CAN) : 2012

ISO 14229-2	Road vehicles – Unified diagnostic services (UDS) – Part 2 : Session layer services: 2013
ISO 14229-1	Road vehicles – Unified diagnostic services (UDS) – Part 1 : Specification and requirements : 2013
ISO 15031-6	Road vehicles - Communication between vehicle and external equipment for emissions-related diagnostics - Part 6: Diagnostic trouble code definitions : 2015
ISO 11898-1	Road vehicles - Controller area network (CAN) - Part 1: Data link layer and physical signaling : 2015
ISO 11898-2	Road vehicles - Controller area network (CAN) – Part 2: High-speed medium access unit : 2016

1.1.2 SAE Publications

SAE J1962	Diagnostic Connector : 2016
SAE J2012	Diagnostic Trouble Code Definitions : 2016

1.1.3 HMC/KMC Publications

ES 95485-00	HMC/KMC Engineering Standard – Keyword Protocol (KWP) on CAN Specification
ES 95480-00	HMC/KMC Engineering Standard – High Speed Controller Area Network (CAN) Design Specification
ES 95489-01	HMC/KMC Engineering Standard – ECU Secure Diagnosis Functional Specification

Document Structure

The standard specification is the general specifications establishing the communications concept. For a given controller, it is possible that not everything in the standard specifications will apply.

The device specification (each controller specific) would contains :

- References to the optional parts of standard specifications that are applicable to the controller.
- Specific information (e.g. signals and messages) received and transmitted by that controller.

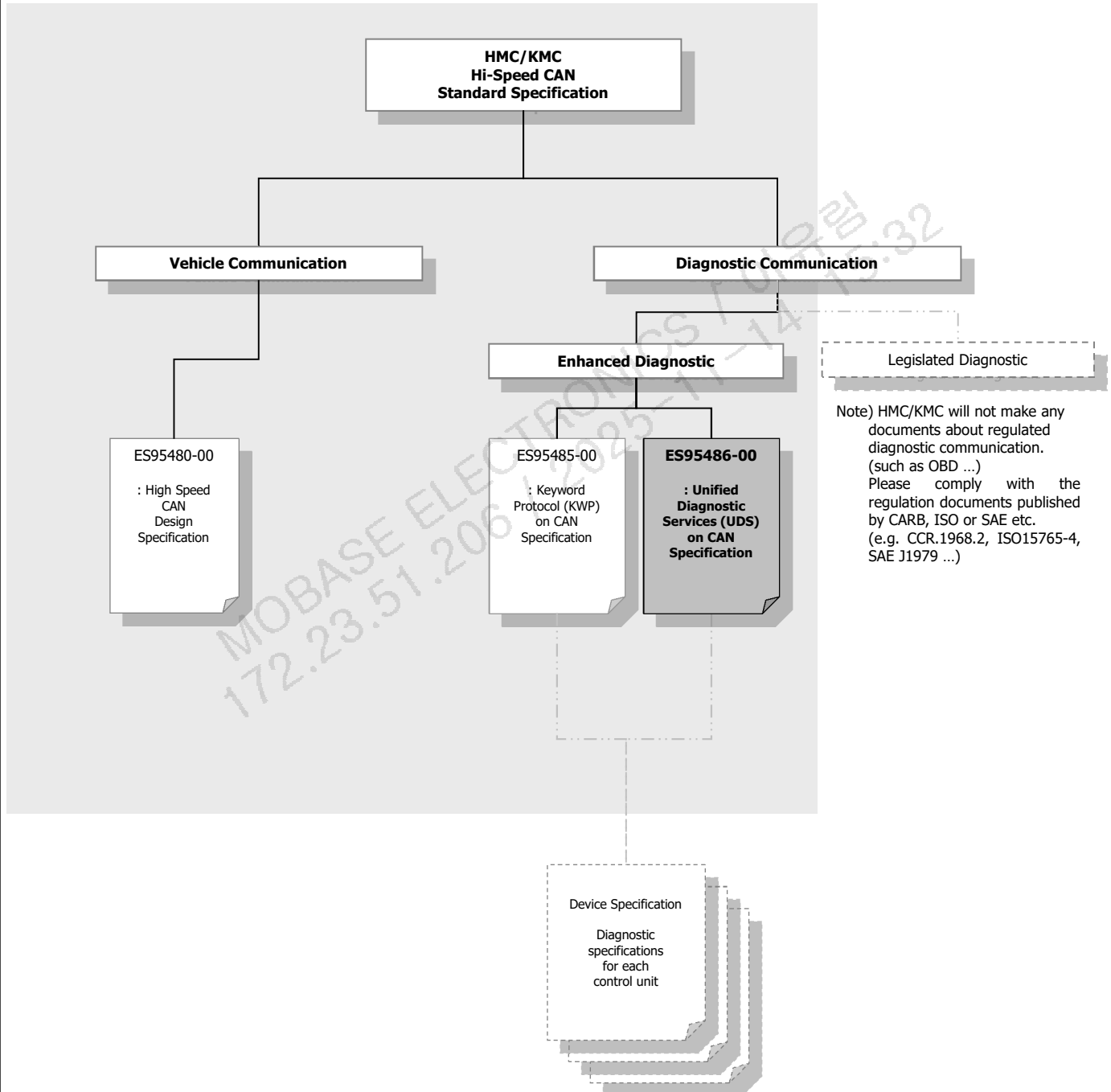


Figure 1-1. HMC/KMC Standard Documents Structure

1.2 Conventions

M	: Mandatory [This parameter has to be present.]
U	: User Optional [This parameter may or may not be present, depending on dynamic usage by the user.]
C	: Conditional [The presence of this parameter depend on other parameters within the service.]
CVT	: Conventions
XXh	: Hexadecimal number
XXd	: Decimal number
XXb	: Binary number

1.3 Definitions And Abbreviations

BS	: Block Size
CAN	: Controller Area Network
CF	: Consecutive Frame
DL(C)	: Data Length (Code)
ECU	: Electrical Control Unit (a generic term for any electrical control unit)
FC	: Flow Control
FF_DL	: First Frame Data Length
FS	: Flow Status
ID	: Identifier
N_Ar	: Network layer timing parameter Ar (r = receiver of the message)
N_As	: Network layer timing parameter As (s = sender of the message)
N_Br	: Network layer timing parameter Br
N_Bs	: Network layer timing parameter Bs
N_Cr	: Network layer timing parameter Cr
N-Cs	: Network layer timing parameter Cs
N_PCI	: network Protocol Control Information
N_PDU	: Network protocol data unit
N_USData	: Network layer Unacknowledged Segmented DATA transfer service name



기술 표준
ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 5/211

SF_DL : Single Frame Data Length
SN : Sequence Number
STmin : Separation Time Minimum

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

2. Protocol Definition

2.1 CAN Physical Layer

The physical layer of diagnostics on CAN shall be compliant to ES95480-0X with the brief definitions as following table.

Parameter	Conditions	
	Chassis - CAN (w/o G/W) Diagnostic - CAN (w/ G/W)	Body - CAN (w/ G/W)
Bus Topology	Line (bus) structure	Star structure
Wiring	Twisted pair wire	Twisted pair wire
Off Board DLC cable Length	Max. 5m	N/A
Data Transfer Rate	500 kbps	100 kbps

Table 2-1. CAN Physical layer definition

2.2 Network Layer

The network layer of the external test equipment and the vehicle ECUs shall be compliant to ISO 15765-2, with the restrictions/additions as defined below.

2.1.1 Addressing

For diagnostic communication on CAN only the normal addressing format as defined in ISO 15765-2 shall be used and 11-bit CAN identifiers and 29-bit CAN identifiers shall be used.

- 11-bit CAN identifiers with normal addressing
- 29-bit CAN identifiers with normal addressing

Two different addressing methods, called physical addressing, and functional addressing are specified for diagnostics.

- Physical addressing (1-to-1 communication) shall be supported for all types of network layer messages.
- Functional addressing (1-to-*n* communication) shall only be supported for Single Frame transmission.
 - ※ Because the response regarding Functional addressing (07DF) request is applied with Physical addressing (Response ID of ECU), ECUs (Server) should transmit multi-frame.

2.1.1.1 CAN identifiers 11bit

- Physical addressing ID : 700-7FF, 580-5FF
- Functional addressing ID : 7DF

2.1.1.2 CAN identifiers 29bit

- Physical addressing ID : 17FF0001~17FFFE00
- Functional addressing ID : 7DF
- Physical addressing ID format :
17FF[TA 1byte][SA 1byte]

*TA : Target address

*SA : Source address

※ SA assign rule

0x00 : External diagnostic tool

0x01~0x10 : Reserved

1F-SG-00002

현대 • 기아 자동차

HYUNDAI • KIA MOTOR

0x11~0xEF : Unique ECU address
0xF0~0xFF : Reserved

e.g.) ECU Source Address : 0xCA
Diagnostic Request ID : 0x17FFCA00
Diagnostic Response ID : 0x17FF00CA

2.1.1.3 CAN ID 2byte Conversion

The Ethernet diagnostic logical address is 2 bytes, and the CAN ID is converted to 2 bytes and used as a logical address. In addition to the Ethernet diagnostic logical address, the local RXSWIN response format CAN ID also requires a 2 byte conversion rule.

11bit CAN Identifier

5Bit of MSB is padded to 0 to extend 11bits to 16bits
Request ID is used for logical address 2byte conversion for Ethernet diagnostic
e.g.) hexadecimal : 0x700₍₁₆₎ -> 0x0700₍₁₆₎
binary : 0b111 0000 0000₍₂₎ -> 0b0000 0111 0000 0000₍₂₎

29bit CAN Identifier

a) Ethdiag

Request ID : 0x17FFCA00
Response ID : 0x17FF00CA
Request Logical address : CA00
Response Logical address : 00CA

Example)

CAN to		Ethdiag		
CAN ID	Payload type	REQ/RES	TargetAddress	SourceAddress
11bit REQ/RES : 700 / 708	8001	REQ	0x0700	-
		RES	0x0708	-
	FCBC	REQ	0x0700	0x1000
		RES	0x1000	0x0708
29bit REQ/RES : 17FFCA00 / 17FF00CA	8001	REQ	0xCA00	-
		RES	0x00CA	-
	FCBC	REQ	0xCA00	0x1000
		RES	0x1000	0x00CA

Ethdiag to				CAN
Payload type	REQ/RES	TargetAddress	SourceAddress	CAN ID
8001	REQ	0x07DF	-	0x7DF
	REQ	0x0700	-	0x700
	RES	0x0708	-	0x708
	REQ	0xCA00	-	0x17FFCA00
	RES	0x00CA	-	0x17FF00CA
FCBC	REQ	0xCA00	0x1000	-
	RES	0x1000	0x00CA	-

b) DoIP

For the bits needed to convert 29 bits to 1 byte, refer to the binary numbers below.
Binary : 1 01DD DDDD DDDD [TA] [SA]



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 8/211

*D is Don't care

Convert 2 bytes by combining the MSB 3bits and the ECU Address(Source Address)
0(Fixed 1bit) + 101(MSB 3bit) + 0000(Reserved 4bit) + SA 1byte

e.g.) ECU Source address is 0xCA
Diagnostic Request : 0x17FFCA00
Diagnostic Response : 0x17FF00CA
Logical address for DoIP diagnostic : 0x50CA

2.1.2 Standardized Diagnostic CAN Identifiers

See Annex. H

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

2.1.3 Data Stream

2.1.3.1 Single frame transmission

Transmission of messages up to six (6) (in case of extended or mixed addressing) or seven (7) (in case of normal addressing) data bytes is performed via transmission of a Single Frame (see Figure 2-1).

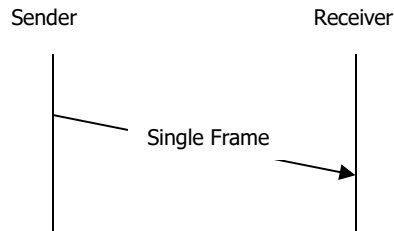


Figure 2-1. Example of unsegmented message

2.1.3.2 Multiple frame transmission

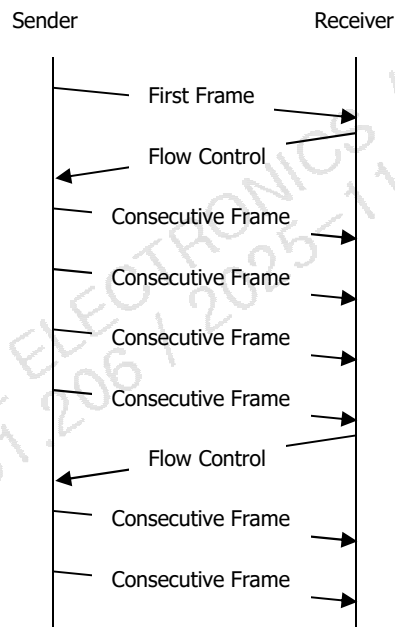


Figure 2-2. Example of segmented message

Transmission of longer messages is performed via segmentation of the message and transmission of multiple messages. Reception of longer messages is performed via reception of multiple messages and reassembly of the received data bytes (concatenation). The multiple messages are called FirstFrame and ConsecutiveFrame. (see Figure 2-2)

The receiver of a multiple message has the possibility of adapting the transmission throughput to its capability by means of the flow control mechanism using the FlowControl protocol data units. Messages longer than six (6) or seven (7) data bytes are segmented into

- a FirstFrame protocol data unit, containing the first five (5) — in the case of extended or mixed addressing — or six (6) — in the case of normal addressing — data bytes, and
- one or more ConsecutiveFrame protocol data units, containing each six (6) or seven (7) data bytes. The CF N_PDU contains only the remaining data bytes, and may therefore be less than six (6) or seven (7) data bytes long.

2.1.4 CAN Message Structure

The DLC (CAN data length code) shall be set to eight (8) and the CAN frame shall include eight (8) bytes . Therefore, all frames must be padded with filler bytes.

- All CAN frames of a request message are padded with a filler byte of 55 hex.
- All CAN frames of a response message are padded with a filler byte of AA hex.

The 8 bytes of the CAN frame defined as follows :

N_AI	N_PCI	N_Data
Address Information (Not used)	Protocol Control Information	Databytes for Application
1 – 8 bytes data of CAN Data Field		

Table 2-3. Data bytes of CAN data field

Because address information(N_AI) is not needed in the normal addressing mode, the above 8 databytes can be defined as follows (Normal Addressing) :

	Byte 1				Bit 3-0	Byte 2	Byte 3	Byte 4 ~ Byte 7	Byte 8
	N_PCIttype								
	7	6	5	4					
Single Frame	0	0	0	0	SF_DL	DATA 1	DATA 2	...	DATA 7
First Frame	0	0	0	1	FF_DL		DATA 1	...	DATA 6
Consecutive Frame	0	0	1	0	SN	DATA 1	DATA 2	...	DATA 7
Flow Control	0	0	1	1	FS	BS	STmin	N/A	

Table2-4. Mapping of parameters and data into CAN frame – Normal Addressing

Hex	Description
0	Single Frame For unsegmented messages, the network layer protocol provides an optimized implementation of the network protocol with the message length embedded in the PCI byte only. SingleFrame (SF) shall be used to support the transmission of messages that can fit in a single CAN frame.
1	First Frame A FirstFrame (FF) shall only be used to support the transmission of messages that cannot fit in a single CAN frame, i.e. segmented messages. The first frame of a segmented message is encoded as an FF. On receipt of an FF, the receiving network layer entity shall start assembling the segmented message.
2	Consecutive Frame When sending segmented data, all consecutive frames following the FF are encoded as ConsecutiveFrame (CF). On receipt of a CF, the receiving network layer entity shall assemble the received data bytes until the whole message is received. The receiving entity shall pass the assembled message to the adjacent upper protocol layer after the last frame of the message has been received without error.
3	Flow Control The purpose of FlowControl (FC) is to regulate the rate at which CF N_PDUs are sent to the receiver. Three distinct types of FC protocol control information are specified to support this function. The type is indicated by a field of the protocol control information called FlowStatus (FS), as defined hereafter.

Table 2-5. Definition of N_PCIttype values

N_PCI	Description	
SF_DL	Single Frame Data Length	
	Value (HEX)	Description
	0	Reserved
	1 ~ 6	The SF_DL is encoded in the low nibble of N_PCI byte #1 value. It shall be assigned the value of the service parameter <Length>.
	7	SF_DL = 7 is only allowed with normal addressing.
	8 ~ F	Invalid
FF_DL	First Frame Data Length	
	Value (HEX)	Description
	0 ~ 6	Invalid
	7	FF_DL = 7 is only allowed with extended or mixed addressing
	8 ~ FFF	The encoding of the segmented message length results in a twelve (12) bit length value (FF_DL) where the least significant bit (LSB) is specified to be bit "0" of the N_PCI byte #2 and the most significant bit (MSB) is bit three (3) of the N_PCI byte #1. The maximum segmented message length supported is equal to 4095 bytes of user data. It shall be assigned the value of the service parameter <Length>
SN	Sequence Number	
	The SequenceNumber (SN) shall be encoded in the lower nibble bits of N_PCI byte #1. The SN shall be set to a value within the range of zero (0) to fifteen (15). - The SN shall start with zero (0) for all segmented messages. - The FF shall be assigned the value zero (0). - The SN of the first CF that immediately follows the FF shall be set to one (1). - The SN shall be incremented by one (1) for each new CF that is transmitted during a segmented message transmission. - When the SN reaches the value of fifteen (15), it shall wraparound and be set to zero (0) for the next CF.	
FS	Flow Status	
	The parameter FlowStatus (FS) indicates whether the sending network entity can proceed with the message transmission.	
	Value (HEX)	Description
	0	ContinueToSend (CTS) The FlowControl ContinueToSend parameter shall be encoded by setting the lower nibble of the N_PCI byte #1 to "0". It shall cause the sender to resume the sending of Consecutive frames. The meaning of this value is that the receiver is ready to receive a maximum of BS number of Consecutive frames.
	1	Wait (WT) The FlowControl Wait parameter shall be encoded by setting the lower nibble of the N_PCI byte #1 to "1". It shall cause the sender to continue to wait for a new FlowControl N_PDU and to restart its N_BS timer.
	2	Overflow (OVFLW) The FlowControl Overflow parameter shall be encoded by setting the lower nibble of the N_PCI byte #1 to "2". It shall cause the sender to abort the transmission of a segmented message and make an N_USData.confirm service call with the parameter <N_Result>=N_BUFFER_OVFLW. (refer to ISO 15765-3) This N_PCI FlowStatus parameter value is only allowed to be transmitted in the FlowControl N_PDU that follows the FirstFrame N_PDU and shall only be used in case the message length FF_DL of the received FirstFrame N_PDU exceeds the buffer size of the receiving entity.
BS	Block Size	

	The units of BS are the absolute number of CF N_PDUs per block.	
	Value (HEX)	Description
	00	The BS parameter value zero (0) shall be used to indicate to the sender that no more FC frames shall be sent during the transmission of the segmented message. The sending network layer entity shall send all remaining consecutive frames without any stop for further FC frames from the receiving network layer entity.
	01 ~ FF	This range of BS parameter values shall be used to indicate to the sender the maximum number of consecutive frames that can be received without an intermediate FC frame from the receiving network entity.
STmin	Minimum Separation Time	
	The STmin parameter value specifies the minimum time gap allowed between the transmission of consecutive frame network protocol data units.	
	Value (HEX)	Description
	00 ~ 7F	0 ms ~ 127 ms, The units of STmin in the range 00 hex ~ 7F hex are absolute milliseconds (ms).
	F1 ~ F9	100 μ s ~ 900 μ s, The units of STmin in the range F1 hex ~ F9 hex are even 100 microseconds (μ s), where parameter value F1 hex represents 100 μ s and parameter value F9 hex represents 900 μ s.
	FA ~ FF	Reserved

Table 2-6. Definition of N_PCI

2.3 Initialization and Timing

2.1.5 Initialization

No separate initialization is required for diagnostic communication on CAN. The communication can be started by giving the request for any service (either functional request or physical request). After power on, the server (ECU) will be in default session. If no other diagnostic session is started, then the default diagnostic session shall be running as long as the service is powered.

2.1.6 Timing

The diagnostic communication on CAN uses two kinds of timing parameters

- Network layer timing parameters and
- Application layer timing parameters

2.1.6.1 Network Layer Timing Parameters

Figure 2-3 shows the CAN message frame flow and timing parameters in the network layer.

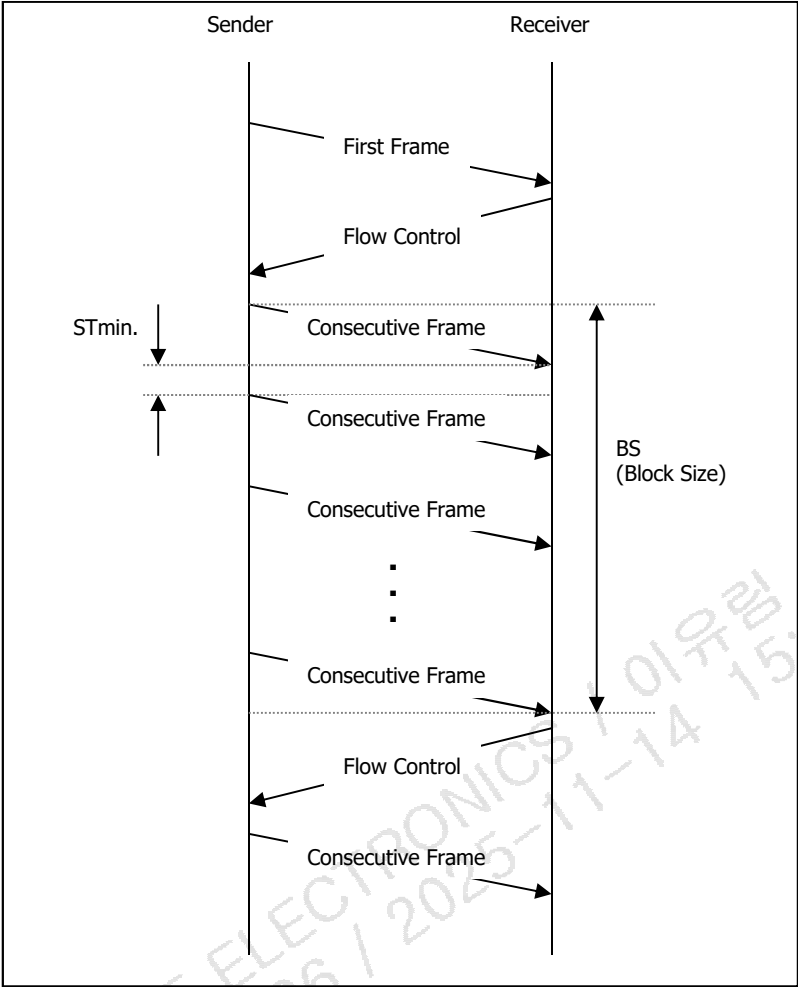


Figure 2-3. Message frame flow diagram

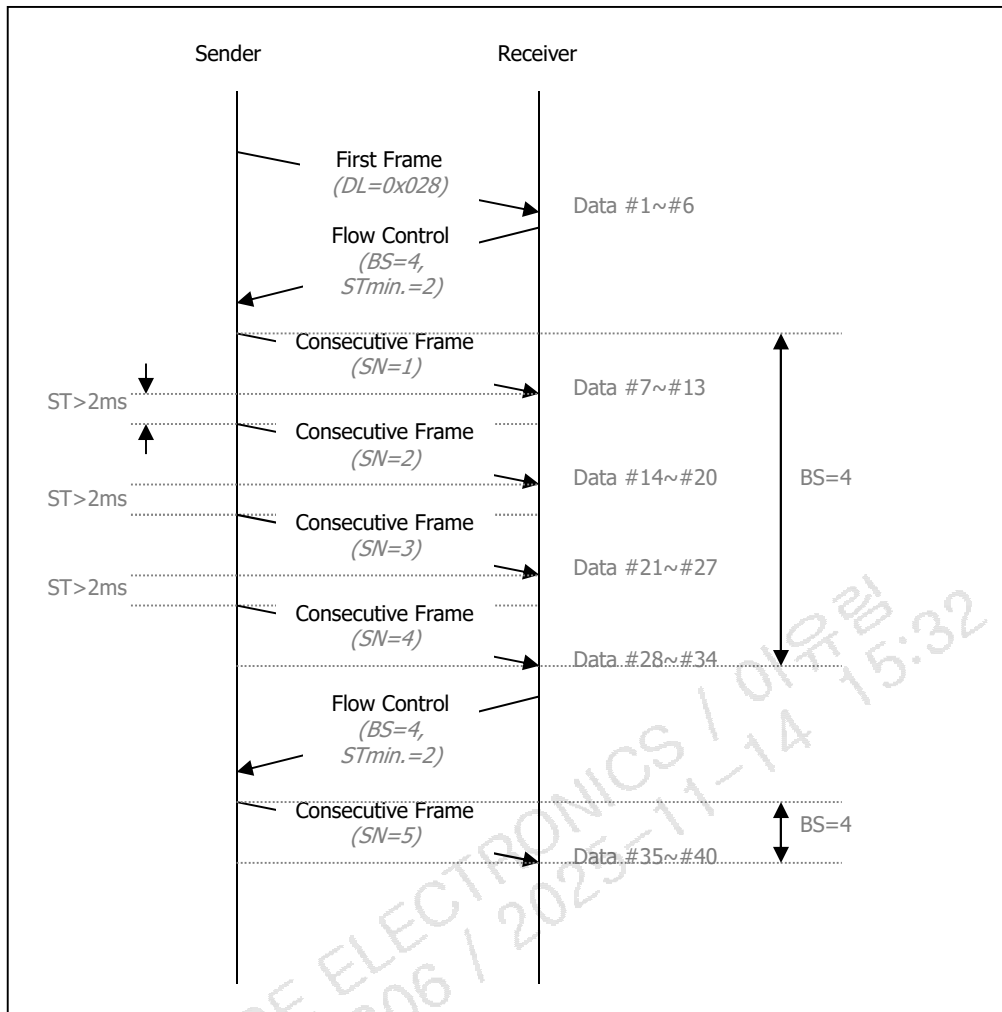


Figure 2-4. Example of message frame flow (STmin = 2, BS = 4)

The timing parameters of Figure 2-3 and each descriptions are defined in the following table respectively.
[STmin]

Diagnostic type	Gateway	Network type	Description	Value
Reprogramming (Boot Area or Programming session)	-	-	ECU shall set the STmin parameter to zero(0). This means that the external test equipment can send consecutive frames as fast as possible.	0 ms
Non reprogramming	No Gateway / Client – Server direct connection	-	The ECU (or external test equipment) can set the STmin value from 0 to 8ms.	0 ~ 8 ms
	Gateway	HS CAN	The ECU (or external test equipment) can set the STmin value from 0.3 to 12ms	0.3ms ~ 12ms
	Gateway	Ethernet	STmin should be equal or larger than 10ms.	10ms ~
※ If ECU receives STmin Value under 0.3ms, ECU shall transmit Consecutive Frame after waiting STmin Value over 0.3ms. (e.g. OBD ECU STmin 0) ※ Background update is not a scope of the Reprogramming because reprogramming is not performed in the BootArea or Programming session with CommunicationControl.				

Table 2-12a. Network layer parameters - STmin

※ In case of transmitting multi-frame, if there were gateway unit(s) between ECU and external test equipment, and Communication Control (DisableRxAndTx) is not able to use, external test equipment can extend STmin as 10~12ms regardless of a response of ECU.

[BS]

Parameter	Description	Recommended Value
BS	The ECU (or external test equipment) can set the BS value up to 8. The external test equipment (or ECU) can send less than 8 Consecutive Frames without additional Flow Control Frames.	0 ~ 8
	In Flash reprogram procedure (when the ECU is in the Boot Area), ECU shall be set the BS parameter to zero(0). This means that the ECU shall be able to receive as many consecutive frames as possible without additional Flow Control Frames. ※ Actually, the number of Consecutive Frames sending from the external test equipment would be set according to the 'maxNumberOfBlockLength' in the ECU's positive response message of RequestDownload service.	0 (Infinite)

Table 2-12b. Network layer parameters - BS

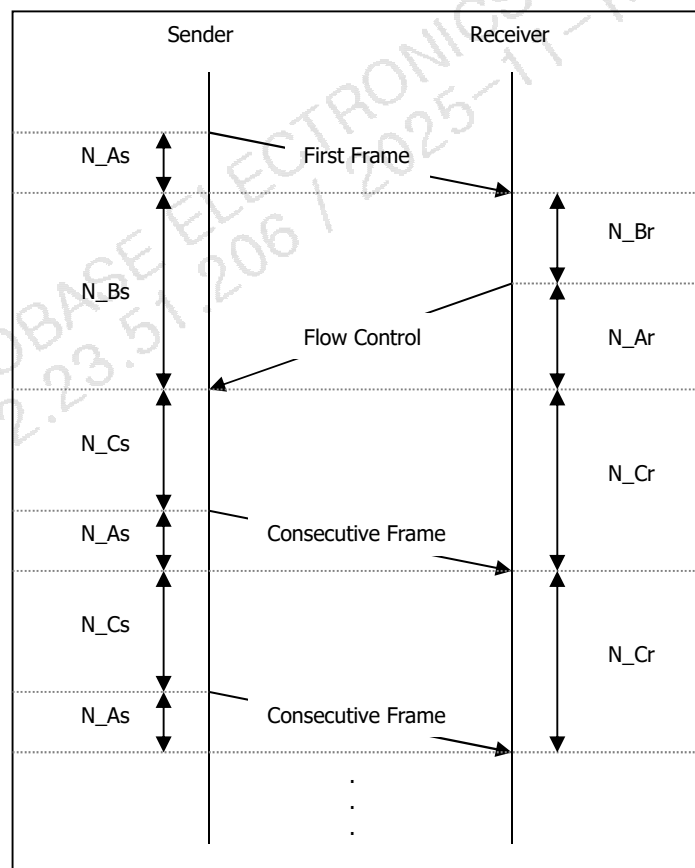


Figure 2-5. Placement of network layer timing parameters

The timing parameters of Figure 2-5 and each descriptions are defined in the following table respectively.

Timing Parameter	Description	Timeout (ms)
------------------	-------------	--------------



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 16/211

N_As	Time for transmission of the CAN frame on the sender side	1000
N_Ar	Time for transmission of the CAN frame on the receiver side	1000
N_Bs	Time until reception of the next Flow Control.	1000
N_Br	Time until transmission of the next Flow Control	N/A
N_Cs	Time until transmission of the next Consecutive Frame.	N/A
N_Cr	Time until reception of the next Consecutive Frame.	1000

Table 2-13. Network layer parameters

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

2.1.6.2 Application Layer Timing Parameters

Figure2-6 shows the diagnostic application message flow and timing parameters in the application layer based on ISO 14229-2.

The interval timing parameters (**P2_{CAN_SERVER}** and **S3_{SERVER}**) and each descriptions are defined in the following table. The **P2_{CAN_SERVER}** parameter is related to performance of a server and **S3_{SERVER}** parameter is related to the timeout of a diagnostic session.

Parameter	Description	Min (ms)	Max (ms)
P2_{CAN_SERVER}	Performance requirement for the server to start with the response message after the reception of a request message.	0	50
P2*_{CAN_SERVER}	Performance requirement for the server to start with the response message after the transmission of a negative response message with response code 78 hex.	0 ¹⁾	5000
P2_{CAN_CLIENT}	Timeout for the client to wait after the successful transmission of a request message for the start of incoming response messages (First Frame of a multi-frame message or a SingleFrame message).	$P2_{CAN_SERVER_max} + \Delta P2_{CAN}$ (80ms for Ethernet ECU)	N/A
P2*_{CAN_CLIENT}	Enhanced timeout for the client to wait after the reception of a negative response message with response code 78 hex for the start of incoming response messages (First Frame of a multiframe message or a SingleFrame message).	$P2*_{CAN_SERVER_max} + \Delta P2_{CAN_rsp}$	N/A
P3_{CAN_CLIENT_Phys}	Minimum time for the client to wait after the successful transmission of a physically addressed request message with no response required before it can transmit the next physically addressed request message.	$P2_{CAN_SERVER_max} + \Delta P2_{CAN}$	N/A
P3_{CAN_CLIENT_Func}	Minimum time for the client to wait after the successful transmission of a functionally addressed request message before it can transmit the next functionally addressed request message in case no response is required or the requested data is only supported by a subset of the functionally addressed servers.	$P2_{CAN_SERVER_max} + \Delta P2_{CAN}$	N/A

1) During the enhanced response timing, the minimum time between the transmission of consecutive negative messages (each with negative response code 0x78 hex) shall be $0.3 * P2*_{CAN_SERVER_max}$, in order to avoid flooding the data link with unnecessary negative response code 0x78 hex messages.

Table 2-14. Application layer timing parameters

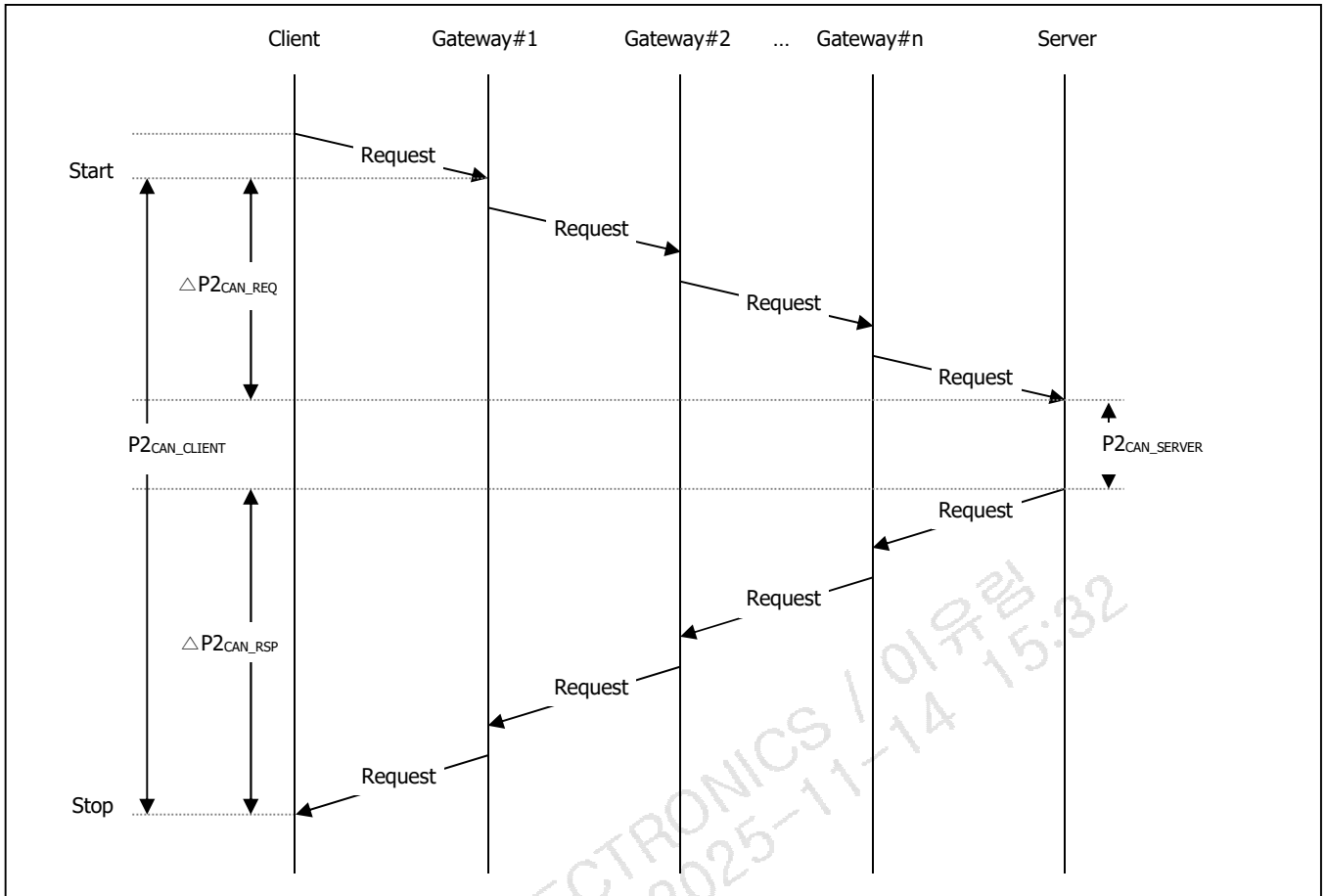


Figure 2-6. Application layer message flow diagram

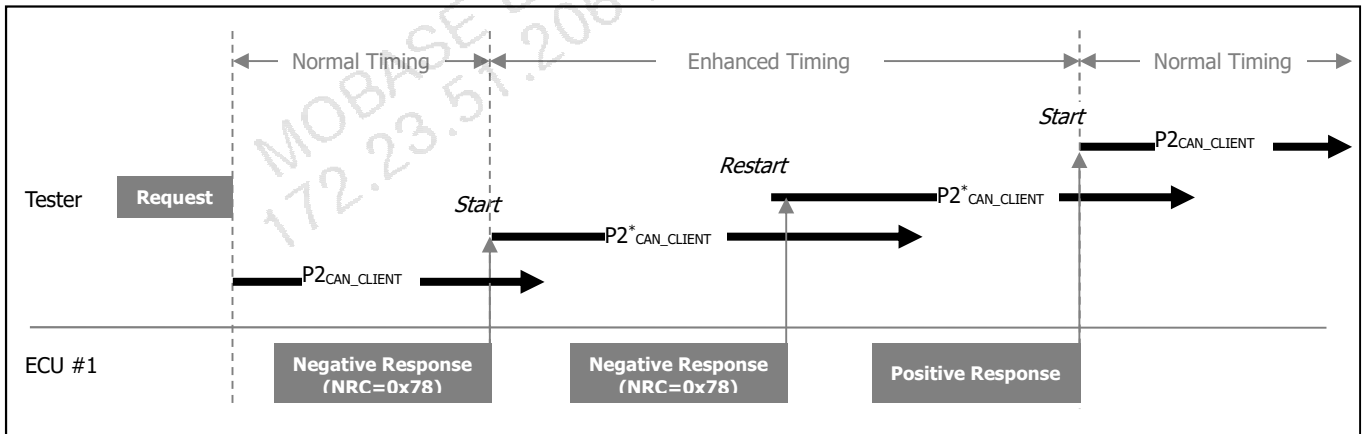


Figure 2-7. P2*CAN_CLIENT timing parameter Definition

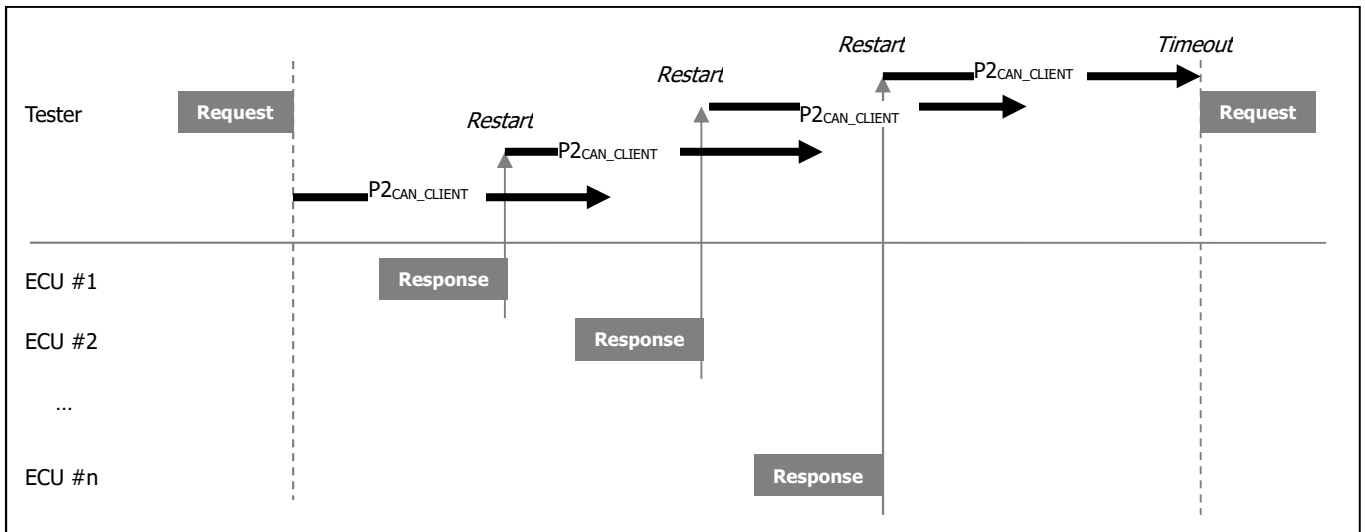


Figure 2-8. P2CAN_CLIENT timing parameter in Functional request

2.1.6.3 Session layer timing parameters

When a diagnostic session other than the defaultSession is started, then a session handling is required which is achieved via the session layer timing parameter given in Table 2-13.

Refer to the ISO15765-3 「6.3.5 Detailed timing parameter descriptions」 to get more information about application & session layer timing parameters.

Timing Parameter	Description	Type	Recommended Value	Timeout
S3 _{Client}	Time between functionally addressed TesterPresent (3E hex) request messages transmitted by the client to keep a diagnostic session other than the defaultSession active in multiple servers (functional communication) or maximum time between physically transmitted request messages to a single server (physical communication). The S3 _{Client} timeout value includes the travel time travel of the message on the network (gateway delays, etc.)	Timer reload value	2000ms	4000ms
S3 _{Server}	Time for the server to keep a diagnostic session other than the defaultSession active while not receiving any diagnostic request message. The tolerance of S3 _{Server} is -0ms, +200ms.	Timer reload value	N/A	5000ms

Table 2-15. Session layer timing parameter definitions

Timing Parameter	Action	Physical and functional communication, using functionally addressed, periodically transmitted TesterPresent request message	Physical communication only, using a physically addressed, sequentially transmitted TesterPresent request message
S3 _{Client}	Initial Start	The completion of the DiagnosticSession Control (10 hex) request message. ※ This is only true for if the session type is a non-default session.	The completion of the DiagnosticSession Control (10 hex) request message in case no response is required. The reception of the DiagnosticSession Control (10 hex) response message in case a response is required.
	Subsequent Start	The completion of the functionally addressed TesterPresent (3E hex)	The completion of any request message in case no response is required.

S3Server		request message, which is transmitted each time the S3Client timer times out.	The reception of any response message in case a response is required.
			An error during the reception of a multi-frame response message.
	Initial Start	The completion of the transmission of a DiagnosticSessionControl positive response message for a transition from the default session to a non-default session, in case a response message is required.	
		Successful completion of the requested action of the service DiagnosticSessionControl (10 hex) for a transition from the default session to a non-default session, in case no response message is required/allowed.	
	Subsequent Stop	The start of a multi-frame request message or the reception of any SingleFrame request message. If the defaultSession is active, the S3Server timer is disabled.	
	Subsequent Start	The completion of any response message that concludes a service execution (final response message) in case a response message is required/allowed to be transmitted (this includes positive and negative response messages). A negative response with response code 78 hex does not restart the S3Server timer.	
		The completion of the requested action (service conclusion) in case no response message (positive and negative) is required/allowed.	
		An error during the reception of a multi-frame request message.	

Table 2-16. Session layer timing parameter definitions

2.1.6.4 Minimum time between client request messages

The minimum time between request messages transmitted by the client is required in order to allow for a polling driven service data interpretation in the server, for example. Based on its normal functionality, a server might process diagnostic request messages with a certain scheduling rate (e.g. 10 ms). The time for the diagnostic service data interpretation scheduler shall be smaller than the performance requirement P2CAN_Server in order to meet the server requirements.

The timing parameter for the minimum time between request message is divided into the following two timing parameters.

- P3CAN_Functional : This timing parameter applies to any functionally addressed request message, because it can be the case that a server is not required to respond to a functionally addressed request message if it does not support the requested data.
- P3CAN_Physical : This timing parameter applies to any physically addressed request message where there is no response required to be transmitted by the server (suppressPosRspMsgIndicationBit = TRUE).

In the case of physical communication where a response is required by the server, the client can transmit the next request immediately after the complete reception of the last response message, because the server has responded completely to the request — which means that the request is completely handled by the server.

The minimum times P3CAN_Physical and P3CAN_Functional, between the end of a physically or functionally addressed request message and the start of a new physically or functionally addressed request message, are defined for the client.

a) The value of P3CAN_Physical will be identical to P2CAN_Server_max for the physically addressed server. The timing applies to any physically addressed request message in any diagnostic session (default and nondefault session) and in case no response is required by the server.

The P3CAN_Physical timer is started in the client each time a physically addressed request message with no response required is successfully transmitted onto the bus, which is indicated via N_USData.con in the client. When the client wants to transmit a new physically addressed request message following a previous



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 21/211

request that was completely handled, then this is only allowed in case the $P3_{CAN_Physical}$ timer is no longer active at the time the client wants to transmit the physically addressed request message. In case $P3_{CAN_Physical}$ would still be active at the point in time the client would like to transmit a new physically addressed request message, then the transmission shall be postponed until $P3_{CAN_Physical}$ is timed out.

b) The value of $P3_{CAN_Functional}$ will be the maximum (worst-case) value of all functionally addressed server's $P2_{CAN_Server_max}$ for any functionally addressed request message in any diagnostic session (default and non-default session).

The $P3_{CAN_Functional}$ timer is started in the client each time a functionally addressed request message with response required or with no response required is successfully transmitted onto the bus, which is indicated via $N_USData.con$ in the client. When the client wants to transmit a new functionally addressed request message following a previous request that was completely handled, then this is only allowed in case the $P3_{CAN_Functional}$ timer is no longer active at the time the client wants to transmit the functionally addressed request message. In case $P3_{CAN_Functional}$ would still be active at the point in time the client would like to transmit a new functionally addressed request message, then the transmission shall be postponed until $P3_{CAN_Functional}$ is timed out.

NOTE) "Completely handled" means that either no response is received in case no response is required, all expected responses to a functionally addressed request are received in case the responding servers are known and responses are required or a $P2_{CAN_Client}$ timeout occurred in case the responding servers are not known and responses are required.

The requirement for the server is that it shall start with its response message within $P2_{CAN_Server}$. This means that the diagnostic data interpretation rate of the server shall be less than $P2_{CAN_Server}$.

MOBASE ELECTRONICS / 172.23.51.206 / 2025-11-14

2.4 Application Layer

2.1.7 Request Message sub-function parameter

The sub-function parameter byte is divided into two part (on-bit-level) as defined in Table below :

Bit position	Description
7	suppressPosRspMsgIndicationBit This bit indicates if a positive response message shall be suppressed by the server. '0' = FALSE, do not suppress a positive response message (a positive response message is required). '1' = TRUE, suppress response message (a positive response message shall not be sent; the server being addressed shall not send a positive response message). Independent of the suppressPosRspMsgIndicationBit, negative response messages are sent by the server(s) according to the restrictions specified 2.4.3~2.4.5.
6 – 0	sub-function parameter value The bits 0-6 of the sub-function parameter contain the sub-function parameter value of the service (00 - 7F hex). Each service utilizing the sub-function parameter byte, but only supporting the suppressPosRspMsgIndicationBit has to support the zeroSubFunction sub-function parameter value (00 hex).

Table 2-17. Sub-function Parameter Structure

NOTE) When requestCorrectlyReceived-ResponsePending (NRC = 78 hex) response code is used, the server shall always send a final response (positive or negative) independent of the suppressPosRspMsgIndicationBit value.

2.1.8 Negative response service

Each diagnostic service has a negative response message according to Table 2-18. The Data Byte #1 is always the specific negative response service identifier. The Data Byte #2 shall be a copy of the service identifier value from the service request message that the negative response message corresponds to.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	Negative Response SID	M	#7Fh	SIDNR
#2d	<Service Name> Request SID	M	#XXh	SIDR!
#3d	Negative Response Code	M	#XXh	NRC_

Table 2-18. Negative response message definition

2.1.9 General server response behaviour

Abbreviation	Description
SuppressPosRspMsgIndicationBit	TRUE ('1') = Server shall NOT send a positive response message FALSE ('0') = Server shall send a positive response message
PosRsp	Abbreviation for positive response message
NegRsp	Abbreviation for negative response message
NoRsp	Abbreviation for NOT sending a positive or negative response message
NRC	Abbreviation for negative response code
ALL	All of the requested data parameters (service without sub-function parameter) of the client request message are supported by the server

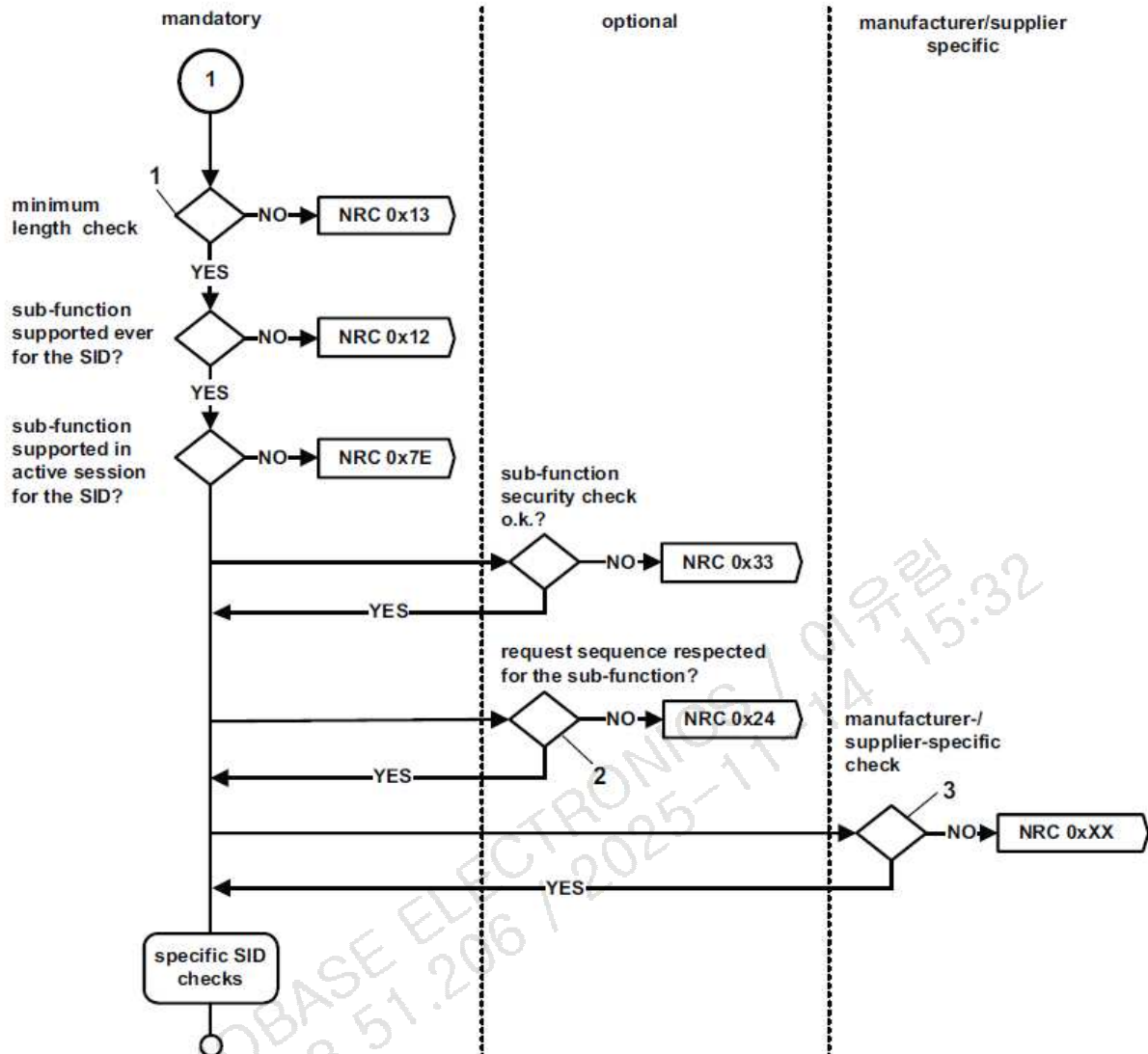


Figure 2-10. General server response behavior for request messages with sub-function parameter

Key

- 1 at least 2 (SID+SubFunction Parameter).
- 2 if sub-function is subject to sequence check, e.g. LinkControl or SecurityAccess.
- 3 refer to the response behaviour (supported negative response codes) of each service.

2.1.10.1 Physically addressed client request message

The server response behaviour specified in this section is referenced in the service description of each service, which supports a sub-function parameter in the physically addressed request message received from the client.

Server Case #	Client Request Message		Server Capability			Server Response		Comments to Server Response
	Addressing Scheme	Sub-Function (SuppressPosRspMsgIndicationBit)	ServiceID Supported	Sub-Function Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Physical	FALSE (bit=0)	YES	YES	At Least 1	PosRsp	-	Server sends positive response
2					At Least 1	NegRsp	NRC=0xXX	Server sends negative response because error occurred reading the data parameters of the request message
3					None		NRC=ROOR	Negative response with NRC 0x31
4			NO	-	-		NRC=SNS or SNSIAS	Negative response with NRC 0x11 or NRC 0x7F
5			YES	NO	-		NRC=SFNS or SFNSIAS	Negative response with NRC 0x12 or NRC 0x7E
6		TRUE (bit=1)	YES	YES	At Least 1	NoRsp	-	Server dose NOT send a response
7					At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
8					None		NRC=ROOR	Negative response with NRC 0x31
9			NO	-	-		NRC=SNS	Negative response with NRC 0x11
10			YES	NO	-		NRC=SFNS	Negative response with NRC 0x12

Table 2-20. Physically addressed request message with sub-function parameter and server response behaviour

Description of server response cases on physically addressed client request messages with subFunction:

- 1) Server sends a positive response message because the service identifier and sub function parameter is supported of the client's request with indication for a response message,
- 2) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier and sub function parameter is supported of the client's request, but some other error appeared (e.g. wrong PDU length according to service identifier and sub function parameter in the request message) during processing of the sub function.
- 3) Server sends a negative response message with the negative response code ROOR (requestOutOfRange) because the service identifier and sub-function parameter are supported but none of the requested data-parameters are supported by the client's request message.
- 4) Server sends a negative response message with the negative response code SNS (serviceNotSupported) or SNSIAS (serviceNotSupportedInActiveSession) because the service identifier is not supported of the client's request with indication for a response message.
- 5) Server sends a negative response message with the negative response code SFNS (subfunctionNotSupported) or SFNSIAS (sub-FunctionNotSupportedInActiveSession) because the service identifier is supported and the sub-function parameter is not supported for the data-parameters (if applicable) of the client's request with indication for a response message.

6) Server sends no response message because the service identifier and sub-function parameter is supported of the client's request with indication for no response message.

NOTE If a negative response code RCRRP (requestCorrectlyReceivedResponsePending) is used, a final response shall be given independent of the suppressPosRspMsgIndicationBit value.

7) Same effect as in 2) (i.e., a negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

8) Same effect as in 3) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon receipt of a physically addressed request messages.

9) Same effect as in 4) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

10) Same effect as in 5) (i.e., the negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response that needs to be sent upon a physically addressed request messages.

2.1.10.2 Functionally addressed client request message

The server response behaviour specified in this section is referenced in the service description of each service, which supports a sub-function parameter in the functionally addressed request message received from the client.

Server Case #	Client Request Message		Server Capability			Server Response		Comments to Server Response
	Addressing Scheme	Sub-Function (SuppressPosRspMsgIndicationBit)	ServiceID Supported	Sub-Function Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Functional	FALSE (bit=0)	YES	YES	At Least 1	PosRsp	-	Server send positive response
2					At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
3					None	NoRsp	-	Server does NOT send a response
4			NO	-	-		-	Server does NOT send a response
5			YES	NO	-		-	Server does NOT send a response
6		TRUE (bit=1)	YES	YES	At Least 1	NoRsp	-	Server does NOT send a response
7					At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
8					None	NoRsp	-	Server does NOT send a response
9			NO	-	-		-	Server does NOT send a response
10			YES	NO	-		-	Server does NOT send a response

Table 2-21. Functionally addressed request message with sub-function parameter and server response behaviour

Description of server response cases on functionally addressed client request messages with subFunction:

1) Server sends a positive response message because the service identifier and sub function parameter is supported of the client's request with indication for a response message,

2) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier and sub function parameter is supported of the client's request, but some other error appeared (e.g. wrong PDU length according to service identifier and sub function parameter in the request message) during processing of the sub function.

3) Server sends no response message because the negative response code ROOR (requestOutOfRange, which is identified by the server because the service identifier and sub function parameter are supported but a required data parameter is not supported of the client's request) is always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

4) Server sends no response message because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

5) Server sends no response message because the negative response codes SFNS (subfunctionNotSupported) and SFNSIAS (sub-functionNotSupportedInActiveSession), which are identified by the server because the service identifier is supported and the sub-function parameter is not supported for the data-parameters (if applicable) of the client's request, are always suppressed in case of a functionally addressed request. The suppressPosRspMsgIndicationBit does not matter in such case.

6) Server sends no response message because the service identifier and sub function parameter is supported of the client's request with indication for no response message.

NOTE) If a negative response code RCRRP (requestCorrectlyReceivedResponsePending) is used, a final response shall be given independent of the suppressPosRspMsgIndicationBit value.

7) Same effect as in 2) (i.e., a negative response message is sent) because the suppressPosRspMsgIndicationBit is ignored for any negative response. This is also true in case the request message is functionally addressed.

8) Same effect as in 3) (i.e., no response message is sent) because the negative response code ROOR (requestOutOfRange, which is identified by the server because the service identifier and sub-function parameter are supported but a required data-parameter is not supported of the client's request) is always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

9) Same effect as in 4) (i.e., no response message is sent) because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

10) Same effect as in 5) (i.e., no response message is sent) because the negative response codes SFNS (sub-functionNotSupported) and SFNSIAS (sub-functionNotSupportedInActiveSession), which are identified by the server because the service identifier is supported and the sub-function parameter is not supported of the client's request, are always suppressed in case of a functionally addressed request message. The suppressPosRspMsgIndicationBit does not matter in such case.

2.1.11 Request message without sub-function parameter and server response behaviour

2.1.11.1 Physically addressed client request message

The server response behaviour specified in this section is referenced in the service description of each service, which does not support a sub-function parameter but a data parameter in the physically addressed request message received from the client.

Server Case #	Client Request Message	Server Capability		Server Response		Comments to Server Response
	Addressing Scheme	ServiceID Supported	Data Parameter Supported (only if applicable)	Message	NRC	
1	Physical	YES	ALL	PosRsp	-	Server send positive response
2			At Least 1		-	Server send positive response
3			At Least 1	NegRsp	NRC=0xXX	Server send negative response because error occurred reading the data parameters of the request message
4			None		NRC = ROOR	Negative response with NRC 0x31
5		NO	-		NRC=SNS or SNSIAS	Negative response with NRC 0x11 or NRC 0x7F

Table 2-22. Physically addressed request message without sub-function parameter and server response behaviour

Description of server response cases on physically addressed client request messages without sub-function (data parameter follows service identifier):

- 1) Server sends a positive response message because the service identifier and all data parameters are supported of the client's request message,
- 2) Server sends a positive response message because the service identifier and a single data parameter is supported of the client's request message,
- 3) Server sends a negative response message (e.g., IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier is supported and at least one data-parameter is supported of the client's request message, but some other error occurred (e.g. wrong length of the request message) during processing of the service.
- 4) Server sends a negative response message with the negative response code ROOR (requestOutOfRange) because the service identifier is supported and none of the requested data parameters are supported of the client's request message,
- 5) Server sends a negative response message with the negative response code SNS (serviceNotSupported) or SNSIAS (serviceNotSupportedInActiveSession) because the service identifier is not supported of the client's request message.

2.1.11.2 Functionally addressed client request message

The server response behaviour specified in this section is referenced in the service description of each service, which does not support a sub-function parameter but a data parameter in the functionally addressed request message received from the client.

Server	Client Request Message	Server Capability	Server Response	Comments to
--------	------------------------	-------------------	-----------------	-------------

Case #	Addressing Scheme	ServiceID Supported	Data Parameter Supported (only if applicable)	Message	NRC	Server Response
1	Functional	YES	YES	PosRsp	-	Server send positive response
2			At Least 1		-	Server send positive response
3			At Least 1	NegRsp	NRC=xx	Server send negative response because error occurred reading the data parameters of the request message
4			None	NoRsp	-	Server does NOT send a response
5		NO	-		-	Server does NOT send a response

Table 2-23. Functionally addressed request message without sub-function parameter and server response behaviour

Description of server response cases on functionally addressed client request messages without sub-function (data parameter follows service identifier):

- 1) Server sends a positive response message because the service identifier and single data parameter is supported of the client's request message,
- 2) Server sends a positive response message because the service identifier and at least one data parameter is supported of the client's request message,
- 3) Server sends a negative response message (e.g. IMLOIF: incorrectMessageLengthOrIncorrectFormat) because the service identifier is supported and at least one, more than one, or all data parameters are supported of the client's request message, but some other error occurred (e.g. wrong length of the request message) during processing of the service,
- 4) Server sends no response message because the negative response code ROOR (request out of range; which would occur because the service identifier is supported, but none of the requested data parameters is supported of the client's request) is always suppressed in case of a functionally addressed request,
- 5) Server sends no response message because the negative response codes SNS (serviceNotSupported) and SNSIAS (serviceNotSupportedInActiveSession), which are identified by the server because the service identifier is not supported of the client's request, are always suppressed in case of a functionally addressed request.

2.1.12 Multiple Concurrent Request Messages with Physical and Functional Addressing

A common server implementation has only one diagnostic protocol instance available in the server which can only handle one request at a time. The rule is that any received message (regardless of addressing mode physical or functional) occupies this resource until the request message is processed (with final response sent or application call without response).

There are only two (2) exceptions which have to be treated separately:

- a) The keep-alive logic is used by a client to keep a previously enabled session active in one or multiple servers. Keep-Alive-Logic is defined as the functionally addressed valid TesterPresent message with suppressPosRspMsgIndicationBit=true and has to be processed by a bypass logic. It is up to the server to make sure that this specific message can not "block" the server's application layer and that an immediately following addressed message can be processed.
- b) If a server supports one or more legislated diagnostic requests and one of these requests is received while a non legislated service (e.g., enhanced diagnostics) is active, then the active service shall be



기술 표준
ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 30/211

aborted, the default session shall be started and the legislated diagnostic service shall be processed.
This requirement does not apply if the programming session is active.

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

2.5 Error handling

Error handling for the application layer and session management to be fulfilled by the client and the server communicating with the client during physical and functional communication shall be in accordance with Tables 2-14, in respect of which it is assumed that the client and the server implement the application and session layer timing according to this part of ISO 14229-1/2/3.

2.1.13 Network / Data Link Layer Error Handling

Error Type			Handling
CAN	DLC	DLC value is smaller than eight (8).	The reception of a CAN frame shall be ignored by the network layer without any further action.
	SF_DL	SF_DL is equal to zero (0) or SF_DL is greater than seven (7).	The network layer shall ignore the received SF.
	FF_DL	FF_DL is greater than the available receiver buffer size.	The network layer shall abort the message reception and send FC with the parameter “Flow Status = Overflow”.
		FF_DL is less than eight (8).	The network layer shall ignore the received FF and not transmit FC.
ST		Reserved ST parameter value.	The sending network entity shall use the longest ST value specified by this document (7F hex = 127 ms) instead of the value received from the receiving network entity for the duration of the ongoing segmented message transmission.
FS		Invalid (reserved) FS parameter	The message transmission shall be aborted.

Table 2-24. Protocol Control Information Error Handling

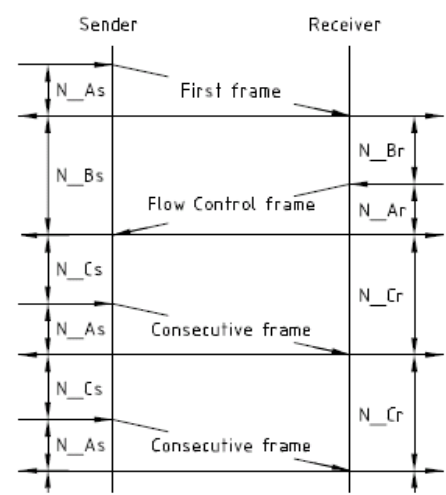
Error Type		Handling	Timing Parameter
N_As	Any message not transmitted in time on the sender side.	Abort message transmission.	
N_Ar	Any message not transmitted in time on the receiver side.	Abort message reception.	
N_Bs	Flow Control not received (lost, overwritten) on the sender side.	Abort message transmission.	
	Preceding First Frame or Consecutive Frame not received (lost, overwritten) on the sender side.		
N_Cr	Consecutive Frame not received (lost, overwritten) on the receiver side.	Abort message reception.	
	Preceding Flow Control not received (lost, overwritten) on the receiver side.		

Table 2-25. Timeout Error Handling

2.1.14 Application Layer Error Handling

Error Type		Client Handling	
		Physical Communication	Functional Communication
Request Transmission	Request message from network layer with a negative result value.	The client shall repeat the last request, after the time $P3_{CAN_Physical}$ following the error indication. Restart $S3_{Client}$ in the case of a physically addressed and sequentially transmitted $TesterPresent$ (because $S3_{Client}$ has been stopped based on the request message transmission).	The client shall repeat the last request, after the time $P3_{CAN_Functional}$ following the error indication.
$P2_{CAN_CLIENT}$ $P2^*_{CAN_CLIENT}$	Timeout	The client shall repeat the last request. Restart $S3_{CLIENT}$ in the case of a physically addressed and sequentially transmitted $TesterPresent$ (because $S3_{CLIENT}$ has been stopped based on the request message transmission).	Where the client does not know the number of servers responding, then this is the indication for the client that no further response messages are expected. No retry of the request message is required. The client shall completely receive all response messages that are in progress until it can continue with further requests. Where the client knows the number of responding servers, then this is the indication for the client that not all expected servers responded. The client shall repeat the request after it has completely received any response message that is in progress at the point in time the timeout occurs.
Response Reception	Response message from network layer with a negative result value.	The client shall repeat the last request. Restart $S3_{Client}$ in the case of a physically addressed and sequentially transmitted $TesterPresent$ (because $S3_{Client}$ has been stopped based on the request message transmission).	The client shall repeat the last request after it has completely received any response message that is in progress at the point in time the error has been indicated.

The client error handling defined shall be performed for a maximum of two (2) times, which means that the worst-case of service request transmissions is three (3).

Table 2-26. Client Error Handling

Error Type		Server Handling
Request Reception	Request message from network layer with a negative result value.	Restart $S3_{Server}$ timer (because it has been stopped based on the previously received $FirstFrame$ indication). The server shall ignore the request.
$P2_{CAN_CLIENT}$ $P2^*_{CAN_CLIENT}$	Timeout	N/A
Response Transmission	Response message from network layer with a negative result value.	Restart $S3_{Server}$ timer (because it has been stopped based on the previously received request message). The server shall <i>not</i> perform a retransmission of the response message.

Table 2-27. Server Error Handling

2.6 External test equipment initialization sequence

The external test equipment shall support the initialization sequence specified in this part.

The purpose of the external test equipment initialization sequence is to prevent the communication collision between external test equipment and a controller which transmits diagnostic request messages to get diagnostic information of other controllers in the vehicle.

If a controller is connected to high-speed CAN network and always transmits diagnostic request message periodically with the same request CAN Identifier of external test equipment, it can cause the communication collision when the external test equipment is connect to the same high-speed CAN network.

To prevent the collision, every time when the external test equipment is connected to the high-speed CAN network and start diagnostic communication, it shall transmit initialization message (functional addressed (7DF hex) DiagnosticSessionControl service (10 hex) with Default Session (81 hex, SPRMIB=True)) at first. Regardless of which service was received, All controllers stop transmit diagnostic request message (if they did) when they received functional addressing (CAN/Ethdiag : 7DF / 18DB33F1 hex, DOIP : E000 / E400 hex) message.

This initialization message has no purpose but the alarm of the external test equipment's connection. Controllers shall always start the standard diagnostic mode when powered up. They still works in the same diagnostic mode (standard diagnostic mode) when the initialization message received.

2.7 External test equipment requirement

The purpose of this section is to unify the operation of external test equipment of HMC/KIA (e.g. EOL). This requirement doesn't affect the ECU functionality.

2.1.15 ECU Identification Data display rules

The content size of the data identifier can be defined as dynamic length. If the data length exceeds the maximum size, external test equipment can display ASCII data with dynamic length.

The display order of Data Identifier is defined in Table C-4.1 The order of DID in the diagnostic tool and Notation. External test equipment shall comply with DID display order. The DID which is not defined in Table C-4.1 is not managed in this specification. It need to be defined in respective ECU specification.

2.1.16 Tester present

External Test Equipment shall send Tester present using functional addressing.

Ofcourse Tester present can be sent using physical addressing. But a number of ECUs can be diagnosed at once in the vehicle diagnostic scenario.

And Diagnostic target ECU can be switchable at all time.

If there is no specific problem, Tester present shall be sent using functional addressing.

3. Diagnostic service implementation

3.1 Diagnostic Service List

Every ECU should support diagnostic services listed below.

Diagnostic Service Name	Req. SID (hex)	Cvt	Diagnostic Modes			Engine / Propulsion system Conditions
			Default Session	Program-ming Session	Extended Diagnostic Session	
			01h	02h	03h	
Diagnostic and Communication Management Functional Unit						
DiagnosticSessionControl	10	M	◆	◆	◆	no restriction, C2
ECUReset	11	M	◆	◆	◆	Engine stop, IGN on
SecurityAccess	27	M	X	◆	◆	no restriction
CommunicationControl	28	M	X	◆	◆	Engine stop
EnableNormalMessageTransmission	29	M	X	◆	◆	no restriction
TesterPresent	3E	M	◆	◆	◆	no restriction
SecuredDataTransmission	84	U	X		◆	no restriction
ControlDTCSetting	85	M	X	◆		no restriction
ResponseOnEvent	86	U	◆		◆	no restriction
LinkControl	87	U	X		◆	no restriction
StopDiagnositcSession	20	M	◆	◆	◆	no restriction
Data Transmission Functional Unit						
ReadDataByIdentifier	22	M	◆		◆	no restriction
ReadMemoryByAddress	23	U	X		◇	no restriction
ReadScalingDataByIdentifier	24	U	◆		◆	no restriction
ReadDataByPeriodicIdentifier	2A	U	X		◆	no restriction
DynamicallyDefineDataIdentifier	2C	U	◆		◆	no restriction
WriteDataByIdentifier	2E	C1	X	◇	◇	Engine stop, IGN on
WriteMemoryByAddress	3D	U	X	◇	◇	Engine stop, IGN on
Stored Data Transmission Functional Unit						
ReadDTCInformation	19	M	◆		◆	no restriction
ClearDiagnosticInformation	14	M	◆		◆	no restriction, C3
Input/Output Control Functional Unit						
InputOutputControlByIdentifier	2F	U	X	X	◆	C4
Remote Activation Of Routine Functional Unit						
RoutineControl	31	C1	○	○	○	Engine stop, IGN on
Upload/Download Functional Unit						
RequestDownload	34	C1	X	◇	X	Engine stop, IGN on
RequestUpload	35	U	X	◇	X	Engine stop, IGN on
TransferData	36	C1	X	◇	X	Engine stop, IGN on
RequestTransferExit	37	C1	X	◇	X	Engine stop, IGN on

◆ : Function possible in the selected diag. session without security access
 ◇ : Function possible in the selected diag. session with security access
 ○ : Specific sub-function possible in the selected diag.Mode (see each function descriptions)
 X : Function impossible in selected diag. session.
 (blank) : No restriction.

C1 : This service function shall be supported if the controller is reprogrammable.

C2 : When the server transitions from any other session (including default session) to Programming session(In this document, Programming session includes Programming mode.), Engine / Propulsion system conditions is Engine stop, IGN on.

C3 : Engine / Propulsion system condition is 'Engine Stop' only for OBD controllers. Refer to J1979, J1979-2/3

C4 : Engine / Propulsion system condition shall be handled in each controller diagnostic specification. The engine/ Propulsion system conditions could be different depending on the DID. Due to functional safety, restriction of the engine / propulsion system conditions could be exist.

- ※ Secured data identifiers, secured memory areas and secured routines require a SecurityAccess service and therefore a non-default diagnostic session
- ※ ACU controllers can or can not support some diagnostic services at default session regardless of security levels. Details refer to the diagnostic specification of the controllers.
- ※ All ECU need to support all diagnostic services at both physical ID and functional ID.
- ※ "engine stop" means "Engine stop" and "not ready" status (HEV, PHEV, EV only)
- ※ In the boot software, 10/11/27/3E/31/34/36/37 hex services must be presented and other services are optionally presented.
- ※ Engine / Propulsion system Condition (Engine stop, IGN on) of some diagnostic services can be changed according to the ECU's specific situation (Exception handling). Refer to the each ECU diagnostic specification in detail.

Table 3-1 – Diagnostic services overview

3.1.1 Summary of Engine / Propulsion system conditions

OTA is not supported to ES95486-00 specification, if you want to refer to OTA related engine condition, See also ES95486-02 specification

Diagnostic Service Name	Req. SID (hex)	Standard Engine / Propulsion system Condition
DiagnosticSessionControl	10	No restriction, Engine stop, IGN on (Programming session : In this document, Programming session includes Programming mode.)
ECUReset	11	Engine stop, IGN on
WriteDataByIdentifier	2E	Engine stop, IGN on
WriteMemoryByAddress	3D	Engine stop, IGN on
InputOutputControlByIdentifier	2F	System specific
RoutineControl	31	Engine stop, IGN on
RequestDownload	34	Engine stop, IGN on
RequestUpload	35	Engine stop, IGN on
TransferData	36	Engine stop, IGN on
RequestTransferExit	37	Engine stop, IGN on
CommunicationControl	28	Engine stop

Table 3-1.1. Diagnostic engine condition overview

3.1.1.1 Engine signal example

This is a general example, the actual signal may be different from the following. Please refer to Network DB

Message	Signal	Vehicle Type
EMS_02_10ms	ENG_EngSta	Emission Engine
HCU_03_10ms	HCU_HevRdySta	Hybrid
VCU_01_10ms	VCU_EvDrvRdySta	EV

Engine Running/Stop : Except Engine running or Drivable, The others are ENGINE STOP

EMS_02		HCU_03		VCU_01	
Value Description	Engine / Propulsion system condition	Value Description	Engine / Propulsion system condition	Value Description	Engine / Propulsion system condition
0x00 Engine stop	STOP	0x00 HEV not ready	STOP	0x00 None	STOP
0x01 Cranking	STOP	0x01 HEV Drivable	Running	0x01 EV Drivable	Running
0x02 Stalled	STOP	0x02 Not Used	STOP	0x02 Not used	STOP
0x03 Running	Running	0x03 Error indicator	STOP	0x03 Error Indicator	STOP
0x07 Fault	STOP				

3.1.1.2 Engine / Propulsion system condition by signal status

Signal status	Engine / Propulsion system Condition
Never received (After start up ECU)	Engine Off
Time out	The last received engine / propulsion system condition
Normal	The last received engine / propulsion system condition

- Never received (After start up ECU) : ECU has never been received engine / propulsion system signal.
- Time out : Cyclic signal has been time out.

3.1.1.3 Limitation and Exception

Bootloader Engine / Propulsion system condition is **Optional**

In the bootloader(FBL), it may be difficult to determine the engine / propulsion system state.
Because FBL(Flash bootloader) supports minimal functionality for the SW update

Engine / Propulsion system Condition (Engine stop, IGN on) of some diagnostic services can be changed according to the ECU's specific situation (Exception handling). Refer to the each ECU diagnostic specification.

E.g.) If some functions regarding EOL(end of line) calibration are needs Engine ON, Engine / Propulsion system condition would be changed by ECU specific requirement.



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 37/211

In the power-on state of the electric vehicle power system, the high voltage main relay is turned on. If the controller using the high voltage power enters the programming session or performs communication control, the application message cannot be transmitted for a while. Therefore, even in the main relay on state The controller using high voltage power, which must transmit the application message, can respond with Negative response code 22 when receiving a request for programming session or communication control. In the electric vehicle, the controllers must perform reprogramming in maintenance service mode.

When a single controller is logically divided into two or more diagnostic IDs and UDS servers for each diagnostic ID, unless there is a specific reason, when the validity of the signals used in the engine condition of each UDS server is determined by a CRC or an alive counter, each UDS server shall match the signal determination criteria to be the same.

e.g.) MCU ID : 05D0 AP ID : 05D1

When an invalid engine signal is received, the MCU may respond with a negative response code 22 and the AP may respond with a positive response. As such, the validity of each may be determined differently, resulting in a difference in response results. Therefore, each UDS server must match the criteria for determining signal validity..

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

4. Diagnostic and Communication Control functional unit

4.1 DiagnosticSessionControl (10 hex) service

The DiagnosticSessionControl service is used to enable different diagnostic sessions in the server(s).

There shall always be exactly one diagnostic session active in a server. A server shall always start the default diagnostic session when powered up. If no other diagnostic session is started, then the default diagnostic session shall be running as long as the server is powered.

If the client has requested a diagnostic session, which is already running, then the server shall send a positive response message and behave as shown in Figure 4-1 that describes the server internal behaviour when transitioning between sessions.

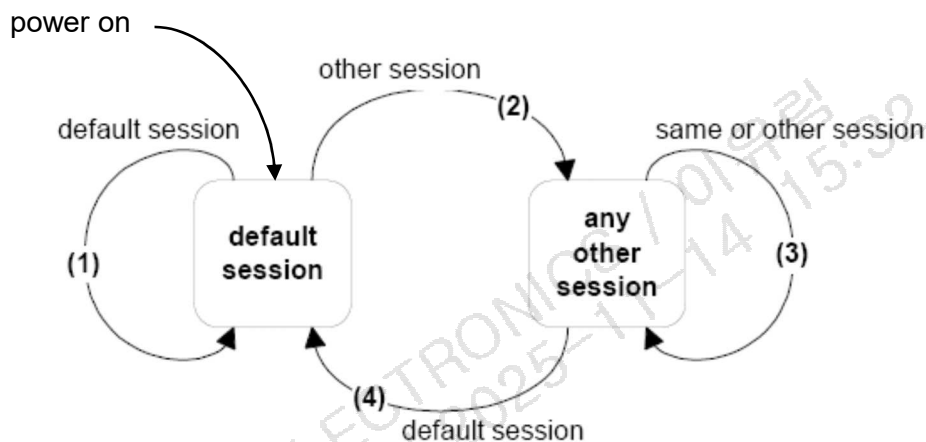


Figure 4-1 – Server diagnostic session state diagram

Diagnostic session transition description:

- 1) When the server is in the defaultSession and the client requests to start the defaultSession then the server shall re-initialize the defaultSession completely. The server shall reset all activated/initiated/changed settings/controls during the activated session. This does not include long term changes programmed into non-volatile memory.
- 2) When the server transitions from the defaultSession to any other session than the defaultSession then the server shall only reset the events that have been configured in the server via the ResponseOnEvent (86 hex) service during the defaultSession.
- 3) When the server transitions from any diagnostic session other than the defaultSession to another session other than the defaultSession (including the currently active diagnostic session) then the server shall (re-) initialize the diagnostic session, which means that each event that has been configured in the server via the ResponseOnEvent (86 hex) service shall be reset and that security shall be enabled. Any configured periodic scheduler shall remain active when transitioning from one non-defaultSession to another or the same non-DefaultSession. The states of the CommunicationControl and ControlDTCSetting services shall not be affected, which e.g. means that normal communication shall remain disabled when it is disabled at the point in time the session is switched.
- 4) When the server transitions from any diagnostic session other than the default session to the defaultSession then the server shall reset each event that has been configured in the server via the ResponseOnEvent (86 hex) service and security shall be enabled. Any configured periodic scheduler shall be disabled. Furthermore the states of the CommunicationControl and ControlDTCSetting services shall be reset, which e.g. means that normal communication shall be reenabled when it was

disabled at the point in time the session is switched to the defaultSession. The server shall reset all activated/initiated/changed settings/controls during the activated session. This does not include long term changes programmed into non-volatile memory.

4.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DiagnosticSessionControl Request Service ID	M	#10h	DSC
#2d	Sub-function = [diagnosticSessionType]	M	#XXh	LEV_ DS_

Table4-1. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	defaultSession This diagnostic session enables the default diagnostic session in the server(s) and does not support any diagnostic application timeout handing provisions. (e.g. no Tester Present service is necessary to keep the session active). If any other session than the defaultSession has been active in the server and the defaultSession in once again started, then the following implementation rules shall be followed : - The server shall stop the current diagnostic session when it has sent the DiagnosticSessionControl positive response message and shall start the newly requested diagnostic session afterwards. - If the server has sent a DiagnosticSessionControl positive response message it shall have re-locked the server if the client unlocked it during the diagnostic session - If the server sends a negative response message with the DiagnostitcSessionControl request service identifier the active session shall be continued.	M	DS
02	programmingSession This diagnosticSession enables all diagnostic services required to support the memory programming of a server. In case the server runs the programmingSession in the boot software, the programmingSession shall only be left via an ECUReset (11 hex) service initiated by the client, a DiagnosticSessionControl (10 hex) service with sessionType equal to defaultSession, or a session layer timeout in the server. In case the server runs in the boot software when it receives the DiagnosticSessionControl (10 hex) service with sessionType equal to defaultSession or a session layer timeout occurs and a valid application software is present for both cases then the server shall restart the application software. This document does not specify the various implementation methods of how to achieve the restart of the valid application software (e.g. a valid application software can be determined directly in the boot software, during the ECU startup phase when performing an ECU reset, etc.).	C	ECUPS
03	extendedDiagnosticSession This diagnosticSession can e.g. be used to enable all diagnostic services required to support the adjustment of functions like "Idle Speed, CO Value, etc." in the server's memory. It can also be used to enable diagnostic services, which are not specifically tied to the adjustment of functions.	M	ECUEDS

04	safetySystemDiagnosticSession This diagnosticSession enables all diagnostic services required to support safety system related functions e.g. airbag deployment.	U	SSDS
05	ECU Programming Mode This diagnosticSession is the same with programmingSession, but both of them should be supported to be compatible with ES95485-00. (Actually, in ES95485-00, ECU Programming Mode would be requested by external test tool with 85 hex. In this case, ECUs which are supported UDS protocol will not response.)	C	ECUPM
06~0F	Reserved	M	
10	Extended Diagnostic Mode This diagnosticSession is the same with extendedDiagnosticSession, but both of them should be supported to be compatible with ES95485-00. (Actually, in ES95485-00, Extended Diagnostic Mode would be requested by external test tool with 90 hex. In this case, ECUs which are supported UDS protocol will not response.)	M	EDM
11~80	Reserved	M	
81	Standard Diagnostic Mode This diagnosticSession is the same with defaultSession, but both of them should be supported to be compatible with ES95485-00.	M	SDM
85	ECU Programming Mode This diagnosticSession is the same with programmingSession, but both of them should be supported to be compatible with ES95485-00.	C	ECUPM
86~8F	Reserved	M	
90	Extended Diagnostic Mode This diagnosticSession is the same with extendedDiagnosticSession, but both of them should be supported to be compatible with ES95485-00.	M	EDM
91~CF	Reserved		
D0	Quiescent-Current Diagnostic-Mode Start The Client requests the Gateway and each Server that Quiescent-Current Diagnostic Mode start. The responses of positive and negative are not necessary.	M	QCDMS
D1	Sleep Mode Start Request The Client requests that the Server transmit from Diagnostic mode to sleep mode. The responses of positive and negative are not necessary.	M	SMSR

C : This session shall be supported if the controller is reprogrammable.

※ In case of Bit7 = 1 of sub-function, a positive response message shall not be sent (81, 85, 90, D0, D1)

Table4-2. diagnosticSessionType definition

Whenever the service STDS (startDiagnosticSession) is called successfully (that includes also calling STDS with the currently active DiagnosticSessionType), the securityAccess rights and the timing parameters will be initialized again. When the CGW(ICU, ESU, CCU) transitions from the extendedDiagnosticSession to the non-default session, the security right should not be initialized (remain unlock condition). (refer to ES95489-01 in detail.)

4.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DiagnosticSessionControl Response Service ID	M	#50h	DSCPR
#2d	diagnosticSessionType	M	#XXh	DS_
	SessionParameterRecord = [	M		SPREC
#3d	P2 _{CAN_SERVER_MAX} (high byte)		#XXh	
#4d	P2 _{CAN_SERVER_MAX} (low byte)		#XXh	
#5d	P2* _{CAN_SERVER_MAX} (high byte)		#XXh	
#6d	P2* _{CAN_SERVER_MAX} (low byte)		#XXh	
	]			
※ P2 _{CAN_SERVER_MAX} : Range = 0~50ms (resolution = 1ms)				
※ P2* _{CAN_SERVER_MAX} : Range = 0~5000ms (resolution = 10ms)				

Table4-3. Positive Response Message definition

4.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request DiagnosticSessionControl are not met.

Table4-4. Supported Negative Response Codes

4.1.4 Implementaion requirements

This service is used by the diagnostic tool to enable different types of diagnostic sessions in a server. In order to execute a diagnostic service the appropriate session has to be started first.

There shall be only one diagnostic session active at a time.

Default Session (01) shall be enabled automatically by the ECU if no diagnostic session has been requested at power up.

An ECU shall return to /Default Session (01) after timeout of another diagnostic session.

An ECU shall be capable of providing all diagnostic functionality defined for the default diagnostic session under normal operating conditions.

The ECU shall first send a DiagnosticSessionControl Positive Response (50 xx) message before the new session becomes active in the ECU.

A DiagnosticSessionControl Positive Response (50 xx) message shall be returned by an ECU if the diagnostic tool requests a session that is already running. If the ECU has already received the same request message previously and performed the requested operation, the ECU shall continue to perform the current operation (i.e. it is not a change of the session).

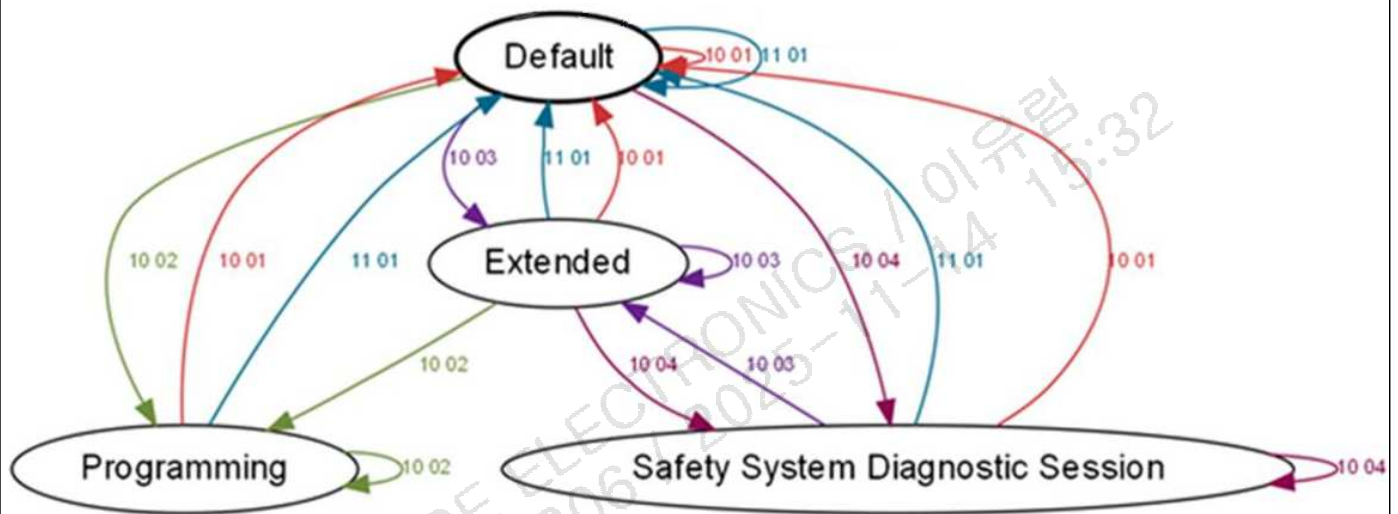
An ECU shall remain in its current diagnostic session if it is not able to switch into the re-requested diagnostic session.

The Tester Present (3E) service shall be used to keep the diagnostic sessions active by re-triggering S3server. Also any other service request keeps the diagnostic session alive.

A functional Tester Present (3E) request without response may be sent at any time, even regardless of any other service in progress.

When receiving or transmitting any diagnostic messages, including \$3E service, the timer will reset. Deactivating a session leads to reinitialisation of all activities, started already.

The following state transitions shall be implemented in the ECU. (In case a session is not available in the ECU, the state transitions from and to the state are omitted):



4.2 ECUReset (11hex) service

The ECUReset service is used by the client to request a server reset.

This service requests the server to effectively perform a server reset based on the content of the resetType parameter value embedded in the ECUReset request message. The ECUReset positive response message (if required) shall be sent before the reset is executed in the server(s). After a successful server reset the server shall activate the defaultSession.

4.1.5 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EcuReset Request Service ID	M	#11h	ER
#2d	Sub-function = [resetType]	M	#XXh	LEV_ RT_

Table4-5. Request Message

Byte pos. In record	Description	Cvt	Mnemonic
00	Reserved	M	
01	hardReset This value identifies a "hard reset" condition which simulates the power-on / start-up sequence typically performed after a server has been previously disconnected from its power supply (i.e. battery). It might result in the re-initialization of both volatile memory and non-volatile memory location to predetermined values.	M	HR
02	keyOffOnReset This value identifies a condition similar to the driver turning the ignition key off and back on. This reset condition should simulate a key-off-on sequence (i.e. interrupting the switched power supply). The performed action is implementation specific and not defined by the standard. Typically the values of non-volatile memory locations are preserved; volatile memory will be initialized.	U	KOFFONR
03	softReset This value identifies a "soft reset" condition, which causes the server to immediately restart the application program if applicable. The performed action is implementation specific and not defined by the standard. A typical action is to restart the application without reinitializing of previously learned configuration data, adaptive factors and other long-term adjustments.	U	SR
04	enableRapidPowerShutDown This value requests the server to enable and perform a "rapid power shut down" function. The server shall execute the function immediately after "key/ignition" is switched off. While the server executes the power down function, it shall transition either directly or after a defined stand-by time to sleep mode. If the client requires a response message and the server is already prepared to execute the "rapid power shut down" function, the server shall send the positive response message prior to the start of the "rapid power shut down" function. The next occurrence of a "key on" or "ignition on" signal terminates the "rapid power shut down" function. The client shall not send any request messages other than the ECUReset with the sub-function disableRapidPowerShutDown in order to not disturb the rapid power shut down function. NOTE) This sub-function is only applicable to a server supporting a stand by-mode	U	ERPSD

05	disableRapidPowerShutDown This value requests the server to disable the previously enabled "rapid power shut down" function.	U	DRPSD
06~7F	Reserved	M	

Table4-6. Request message sub-function parameter definition

4.1.6 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EcuReset Response Service ID	M	#51h	ERPR
#2d	resetType	M	#XXh	RT_
#3d	powerDownTime	C	#XXh	PDT

C : This parameter is present if the sub-function parameter is set to the enableRapidPowerShutDown value (04hex);

Table4-7. diagnosticSessionControl Positive Response Message definition

Definition	Value	Description
powerDownTime This parameter indicates to the client the minimum time of the stand-by-sequence the server will remain in the power down sequence.	00 – FE hex	1 second per count (0-254 seconds powerDowntime)
	FF hex	Failure or time not available

Table4-8. powerDownTime parameter definition

4.1.7 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the ECUReset request is not met.

Table4-9. Supported Negative Response Codes

4.1.8 Implementation rules

The ECUReset positive response message (if required) shall be sent before the reset is executed in the server(s). After a successful server reset the server shall activate the defaultSession.

	Limit(Att_Cnt == Att_Cnt_Limit). A server shall implement a single timer for all security levels(e.g. ASK, OEUK), which means that all security levels share a single timer. When memory initialization such as ECU reset occurs when the delay timer has not yet expired, the controller starts the timer again from 180 seconds. Therefore, the parameter may be managed as volatile. IMPORTANT : this value shall not be initialized by Diagnostic Session Control Service(0x10)	
Att_Cnt_Limit	This value represents the number of false security access attempts before a delay time (Start_Delay) is inserted.	(constant) 3 times
Att_Cnt	this variable tracks the current value of the number of false access attempts. A separate counter is required for each individual security level. A server have same limits for each security level. Att_Cnt shall be implemented for a given SubFunction. If a valid key is received, the count is initialized to zero. And Delay_Timer expired then Att_cnt of security level which have reached Att_Cnt_Limit shall be initialized to zero. IMPORTANT : this value shall not be initialized by ECU reset or Diagnostic Session Control Service(0x10) NOTE : Counters can be managed separately for the boot loader and application.	(non-volatile)0~3 NOTE : This parameters can be used.

Table 4-80 SecurityAccess parameters

The state transition diagram of SecurityAccess is shown in Figure 4-11, and the detailed operation is defined as shown in Table 4-81.

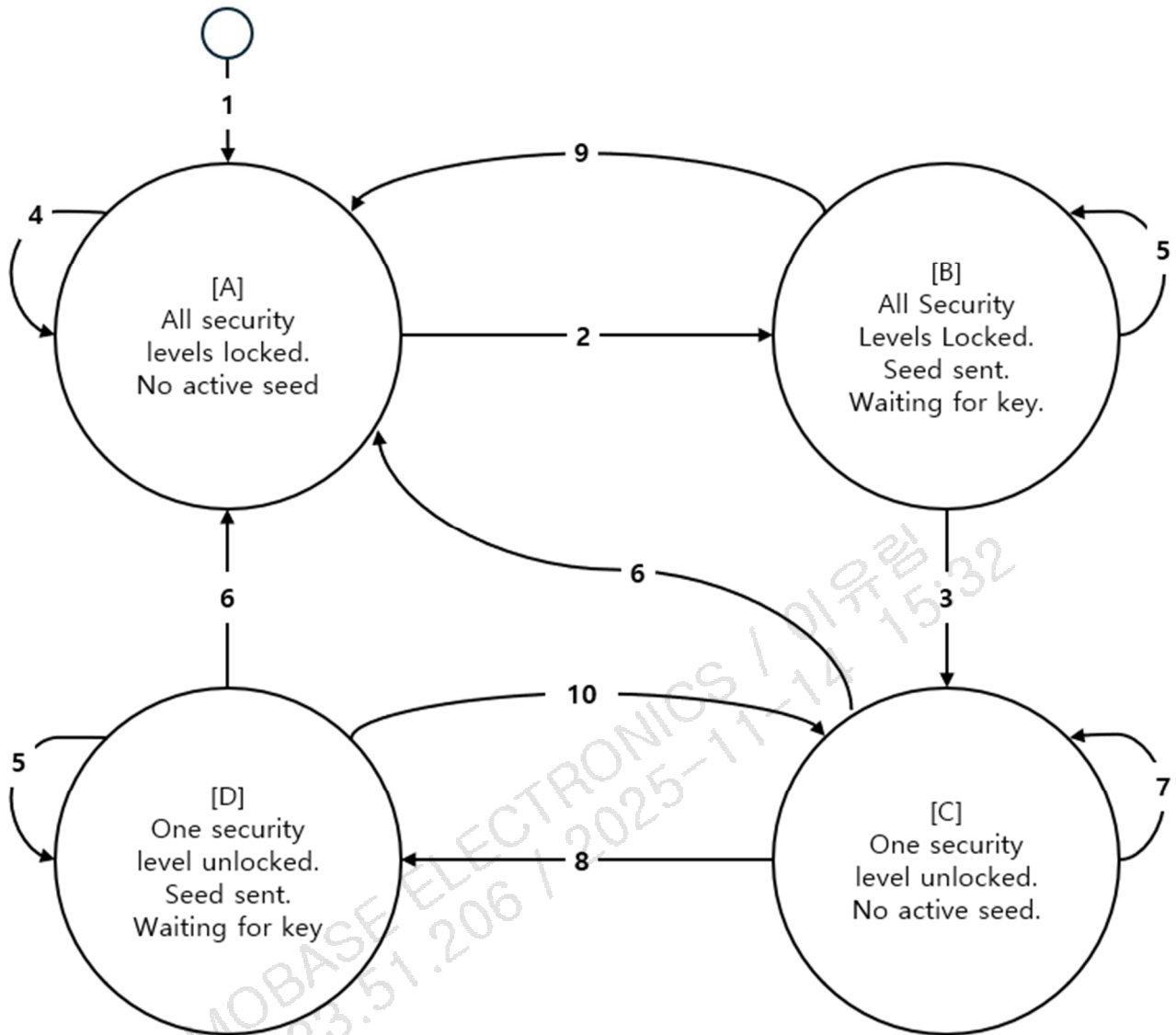


Figure 4-11 SecurityAccess state chart

No	Operation	Condition	Action
1		Start/Restart of ECU Application(e.g. ECU reset, Power cycle, Key cycle, Sleep - > wake transition, etc)	Initialize Att_Cnt (Load Att_Cnt from NVM) If at least one of the Att_Cnt reaches Att_Cnt_Limit then Delay_Timer shall be started.
2		AND SecurityAccess requestSeed received Message length OK Delay expired	Generate and store Seed for the requested securityAccessType(e.g. ASK, OEUK). Transmit SecurityAccess positive response on requestSeed request with Seed as active for the requested securityAccessType.
3		AND SecurityAccess sendKey received SubFunction: yy == xx+1 Sendkey Subfunction must be of the expected securityAccessType (the active stored seed is for the corresponding requestSeed securityAccessType)	Att_Cnt = 0 for SubFunction xx Store Att_Cnt in non-volatile memory Unlock security level for SubFunction xx. Transmit SecurityAccess positive response on sendKey request.



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 48/211

			Message length OK	
			Key OK	
4	OR	AND	SecurityAccess requestSeed received	Transmit negative response NRC13
			Message Length NOK	
			SecurityAccess sendKey received	Transmit negative response NRC 24
		AND	SecurityAccess requestSeed received	Transmit negative response NRC 37
			Message length OK	
			Delay NOT expired	
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A
			Delay Timer Expiration Occurs for SubFunction xx.	Att_Cnt = 0 for SubFunction xx. Store Att_Cnt in non-volatile memory
5		AND	SecurityAccess requestSeed received	Generate new seed and transmit SecurityAccess positive response with the new seed for the requested securityAccessType Save SubFunction: xx = securityAccess Type.
			Static_Seed == False NOTE : Only randomseed (Static_Seed == False) is allowed for HMC. For Static seed is described in ISO14229-1	
6			DiagnosticSessionControl accepted or session timeout occurs.	Start appropriate diagnostic session. Lock ECU.
7		AND	SecurityAccess requestSeed received	Transmit negative response NRC13
			Message Length NOK	
			SecurityAccess sendKey received	Transmit negative response NRC24
		AND	SecurityAccess requestSeed received	Transmit negative response NRC37
			Message length OK	
			Delay NOT expired	
		AND	SecurityAccess requestSeed received	Transmit SecurityAccess positive response with zero seed.
			Requested level is unlocked	
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A
			Delay Timer Expiration Occurs for SubFunction xx.	Att_Cnt = 0 for SubFunction xx Store Att_Cnt in non-volatile memory
8		AND	SecurityAccess requestSeed received	Generate and store Seed for the requested securityAccessType Save SubFunction: xx = securityAccess Type. Transmit SecurityAccess positive response on requestSeed request with Seed as active for the requested securityAccessType.
			Request level is NOT unlocked	
			Message length OK	
			Delay expired	

1F-SG-00002

현대·기아 자동차

HYUNDAI · KIA MOTOR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 49/211

9	OR	AND	SecurityAccess sendKey received	Att_Cnt++ for SubFunction xx Store Att_Cnt in non-volatile memory Transmit negative response NRC 35.
			SubFunction: yy == xx+1	
			Message length OK	
			Key NOK	
			(Att_Cnt+1) < Att_Cnt_Limit	
		AND	SecurityAccess sendKey received	Att_Cnt = Att_Cnt_Limit for SubFunction xx. Start Delay_Timer. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 36.
			SubFunction:yy==xx+1	
			Message length OK	
			Key NOK	
			(Att_Cnt+1) >= Att_Cnt_Limit	
		AND	SecurityAccess sendKey received	Transmit negative response NRC 24
			SubFunction:yy<>xx+1	
		AND	SecurityAccess sendKey received	Transmit negative response NRC 13
			SubFunction:yy==x+1	
			Message length NOK	
			DiagnosticSessionControl accepted or session timeout occurs.	Start appropriate diagnostic session.
			SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A.
		AND	SecurityAccess requestSeed received	Transmit negative response NRC13
			Message Length NOK	
10	OR	AND	SecurityAccess requestSeed received	Transmit SecurityAccess positive response with zero seed
			Requested level is unlocked	
		AND	SecurityAccess sendKey received	Att_Cnt = 0 forSubFunction xx. Store Att_Cnt in non-volatile memory. Lock currently unlocked security level. Unlock security level forSubFunction xx.
			SubFunction: yy==xx+1	
			Message length OK	
			Key OK	
		AND	SecurityAccess sendKey received	Att_Cnt++ for SubFunction xx. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 35.
			SubFunction: yy==xx+1	
			Message length OK	
			Key NOK	
			(Att_Cnt+1) < Att_Cnt_Limit	
			SecurityAccess sendKey received	
		AND	SecurityAccess sendKey received	Att_Cnt = Att_Cnt_Limit for SubFunction xx. Start Delay_Timer for SubFunction xx. Store Att_Cnt in non-volatile memory. Transmit negative response NRC 36.
			SubFunction:yy=xx+1	
			Message length OK	
			Key NOK	
			(Att_Cnt+1) >= Att_Cnt_Limit	

	AND	SecurityAccess sendKey received	Transmit negative response NRC 24
		SubFunction: yy <> xx+1	
	AND	SecurityAccess sendKey received	Transmit negative response NRC 13
		SubFunction: yy == xx+1	
		Message length NOK	
		SecurityAccess request results in a general negative response code (e.g. minimum length, SubFunction supported) according to the general negative response handling (see Annex A).	Transmit negative response code as defined in Annex A.
	AND	SecurityAccess requestSeed received	Transmit negative response NRC 13
		Message Length NOK	

Table 4-81 SecurityAccess State transition

When the Delay Timer expires, the value must be initialized to 0 only for Att_Cnt(X) that started the Delay Timer.

Example)

Sequence	Att_Cnt(ASK)	Att_Cnt(OEUK)	DelayTimer	Action
1	0	0	-	-
2	1	0	-	ASK key NOK
3	2	0	-	ASK key NOK
4	2	1	-	OEUK key NOK
5	2	2	-	OEUK key NOK
6	3	2	Start 180sec Delay_Timer	ASK key NOK
7	0	2	Timer Expired	-
8	0	3	Start 180sec Delay_Timer	OEUK key NOK
9	0	0	Timer Expired	

Table 4-82 Att_Cnt(X) Example

DelayTimer uses Single Time and shares DelayTimer at all levels.

That is, in the case of the controller applying the ASK and the OEUK, the timer of the ASK and the timer of the OEUK are the same, and if the DelayTimer is operated by a specific security level, it is judged that the timer is also operating at another level. Therefore, if there is a request for seed when the DelayTimer does not expire, Respond with NRC 37 regardless of the security level unlock status.

NOTE: Since Att_Cnt is managed separately, only Att_Cnt of the security level that started DelayTimer is initialized to 0 when DelayTimer expires.

Boot SW (Programming session) and Application SW can be allowed to manage Att_Cnt(X) values separately from Boot SW and Application if they are separated by separate SW. However, it is strongly recommended that Att_Cnt(X) values be shared between Boot SW and Application.

If a server supported security, but is already unlocked when a SecurityAccess 'requestSeed' message is received, that server shall respond with a SecurityAccess 'requestSeed' positive response message service with a seed value equal to zero (0). The client shall use this method to determine if a server is locked by checking for a non-zero seed.

The 'requestSeed' subfunction parameter value shall always be an odd number and the corresponding

'sendKey' subfunction parameter value for the same security level shall equal the 'requestSeed' sub-function parameter value plus one.

4.1.9 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = requestSeedType#1 requestSeedType#2]	M	#XXh=[#01h/0Bh #11h/#13h]	LEV_ SAT_RSD1 SAT_RSD2
※ Type#1 : Normal Seed/Key Algorithm (size ≤ 5-byte) If a ECU supports a specific state for flash(Unlock flash state), 0B hex can be used. ※ Type#2 : Advanced Seed/Key (ASK) Algorithm (size > 5-byte) ※ ASK is an improved version of Normal Seed/Key Algorithm. Type#1 can be replaced with Type#2 to improve Security. ※ Type#3 : OEUK Algorithm - 13h				

Table4-10. Request Message definition – sub-function = #01h / #0Bh / #11h / #13h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = sendCertificateandrequestSeed]	M	#41h	LEV_ SAT_SCR SD
#3d .. #nd	securityAccessDataRecord[] = [Certificate]	M	#XXh .. #XXh	SECACCD R
※ In case of application of Secure Access function in CGW, sub-function #41h must be used. (refer to ES95489-01)				

Table4-11. Request Message definition – sub-function = #41h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecurityAccess Request Service ID	M	#27h	SA
#2d	Sub-function = [securityAccessType = sendKey]	M	#XXh=[#02h/0Ch #12h/#14h #42h]	LEV_ SAT_SK
#3d .. #nd	sendKey[] = [key (high byte) .. key (low byte)]	M .. U	#XXh .. #XXh	SEKEY_ KEYHB .. KEYLB
The data size of key from the client if the securityAccessType parameter(=02/0C hex) shall grater than 2-byte / securityAccessType parameter(=12 hex) shall be 8-byte / securityAccessType parameter(= 14/42 hex) shall be 256-byte. ※ 256-byte Key is the value of “PKCS#1 Signature” by “the tester's private key”				

Table4-12. Request Message definition – sub-function = #02h / #0Ch / #12h / #14h / #42h

4.1.10 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
-----------	----------------	-----	-----------	----------

#1d	SecurityAccess Response Service ID	M	#67h	SAPR
#2d	securityAccessType	M	#XXh	SAT_
#3d	securitySeed[] = [	C	#XXh	SECSEED_
..	seed (high byte)	..	..	SEEDHB
#6d	seed (low byte)]	C	#XXh	SEEDLB

C : The presence of this parameter depends on the securityAccessType parameter. It is mandatory to be present if the securityAccessType parameter(=01/0B/11/13/41 hex) indicates that the client wants to retrieve the seed from the server. The data size of seed from the server if the securityAccessType parameter (=01/0B hex) shall greater than 2-byte. The data size of seed from the server if the securityAccessType parameter(=11/13/41 hex) shall be 8-byte.

Table4-13. Positive Response Message definition

4.1.11 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request Security Access are not met.
24	requestSequenceError	M	Send if the 'sendKey' sub-function is received without first receiving a 'requestSeed' request message.
31	requestOutOfRange	M	This code shall be sent if the user optional securityAccessDataRecord contains invalid data.
35	invalidKey	M	Send if an expected 'sendKey' sub-function value is received and the value of the key does not match the server's internally stored/calculated key.
36	exceededNumberOfAttempts	M	Send if the delay timer is active due to exceeding the maximum number of allowed false access attempts.
37	requiredTimeDelayNotExpired	M	Send if the delay timer is active and request is transmitted.

Table4-14. Supported Negative Response Codes

4.1.12 Message Flow Example SecurityAccess – server is in a “locked” state

In the following example, it is assumed:

- the node has a physical request CANId of \$7E0
- the node has a physical response CANId of \$7E8
- AABB hex is a valid seed value for the target node
- CCDD hex is a valid key value for the target node

4.1.12.1 SecurityAccess – requestSeed

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	02 27 01 55 55 55 55 55	Single Frame
7E8	Response	8	04 67 01 AA BB AA AA AA	Single Frame

Table 4-15 SecurityAccess(requestSeed) Message Example



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 53/211

4.1.12.2 SecurityAccess – sendKey

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	04 27 02 CC DD 55 55 55	Single Frame
7E8	Response	8	02 67 02 AA AA AA AA AA	Single Frame

Table 4-16 SecurityAccess(sendKey) Message Example

4.1.13 Message Flow Example SecurityAccess – server is in a “unlocked” state

4.1.13.1 SecurityAccess – requestSeed

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	02 27 01 55 55 55 55 55	Single Frame
7E8	Response	8	04 67 01 00 00 AA AA AA	Single Frame

Table 4-17 SecurityAccess(requestSeed) Message Example

MOBASE ELECTRONICS / 이우림
172.23.51.206 / 2025-11-14 15:32

4.4 CommunicationControl (28 hex) service

The purpose of this service is to switch on/off the transmission and/or the reception of certain messages of (a) server(s) (e.g. application communication messages).

After DisableNormalMessageTransmission request, if the server does not receive any request for S3_{server}, it returns to Default Session and resumes Normal message transmission and “Communication Time Out” monitoring within 500ms.

CAN Fault Diagnosis corresponding server(s) is conducted after communication resumption, because communication resumption time is different depending on the server(s). (Communication is resumed within 500ms, and CAN Fault Diagnosis is conducted after 500ms.)

4.1.14 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	CommunicationControl Request Service ID	M	#28h	CC
#2d	Sub-function = [controlType]	M	#XXh	LEV_CTRLTP
#3d	communicationType	C	#XXh	CTP

Communication Type : See annex B.1 for the coding of the communicationType data parameter in detail. This parameter is present if the sub-function parameter is set to the enableRxAndTx (00 hex) or disableRxAndTx (03hex).

Table4-18. Request Message definition

Hex	Description	Cvt	Mnemonic
00	enableRxAndTx This value indicates that the reception and transmission of messages shall be enabled for the specified communicationType.	M	ERXTX
01	disableNormalMessageTransmission (responseRequired) This value is the same with disableRxAndTx (03 hex) set to suppressPos RspMsgIndicationBit 'FALSE', but both of them have to be supported to be compatible with ES95485-00. This value does not need communicationType (byte#3). This value is the same with communication Type (03hex) which means disable not only normal Messages but also NM Messages.	M	DNMTRR
02	disableNormalMessageTransmission (noResponseRequired) This value is the same with disableRxAndTx (03 hex) set to suppressPos RspMsgIndicationBit 'TRUE', but both of them should be supported to be compatible with ES95485-00. This value does not need communicationType (byte#3). This value is the same with communication Type (03hex) which means disable not only normal Messages but also NM Messages.	M	DNMTNRR
03	disableRxAndTx This value indicates that the reception and transmission of messages shall be disabled for the specified communicationType.	M	DRXTX
04~7F	Reserved	M	

Table4-19. Request message sub-function (controlType) parameter definition

HEX	Parameter Name	Cvt	Mnemonic
01	normalCommunicationMessages This value references all application-related communication (interapplication signal exchange between multiple in-vehicle servers).	M	NCM

02	networkManagementCommunicationMessages This value references all network management related communication.	M	NMCM
03	networkManagementCommunicationMessages and normalCommunicationMessages This value references all network management and application-related communication.	M	NMCMNCM

Table4-20. Request message communication Type definition

4.1.15 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	CommunicationControl Response Service ID	M	#68h	CCPR
#2d	controlType	M	#XXh	CTRLTP

Table4-21. Positive Response Message definition

4.1.16 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot disable/enable the requested communication type.
31	requestOutOfRange	M	The server shall use this response code, if it detects and error in the communicationType parameter.

Table4-22. Supported Negative Response Codes

4.1.17 Message Flow Example

Not only 4.4.4.1 but also 4.4.4.2 would be used to disable server's normal message transmission.

NOTE) 4.4.4.1 is temporarily used to be compatible with ES95485-00

4.1.17.1 Communication Control – Disable Tx and Rx

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7DF	Functional Request	8	02 28 01 55 55 55 55 55	Single Frame
7E8	Response	8	02 68 01 AA AA AA AA AA	Single Frame
7E9	Response	8	02 68 01 AA AA AA AA AA	Single Frame

Table 4-23 CommunicationControl Message Example

4.1.17.2 Communication Control – Disable Tx and Rx

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7DF	Functional Request	8	03 28 03 03 55 55 55 55	Single Frame
7E8	Response	8	02 68 03 AA AA AA AA AA	Single Frame
7E9	Response	8	02 68 03 AA AA AA AA AA	Single Frame

Table 4-24 CommunicationControl Message Example

4.5 EnableNormalMessageTransmission (29 hex) service

This service is used temporarily to compatible with ES95485-00. (It will be terminated in the future.)

This service is used to enable the normal (non diagnostic) message transmission i.e., to resume the activities which are cancelled by successful CommunicationControl request (SID #28h).

The EnableNormalMessageTransmission request message shall be used to indicate the server that normal transmission shall be resumed. The service is used in combination with the CommunicationControl service to once again start the normal message transmission from the server.

The EnableNormalMessageTransmission request message can be used with both physical and functional addressing to resume normal message transmission from an individual server or a group of servers.

The server(s) being addressed shall respond with a negative response message, if it is not possible to enable the normal message transmission. The server(s) shall return positive response message, if the request was called with the parameter responseRequired (#01h) and it is able to enable the normal message transmission. The server(s) shall not return positive response, if the request was called with the parameter noResponseRequired (#02h).

The normal message of this service includes normal message other than diagnostic and Network Management (NM) message. In case enable the message separately, Communication Control service shall be used.

According to the ES95480-00 (Initialization Time Parameter), after 'Enable Normal Message Transmission' service, the server(s) shall restart its communication and "Communication Time Out" function within 500msec.

4.1.18 EnableNormalMessageTransmission service

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EnableNormalMessageTransmission service	M	#29h	ENMT
#2d	Sub-function = [responseRequired, noResponseRequired]	M M M	#XXh [#01h, #02h]	

Table 4-25. EnableNormalMessageTransmission Request Message

4.1.19 EnableNormalMessageTransmission positive Response

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	EnableNormalMessageTransmission service Response Service Id	M	#69h	ENMTPR

Table 4-26. EnableNormalMessageTransmission Positive Response Message

4.1.20 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot enable normal message transmission.

Table4-22. Supported Negative Response Codes

4.6 TesterPresent (3E hex) service

This service is used to indicate to a server (or servers) that a client is still connected to the vehicle and that certain diagnostic services and/or communication that have been previously activated are remain active.

This service is used to keep one or multiple servers in a diagnostic session other than defaultSession.

This can either be done by transmitting the TesterPresent request message periodically or in case of the absence of other diagnostic services to prevent the server(s) from automatically returning to the defaultSession.

4.1.21 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TesterPresent Request Service ID	M	#3Eh	TP
#2d	Sub-function = [zeroSubFunction]	M	#00h or #80h	LEV_ZSUBF

Table4-28. Request Message

Hex	Description	Cvt	Mnemonic
00	zeroSubFunction This parameter value is used to indicate that no sub-function value beside the suppressPosRspMsgIndicationBit is supported by this service.	M	ZSUBF

Table4-29. Request message sub-function parameter definition

4.1.22 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TesterPresent Response Service ID	M	#7Eh	TPPR
#2d	zeroSubFunction	M	#00h	ZSUBF

Table4-30. Positive Response Message definition

4.1.23 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported.
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong

Table4-31. Supported Negative Response Codes

4.7 SecuredDataTransmission (84 hex) service

4.1.24 Security Sub-layer

The purpose of this service is to transmit data that is protected against attacks from third parties – which could endanger data security – according to ISO 15764.

The SecuredDataTransmission service is applicable if a client intends to use diagnostic services defined in this document in a secured mode. It may also be used to transmit external data, which conform to some other application protocol, in a secured mode between a client and a server. A secured mode in this context means that the data transmitted is protected by cryptographic methods.

This section briefly describes the security sub-layer as defined in ISO 15764 (see ISO 15764 for details).

Figure 4-3 illustrates the security sub-Layer as defined in ISO 15764. The security sub-layer has to be added in the server and client application for the purpose of performing diagnostic services in a secured mode.

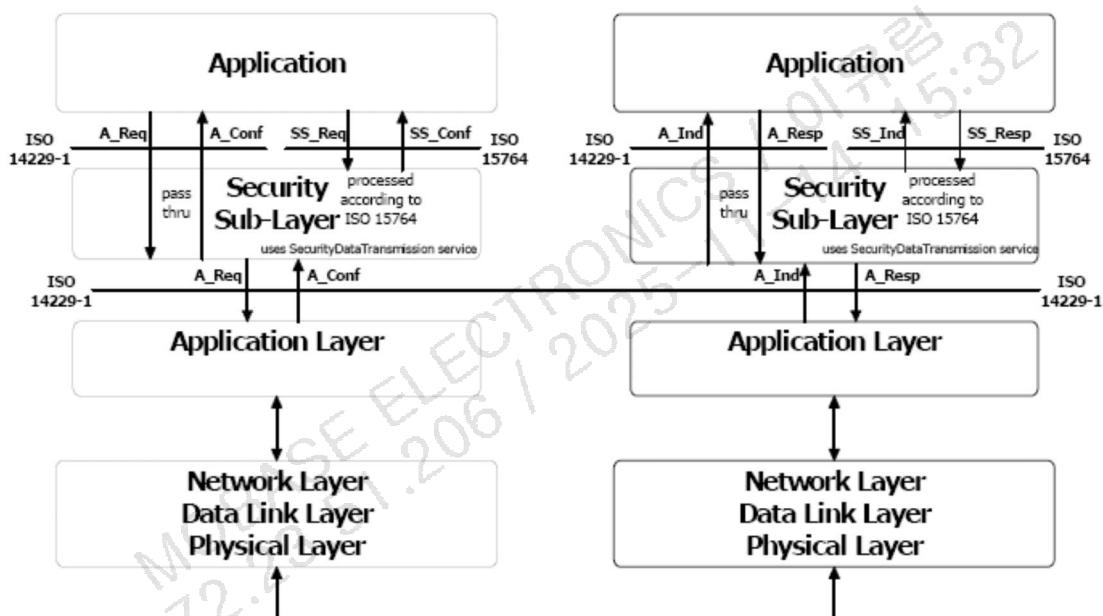


Figure 4-3. Security Sub-Layer implementation

There are two (2) methods to perform diagnostic service data transfer between the client and server(s):

- Unsecured data transmission mode

The application uses the diagnostic Services and Application Layer Service Primitives defined in this document to exchange data between a client and a server. The Security Sub-Layer performs a "Pass-Thru" of data between "Application" and "Application Layer" in the client and the server.

- Secured data transmission mode

The application uses the diagnostic Services or external services and the Security Sub-Layer Service Primitives defined in ISO 15764 to exchange data between a client and a server. The security sub-layer uses the SecuredDataTransmission service for the transmission/reception of the secured data. Secured links must be point-to-point communication. Therefore only physical addressing is allowed, which means that only one server is involved.

The interface of the security sub-layer to the application is according to the ISO/OSI model conventions and therefore provides the following four (4) security sub-layer (SS_) service primitives

- SS_SecuredMode.req : Security Sub-Layer Request
- SS_SecuredMode.ind : Security Sub-Layer Indication
- SS_SecuredMode.resp : Security Sub-Layer Response
- SS_SecuredMode.conf : Security Sub-Layer Confirmation

ISO 14229-1 defines both, confirmed and unconfirmed services. In a secured mode only confirmed services are allowed (suppressPosRspMsgIndicationBit = FALSE). Based on this requirement the following services are not allowed to be executed in a secured mode:

- ResponseOnEvent (86 hex)
- ReadDataByPeriodicIdentifier (2A hex)
- TesterPresent (3E hex)

The confirmed services (suppressPosRspMsgIndicationBit = FALSE) use the four (4) application layer service primitives request, indication, response and confirmation. Those are mapped onto the four (4) security sublayer service primitives and vice versa when executing a confirmed diagnostic service in a secured mode.

The task of the Security Sub-Layer when performing a diagnostic service in a secured mode is to encrypt data provided by the "Application", to decrypt data provided by the "Application Layer" and to add, check, and remove security specific data elements. The Security Sub-Layer uses the SecuredDataTransmission (84 hex) service of the application layer to transmit and receive the entire diagnostic message or message according to an external protocol (request and response), which shall be exchanged in a secured mode.

4.1.25 Security Sub-layer Access

The concept of accessing the security sub-layer for a secured service execution is similar to the application layer interface as described in this document. The security sub-layer makes use of the application layer service primitives.

The following describes the execution of confirmed diagnostic service in a secured mode:

- The client application uses the security sub-layer SecuredServiceExecution service request to perform a diagnostic service in a secured mode. The security sub-layer performs the required action to establish a link with the server(s), adds the specific security related parameters, encrypts the service data of the diagnostic service to be executed in a secured mode if needed and uses the application layer SecuredDataTransmission service request to transmit the secured data to the server.
- The server receives an application layer SecuredDataTransmission service indication, which is handled by the security sub-layer of the server. The security sub-layer of the server checks the security specific parameters, decrypts encrypted data and presents the data of the service to be executed in a secured mode to the application via the security sub-layer SecuredServiceExecution service indication. The application executes the service and uses the security sub-layer SecuredServiceExecution service response to respond to the service in a secured mode. The security sub-layer of the server adds the specific security related parameters, encrypts the response message data if needed and uses the application layer SecuredDataTransmission service response to transmit the response data to the client.
- The client receives an application layer SecuredDataTransmission service confirmation primitive, which is handled by the security sub-layer of the client. The security sub-layer of the client checks the security specific parameters, decrypts encrypted response data and presents the data via the security sub-layer SecuredServiceExecution confirmation to the application.

Figure 11 graphically shows the interaction of the security sub-layer, the application layer, and the application when executing a confirmed diagnostic service in a secured mode.

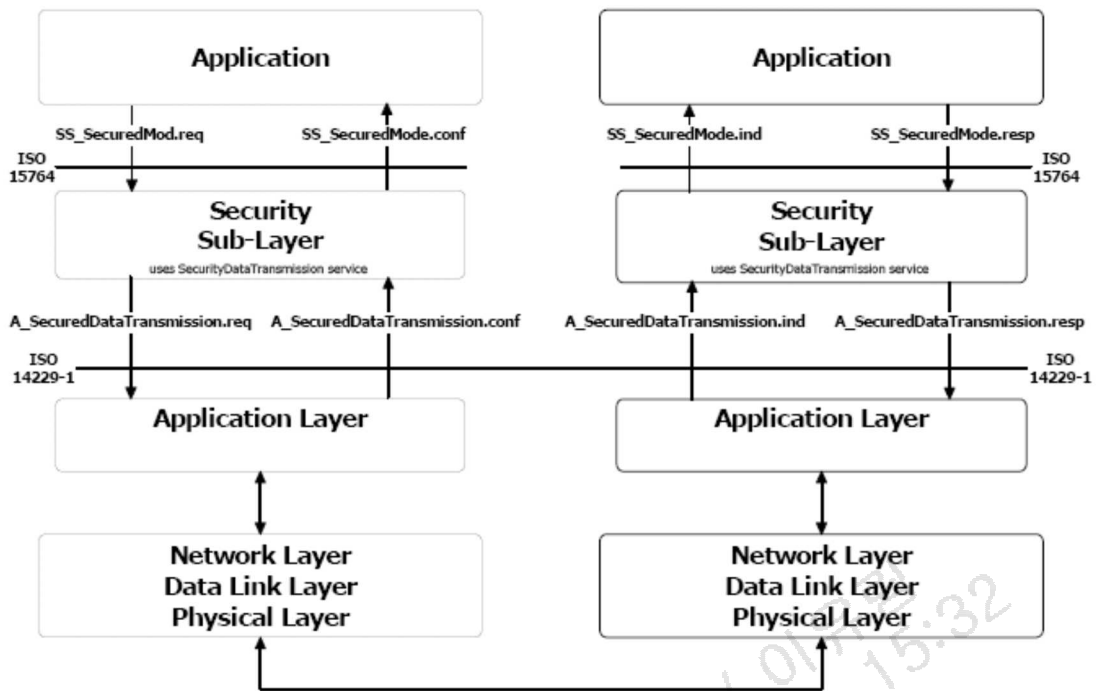


Figure 4-4 Security sub-layer, application layer, and application interaction

4.1.26 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecuredDataTransmission Request Service ID	M	#84h	SDT
#2d	securityDataRequestRecord = [securityDataParameter#1	M	#XXh	SECDRQR_SDP_
..	..	..	..	..
#nd	.. securityDataParameter#m]	M	#XXh	SDP_

Table4-32. Request Message

4.1.27 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	SecuredDataTransmission Response Service ID	M	#C4h	SDTPR
#2d	securedDataResponseRecord[] = [securityDataParameter#1	M	#XXh	SECDRQR_SDP_
..	..	..	..	..
#nd	.. securityDataParameter#m]	M	#XXh	SDP_

Table4-33. Positive Response Message definition

4.1.28 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
38-4F	reservedByExtendedDataLinkSecurityDocu ment	M	This range of values is reserved by extended data link security

Table4-34. Supported Negative Response Codes

4.8 ControlDTCSetting (85 hex) service

The ControlDTCSetting service shall be used by a client to stop or resume the setting of diagnostic trouble codes (DTC's) in the server(s).

The ControlDTCSetting request message can be used to stop the setting of diagnostic trouble codes in an individual server or a group of servers. If the server being addressed is not able to stop the setting of diagnostic trouble codes, it shall respond with a ControlDTCSetting negative response message indicating the reason for the reject.

The update of the DTC status bit information shall continue once a ControlDTCSetting request is performed with sub-function set to "on" or a session layer timeout occurs (server transitions to defaultSession).

If a clearDiagnosticInformation (14 hex) service is sent by the client the ControlDTCSetting shall not prohibit resetting the server's DTC memory. In case a successful ECUReset is performed then this re-enables the setting of DTC's.

4.1.29 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ControlDTCSetting Service ID	M	#85h	CDTCS
#2d	Sub-function = [DTCSettingType]	M	#XXh	LEV_ DTCSTP_

Table4-35. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	On	M	ON
02	Off	M	OFF
03~7F	Reserved	M	

Table4-36. Request message sub-function parameter definition

4.1.30 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ControlDTCSetting Response Service ID	M	#C5h	CDTCSRP
#2d	DTCSettingType	M	#XXh	DTCSTP

Table4-37. Positive Response Message definition

4.1.31 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot perform the requested DTC control functionality

Table4-38. Supported Negative Response Codes

4.9 ResponseOnEvent (86 hex) service

The ResponseOnEvent service requests a server to start or stop transmission of response on a specified event. This service provides the possibility to automatically execute a diagnostic service in case a specified event occurs in the server. The client specifies the event (including optional event parameters) and the service (including service parameters) to be executed in case the event occurs.

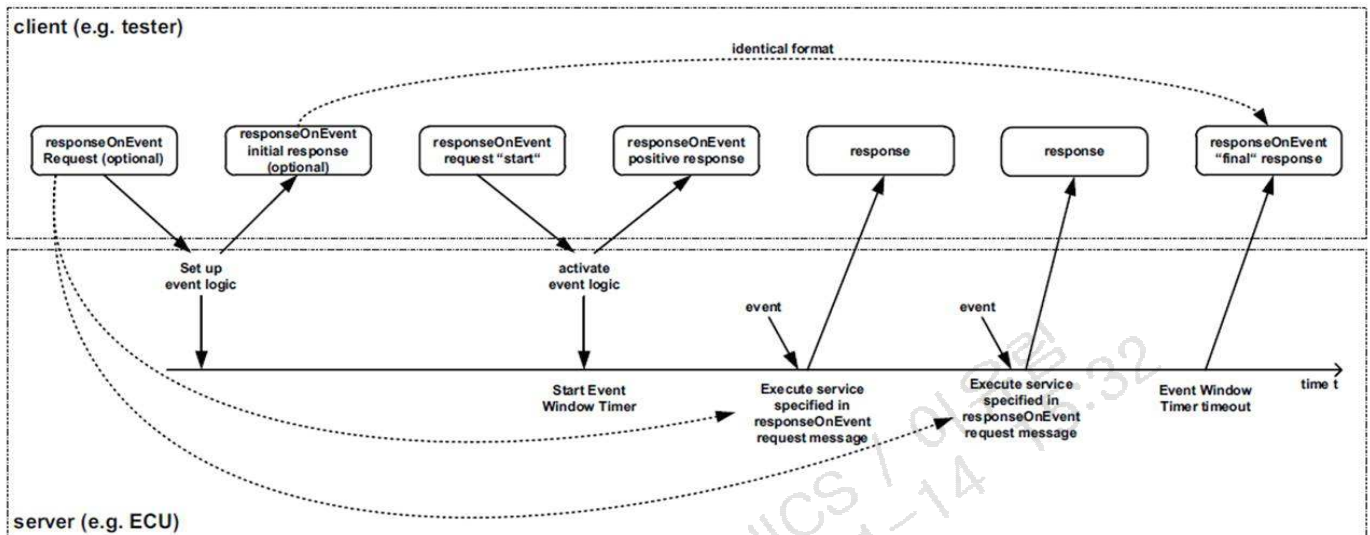


Figure 4-5 ResponseOnEvent service - client and server behaviour

The server shall evaluate the sub-function and data content of the ResponseOnEvent request message at the time of the reception. This includes the following sub-function and parameters:

- eventType,
- eventWindowTime,
- eventTypeRecord (eventTypeParameter #1-#m).

In case of invalid data in the ResponseOnEvent request message a negative response with the negative response code 31 hex shall be sent. The serviceToRespondToRecord is not part of this evaluation. The serviceToRespondToRecord parameter will be evaluated when the specified event occurs, which triggers the execution of the service contained in the serviceToRespondToRecord. At the time the event occurs the serviceToRespondToRecord (diagnostic service request message) shall be executed. In case conditions are not correct a negative response message with the appropriate negative response code shall be sent. Multiple events shall be signalled in the order of their occurrence.

The following requirements shall apply for this service when implemented on CAN.

- a) Multiple ResponseOnEvent services may run concurrently with different requirements (different EventTypes, serviceToRespondTo-Records, ...) to start and stop diagnostic services.
- b) While the ResponseOnEvent service is active, the server shall be able to process concurrent diagnostic request and response messages accordingly. This should be accomplished with a (different) pair of serviceToRespondTo-request/response CAN identifiers. See Figure 4-2. If the same diagnostic request/response CAN identifiers are used for diagnostic communication and the serviceToRespondToresponses, the following restrictions shall apply.
 - 1) The server shall ignore an incoming diagnostic request after an event has occurred and the serviceToRespondTo-response is in progress, until the serviceToRespondTo-response is completed.

- 2) After the client receives any response after sending a diagnostic request, the response shall be classified according to the possible serviceToRespondTo-responses and the expected diagnostic responses that have been sent.
 - 3) If the response is a serviceToRespondTo-response (one of the possible responses set up with ResponseOnEvent-service), the client shall repeat the request after the serviceToRespondToresponse has been received completely.
 - 4) If the response is ambiguous (i.e. the response could originate from the serviceToRespondTo initiated by an event or from the response to a diagnostic request), the client shall present the response both as a serviceToRespondTo-response and as the response to the diagnostic request. The client shall not repeat the request with the exception of NegativeResponseCode busyRepeatRequest (21 hex). (See the negative response code definitions in ISO 14229-1.)
- c) The ResponseOnEvent service shall only be allowed to use those diagnostic services available in the active diagnostic session.
- d) While the ResponseOnEvent service is active, any change in a diagnostic session shall terminate the current ResponseOnEvent service(s). For instance, if a ResponseOnEvent service has been set up during extendedDiagnosticSession, it shall terminate when the server switches to the defaultSession.
- e) If a ResponseOnEvent (86 hex) service has been set up during defaultSession, then the following shall apply :
- 1) If Bit 6 of the eventType subfunction parameter is set to 0 (do not store event), then the event shall terminate when the server powers down. The server shall not continue a ResponseOnEvent diagnostic service after a reset or power on (i.e. the ResponseOnEvent service is terminated).
 - 2) If Bit 6 of the eventType subfunction parameter is set to 1 (store event), it shall resume sending serviceToRespondTo-responses according to the ResponseOnEvent-set up after a power cycle of the server.

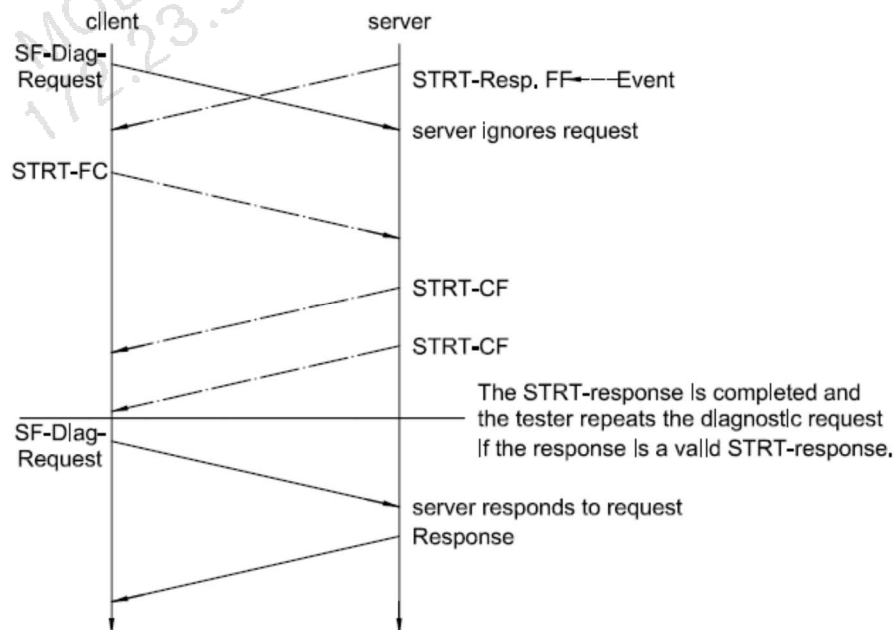


Figure 4-6 Concurrent request when the event occurs

- f) The subfunction parameter value responseRequired = "no" should only be used for the eventType = stopResponseOnEvent, startResponseOnEvent or clearResponseOnEvent. The server shall always

return a response to the event-triggered response when the specified event is detected.

g) The server shall return a final positive response to indicate the ResponseOnEvent (86 hex) service has reached the end of the finite event window, unless one of the following conditions apply:

- 1) if eventTypes do not setup ResponseOnEvent, such as stopResponseOnEvent, startResponseOnEvent, clearResponseOnEvent or reportActivatedEvents;
- 2) if the infinite event window was established
 - if the Service has been deactivated before the event window was closed,
 - Bit 6 of the eventType subfunction parameter is set to 0 (do not store) and the server powers down or resets.

h) When the specified event is detected, the server shall respond immediately with the appropriate serviceToRespondTo-response message. The immediate serviceToRespondTo-response message shall not disrupt any other diagnostic request or response transmission already in progress (i.e. the serviceToRespondTo-response shall be delayed until the current message transmission has been completed — see Figure 4-7).

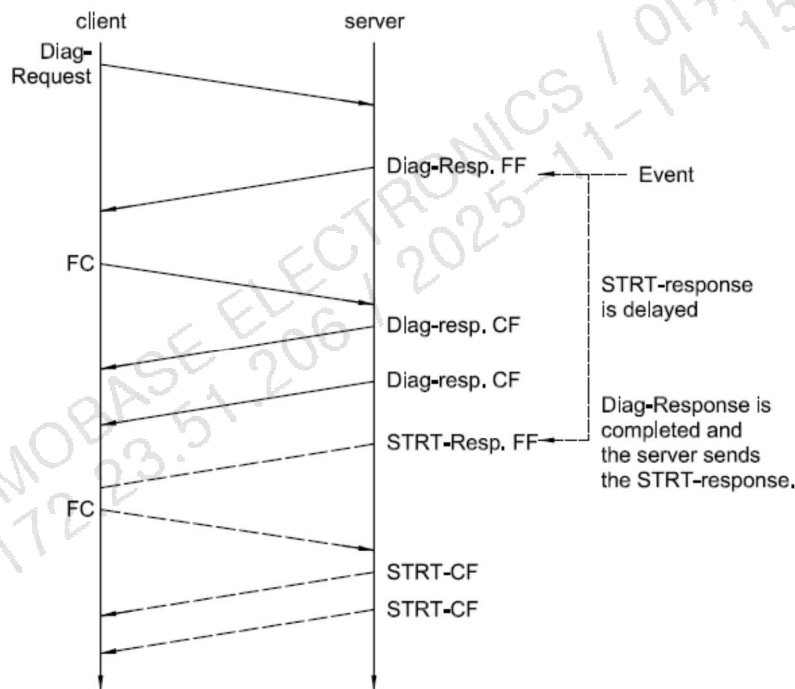


Figure 4-7 Event occurrence during a message in progress

i) The ResponseOnEvent service shall only apply to transient events and conditions. The server shall return a response once per event occurrence. For a condition that is continuously sustained over a period of time, the response service shall be executed only one time at the initial occurrence. In case the eventType is defined so that serviceToRespondTo-responses could occur at a high frequency, then appropriate measures have to be taken in order to prevent back to back serviceToRespondTo-responses. A minimum separation time between serviceToRespondTo-responses could be part of the eventTypeRecord (vehicle-manufacturer-specific).

The following table defines the data parameters applicable for the implementation of this service on CAN.

Recommended services (ServiceToRespondTo)	RequestService Identifier (SId)
ReadDataByIdentifier	22 hex

ReadDTCInformation	19 hex
RoutineControl	31 hex
InputOutputControlByIdentifier	2F hex

Table4-39. Recommended services to be used with the ResponseOnEvent service

4.1.32 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Request Service ID	M	#86h	ROE
#2d	Sub-function = [eventType]	M	#XXh	LEV_ETP
#3d	eventWindowTime	M	#XXh	EWT
#4d	eventTypeRecord[] = [eventTypeParameter 1	C1	#XXh	ETR_ETP1
..	..	..	..	..
#(m-1)+4d	eventTypeParameter m]	C1	#XXh	ETPm
#n-(r-1)-1d	servieToRespondToRecord[] = [serviceId	C2	#XXh	STRTR_SI
#n-(r-1)d	serviceParameter 1	C3	#XXh	SP1
..	..	..	..	..
#nd	serviceParameter r]	C3	#XXh	SPr

C1 : present if the eventType requires additional parameters to be specified for the event to respond to.
C2 : mandatory to be present if the sub-function parameter is not equal to reportActivateEvents, stopResponseOnEvent, startResponseOnEvent, ClearResponseOnEvent
C3 : present if the service request of the service to respond to requires additional service parameters
eventWindowTime : See annex B-2 for this parameter
eventTypeRecord : This parameter record contains additional parameters for the specified eventType.
See definition of each eventType(table 4-33).

Table4-40. Request Message definition

Bit 6 value	Description	Cvt	Mnemonic
0	doNotStoreEvent This value indicates that the event shall terminate when the server powers down and the server shall not continue a ResponseOnEvent diagnostic service after a reset or power on (i.e. the ResponseOnEvent service is terminated).	M	DNSE
1	storeEvent This value indicates that the event shall resume sending serviceToRespondTo-responses according to the ResponseOnEvent-set up after a power cycle of the server.	U	SE

Table4-41. sub-function (eventType) bit 6 definition - StorageState

Hex (bit 5-0)	Description	Cvt	Mnemonic
00	stopResponseOnEvent This value is used to stop the server sending responses on event. The event logic that has been set up is not cleared but can be restarted with the startResponseOnEvent sub-function parameter. Length of eventTypeRecord: 0 byte	U	STPROE

01	onDTCStatusChange This value identifies the event as a new DTC detected matching the DTCStatusMask specified for this event. This eventType requires the specification of the DTCStatusMask in the request message (eventTypeParameter#1). Length of eventTypeRecord: 1 byte	U	ONDTCS
02	onTimerInterrupt This value identifies the event as a timer interrupt, but the timer and its values are not part of the ResponseOnEvent service. This eventType requires the specification of more details in the request message (eventTypeRecord). Length of eventTypeRecord: 1 byte	U	OTI
03	onChangeOfDataIdentifier This value identifies the event as a new internal data record identified by dataIdentifier. The data values are vehicle manufacturer specific. This eventType requires the specification of more details in the request message (eventTypeRecord). Length of eventTypeRecord: 3 bytes	U	OCOCID
04	reportActivatedEvents This value is used to indicate that in the positive response all events are reported that have been activated in the server with the ResponseOnEvent service (and are currently active). Length of eventTypeRecord: 0 bytes	U	RAE
05	startResponseOnEvent This value is used to indicate to the server to activate the event logic (including event window timer) that has been set up and start sending responses on event. Length of eventTypeRecord: 0 byte.	M	STRTROE
06	clearResponseOnEvent This value is used to clear the event logic that has been set up in the server (This also stops the server sending responses on event.) Length of eventTypeRecord: 0 byte.	U	CLRROE
07	onComparisonOfValues A defined alteration of a data value out of a specific record identified by a dataIdentifier which identifies a data value event. With this sub-function the user shall have the possibility to define an event at the occurrence of a specific result gathered from a defined measurement value comparison. A specific measurement value included in a data record assigned to a defined dataIdentifier is compared with a given comparison value. The specified operator defines the kind of comparison. The event occurs if the comparison result is positive. Length of eventTypeRecord: 10 Byte	U	OCOV
08~3F	Reserved	M	

Table4-42. sub-function (eventType) bit5-0 definition

4.1.33 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Response Service ID	M	#C6h	ROEPR
#2d	eventType	M	#XXh	ETP
#3d	numberOfIdentifiedEvents	M	#XXh	NOIE
#4d	eventWindowTime	M	#XXh	EWT
#5d	eventTypeRecord[] = [	C1	#XXh	ETR_
..	eventTypeParameter 1	..	..	ETP1
..	..	..	..	..
#(m-1)+5d	eventTypeParameter m]	C1	#XXh	ETPm
#n-(r-1)-1d	servieToRespondToRecord[] = [	M	#XXh	STRTR_
#n-(r-1)d	serviceld	C2	#XXh	SI
..	serviceParameter 1	..	..	SP1
..	..	..	..	..
#nd	serviceParameter r]	C2	#XXh	SPr

C1: present if the eventType required additional parameters to be specified for the event to respond to.
C2: present if the service request of the service to respond to required additional service parameters

Table4-43. Positive Response Message definition for all sub-functions but reportActivatedEvents

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ResponseOnEvent Response Service ID	M	#C6h	ROEPR
#2d	eventType = reportActivatedEvents	M	#04h	ETP_RAE
#3d	numberOfIdentifiedEvents	M	#XXh	NOIE
#4d	eventTypeOfActivateEvent #1	C1	#XXh	EVOAE
#5d	eventWindowTime #1	C1	#XXh	EWT
#6d	eventTypeRecord#1[] = [	C2	#XXh	ETR_
..	eventTypeParameter 1	..	..	ETP1
..	..	..	..	..
#(m-1)+6d	eventTypeParameter m]	C2	#XXh	ETPm
#p-(o-1)-1d	servieToRespondToRecord#1[] = [	C3	#XXh	STRTR_
#p-(o-1)d	serviceld	C4	#XXh	SI
..	serviceParameter 1	..	..	SP1
..	..	..	..	..
#pd	serviceParameter o]	C4	#XXh	SPo
..	..	..	..	..
#n-(r-1)-4-(q-1)d	eventTypeOfActivateEvent #k	C1	#XXh	EVOAE
#n-(r-1)-3-(q-1)d	eventWindowTime #k	C1	#XXh	EWT
#n-(r-1)-2-(q-1)d	eventTypeRecord#k[] = [	C2	#XXh	ETR_
..	eventTypeParameter 1	..	..	ETP1
..	..	..	..	..
#n-(r-1)-2d	eventTypeParameter q]	C2	#XXh	ETPq
#n-(r-1)-1d	servieToRespondToRecord#k[] = [	C3	#XXh	STRTR_
#n-(r-1)d	serviceld	C4	#XXh	SI
..	serviceParameter 1	..	..	SP1
..	..	..	..	..
#nd	serviceParameter r]	C4	#XXh	SPr

C1: present if an active event is reported
C2: present if the reported eventType of the active event (eventTypeOfActiveEvent) requires additional parameters to be specified for the event to respond to.
C3: mandatory to be present when reporting an active event.
C4: present if the report service request of the service to respond to requires additional service parameters.

Table4-44. Positive Response Message definition, sub-function = reportActivatedEvents

4.1.34 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	Used when the server is condition that cannot perform the requested functionality
31	requestOutOfRange	M	The server shall use this response code : 1. if it detects an error in the eventTypeRecord parameter 2. if the specified eventWindowTime is invalid.

Table4-45. Supported Negative Response Codes

4.1.35 Message flow example(s) ResponseOnEvent (Finite Event Window)

In the following example, it is assumed:

- the node has a physical request CANId of \$7E0
- the node has a physical response CANId of \$7E8
- storageStatus = doNotStoreEvent (0)
- suppressPosRspMsgIndicationBit = FALSE (0)

4.1.35.1 Step #1 : Set Up Event Logic

- eventWinowTime equal to 08 hex defines an event window of 80 seconds (eventWindowTime * 10 s)
- eventType = onDTCStatusChange (01 hex)
- eventTypeParameter = testFailed status (01 hex)
- serviceToRespondToRecord :
serviceld = ReadDTCInformation (19 hex)
sub-function = reportNumberOfDTCByStatusMask (01 hex)
DTCStatusMask = testFailed status (01 hex)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	07 86 01 08 01 19 01 01	Single Frame
7E8	Response	8	10 08 C6 01 00 08 01 19	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 01 01 AA AA AA AA AA	1st Consecutive Frame

Table 4-46 ResponseOnEvent (FiniteEventWindow) Message Example – Set Up Event Logic

4.1.35.2 Step #2 : Activate Event Logic

- eventType = startResponseOnEvent (05 hex)

- DTCStatusAvailabilityMask = FF hex
- DTCCount = 4 (00 04 hex)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	02 86 04 55 55 55 55 55	Single Frame
7E8	Response	8	04 C6 01 00 08 AA AA AA	Single Frame
7E8	Response	8	04 59 FF 00 04 AA AA AA	Single Frame
7E8	Response	8	04 59 FF 00 04 AA AA AA	Single Frame

Table 4-47 ResponseOnEvent (FiniteEventWindow) Message Example – Activate Event Logic

4.1.35.3Step #3-1 (during the active event window) : Report Currently Active Events

- eventType = reportActivatedEvents (04 hex)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	03 86 05 08 55 55 55 55	Single Frame
7E8	Response	8	10 09 C6 04 02 01 08 01	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 19 01 01 AA AA AA AA	1st Consecutive Frame

Table 4-48 ResponseOnEvent (FiniteEventWindow) Message Example –Report Currently Active Events

4.1.35.4Step #3-2 (Event Window Time has expired) : Event Window Timer Timeout

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E8	Response	8	10 08 C6 01 02 08 01 19	First Frame
7E0	Physical Request	8	30 00 00 55 55 55 55 55	Flow Control
7E8	Response	8	21 01 01 AA AA AA AA AA	1st Consecutive Frame

Table 4-49 ResponseOnEvent (FiniteEventWindow) Message Example –Event Window Timer Timeout

4.10 LinkControl (87 hex) service

The LinkControl service is used to control the communication link baudrate between the client and the server(s) for the exchange of diagnostic data. This service optionally applies to those data link layers, which allow for a baudrate transition during an active diagnostic session.

- Step #1 : The client verifies if the transition can be performed and informs the server(s) about the baudrate to be used. Each server has to respond positively (suppressPosRspMsgIndicationBit = FALSE) before the client performs step#2. This step actually does not perform the baudrate transition.
- Step #2: The client actually requests the transition of the baudrate. This step shall only be performed if it is verified that the baudrate transition can be performed (step #1 performed). In case of functional communication it is recommended that there shall not be any response from a server when the baudrate is transitioned (suppressPosRspMsgIndicationBit = TRUE), because one server might already have been transitioned to the new baudrate while others still need to transmit their response message(s) (baudrate mismatch avoidance).

Any baudrate transition shall occur as follows:

- suppressPosRspMsgIndicationBit = TRUE: after the successful transmission/reception of the client request message, which requests the baudrate transition.
- suppressPosRspMsgIndicationBit = FALSE: after the successful transmission/reception of the server positive response message, which confirms the successful reception of the request, which requests the baudrate transition.

This service is tied to a non-defaultSession. A session layer timer timeout will transition the server(s) back to its(their) normal speed of operation. The same applies in case an ECUReset service (11 hex) is performed. The transition into another non-defaultSession shall not influence the baudrate.

4.1.36 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	LinkControl Request Service ID	M	#87h	LC
#2d	Sub-function = [linkControlType]	M	#XXh	LEV_ LCTP_
#3d	baudrateIdentifier	C1	#XXh	BI_

Table4-50. Request Message definition
 (linkControlType = verifyBaudrateTransitionWithFixedBaudrate)

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	LinkControl Request Service ID	M	#87h	LC
#2d	Sub-function = [linkControlType]	M	#XXh	LEV_ LCTP_
#3d	linkBaudrateRecord[] = [baudrateHighByte baudrateMiddleByte baudrateLowByte]	C	#XXh	LBR_ BRHB
#4d		C	#XXh	BRMB
#5d		C	#XXh	BRLB

Table4-51. Request Message definition
 (linkControlType = verifyBaudrateTransitionWithSpecificBaudrate)

Hex	Description	Cvt	Mnemonic
-----	-------------	-----	----------

00	Reserved	M	
01	verifyBaudrateTransitionWithFixedBaudrate This parameter is used to verify if a transition to a pre-defined baudrate, which is specified by the baudrateIdentifier data parameter can be performed.	C	VBTFWBR
02	verifyBaudrateTransitionWithSpecificBaudrate This parameter is used to verify if a transition to a specially defined baudrate, which is specified by the linkBaudrateRecord data parameter can be performed.	C	VBTFWSBR
03	transitionBaudrate This sub-function parameter requests the server(s) to transition the baudrate to the one that was specified in the preceding verification message.	U	TB
04 ~ 7F	Reserved	M	

C : More than one identifier shall be supported.

Table4-52. Request message sub-function (LinkControlType) parameter definition

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	PC9600Baud This value specifies the standard PC baudrate of 9.6KBaud.	U	PC9600
02	PC19200Baud This value specifies the standard PC baudrate of 19.2KBaud.	U	PC19200
03	PC38400Baud This value specifies the standard PC baudrate of 38.4KBaud.	U	PC38400
04	PC57600Baud This value specifies the standard PC baudrate of 57.6KBaud.	U	PC57600
05	PC115200Baud This value specifies the standard PC baudrate of 115.2KBaud.	U	PC115200
10	CAN125000Baud This value specifies the standard CAN baudrate of 123KBaud.	U	CAN125000
11	CAN250000Baud This value specifies the standard CAN baudrate of 250KBaud.	U	CAN250000
12	CAN500000Baud This value specifies the standard CAN baudrate of 500KBaud.	U	CAN500000
13	CAN1000000Baud This value specifies the standard CAN baudrate of 1MBaud.	U	CAN1000000
14~FF	Reserved	M	

Table4-53. Definition of baudrateIdentifier value

4.1.37 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
-----------	----------------	-----	-----------	----------

#1d	LinkControl Response Service ID	M	#C7h	LCPR
#2d	linkControlType	M	#XXh	LCTP

Table4-54. Positive Response Message definition

4.1.38 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	Send if the sub-function parameter in the request message is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the requested LinkControl are not met.
24	requestSequenceError	M	This code shall be returned if the client requests the transition of the baudrate without a preceding verification step, which specifies the baudrate to transition to.
31	requestOutOfRange	M	This code shall be returned if, 1. the requested fixed baudrate (baudrateIdenfitier) is invalid 2. the specific baudrate (linkBaudrateRecord) is invalid.

Table4-55. Supported Negative Response Codes

4.1.39 Message Flow Examples LinkControl

In the following example, it is assumed:

- the node has a physical request CANId of \$7E0
- the node has a physical response CANId of \$7E8

4.1.39.1 Example #1 – Transition baudrate to fixed baudrate

- PC baudrate = 115200 kBit/s (05 hex)

4.1.39.1.1 Step #1 : Verify if all criteria are met for a baudrate switch

- linkControlType = verifyBaudrateTransitionWithFixedBaudrate (01 hex)
- suppressPosRspMsgIndicationBit = FALSE (0)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	03 87 01 05 55 55 55 55	Single Frame
7E8	Response	8	02 C7 01 AA AA AA AA AA	Single Frame

Table 4-56 LinkControl Message Example – VerifyBaudrateTransitionWithFixedBaudrate

4.1.39.1.2 Step #2 : Transition the baudrate

- linkControlType = transitionBaudrate (03 hex)
- suppressPosRspMsgIndicationBit = True (1)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
--------	----------------------------------	-----	------------	----------



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 73/211

7E0	Physical Request	8	02 87 83 55 55 55 55 55	Single Frame
-----	------------------	---	-------------------------	--------------

Table 4-57 LinkControl Message Example – TransitionBaudrateWithFixedBaudrate

4.1.39.2 Example #2 – Transition baudrate to specific baudrate

- PC baudrate = 150 kBit/s (02 49 F0 hex)

4.1.39.2.1 Step #1 : Verify if all criteria are met for a baudrate switch

- linkControlType = verifyBaudrateTransitionWithSpecificBaudrate (02 hex)
- suppressPosRspMsgIndicationBit = FALSE (0)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	05 87 02 02 49 F0 55 55	Single Frame
7E8	Response	8	02 C7 02 AA AA AA AA AA	Single Frame

Table 4-58 LinkControl Message Example – VerifyBaudrateTransitionWithSpecificBaudrate

4.1.39.2.2 Step #2 : Transition the baudrate

- linkControlType = transitionBaudrate (03 hex)
- suppressPosRspMsgIndicationBit = True (1)

CAN ID	Client Request / Server Response	DLC	Data Bytes	Comments
7E0	Physical Request	8	02 87 83 55 55 55 55 55	Single Frame

Table 4-59 LinkControl Message Example – TransitionBaudrateWithSpecificBaudrate

4.11 StopDiagnosticSession (20 hex) service

4.1.40 StopDiagnosticSession Request

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	stopDiagnosticSessionRequest Service Id	M	#20h	SPDS

Table 4-60. StopDiagnosticSession Request Message

4.1.41 StopDiagnosticSession positive Response

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	stopDiagnosticSession Positive response	M	#60h	SPDSPR

Table 4-61. StopDiagnosticSession Positive Response Message

After positive response, the following conditions will be found :

- Default Session (\$01) (or Standard Diagnostic Session (\$81)) is active
- Default timing of the Default Session (or Standard Diagnostic Session) is active
- Security Access is (re)initialized

Even if there would have been sent further parameters (#2 etc.) in the request, the positive response would just include them (and have the same length as the request)

4.1.42 StopDiagnosticSession negative Response

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	negativeResponse Service Id	M	#7Fh	NR
#2d	stopDiagnosticSessionRequest Service Id	M	#20h	SPDS
#3d	responseCode= [GeneralReject (Programming session is running – Reprogramming routine is not completed yet.)]	M	#XXh= [#10h]	RC

Table 4-62. stopDiagnosticSession Negative Response Message

5. Data Transition functional unit

5.1 ReadDataByIdentifier (22 hex) service

The ReadDataByIdentifier service allows the client to request data record values from the server identified by one or more dataIdentifiers. Upon receiving a ReadDataByIdentifier request, the server shall access the data elements of the records specified by the dataIdentifier parameter(s) and transmit their value in one single ReadDataByIdentifier positive response containing the associated dataRecord parameter(s). The request message may contain the same dataIdentifier multiple times. The server shall treat each dataIdentifier as a separate parameter and respond with data for each dataIdentifier as often as requested.

5.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByIdentifier Request Service ID	M	#22h	RDBI
#2d	dataIdentifier[]#1 = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB
..	..	..	..	..
#(n-1)d	dataIdentifier[]#m = [byte#1 (MSB) byte#2]	U	#XXh	DID_ HB
#nd		U	#XXh	LB
dataIdentifier : see Annex C.1				

Table5-1. Request Message

5.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByIdentifier Response Service ID	M	#62h	RDBIPR
#2d	dataIdentifier[] #1 = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB
#4d	dataRecord[] #1 = [data#1 .. data#k]	M	#XXh	DREC_ DATA_1
..		..	..	..
#(k-1)+4d	data#k]	U	#XXh	DATA_k
..	..	..	..	..
#n-(o-1)-2d	dataIdentifier[] #m = [byte#1 (MSB) byte#2]	U	#XXh	DID_ HB
#n-(o-1)-1d		U	#XXh	LB
#n-(o-1)d	dataRecord[] #m = [data#1 .. data#k]	U	#XXh	DREC_ DATA_1
..		..	..	..
#nd	data#k]	U	#XXh	DATA_o

Table5-2. Positive Response Message definition

5.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
-----	----------------------	-----	---------------------



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 76/211

13	incorrectMessageLengthOrInvalidFormat	M	This response code shall be sent if the length of the request message is invalid or the client exceeded the maximum number of dataIdentifiers allowed to be requested at a time.
22	conditionNotCorrect	U	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	This code shall be sent if none of the requested dataIdentifier values are supported by the device; none of the requested dataIdentifiers are supported in the current session; the requested dynamicDefinedDataIdentifier has not been assigned yet;
33	securityAccessDenied	M	This code shall be sent if at least one of the dataIdentifier is secured and the server is not in and unlocked state.
14	responseTooLong	M	This NRC shall be if the total length of the response exceeds the limit of the underlying transport protocol (e.g., when multiple DIDs are requested in a single request)

Table5-3. Supported Negative Response Codes

MOBASE ELECTRONICS / 2025-11-14 15:32
172.23.51.206 / 2025-11-14 15:32

5.2 ReadMemoryByAddress (23 hex) service

The ReadMemoryByAddress service allows the client to request memory data from the server via provided starting address and size of memory to be read.

5.1.4 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadMemoryByAddress Request Service ID	M	#23h	RMBA
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#3d	memoryAddress[] = [byte#1 (MSB)	M	#XXh	MA_ B1
..	..	..	..	..
#(m-1)+3	byte#m]	C1	#XXh	Bm
#n-(k-1)d	memorySize[] = [byte#1 (MSB)	M	#XXh	MS_ B1
..	..	..	..	..
#nd	byte#k]	C2	#XXh	Bk

addressAndLengthFormatIdentifier : This parameter is a one byte value with each nibble encoded separately.
 - bit 7-4 : Length(number of byte) of the memorySize parameter
 - bit 3-0 : Length(number of byte) of the memoryAddress parameter
 C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier.
 C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.

Table5-4. Request Message

5.1.5 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadMemoryByAddress Response Service ID	M	#63h	RMBAPR
#2d	dataRecord[] = [data#1	M	#XXh	DREC_ DATA_1
..	..	..	..	..
#nd	data#m]	U	#XXh	DATA_m

Table5-5. Positive Response Message definition

5.1.6 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating conditions of the server are not met to perform the required action



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 78/211

31	requestOutOfRange	M	This response code shall be sent if ; 1. any memory address within the interval [\$MA, (\$MA+\$MS-\$1)] is invalid. 2. any memory address within the interval [\$MA, (\$MA+\$MS-\$1)] is restricted. 3. the memorySize parameter value in the request message is greater than the maximum value supported by the server. 4. the specified addressAndLengthFormatIdentifier is not valid.
33	SecurityAccessDenied	M	This code shall be sent if any memory address within the interval [\$MA, (\$MA+\$MS-\$1)] is secure and the server is locked.

Table5-6. Supported Negative Response Codes

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

5.3 ReadScalingDataByIdentifier (24 hex) service

The ReadScalingDataByIdentifier service allows the client to request scaling data record information from the server identified by a dataIdentifier.

Upon receiving a ReadScalingDataByIdentifier request, the server shall access the scaling information associated with the specified dataIdentifier parameter and transmit the scaling information values in one ReadScalingDataByIdentifier positive response.

5.1.7 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadScalingDataByIdentifier Request Service ID	M	#24h	RSDBI
#2d	dataIdentifier[] = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB
dataIdentifier : see Annex C.1				

Table5-7. Request Message

5.1.8 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadScalingDataByIdentifier Response Service ID	M	#64h	RSDBIPR
#2d	dataIdentifier[] = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB
#4d	scalingByte#1	M	#XXh	SB_1
#3d	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#p]	C1	#XXh	SBE_ PAR1
..		..	..	..
..		..	..	..
..	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#r]	C1	#XXh	SBE_ PAR1
..		..	..	..
..		..	..	..
..	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#r]	C1	#XXh	SBE_ PAR1
..		..	..	..
..		..	..	..
..	scalingByteExtension[]#1 = [scalingByteExtensionParameter#1 .. scalingByteExtensionParameter#r]	C1	#XXh	SBE_ PAR1
..		..	..	..
..		..	..	..

C1 : The presence of this parameter depends on the scalingByte high nibble. It is mandatory to be present if the scalingByte high nibble is encoded as formula, unit/format, or bitMappedReported/WithOutMask.
 C2 : The presence of this parameter depends on whether the encoding of the scaling information requires more than one byte
 scalingByte : The parameter scalingByte consists of one byte (high and low nibble). The scalingByte high nibble defines the data type, which is used to represent the dataIdentifier (DID). The scalingByte low nibble defines the number of bytes used to represent the parameter in a datastream. (see Annex C.2)
 scalingByteExtension : This parameter is used to provide additional information for scalingBytes with a high nibble encoded as formula, unit/format, or bitmappedReportedWithOutMask. (see Annex C.3)

Table5-8. Positive Response Message definition

5.1.9 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
-----	----------------------	-----	---------------------



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 80/211

13	incorrectMessageLengthOrInvalidFormat	M	This response code shall be sent if the length of the request message is invalid.
22	conditionNotCorrect	M	This code shall be sent if the operating conditions of the server are not met to perform the required action.
31	requestOutOfRange	M	This return code shall be sent if ; 1. the requested dataIdentifier value is not supported by the device (physical addressing only) 2. the requested dataIdentifier value is supported by the device, but no scaling information is available for the specified dataIdentifier.
33	securityAccessDenied	M	This code shall be sent if the dataIdentifier is secured and the server is not in an unlocked state.

Table5-9. Supported Negative Response Codes

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

5.4 ReadDataByPeriodicIdentifier (2A hex) service

The ReadDataByPeriodicIdentifier service allows the client to request the periodic transmission of data record values from the server identified by one or more periodicDataIdentifiers.

The client request message contains one or more 1-byte periodicDataIdentifier values that identify data record(s) maintained by the server. The periodicDataIdentifier represents the low byte of a dataIdentifier out of the dataIdentifier range reserved for this service (0xF2XX, see table C-4 for allowed periodicDataIdentifier values), e.g. the periodicDataIdentifier 0xE3 used in this service is the identifier 0xF2E3.

The format and definition of the dataRecord shall be vehicle manufacturer specific, and may include analog input and output signals, digital input and output signals, internal data, and system status information if supported by the server.

Upon receiving a ReadDataByPeriodicIdentifier request other than stopSending the server shall check whether the conditions are correct to execute the service.

A periodicDataIdentifier shall only be supported with a single transmissionMode at a given time. A change to the schedule of a periodicDataIdentifier shall be performed on reception of a request message with the transmissionMode parameter set to a new schedule for the same periodicDataIdentifier. Multiple schedules for different periodicDataIdentifiers shall be supported upon vehicle manufacturer's request.

IMPORTANT — If the conditions are correct then the server shall transmit a positive response message, including only the service identifier. The server shall never transmit a negative response message once it has accepted the initial request message by responding positively.

Following the initial positive response message the server shall access the data elements of the records specified by the periodicDataIdentifier parameter(s) and transmit their value in separate periodic data response messages for each periodicDataIdentifier containing the associated dataRecord parameters.

The separate periodic data response messages defined to transmit the periodicDataIdentifier data to the client following the initial positive response message shall include the periodicDataIdentifier and the data of the periodicDataIdentifier, but not the positive response service identifier. The mapping of the periodic response message onto certain data link layers is described in the appropriate implementation specifications of ISO 14229.

The documented periodic rate for a specific transmissionMode is defined as the time between any two consecutive response messages with the same periodicDataIdentifier, when only a single periodicDataIdentifier is scheduled. If multiple periodicDataIdentifiers are scheduled concurrently, the effective period between the same periodicDataIdentifier will vary based upon the following design parameters:

- the call rate of the periodic scheduler,
- the number of available protocol specific periodic data response message address information IDs allocated per scheduler call (e.g., CAN identifier on CAN)
- the number of periodicDataIdentifiers that can be defined in parallel to be transmitted concurrently.

These parameter values will impact the extent of how much the effective period between the same periodicDataIdentifier will increase if multiple periodicDataIdentifiers are transmitted concurrently. Therefore, all of the previously mentioned design parameters shall be specified by the vehicle manufacturer. Each time the periodic scheduler is called it shall determine if any periodicDataIdentifiers are ready to transmit.

Example, two distinct ECU implementations may both support a fast transmissionMode with a periodic rate of 10 ms and a single unique periodic data response message address information ID. If the first implementation calls the periodic scheduler every 10ms, the time between the same periodicDataIdentifier

would increase to 20 ms when two periodicDataIdentifiers are scheduled and would increase to 40 ms when four periodicDataIdentifiers are scheduled. If the second implementation calls the periodic scheduler every 5 ms, the time between the same periodicDataIdentifier would remain at 10 ms when two periodicDataIdentifiers are scheduled and would increase to 20 ms when four periodicDataIdentifiers are scheduled.

Upon receiving a ReadDataByPeriodicIdentifier request including the transmissionMode stopSending the server shall either stop the periodic transmission of the periodicDataIdentifier(s) contained in the request message or stop the transmission of all periodicDataIdentifier if no specific one is specified in the request message. The response message to this transmissionMode only contains the service identifier.

The server may limit the number of periodicDataIdentifiers that can be simultaneously supported as agreed upon by the vehicle manufacturer and system supplier. Exceeding the maximum number of periodicDataIdentifier that can be simultaneously supported shall result in a single negative response and none of the periodicDataIdentifiers in that request shall be scheduled. Repetition of the same periodicDataIdentifier in a single request message is not allowed and the server shall ignore them all except one periodicDataIdentifier if the client breaks this rule.

5.1.10 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByPeriodicIdentifier Request Service ID	M	#2Ah	RDBPI
#2d	transmissionMode	M	#XXh	TM
#3d	periodicDataIdentifier[] #1	C	#XXh	PDID1
..	..	..	..	..
#m+2d	periodicDataIdentifier[] #m	U	#XXh	PDIDm

C: The first periodicDataIdentifier is mandatory to be present in the request message if the transmissionMode is equal to sendAtSlowRate, sendAtMediumRate, or sendAtFastRate. In case the transmissionMode is equal to stopSending there can either be no periodicDataIdentifier present in order to stop all scheduled periodicDataIdentifier or the client can explicitly specify one or more periodicDataIdentifier(s) to be stopped.

Table5-10. Request Message

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	sendAtSlowRate This parameter specifies that the server shall transmit the requested dataRecord information at a slow rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter slow is vehicle manufacturer specific, and predefined in the server.	C	SASR
02	sendAtMediumRate This parameter specifies that the server shall transmit the requested dataRecord information at a medium rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter medium is vehicle manufacturer specific, and pre-defined in the server.	C	SAMR

03	sendAtFastRate This parameter specifies that the server shall transmit the requested dataRecord information at a fast rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter fast is vehicle manufacturer specific, and predefined in the server.	C	SAFR
04	stopSending The server stops transmitting positive response messages send periodically/repeatedly. Note that maximumNumberOfResponsesToSend parameter should be set to 0x01 if transmissionMode = stopSending (otherwise, server operation could be undefined).	M	SS
05~FF	Reserved	M	

C : More than one identifier shall be supported.

Table5-11. Data parameter definition - TransmissionMode

5.1.11 Positive response message

It has to be distinguished between the initial positive response, which indicates that the server accepts the service and subsequent positive response message, which include periodicDataIdentifier data.

Table5-12/13 defines the initial positive response message to be transmitted by the server when it accepts the request.

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDataByPeriodicIdentifier Response Service ID	M	#6Ah	RDBPIPR

Table5-12. Positive Response Message definition

The data of a periodicDataIdentifier is transmitted periodically (with updated data) at a rate determined by the transmissionMode parameter of the request.

After the initial positive response, for each supported periodicDataIdentifier in the request the server shall start sending a single periodic data response message as defined below.

Table 5-13 defines the periodic data response message data definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	periodicDataIdentifier	M	#XXh	PDID
#2d	dataRecord[] = [data#1 .. data#k]	M	#XXh	DREC_ DATA_1
..		..	..	..
#k+2d		U	#XXh	DATA_k

Table5-13. Periodic data response message data definition

5.1.12 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	This NRC shall be sent if the length of the request message is invalid or the client exceeded the maximum number of periodicDataIdentifiers allowed to be requested at a time.



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 84/211

22	conditionNotCorrect	M	This NRC shall be sent if the operating conditions of the server are not met to perform the required action. E.g. this could occur if the client requests periodicDataIdentifiers with different transmissionModes and the server does not support multiple transmissionModes simultaneously.
31	requestOutOfRange	M	This NRC shall be sent if none of the requested periodicDataIdentifier values are supported by the device; none of the requested periodicDataIdentifiers are supported in the current session; The specified transmissionMode is not supported by the device; the requested dynamicDefinedDataIdentifier has not been assigned yet; the client exceeded the maximum number of periodicDataIdentifiers allowed to be scheduled concurrently
33	securityAccessDenied	M	This NRC shall be sent if at least one of the periodicDataIdentifier is secured and the server is not in an unlocked state.

Table5-14. Supported Negative Response Codes

MOBASE ELECTRONICS / 172.23.51.206 / 2025-11-14 15:32

5.5 DynamicallyDefinedDataIdentifier (2C hex) service

The DynamicallyDefinedDataIdentifier service allows the client to dynamically define in a server a data identifier that can be read via the ReadDataByIdentifier service at a later time. The intention of this service is to provide the client with the ability to group one or more data elements into a data superset that can be requested en masse via ReadDataByIdentifier or ReadDataByPeriodicIdentifier service. The data elements to be grouped together can either be referenced by ;

- a source data identifier, a position and size or,
- a memory address and a memory length, or,
- a combination of the two methods listed above using multiple requests to define the single data element. The dynamically defined dataIdentifier will then contain a concatenation of the data parameter definitions.

This service allows greater flexibility in handling ad hoc data needs of the diagnostic application that extend beyond the information that can be read via statically defined data identifiers, and can also be used to reduce bandwidth utilization by avoiding overhead penalty associated with frequent request/response transactions. The definition of the dynamically defined data identifier can either be done via a single request message or via multiple request messages. This allows for the definition of a single data element referencing source identifier(s) and memory addresses. The server has to concatenate the definitions for the single data element. A redefinition of a dynamically defined data identifier can be achieved by clearing the current definition and start over with the new definition.

Although this service does not prohibit such functionality, it is not recommended for the client to reference one dynamically defined data record from another, because deletion of the referenced record could create data consistency problems within the referencing record. This service also provides the ability to clear an existing dynamically defined data record. Requests to clear a data record shall be positively responded to if the specified data record identifier is within the range of valid dynamic data identifiers supported by the server (see ISO 14229-1.2 annex C.1 for more details).

The server shall maintain the dynamically defined data record until it is cleared or as specified by the vehicle manufacturer (e.g., deletion of dynamically defined data records upon session transition or upon power down of the server).

The server can implement data records in two different ways:

- 1) Composite data records containing multiple elemental data records which are not individually referenced.
- 2) Unique 2-byte identification "tag" or parameter identifier (PID) value for individual, elemental data records supported within the server (an example elemental data record, or PID, is engine speed or intake air temperature). This implementation of data records is a subset of a composite data record implementation, because it only references a single elemental data record instead of a data record including multiple elemental data records.

Both types of implementing data records are supported by the DynamicallyDefineDataIdentifier service to define a dynamic data identifier.

- 1) Composite block of data: The position parameter has to reference the starting point in the composite block of data and the size parameter has to reflect the length of data to be placed in the dynamically defined data identifier. The tester is responsible to not include only a portion of an elemental data record of the composite block of data in the dynamic data record.
- 2) 2-byte PID: The position parameter has to be set to one (1) and the size parameter has to reflect the length of the PID (length of the elemental data record). The tester is responsible to not include only a portion of the 2-byte PID value in the dynamic data record.

The ordering of the data within the dynamically defined data record shall be of the same order as it was specified in the client request message(s). Also, first position of the data specified in the client's request

shall be oriented such that it occurs closest to the beginning of the dynamic data record, in accordance with the ordering requirement mentioned in the preceding sentence.

In addition to the definition of a dynamic data identifier via a logical reference (a record data identifier) this service provides the capability to define a dynamically defined data identifier via an absolute memory address and a memory length information. This mechanism of defining a dynamic data identifier is recommended to be used only during the development phase of a server.

5.1.13 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI
#2d	sub-function = [defineByIdentifier]	M	#01h	LEV_ DBID
#3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	M M	#XXh #XXh	DDDDI_ HB LB
#5d #6d	sourceDataIdentifier[] #1 = [byte#1 (MSB) byte#2 (LSB)]	M M	#XXh #XXh	SDI_ HB LB
#7d	positionInSourceDataRecord #1	M	#XXh	PISDR#1
#8d	memorySize #1	M	#XXh	MS#1
..	..	..	..	..
#n-3d #n-2d	sourceDataIdentifier[] #m = [byte#1 (MSB) byte#2 (LSB)]	U U	#XXh #XXh	SDI_ HB LB
#n-1d	positionInSourceDataRecord #m	U	#XXh	PISDR#m
#nd	memorySize #m	U	#XXh	MS#m

positionInSourceDataRecord : This 1-byte parameter is used to specify the starting byte position of the except of the source data record to be included in the dynamic data record. A position of one(1) shall reference the first byte of the data record referenced by the sourceDataIdentifier

Table5-15. Request message definition – sub-function = defineByIdentifier

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI
#2d	sub-function = [defineByMemoryAddress]	M	#02h	LEV_ DBMA
#3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	M M	#XXh #XXh	DDDDI_ HB LB
#5d	addressAndLengthFormatIdentifier	M1	#XXh	ALFID
#6d .. #(m-1)+6d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M .. C	#XXh .. #XXh	MA_ B1 .. Bm
#m+6d .. #m+6+(k-1)d	memorySize[] = [byte#1 (MSB) .. byte#k]	M .. C	#XXh .. #XXh	MS_ B1 .. Bk
..	..	..	..	..

#n-k-(m-1)d .. #n-kd	memoryAddress[] = [byte#1 (MSB) .. byte#m]	U .. U/C	#XXh .. #XXh	MA_ B1 .. Bm
#n-(k-1)d .. #nd	memorySize[] = [byte#1 (MSB) .. byte#k]	U .. U/C	#XXh .. #XXh	MS_ B1 .. Bk

M1 : The addressAndLengthFormatIdetifier parameter is only present once at the very beginning of the request message and defines the length of the address and length information for each memory location reference throughout the whole request message.

Table5-16. Request message definition – sub-function = defineByMemoryAddress

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefinedDataIdentifier Request Service ID	M	#2Ch	DDDI
#2d	sub-function = [clearDynamicallyDefinedDataIdentifier]	M	#03h	LEV_ CDDID
#3d #4d	dynamicallyDefinedDataIdentifier[] = [byte#1 (MSB) byte#2 (LSB)]	C C	#XXh #XXh	DDDDI_ HB LB

C : The present of this parameter requires the server to clear the dynamicallydefinedDataIdentifier included in byte#1 and byte#2. If the parameter is not present all dynamicallyDefinedDataIdentifier in the server shall be cleared.

Table5-17. Request message definition – sub-function = clearDynamicallyDefinedDataIdentifier

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	defineByIdentifier	U	DBID
02	defineByMemoryAddress	U	DBMA
03	clearDynamicallyDefineDataIdentifier	U	CDDDI
04~7F	Reserved	M	

Table5-18. request message sub-function parameter definition

Parameter	Description	# of bytes	Resolution	Min value	Max value
P2_{CAN_SERVER_MAX}	Default P2_{CAN_SERVER_MAX} timing supported by the server for the activated diagnostic session	2	1ms	0ms	65535ms
P2*_{CAN_SERVER_MAX}	Enhanced (NRC 78h) P2_{CAN_SERVER_MAX} supported by the server for the activated diagnostic session.	2	10ms	0ms	655350ms

Table5-19. sessionParameterRecord definition

5.1.14 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	DynamicallyDefineDataIdentifier Response Service ID	M	#6Ch	DDDI_{PR}
#2d	definitionType	M	#XXh	DM
#3d #4d	dynamicallyDefinedDataIdentifier [] = [byte#1 (MSB) byte#2 (LSB)]	C C	#XXh #XXh	DDDDI_ HB LB

Table5-20. Positive Response Message definition

5.1.15 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	This response code shall sent if the sub-function parameter is not supported.
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating conditions of the server are not met to perform the required action.
31	requestOutOfRange	M	This response code shall be sent if ; 1. Any data identifier (dynamicallyDefinedData Identifier or any sourceDataIdentifier) in the request message is not supported/invalid. 2. The positionSourceDataRecord was incorrect (less than 1, or greater than maximum allowed by server) 3. Any memory address in the request message is not supported in the server 4. The specified memorySize was invalid. 5. The amount of data to be packed into the dynamic data exceeds the maximum allowed by server. 6. The specified addressAndLengthFormat Identifier is not valid.
33	securityAccessDenied	M	This code shall be sent if; 1. any data identifier (dynamicallyDefinedData Identifier) in the request message is secured and the server is not in an unlocked state. 2. any memory address in the request message is secured and the server is not in unlocked state.

Table5-21. Supported Negative Response Codes

5.6 WriteDataByIdentifier (2E hex) service

The WriteDataByIdentifier service allows the client to write information into the server at an internal location specified by the provided data identifier.

Dynamically defined dataIdentifier(s) shall not be used with this service.

5.1.16 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteDataByIdentifier Request Service ID	M	#2Eh	WDBI
#2d	dataIdentifier = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB
#4d	dataRecord = [data#1 .. data#m]	M	#XXh	DREC_ DATA_1
..		..	..	..
#m+3d		U	#XXh	DATA_m

dataIdentifier : see Annex C.1

Table5-22. Request Message

5.1.17 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteDataByIdentifier Response Service ID	M	#6Eh	WDBIPR
#2d	dataIdentifier = [byte#1 (MSB) byte#2]	M	#XXh	DID_ HB
#3d		M	#XXh	LB

Table5-23. Positive Response Message definition

5.1.18 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	This response code shall be sent if 1. The dataIdentifier in the request message is not supported in the server or the dataIdentifier is supported for read only purpose (via ReadDataByIdentifier service). 2. Any data transmitted in the request message after the dataIdentifier is invalid (if applicable to the node).
33	securityAccessDenied	M	This code shall be sent if the dataIdentifier, which reference a specific address, is secured and the server is not in an unlocked state.
72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when writing to a memory location.

Table5-24. Supported Negative Response Codes

5.7 WriteMemoryByAddress (3D hex) service

The WriteMemoryByAddress service allows the client to write information into the server at one or more contiguous memory locations.

The WriteMemoryByAddress request message writes information specified by the parameter dataRecord[] into the server at memory locations specified by parameters memoryAddress and memorySize. It is also possible to use a fixed addressAndLengthFormatIdentifier and unused bytes within the memoryAddress or memorySize parameter are padded with the value 00 hex in the higher range address locations.

5.1.19 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteMemoryByAddress Request Service ID	M	#3Dh	WMBA
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#3d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M	#XXh	MA_ B1
.. #m+2d		.. C1	.. #XXh	.. Bm
#n-r-2-(k-1)d	memorySize[] = [byte#1 (MSB) .. byte#k]	M	#XXh	MS_ B1
.. #n-r-2d		.. C2	.. #XXh	.. Bk
#n-(r-1)d	dataRecord[] = [data#1 .. data#r]	M	#XXh	DREC_ DATA_1
.. #nd		.. U	.. #XXh	.. DATA_r

C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier.
C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.

Table5-25. Request Message

5.1.20 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	WriteMemoryByAddress Response Service ID	M	#7Dh	WMBAPR
#2d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#3d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M	#XXh	MA_ B1
.. #(m-1)+3		.. C	.. #XXh	.. Bm
#n-(k-1)d	memorySize[] = [byte#1 (MSB) .. byte#k]	M	#XXh	MS_ B1
.. #nd		.. C	.. #XXh	.. Bk

C1: The presence of this parameter depends on address length information parameter of the addressAndLengthFormatIdentifier.
C2: The presence of this parameter depends on the memory size length information of the addressAndLengthFormatIdentifier.

Table5-26. Positive Response Message definition

5.1.21 Supported negative response codes



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 91/211

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be sent if the operating condition of the server are not met to perform the required action.
31	requestOutOfRange	M	1. Any memory address within the interval [\$MA, (\$MA + \$MS -\$1)] is invalid. 2. Any memory address within the interval [\$MA, (\$MA + \$MS -\$1)] is restricted. 3. The memorySize parameter value in the request message is greater than the maximum value supported by the server. 4. The specified addressAndLengthFormat Identifier is not valid.
33	securityAccessDenied	M	This code shall be sent if any memory address within the interval [\$MA, (\$MA + \$MS -\$1)] is secure and the server is locked.
72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when writing to a memory location.

Table5-27. Supported Negative Response Codes

MOBASE ELECTRONICS / 01-15-15
172.23.51.206 / 2025-11-14

6.1 ClearDiagnosticInformation (14 hex) service

The ClearDiagnosticInformation service is used by the client to clear diagnostic information in one or multiple servers' memory. The server shall send a positive response even if no DTC's are stored. If a server supports multiple copies of DTC status information in memory (e.g. one copy in RAM and one copy in EEPROM), the server shall clear the copy used by the ReadDTCInformation status reporting service. Additional copies, e.g. backup copy in long-term memory, are updated according to the appropriate backup strategy (e.g. in the power-latch phase).

The request message of the client contains 1 parameter. The parameter groupOfDTC allows the client to clear a group of DTC's (e.g. Powertrain, Body, Chassis, etc.), or a specific DTC. Unless otherwise stated, the server shall clear both emission-related and non emission-related DTC information from memory for the requested group.

DTC information reset / cleared via this service includes but is not limited to the following :

- DTC status byte
- Captured DTC snapshot data
- Captured DTC extended data
- Other DTC related data such as first/most recent DTC, flags, counters, timers, etc. specific to DTC's

Any DTC information stored in an optionally available DTC mirror memory in the server is not affected by this service.

Note) Mirror Memory : The DTC mirror memory is an additional optional error memory in the server that cannot be erased by the ClearDiagnosticInformation (14 hex) service. The DTC mirror memory mirrors the normal DTC memory and can be used for example if the normal error memory is erased. To clear Mirror Memory an according Routine Identifier shall be used.

6.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ClearDiagnosticInformation Request Service ID	M	#14h	CDTCI
#2d	groupOfDTC[] = [groupOfDTCHighByte groupOfDTCMiddleByte groupOfDTCLowByte]	M	#XXh	GODTC_ HB
#3d		M	#XXh	MB
#4d		M	#XXh	LB
groupOfDTC : see Annex D.1				

Table6-2. Request Message

6.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ClearDiagnosticInformation Response Service ID	M	#54h	CDTCIPR

Table6-3. Positive Response Message definition

6.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This response code shall be used if internal conditions within the server prevent the clearing of DTC related information stored in the server.



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 94/211

31	requestOutOfRange	M	The return code shall be sent if the specified groupOfDTC parameter is not supported.
72	generalProgrammingFailure	U	This NRC shall be returned if the server detects an error when writing to a memory location.

Table6-4. Supported Negative Response Codes

6.1.4 Implementaion rule

The ECU behavior shall be consistent: The DTCs shall be cleared before the positive response is sent. This does not require that the non volatile memory is cleared, but any memory cache has to be.

With the parameter groupOfDTC it is also possible to clear only a specific group of DTCs (e.g. Powertrain) or a specific DTC.

A subsequent call to read out the fault memory shall not retrieve any DTC which has been set before the last execution of ClearDiagnosticInformation.

Even if no DTC was stored, the ECU shall return with a positive response.

There shall be no sequence dependency to any other service. Even if the fault memory was not read, it may be cleared.

A DTC can just be cleared by tester with service #14h.

6.2 ReadDTCInformation (19 hex) service

This service allows a client to read the status of server resident Diagnostic Trouble Code (DTC) information from any server, or group of servers within a vehicle. Unless otherwise stated, the server shall return both emission-related and non emission-related DTC information. This service allows the client to do the following :

- Retrieve the number of DTC's matching a client defined DTC status mask (at the point of the request),
- Retrieve the list of all DTC's matching a client defined DTC status mask,
- Retrieve DTCSnapshot data associated with a client defined DTC and status mask combination: DTC Snapshots are specific data records associated with a DTC, that are stored in the server's memory. The content of the DTC Snapshots is not defined by this standard, but typical usage of DTC Snapshots is to store data upon detection of a system malfunction. The DTC Snapshots will act as a snapshot of data values from the time of the system malfunction occurrence.
- Retrieve DTCExtendedData associated with a client defined DTC and status mask combination out of the DTC memory or the DTC mirror memory. DTC Extended Data consist of extended status information associated with a DTC. DTC Extended Data contains DTC parameter values, which have been identified at the time of the request. A typical use of DTC Extended Data is to store dynamic data associated with the DTC, e.g.
 - DTC occurrence counter,
 - current threshold values,
 - time of last occurrence (etc.).
 - fault validation counters (e.g. counts number of reported "test failed" and possible other counters if the validation is performed in several steps),
 - uncompleted test counters (e.g. counts numbers of driving cycles since the test was latest completed i.e. since the test reported "test passed" or "test failed"),
 - fault occurrence counters (e.g. counts number of driving cycles in which "testFailed" have been reported),
 - DTC aging counter (e.g. counts number of driving cycles since the fault was latest failed excluding the driving cycles in which the test haven't report "test pass" or the test report "test fail"),
 - specific counters for OBD (e.g. number of remaining driving cycles until the "check engine" lamp is switched off).
- Retrieve the number of DTC's matching a client defined severity mask (at the point of the request),
- Retrieve the list of DTC's matching a client defined severity mask record,
- Retrieve severity information for a client defined DTC,
- Retrieve the status of all DTC's supported by the server,
- Retrieve the first DTC failed by the server,
- Retrieve the most recently failed DTC within the server,
- Retrieve the first DTC confirmed by the server,
- Retrieve the most recently confirmed DTC within the server,
- Retrieve the list of DTC's out of the DTC mirror memory matching a client defined DTC status mask,
- Retrieve mirror memory DTCExtendedData record data for a client defined DTC mask and a client defined DTCExtendedData record number out of the DTC mirror memory,
- Retrieve the number of DTC's out of the DTC mirror memory matching a client defined DTC status mask,
- Retrieve the number of "only" emissions-related OBD DTC's matching a client defined DTC status mask. Emissions-related OBD DTC's cause the malfunction indicator to be turned on/display a message in case such DTC is detected,
- Retrieve the status of "only" emissions-related OBD DTC's matching a client defined DTC status mask. Emissions-related OBD DTC's cause the malfunction indicator to be turned on/display a message in case such DTC is detected,
- Retrieve all current "prefailed" DTCs which have not yet been detected as "pending" or "confirmed",
- Retrieve all DTCs with "permanentDTC" status. These DTCs have been previously cleared by the clearDiagnosticInformation service but remain in the non-volatile memory of the server until the appropriate monitors for each DTC have successfully passed.

This service uses a sub-function to determine which type of diagnostic information the client is requesting. Further details regarding each sub-function parameter are provided in the following sections.

This service makes use of the following terms:

- Enable Criteria: Server/vehicle manufacturer/system supplier specific criteria used to control when the server actually performs a particular internal diagnostic.
- Test Pass Criteria: Server/vehicle manufacturer/system supplier specific conditions, that define, whether a system being diagnosed is functioning properly within normal, acceptable operating ranges (e.g. no failures exist and the diagnosed system is classified as "OK").
- Test Failure Criteria: Server/vehicle manufacturer/system supplier specific failure conditions, that define, whether a system being diagnosed has failed the test.
- Confirmed Failure Criteria: Server/vehicle manufacturer/system supplier specific failure conditions that define whether the system being diagnosed is definitively problematic (confirmed), warranting storage of the DTC record in long term memory.
- Occurrence Counter: A counter maintained by certain servers that records the number of instances in which a given DTC test reported a unique occurrence of a test failure.
- Aging: A process whereby certain servers evaluate past results of each internal diagnostic to determine if a confirmed DTC can be cleared from long-term memory, e.g. in the event of a calibrated number of failure free cycles. (see ISO 14229-1.2 "10.3 ReadDTCInformation Service" for more details)

6.1.5 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportNumberOfDTCByStatusMask reportDTCByStatusMask	M	#01h #02h	LEV_ RNODTCBSM RDTCSBM
#3d	DTCStatusMask	M	#XXh	DTCSTM
DTCStatusMask : see Annex D.2				

Table6-5. Request Message definition – sub-function = #01h / 02h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCSnapshotIdentification reportDTCSnapshotRecordByDTCNumber]	M	#03h #04h	LEV_ RDTCSSI RDTCSSBDTC
#3d	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	C	#XXh	DTCMREC_ DTCHB
#4d		C	#XXh	DTCMB
#5d		C	#XXh	DTCLB
#6d	DTCSnapshotRecordNumber	C	#XXh	DTCSSRN
DTCSnapshotRecordNumber : DTCSnapshotRecordNumber is a 1-byte value indicating the number of the specific DTCSnapshot data record requested for a client defined DTCMaskRecord via the reportDTCSnapshotByDTCNumber / reportDTCSnapshotByRecordNumber sub-functions. For emissions-related servers (OBD compliant ECUs) the DTCSnapshot data record number 00 hex shall be the equivalent data record as specified in ISO 15031-5 service 02 hex frame number 00 hex. If the server supports multiple DTCSnapshot data records the range of 01 hex through FE hex shall be used. A value of FF hex requests the server to report all stored DTCSnapshot data records at once.				

Table6-6. Request Message definition – sub-function = #03/ 04h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI

#2d	Sub-function = [reportDTCSnapshotRecordByRecordNumber]	M	#05h	LEV_ RDTCSSBRN
#3d	DTCSnapshotRecordNumber	M	#XXh	DTCSSRN

Table6-7. Request Message definition – sub-function = #05h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCExtendedDataRecordByDTCNumber]	M	#06h	LEV_ RDTCEDRBDN RMDedrBDN
#3d	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	M	#XXh	DTCMREC_ DTCHB
#4d		M	#XXh	DTCMB
#5d		M	#XXh	DTCLB
#6d	DTCExtendedDataRecordNumber	M	#XXh	DTCEDRN

DTCExtendedDataRecordNumber : DTCExtendedDataRecordNumber is a 1-byte value indicating the number of the specific DTCExtendedData record requested for a client defined DTCMaskRecord via the reportDTCExtendedDataRecordByDTCNumber sub-function. For emissions-related servers (OBD compliant ECUs) the DTCExtendedDataRecordNumber 00 hex shall be reserved for future OBD use.

Table6-8. Request Message definition – sub-function = #06h

Hex	Description	Cvt
00	Reserved (for future OBD use)	M
01~8F	Vehicle Manufacturer Specific Stored DTCExtendedData Records	(To be defined)
90~EF	Reserved (for Legislated OBD Stored DTCExtendedData Records)	M
F0~FD	Reserved (for future)	M
FE	Reserved (for All Legislated OBD Stored DTCExtendedData Records in a Single Response Message)	M
FF	All Stored DTCExtendedData Records in a Single Response Message	M

Table6-9. DTCExtendedDataRecordNumber definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-Function = [reportSupportedDTC reportFirstTetFailedDTC reportFirstConfirmedDTC reportMostRecentTestFailedDTC reportMostRecentConfirmedDTC reportDTCFaultDetectionCounter reportDTCWithPermanentStatus]	M	#0Ah #0Bh #0Ch #0Dh #0Eh #14h #15h	LEV_ RSUPDTC RFTFDTC RFCDDTC RMRTFDTC RMRCDDTC RDTCFDC RDTCWPS

Table6-10. Request Message definition – sub-function = #0A~15

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportNumberOfDTCBySeverityMaskRecord reportDTCBySeverityMaskRecord]	M M	#07h #08h	LEV_ RNODTCBSMR RDTCBSMR

#3d	DTCSeverityMaskRecord[] = [DTCSeverityMask DTCStatusMask]	M	#XXh	DTCSVMREC_
#4d		M	#XXh	DTCSVM DTCSM
DTCSeverityMask, DTCSeverity : See Annex D.3				

Table6-11. Request Message definition – sub-function = #07 / 08h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportSeverityInformationOfDTC]	M	#09h	LEV_ RSIODTC
#3d	DTCMaskRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte]	M	#XXh	DTCMREC_ DTCHB
#4d		M	#XXh	DTCMB
#5d		M	#XXh	DTCLB

Table6-12. Request Message definition – sub-function = #09h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportSupportedDTCExtDataRecord]	M	#1Ah	LEV_ RDTCEDI
#3d	DTCExtDataRecordNumber	M	#01~FDh	DTCEDRN

Table6-23. Request Message definition – sub-function = #1Ah

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportWWHOBDDTCByMaskRecord]	M	#42h	LEV_ ROBDDTCBMR
#3d	FunctionalGroupIdentifier	M	#XX	FGID
#4d	DTCSeverityMaskRecord[] = [DTCStatusMask DTCSeverityMask]	M	#XX	DTCSVMREC_ DTCSM
#5d			#XX	DTCSVM

Table6-24. Request Message definition – sub-function = #42h

Hex	Description	Cvt	Mnemonic
00~32	ISO/SAE reserved	M	ISOSAERESRVD
33	Emissions-system group	M	EMSYSGRP
34~CF	ISO/SAE reserved	M	ISOSAERESRVD
D0	Safety-system group	M	SAFESYSGRP
D1~DF	Legislative system group	M	LEGSYSGRP
E0~FD	ISO/SAE reserved	M	ISOSAERESRVD
FE	VOBD system	M	VOBDSYSGRP
FF	ISO/SAE reserved	M	ISOSAERESRVD

Table6-25. FunctionalGroupIdentifiers definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportWWHOBDDTCWithPermanentStatus]	M	#55h	LEV_ RWWHOBDDT CWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID

Table6-26. Request Message definition – sub-function = #55h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Request Service ID	M	#19h	RDTCI
#2d	Sub-function = [reportDTCInformationByDTCReadinessGroupIdentifier]	M	#56h	LEV_ RDTCBRGI
#3d	FunctionalGroupIdentifier	M	#XX	FGID
#4d	DTCReadinessGroupIdentifier	M	#XX	DTCRGI

Table6-27. Request Message definition – sub-function = #56h

Hex	Description	Cvt	Mnemonic
00	Reserved	M	
01	reportNumberOfDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of DTC's matching a client defined status mask.	M	RNODTCBSM
02	reportDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of DTC's and corresponding statuses matching a client defined status mask.	M	RDTCBM
03	reportDTCSnapshotIdentification This parameter specifies that the server shall transmit to the client all DTCSnapshot data record identifications (DTC number(s) and DTCSnapshot record number(s)).	U	RDTCCSI
04	reportDTCSnapshotRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCSnapshot record(s) associated with a client defined DTC number and DTCSnapshot record number (FF hex for all records).	U	RDTCCSBDTC
05	reportDTCSnapshotRecordByRecordNumber This parameter specifies that the server shall transmit to the client the DTCSnapshot record(s) associated with a client defined DTCSnapshot record number (FF hex for all records). Note that this sub-function parameter can only be supported if the DTCSnapshotRecordNumber is unique to the server (each record number exists only once in the server) and not unique to a DTC.	U	RDTCCSBRN
06	reportDTCExtendedDataRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCExtendedData record(s) associated with a client defined DTC number and DTCExtendedData record number (FF hex for all records, FE hex for all OBD records).	U	RDTCCEDRBDN
07	reportNumberOfDTCBySeverityMaskRecord This parameter specifies that the server shall transmit to the client the number of DTC's matching a client defined severity mask record.	U	RNODTCBSMR
08	reportDTCBySeverityMaskRecord This parameter specifies that the server shall transmit to the client a list of DTC's and corresponding statuses matching a client defined severity mask record.	U	RDTCBMR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 100/211

09	reportSeverityInformationOfDTC This parameter specifies that the server shall transmit to the client the severity information of a specific DTC specified in the client request message.	U	RSIODTC
0A	reportSupportedDTC This parameter specifies that the server shall transmit to the client a list of all DTC's and corresponding statuses supported within the server.	M	RSUPDTC
0B	reportFirstTestFailedDTC This parameter specifies that the server shall transmit to the client the first failed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of whether or not the DTC was confirmed or aged.	U	RFTFDTC
0C	reportFirstConfirmedDTC This parameter specifies that the server shall transmit to the client the first confirmed DTC to be detected by the server since the last clear of diagnostic information. The information reported via this sub-function parameter shall be independent of the aging process of confirmed DTC's (e.g. if a DTC ages such that its status is allowed to be reset, the first confirmed DTC record shall continue to be preserved by the server, regardless of any other DTC's that become confirmed afterwards).	U	DFCDTC
0D	reportMostRecentTestFailedDTC This parameter specifies that the server shall transmit to the client the most recent failed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of whether or not the DTC was confirmed or aged.	U	RMRVDTC
0E	reportMostRecentConfirmedDTC This parameter specifies that the server shall transmit to the client the most recent confirmed DTC to be detected by the server since the last clear of diagnostic information. Note that the information reported via this sub-function parameter shall be independent of the aging process of confirmed DTC's (e.g. if a DTC ages such that its status is allowed to be reset, the first confirmed DTC record shall continue to be preserved by the server assuming no other DTC's become confirmed afterwards).	U	RMRC DTC
0F	reportMirrorMemoryDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of DTC's out of the DTC mirror memory and corresponding statuses matching a client defined status mask.	U	RMMDTCBSM
10	reportMirrorMemoryDTCExtendedDataRecordByDTCNumber This parameter specifies that the server shall transmit to the client the DTCExtendedData record(s) - out of the DTC mirror memory - associated with a client defined DTC number and DTCExtendedData record number (FF hex for all records) DTC's.	U	RMMDED RBDN

11	reportNumberOfMirrorMemoryDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of DTC's out of mirror memory matching a client defined status mask.	U	RNOMMDTCBSM
12	reportNumberOfEmissionsRelatedOBDDTCByStatusMask This parameter specifies that the server shall transmit to the client the number of emissionsrelated OBD DTC's matching a client defined status mask. The number of OBD DTC's reported shall only be those which are required to be compatible with emissions-related legal requirements.	U	RNOOBDDTCBSM
13	reportEmissionsRelatedOBDDTCByStatusMask This parameter specifies that the server shall transmit to the client a list of emissions-related OBD DTC's and corresponding statuses matching a client defined status mask. The list of OBD DTC's reported shall only be those which are required to be compatible with emissions-related legal requirements.	U	ROBDDTCBSM
14~19	Reserved	U	RESRVD
1A	reportDTCExtendedDataRecordIdentification This parameter specifies that the server shall transmit to the client the DTCs which supports a DTCExtendedDataRecord.	U	
1B~41	ISOSAEReserved	M	ISOSAERESRVD
42	reportWWHOBDDTCByMaskRecord This parameter specifies that the server shall transmit to the client a list of WWH OBD DTCs and corresponding status and severity information matching a client defined status mask and severity mask record.	U	RWWHOBDDTCBMR
43~54	ISOSAEReserved	M	ISOSAERESRVD
55	reportWWHOBDDTCWithPermanentStatus This parameter specifies that the server shall transmit to the client a list of WWH OBD DTCs with "permanent DTC" status	U	RWWHOBDDTCWPS
56	reportDTCInformationByDTCReadinessGroupIdentifier This parameter specifies that the server shall transmit to the client a list of OBD DTCs which matches the DTCReadiness Group Identifier.	U	RDTCBRGI
57~7F	ISOSAEReserved	M	ISOSAERESRVD

Table6-13. Request message sub-function definition

6.1.6 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPLR
#2d	reportType = [reportNumberOfDTCByStatusMask reportNumberOfDTCBySeverityMaskRecord reportNumberOfMirrorMemoryDTCByStatusMask reportNumberOfEmissionRelatedOBDDTCByStatusMask]	M	#01h #07h #11h #12h	LEV_ RNODTCBSM RNODTCBSMR RNOMMDTCBSM RNOOBDDTCBSM
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM

#4d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ SAE_J2012- DA DTC Format_04
#5d	DTCCount[] = [DTCCountHighByte DTCCountLowByte]	M	#XXh	DTCC_ DTCCHB
#6d		M	#XXh	DTCCLB

Table6-14. Positive Response Message definition – sub-function = #01 / 07 / 11 / 12h

Hex	Description	Mnemonic
00	SAE_J2012-DA_DTCFormat_00	J2012-DADTCF00
01	ISO 14229-1 DTC Format	14229-1DTCF
02	SAE J1939-73 DTC Format	J1939-73DTCF
03	ISO 11992-4 DTC Format	11992-4DTCF
04	SAE_J2012-DA DTC Format_04	J2012-DADTCF04
05~FF	Reserved	

Table6-15. DTCFormat Identifier definition

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPLR
#2d	reportType = [reportDTCByStatusMask reportSupportedDTCs reportFirstTestFailedDTC reportFirstConfirmedDTC reportMostRecentTestFailedDTC reportMostRecentConfirmedDTC reportDTCWithPermanentStatus]	M	#02h #0Ah #0Bh #0Ch #0Dh #0Eh #15h	LEV_ RDTCBMS RSUPDTC RFTFDTC RFCDDTC RMRTFDTC RMRCDDTC RDTCWPS
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#4d	DTCAndStatusRecord[] = [DTCHighByte#1 DTCMiddleByte#1 DTCLowByte#1 statusOfDTC#1 DTCHighByte#2 DTCMiddleByte#2 DTCLowByte#2 statusOfDTC#2 .. DTCHighByte#m DTCMiddleByte#m DTCLowByte#m statusOfDTC#m	C	#XXh	DTCASR_ DTCHB
#5d		C	#XXh	DTCMB
#6d		C	#XXh	DTCLB
#7d		C	#XXh	SODTC
#8d		C	#XXh	DTCHB
#9d		C	#XXh	DTCMB
#10d		C	#XXh	DTCLB
#11d		C	#XXh	SODTC
..		..	..	..
#n-3d		C	#XXh	DTCHB
#n-2d		C	#XXh	DTCMB
#n-1d		C	#XXh	DTCLB
#nd		C	#XXh	SODTC

Table6-16. Positive Response Message definition – sub-function = #02 / 0A~0E / 15

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPLR
#2d	reportType = [reportDTCSnapshotIdentification]	M	#03h	LEV_ RDTCSSI

#3d	DTCRecord[] #1 = [DTCHighByte#1 DTCMiddleByte#1 DTCLowByte#1	C	#XXh	DTCASR_ DTCHB
#4d		C	#XXh	DTCMB
#5d		C	#XXh	DTCLB
#6d	DTCSnapshotRecordNumber #1	C	#XXh	DTCSSRN
..	..	..	..	..
#9d	DTCRecord[] #m = [DTCHighByte#m DTCMiddleByte#m DTCLowByte#m]	C	#XXh	DTCASR_ DTCHB
#10d		C	#XXh	DTCMB
#11d		C	#XXh	DTCLB
#nd	DTCSnapshotRecordNumber #m	C	#XXh	DTCSSRN

Table6-17. Positive Response Message definition – sub-function = #03h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportDTCSnapshotRecordByDTCNumber]	M	#04h	LEV_ RDTCSSBDTC
#3d	DTCAndStatusRecord[] = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	M	#XXh	DTCASR_ DTCHB
#4d		M	#XXh	DTCMB
#5d		M	#XXh	DTCLB
#6d		M	#XXh	SODTC
#7d	DTCSnapshotRecordNumber #1	C	#XXh	DTCSSRN
#8d	DTCSnapshotRecordNumberOfIdentifiers #1	C	#XXh	DTCSSRNI
#9d	DTCSnapshotRecord[] #1 = [dataIdentifier#1 byte#1 (MSB) .. dataIdentifier#1 byte#k snapshotData#1 byte#1 .. snapshotData#1 byte#p .. dataIdentifier#w byte#1 (MSB) .. dataIdentifier#2 byte#k snapshotData#w byte#1 .. snapshotData#w byte#m]	C	#XXh	DTCSSR_ DIDB11
..		..	..	..
#9+k-1d		C	#XXh	DIDB1k
#9+kd		C	#XXh	SSD11
..		..	..	..
#9+k+(p-1)d		C	#XXh	SSD1p
..		..	..	..
#r-(m-1)-2d		C	#XXh	DIDBw1
..		..	..	..
#r-(m-1)-1d		C	#XXh	DIDBwk
#r-(m-1)d		C	#XXh	SSDw1
..		..	..	..
#rd		C	#XXh	SSDwm
..	..	..	..	..
#t	DTCSnapshotRecordNumber#x	C	#XXh	DTCSSRN
#t+1d	DTCSnapshotRecordNumberOfIdentifiers #x	C	#XXh	DTCSSRNI

#t+2d	DTCSnapshotRecord[] #x = [	dataIdentifier#1 byte#1 (MSB)	C	#XXh	DTCSSR_ DIDB11
..	..	..	..	..	..
#t+2+k-1d	..	dataIdentifier#1 byte#k	C	#XXh	DIDB1k
#t+2+kd	..	snapshotData#1 byte#1	C	#XXh	SSD11
..	..	..	..	..	..
#t+2+k+(p-1)d	..	snapshotData#1 byte#p	C	#XXh	SSD1p
..	..	..	..	..	..
#n-(u-1)-2d	..	dataIdentifier#w byte#1 (MSB)	C	#XXh	DIDBw1
..	..	..	..	..	..
#n-(u-1)-1d	..	dataIdentifier#2 byte#k	C	#XXh	DIDBwk
#n-(u-1)d	..	snapshotData#w byte#1	C	#XXh	SSDw1
..	..	..	..	..	..
#nd	..	snapshotData#w byte#u]	C	#XXh	SSDwu

Table6-18. Positive Response Message definition – sub-function = #04h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTCIPLR
#2d	reportType = [reportDTCSnapshotRecordByRecordNumber]	M	#05h	LEV_ RDTCSBRN
#3d	DTCSnapshotRecordNumber#1	M	#XXh	DTCEDRN
#4d	DTCSnapshotRecord[] #1 = [DTCHighByte	M	#XXh	DTCASR_ DTCHB
#5d	DTCMiddleByte	M	#XXh	DTCMB
#6d	DTCLowByte	M	#XXh	DTCLB
#7d	statusOfDTC]	M	#XXh	SODTC
#8d	DTCSnapshotRecordNumberOfIdentifiers #1	C	#XXh	DTCSSRNI
#9d	DTCSnapshotRecord[] #1 = [dataIdentifier#1 byte#1 (MSB)	C	#XXh	DTCSSR_ DIDB11
..	..	..	..	..
#9+k-1d	..	C	#XXh	DIDB1k
#9+kd	..	C	#XXh	SSD11
..	..	..	..	..
#9+k+(p-1)d	..	C	#XXh	SSD1p
..	..	..	..	..
#r-(m-1)-2d	..	C	#XXh	DIDBw1
..	..	..	..	..
#r-(m-1)-1d	..	C	#XXh	DIDBwk
#r-(m-1)d	..	C	#XXh	SSDw1
..	..	..	..	..
#rd	..	C	#XXh	SSDwm
..	..	..	..	..
#t	DTCSnapshotRecordNumber#x	C	#XXh	DTCSSRN
#t+1d	DTCSnapshotRecord[] #x = [DTCHighByte	M	#XXh	DTCASR_ DTCHB
#t+2d	DTCMiddleByte	M	#XXh	DTCMB
#t+3d	DTCLowByte	M	#XXh	DTCLB
#t+4d	statusOfDTC]	M	#XXh	SODTC
#t+5d	DTCSnapshotRecordNumberOfIdentifiers #x	C	#XXh	DTCSSRNI

#t+6d	DTCSnapshotRecord[] #x = [	dataIdentifier#1 byte#1 (MSB)	C	#XXh	DTCSSR_ DIDB11
..		..	..	..	..
#t+6+k-1d		dataIdentifier#1 byte#k	C	#XXh	DIDB1k
#t+6+kd		snapshotData#1 byte#1	C	#XXh	SSD11
..		..	..	..	..
#t+6+k+(p-1)d		snapshotData#1 byte#p	C	#XXh	SSD1p
..		..	..	..	..
???		dataIdentifier#w byte#1 (MSB)	C	#XXh	DIDBw1
..		..	..	..	..
#n-(u-1)-1d		dataIdentifier#2 byte#k	C	#XXh	DIDBwk
#n-(u-1)d		snapshotData#w byte#1	C	#XXh	SSDw1
..		..	..	..	..
#nd		snapshotData#w byte#u]	C	#XXh	SSDwu

Table6-19. Positive Response Message definition – sub-function = #05h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportDTCExtendedDataRecordByDTCNumber reportMirrorMemoryDTCExtendedDataReordByDTCNumber]	M	#06h #10h	LEV_ RDTCEDRBDN RMMDEDRBDN
#3d	DTCAndStatusRecord[] = [DTCHighByte	M	#XXh	DTCASR_ DTCHB
#4d	DTCMiddleByte	M	#XXh	DTCMB
#5d	DTCLowByte	M	#XXh	DTCLB
#6d	statusOfDTC]	M	#XXh	SODTC
#7d	DTCExtendedDataRecordNumber#1	C	#XXh	DTCEDRN
#8d	DTCExtendedDataRecord[] #1 = [extendedData#1 byte#1	C	#XXh	DTCSSR_ EDD11
..	..	..	..	..
#8+(p-1)d	extendedData#1 byte#p]	C	#XXh	EDD1p
..	..	..	..	..
#td	DTCExtendedDataRecord#x	C	#XXh	DTCEDRN
#t+1d	DTCExtendedDataRecord[] #x = [extendedData#1 byte#1	C	#XXh	DTCSSR_ EDDx1
..	..	..	..	..
#t+1+(q-1)d	extendedData#1 byte#q]	C	#XXh	EDDxq

Table6-20. Positive Response Message definition – sub-function = #06 / 10h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportDTCBySeverityMaskRecord reportSeverityInformationOfDTC]	M	#08h #09h	LEV_ RDTCBSMR RSIODTC
#3d	DTCStatusAvailabilityMask	C	#XXh	DTCSAM

#4d	DTCAndSeverityRecord[] = [	DTCSeverity #1	C	#XXh	DTCASR_
#5d		DTCFunctionalUnit #1	C	#XXh	DTCS
#6d		DTCHighByte #1	C	#XXh	DTCFU
#7d		DTCMiddleByte #1	C	#XXh	DTCHB
#8d		DTCLowByte #1	C	#XXh	DTCMB
#9d		statusOfDTC #1	C	#XXh	DTCLB
..		..	..	..	SODTC
#n-5d		DTCSeverity #m	C	#XXh	..
#n-4d		DTCFunctionalUnit #m	C	#XXh	DTCS
#n-3d		DTCHighByte #m	C	#XXh	DTCFU
#n-2d		DTCMiddleByte #m	C	#XXh	DTCHB
#n-1d		DTCLowByte #m	C	#XXh	DTCMB
#nd		statusOfDTC #m	C	#XXh	DTCLB
					SODTC

Table6-21. Positive Response Message definition – sub-function = #08 / 09h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportSupportedDTCExtDataRecord]	M	#1Ah	LEV_ RDTCEDI
#3d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#4d	DTCExtDataRecordNumber	C1	#01~FD	DTCEDRN
#5d	DTCAndStatusRecord[]#1 = [DTCHighByte #1	C1	#XXh	DTCASR_ DTCHB
#6d	DTCMiddleByte #1	C1	#XXh	DTCMB
#7d	DTCLowByte #1	C1	#XXh	DTCLB
#8d	statusOfDTC #1]	C1	#XXh	SODTC
..	..	..	..	..
#n-3d	DTCAndStatusRecord[]#m = [DTCHighByte #m	C2	#XXh	DTCASR_ DTCHB
#n-2d	DTCMiddleByte #m	C2	#XXh	DTCMB
#n-1d	DTCLowByte #m	C2	#XXh	DTCLB
#nd	statusOfDTC #m]	C2	#XXh	SODTC

C1: This parameter shall be present if at least one DTC supports the DTCExtendedDataRecord
C2: This parameter record is only present if more than one DTC support the DTCExtendedDataRecord

표 6-28. Positive Response Message definition – sub-function = #1Ah

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportWWHOBDDTCByMaskRecord]	M	#42h	LEV_ RWWHOBDDT CBSMR
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCSeverityAvailabilityMask	M	#XXh	DTCSVAM
#6d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012- DADTCF04

#7d	DTCAndSeverityRecord[] = [	DTCSeverity#1	C1	#XXh	DTCASR_
#8d		DTCHighByte#1 (MSB)	C1	#XXh	DTCS
#9d		DTCMiddleByte#1	C1	#XXh	DTCHB
#10d		DTCLowByte#1	C1	#XXh	DTCMB
#11d		statusOfDTC#1	C1	#XXh	DTCLB
..		..	..	..	SODTC
#n-4d		DTCSeverity#m	C2	#XXh	..
#n-3d		DTCHighByte#m (MSB)	C2	#XXh	DTCS
#n-2d		DTCMiddleByte#m	C2	#XXh	DTCHB
#n-1d		DTCLowByte#m	C2	#XXh	DTCMB
#nd		statusOfDTC#m]	C2	#XXh	DTCLB
					SODTC

C1: This parameter is only present if DTC information is available to be reported.
C2: This parameter is only present if more than one DTC information is available to be reported.

표 6-29. Positive Response Message definition – sub-function = #42h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportWWHOBDDTCWithPermanentStatus]	M	#55h	LEV_ RWWHOBDDT CWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012- DADTCF04
#6d	DTCAndStatusRecord[] = [DTCHighByte#1 (MSB) DTCMiddleByte#1 DTCLowByte#1 statusOfDTC#1 .. DTCHighByte#m (MSB) DTCMiddleByte#m DTCLowByte#m statusOfDTC#m]			DTCASR_
#7d		C1	#XXh	DTCHB
#8d		C1	#XXh	DTCMB
#9d		C1	#XXh	DTCLB
		C1	#XXh	SODTC
..		..	..	..
#n-3d		C2	#XXh	DTCHB
#n-2d		C2	#XXh	DTCMB
#n-1d		C2	#XXh	DTCLB
#nd	C2	#XXh	SODTC	

C1: This parameter is only present if DTC information is available to be reported.

C2: This parameter is only present if more than one DTC information is available to be reported.

표 6-30. Positive Response Message definition – sub-function = #55h

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	ReadDTCInformation Response Service ID	M	#59h	RDTICIPR
#2d	reportType = [reportDTCByReadinessGroupIdentifier]	M	#56h	LEV_ RWWHOBDDT CWPS
#3d	FunctionalGroupIdentifier	M	#XXh	FGID
#4d	DTCStatusAvailabilityMask	M	#XXh	DTCSAM
#5d	DTCFormatIdentifier = [SAE_J2012-DA_DTCFormat_04]	M	#04h	DTCFID_ J2012- DADTCF04
#6d	DTCReadinessGroupIdentifier	M	#XXh	RGI

#7d	DTCAndStatusRecord[] = [	DTCHighByte#1 (MSB)	C1	#XXh	DTCASR_
#8d		DTCMiddleByte#1	C1	#XXh	DTCHB
#9d		DTCLowByte#1	C1	#XXh	DTCMB
#10d		statusOfDTC#1	C1	#XXh	DTCLB
..		..	..	..	SODTC
#n-3d		DTCHighByte#m (MSB)	C2	#XXh	DTCHB
#n-2d		DTCMiddleByte#m	C2	#XXh	DTCMB
#n-1d		DTCLowByte#m	C2	#XXh	DTCLB
#nd		statusOfDTC#m]	C2	#XXh	SODTC

C1: This parameter shall be present if at least one DTC matches the client defined Readiness Group Identifier.
C2: This parameter record is only present if more than one DTC matches the client defined Readiness Group Identifier

표 6-31. Positive Response Message definition – sub-function = #56h

6.1.7 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
12	subFunctionNotSupported	M	This code is returned if the requested sub-function is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
31	requestOutOfRange	M	This code is returned if : 1. The client specified a DTCMaskRecord / DTCSeverityMaskRecord that was not recognized by the server. 2. The client specified an invalid DTCSnapshotRecordNumber / DTCExtendedDataRecordNumber.

Table6-22. Supported Negative Response Codes

7. InputOutput Control functional unit

7.1 InputOutputControlByIdentifier (2F hex) service

The InputOutputControlByIdentifier service is used by the client to substitute a value for an input signal, internal server function and/or control an output (actuator) of an electronic system.

The server shall send a positive response message to a request message with an inputOutputControlParameter of returnControlToECU even if the dataIdentifier is currently not under tester control.

7.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	InputOutputControlByIdentifier Request Service ID	M	#2Fh	IOCB
#2d	dataIdentifier #1[] = [byte #1 (MSB) byte #2 (LSB)]	M	#XXh	IOI_ B1
#3d		M	#XXh	B2
#4d	controlOptionRecord #1[] = [inputOutputControlParameter	M	#XXh	CSR_ IOCP
#5d	controlOptionRecord #2[] = [controlState#1 .. controlState#m	C1	#XXh	CSR_ CS_ ..
..		..	..	..
#5+(m-1)d	controlState#m	C1	#XXh	CS_ ..
#5+md	controlEnableMaskRecord #1[] = [controlMask#1 .. controlMask#r]	C2	#XXh	CEM_ CM_ ..
..		..	..	..
#5+m+(r-1)d	controlMask#r]	C2	#XXh	CM_ ..

C1 : The presence of this parameter depends on the dataIdentifier#1 and the inputOutputControlParameter of controlOptionRecord#1. (See Table 7-2)

C2 : The presence of this parameter depends on the dataIdentifier#1.

dataIdentifier : see Annex C.1

controlEnableMaskRecord : see Annex C.1.1

The value of each bit shall determine whether the corresponding parameter in the dataIdentifier will be affected by the request. A bit value of '0' in the ControlEnableMask shall represent that the corresponding parameter is not affected by this request and a bit value of '1' shall represent that the corresponding parameter is affected by this request. The most significant bit of ControlMask#1 shall correspond to the first parameter in the ControlState starting at the most significant bit of ControlState#1, the second most significant bit of ControlMask#1 shall correspond to the second parameter in the ControlState, and continuing on in this fashion utilising as many ControlMask bytes as necessary to mask all parameters. For example, the least significant bit of ControlMask#2 would correspond to the 16th parameter in the controlState. For bitmapped dataIdentifiers, unsupported bits shall also have a corresponding bit in the ControlEnableMask so that the position of the mask bit of every parameter in the ControlEnableMask shall exactly match the position of the corresponding parameter in the controlState.

Table7-1. Request Message

Hex	Description	Cvt	Mnemonic
-----	-------------	-----	----------

00	returnControlToECU This value shall indicate to the server that the client does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: if the dataIdentifier consists of more than one parameter	U	RCTECU
01	resetToDefault This value shall indicate to the server that it is requested to reset the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier to its default state. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RTD
02	freezeCurrentState This value shall indicate to the server that it is requested to freeze the current state of the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. Number of controlState bytes in request: 0 Number of controlState bytes in pos. response: depends on the dataIdentifier	U	FCS
03	shortTermAdjustment This value shall indicate to the server that it is requested to adjust the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set pulse width of valve to a specific value/duty cycle). Number of controlState bytes in request: depends on the dataIdentifier Number of controlState bytes in pos. response: depends on the dataIdentifier	U	STA

Table7-2. inputOutputControlParameter definition

Unit	Hex	Physical Value	Remark
Time	00~3F	0~63 (sec)	
	40~7F	Reserved	
	80~F4	0~1000 (ms)	
	F5~FF	Reserved	

Table7-3. controlState definition

7.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	InputOutputControlByIdentifier Response Service ID	M	#6Fh	IOCBIPR
#2d	dataIdentifier #1[] = [byte #1 (MSB) byte #2 (LSB)]	M	#XXh	IOI_ B1
#3d		M	#XXh	B2
#4d	controlOptionRecord #1[] = [inputOutputControlParameter	M	#XXh	CSR_ IOCP
#5d	controlOptionRecord #2[] = [controlState#1 .. controlState#m	C	#XXh	CSR_ CS_ ..
..		..	..	..
#5+(m-1)d		C	#XXh	CS_ ..

Table7-4. Positive Response Message definition



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 111/211

7.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request InputOutputControlByIdentifier are not met.
31	requestOutOfRange	M	This code shall be returned if : 1. the requested dataIdentifier value is not supported by the device. 2. the dataIdentifier uses the controlState#1 parameter as an inputOutputControlParameter and the value contained in this parameter is invalid. 3. the one or multiple of the applicable controlStates of the controlOptionRecord record are invalid. 4. the combination of bits enabling control in the ControlEnableMaskRecord is not supported by the device.
33	securityAccessDenied	M	This code shall be returned if a client sends a request with a valid secure dataIdentifier and the server's security feature is currently active.

Table7-5. Supported Negative Response Codes

8. Remote Activation of Routine functional unit

The following section describes two (2) different methods of implementation (Methods “A” and “B”).

- Method “A” ;

- This method is based on the assumption that after a routine has been started by the client in the server's memory the client shall be responsible to stop the routine.
- The server routine shall be started in the server's memory some time between the completion of the RoutineControl request message that starts the routine and the completion of the first response message (if “positive” based on the server's conditions).
- The server routine shall be stopped in the server's memory some time after the completion of the StopRoutine request message and the completion of the first response message (if “positive” based on the server's conditions).
- The client may request routine results after the routine has been stopped.

- Method “B” ;

- This method is based on the assumption that after a routine has been started by the client in the server's memory that the server shall be responsible to stop the routine.
- The server routine shall be started in the server's memory some time between the completion of the RoutineControl request message that starts the routine and the completion of the first response message (if “positive” based on the server's conditions).
- The server routine shall be stopped any time as programmed or previously initialized in the server's memory.

8.1 RoutineControl (31 hex) service

The RoutineControl service is used by the client to ;

- start a routine,
- stop a routine, and
- request routine results

8.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [			

Table8-1. Request Message

Hex	Description	Cvt	Mnemonic
01	startRoutine	M	STR

02	stopRoutine	U	STPR
03	requestRoutineResults	U	RRR

Table8-2. Request message sub-function definition

Hex	Description	Cvt	Mnemonic
0000 – 00FF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD
0100 – 01FF	Reserved (Tachograph TestIds)	M	TACHORI_
0200 - DFFF	Reserved (vehicleManufacturerSpecific)	M	VMS_
E000 – E1FF	Reserved (OBDTestIds) IMPORTANT : Do not use other than legislative ECU	M	OBDRI_
E200 - EFFF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD
F000 - FFFF	Reserved (SystemSupplierSpecific)	M	SSS_
FF00	eraseMemory (Calibration Data & Program)	U	EM_
FF01	checkProgrammingDependencies	U	CPD_
FF02 - FFFF	Reserved (ISOSAE Reserved)	M	ISOSAERESRVD

Table8-3. Request message routineIdentifier definition

8.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl Response Service ID	M	#71h	RCPR
#2d	routineControlType	M	#XXh	RCTP_
#3d	routineIdentifier [] = [byte #1 (MSB) byte #2]	M	#XXh	RI_ B1
#4d		M	#XXh	B2
#5d	routineStatusRecord [] = [routineStatus #1 .. routineStatus #m]	U	#XXh	RSR_ RS_
..		..	..	..
#nd		U	#XXh	RS_

Table8-4. Positive Response Message definition

8.1.3 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
10	generalReject	M	This response code indicates that the requested action has been rejected by the server. The generalReject response code shall only be implemented in the server if none of the negative response codes defined in this document meet the needs of the implementation.
12	subFunctionNotSupported	M	This code is returned if the requested sub-function is not supported
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This code shall be returned if the criteria for the request RoutineControl are not met.
24	requestSequenceError	M	This code shall be returned if the 'stopRoutine' or 'requestRoutineResults' sub-function is received without first receiving a 'startRoutine' for the requested routineIdentifier.



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 114/211

31	requestOutOfRange	M	The code shall be returned if ; 1. The server does not supported the requested routineIdentifier. 2. The user optional routineControlOptionRecord contains invalid data for the requested routineIdentifier.
33	seurityAccessDenied	M	This NRC shall be sent if a client sends a requested with a valid secure routineIdentifier and the server's security feature is currently active.
72	GeneralProgrammingFailure	M	This NRC shall be returned if the server detects an error when performing a routine, which accesses server internal memory. An example is when the routine erases or prgrams a certain memory location in the permanent memory device (e.g. Flash Memory) and the access to that memory location fails. e.g.) CheckProgrammingDependancy or CheckMemory : SW integrity fails
F4	LowerVersionDownloadAttempt	M	This response code indicates that the requested action will not be taken because the downloaded software is a lower version than the existing version. e.g.) CheckProgrammingDependancy : ECU downgrade attempts IMPORTANT) The memory swap ECU supporting SwapActiveArea should determine the downgrade in SwapActiveArea instead of the CheckProgrammingdependencies step.

Table8-5. Supported Negative Response Codes

9. Upload Download functional unit

9.1 RequestDownload (34 hex) service

The RequestDownload service is used by the client to initiate a data transfer from the client to the server (download). After the server has received the requestDownload request message the server shall take all necessary actions to receive data before it sends a positive response message.

9.1.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestDownload Request Service ID	M	#34h	RD
#2d	dataFormatIdentifier	M	#00h	DFI_
#3d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#4d	memoryAddress[] = [byte#1 (MSB) .. byte#m]	M	#XXh	MA_ B1
..		..	..	..
#(m-1)+4d		C	#XXh	Bm
#n-(k-1)d	memorySize[] = [byte#1 (MSB) .. byte#k]	M	#XXh	MS_ B1
..		..	..	..
#nd		C	#XXh	Bk

dataFormatIdentifier : This parameter is a one byte value with each nibble encoded separately. The high nibble specifies the "compressionMethod, and the low nibble specifies the "encryptingMethod". The value 00 hex specifies that no compressionMethod nor encryptingMethod is used. Value other than 00 hex are vehicle manufacturer specific.

IMPORTANT : Since secure flash is mandatory and its algorithm in use is one (AES-128 CTR), HMC/KIA doesn't use encryptingMethod of dataFormatIdentifier. ECU shall decrypt data (e.g. Secure flash 2.0 method) regardless of the encryptingMethod which is given from parameter of dataFormatIdentifier.

Exception : If an ECU needs to download data without decrypting for development, 'No encrypting method'(0x0) can be used. E.g.) 0x0 : No encrypting, 0x1~F : Secure flash encrypting method.

Table9-1. Request Message definition

9.1.2 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestDownload Response Service ID	M	#74h	RDPR
#2d	lengthFormatIdentifier	M	#XXh	LFID
#3d	maxNumberOfBlockLength [byte#1 (MSB) .. byte#m]	M	#XXh	MNROB_ B1
..		..	..	..
#nd		M	#XXh	Bm

maxNumberOfBlockLength : This parameter is used by the requestDownload positive response message to inform the client how many data bytes (maxNumberOfBlockLength) shall be included in each TransferData request message from the client. This length reflects the complete message length, including the service identifier and the data parameters present in the TransferData request message. This parameter allows the client to adapt to the receive buffer size of the server before it starts transferring data to the server.

Table9-2. Positive Response Message definition

9.1.3 Supported negative response codes



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 116/211

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This return code shall be sent if a server receives a request for this service while in the process of receiving a download of a software or calibration module. This could occur if there is a data size mismatch between the server and the client during the download of a module.
24	requestSequenceError	M	The server shall return this response code in the event that the requested memory area includes a reprogramming zone, and the 'Erase Memory' operation for the requested memory area has not been successfully completed prior to this request.
31	requestOutOfRange	M	This return code shall be sent if ; 1. The specified dataFormatIdentifier is not valid. 2. The specified addressAndLengthFormatIdentifier is not valid. 3. The specified memoryAddress / memorySize is not valid.
33	securityAccessDenied	M	This return code shall be sent if the server is secure (for server's that support the SecurityAccess service) when a request for this service has been received.
70	uploadDownloadNotAccepted	M	This response code indicates that an attempt to download to a server's memory cannot be accomplished due to some fault conditions.

Table9-3. Supported Negative Response Codes

9.2 RequestUpload (35 hex) service

The RequestUpload service is used by the client to initiate a data transfer from the server to the client (upload). After the server has received the requestUpload request message the server shall take all necessary actions to send data before it sends a positive response message.

9.1.4 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestUpload Request Service ID	M	#35h	RU
#2d	dataFormatIdentifier	M	#00h	DFI_
#3d	addressAndLengthFormatIdentifier	M	#XXh	ALFID
#4d	memoryAddress[] = [	M	#XXh	MA_
..	byte#1 (MSB)	..	..	B1
..	..	..	..	..
#(m-1)+4d	byte#m]	C	#XXh	Bm
#n-(k-1)d	memorySize[] = [	M	#XXh	MS_
..	byte#1 (MSB)	..	..	B1
..	..	..	..	..
#nd	byte#k]	C	#XXh	Bk

Table9-4. Request Message definition

9.1.5 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestUpload Response Service ID	M	#75h	RUPR
#2d	lengthFormatIdentifier	M	#XXh	LFID
#3d	maxNumberOfBlockLength [	M	#XXh	MNROB_
..	byte#1 (MSB)	..	..	B1
..	..	..	..	..
#nd	byte#m]	M	#XXh	Bm

Table9-5. Positive Response Message definition

9.1.6 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
22	conditionNotCorrect	M	This return code shall be sent if the criteria for the requestUpload are not met. This could occur if a server receives a request for this service while a requestUpload is already active, but not yet completed.
31	requestOutOfRange	M	This return code shall be sent if ; 1. The specified dataFormatIdentifier is not valid. 2. The specified addressAndLengthFormatIdentifier is not valid. 3. The specified memoryAddress / memorySize is not valid.
33	securityAccessDenied	M	This return code shall be sent if the server is secure (for server's that support the SecurityAccess service) when a request for this service has been received.



기술 표준
ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 118/211

70	uploadDownloadNotAccepted	M	This response code indicates that an attempt to upload to a server's memory cannot be accomplished due to some fault conditions.
----	---------------------------	---	--

Table9-6. Supported Negative Response Codes

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

9.3 TransferData (36 hex) service

The TransferData service is used by the client to transfer data either from the client to the server (download) or from the server to the client (upload).

The data transfer direction is defined by the preceding RequestDownload or RequestUpload service. If the client initiated a RequestDownload the data to be downloaded is included in the parameter(s) transferRequestParameter in the TransferData request message(s). If the client initiated a RequestUpload the data to be uploaded is included in the parameter(s) transferResponseParameter in the TransferData response message(s).

The TransferData service request includes a blockSequenceCounter to allow for an improved error handling in case a TransferData service fails during a sequence of multiple TransferData requests. The blockSequenceCounter of the server shall be initialized to one (1) when receiving a RequestDownload (34 hex) or RequestUpload (35 hex) request message. This means that the first TransferData (36 hex) request message following the RequestDownload (34 hex) or RequestUpload (35 hex) request message starts with a blockSequenceCounter of one (1).

Example use cases:

- If a TransferData request to download data is correctly received and processed in the server but the positive response message does not reach the client then the client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this TransferData request is repeated. The server would send the positive response message immediately without writing the data once again into its memory.
- If the TransferData request to download data is not received correctly in the server then the server would not send a positive response message. The client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this is a new TransferData. The server would process the service and would send the positive response message.
- If a TransferData request to upload data is correctly received and processed in the server but the positive response message does not reach the client then the client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this TransferData request is repeated. The server would send the positive response message immediately accessing the previously provided data once again in its memory.
- If the TransferData request to upload data is not received correctly in the server then the server would not send a positive response message. The client would determine an application layer timeout and would repeat the same request (including the same blockSequenceCounter). The server would receive the repeated TransferData request and could determine based on the included blockSequenceCounter that this is a new TransferData. The server would process the service and would send the positive response message.

9.1.7 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TransferData Request Service ID	M	#36h	TD
#2d	blockSequenceCounter	M	#XXh	BSC

#3d	transferRequestParameterRecord[] = [	C	#XXh	TRPR_
..	transferRequestParameter#1	..	..	TRTP_
#nd	transferRequestParameter#m]	U	#XXh	TRTP_

C = Conditional : this parameter is mandatory if a download is in progress.

blockSequenceCounter
The blockSequenceCounter parameter value starts at 01 hex with the first TransferData request that follows the RequestDownload (34 hex) or RequestUpload (35 hex) service. Its value is incremented by 1 for each subsequent TransferData request. At the value of FF hex the blockSequenceCounter rolls over and starts at 00 hex with the next TransferData request message.

Table9-7. Request Message definition

9.1.8 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	TransferData Response Service ID	M	#76h	TDPR
#2d	blockSequenceCounter	M	#XXh	BSC
#3d	transferResponseParameterRecord[] = [	C	#XXh	TREPR_
..	transferResponseParameter#1	..	..	TREP_
#nd	transferResponseParameter#m]	U	#XXh	TREP_

C = Conditional : this parameter is mandatory if an upload is in progress.

Table9-8. Positive Response Message definition

9.1.9 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	icorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong (e.g., message length does not meet requirements of maxNumberOfBlockLength parameter returned in the positive response to requestDownload).
24	requestSequenceError	M	The server shall use this response code 1. If the RequestDownload or RequestUpload service is not active when a request for this service is received. 2. If the RequestDownload or RequestUpload service is active, but the server has already received all data as determined by the memorySize parameter in the active RequestDownload or RequestUpload service. NOTE) The repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.
31	requestOutOfRange	M	This return code shall be sent if the transferRequestParameterRecord contains additional control parameters (e.g. additional address information) and this control information is invalid.



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 121/211

71	transferDataSuspended	M	This return code shall be sent if 1. The response code indicates that a data transfer operation was halted due to some fault. 2. The download module length does not meet the requirements of the memorySize parameter sent in the request message of the requestDownload service.
72	generalProgrammingFailure	M	This return code shall be sent if the server detects an error when erasing or programming a memory location in the permanent memory device (e.g. Flash Memory) during the download of data.
73	wrongBlockSequenceCounter	M	This return code shall be sent if the server detects an error in the sequence of the blockSequenceCounter. NOTE) The repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.
92	voltageTooHigh	M	This return code shall be sent as applicable if the voltage measured at the primary power pin of the server is out of the acceptable range for downloading data into the server's permanent memory (e.g. Flash Memory).
93	voltageTooLow	M	

Table9-9. Supported Negative Response Codes

9.1.10 Implementation rules

The BlockSequenceCounter allows the diagnostic server to detect and handle transmission errors. It shall start at 1. Its value is incremented by 1 for each subsequent request of this service. When the block number 255 (0xFF) is transmitted, it shall continue with block number 0.

9.4 RequestTransferExit (37 hex) service

This service is used by the client to terminate a data transfer between client and server (upload or download).

9.1.11 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestTransferExit Request Service ID	M	#37h	RTE
#2d	transferRequestParameterRecord[] = [transferRequestParameter#1	U	#XXh	TRPR_ TRTP_
..	..	..	..	..
#nd	trans ferRequestParameter#m]	U	#XXh	TRTP_
transferReqeustParameterRecord - This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.				

Table9-10. Request Message

9.1.12 Positive response message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RequestTransferExit Response Service ID	M	#77h	RTEPR
#2d	transferResponseParameterRecord[] = [transferResponseParameter#1	U	#XXh	TREPR_ TREP_
..	..	..	..	..
#nd	trans ferResponseParameter#m]	U	#XXh	TREP_
transferResponseParameterRecord - This parameter shall contain parameter(s) which are required by the client to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.				

Table9-11. Positive Response Message definition

9.1.13 Supported negative response codes

Hex	NegativeResponseCode	Cvt	Cause of Occurrence
13	incorrectMessageLengthOrInvalidFormat	M	The length of the message is wrong
24	requestSequenceError	M	This return code shall be sent if ; 1. The programming process is not completed when a request for this service is received. 2. The RequestDownload or ReqeustUpload service is not active.
31	requestOutOfRange	U	This NRC shall be returned if the transferRequestParameterRecord contains invalid data.
72	GeneralProgrammingFailure	U	This NRC shall be returned if the server detects an error when finalizing the data transfer between the client and server (e.g., via an integrity check).

Table9-12. Supported Negative Response Codes

10. Non-volatile server memory programming process

10.1 Requirements for Flash Reprogramming

NOTE : This specification describes Legacy reprogramming.
Refer to the ES98765-02 specification for reprogramming standards

10.1.1 General Requirement

Identification information¹⁾, which would be compared with tool's one during the programming process or have to be always available, should be located in a non-erasable area. It will enable reprogramming process even if the server has incomplete set of application software and application data caused by failure of Flash Reprogramming (erasing identification information).

10.1.2 Boot Area

- 1) All programmable servers that support programming of the application software shall contain boot software in a boot memory region.
- 2) The boot memory shall be protected against inadvertent erasure.
- 3) The boot software can be protected via hardware or software.
- 4) Servers that support boot software shall continue to execute out of the boot until a complete set of application software and/or application data is programmed.
- 5) The server shall be able to recover and be reprogrammed if any of the following error conditions occur during the programming process:
 - ① Loss of supplied power connection
 - ② Loss of the ground connection
 - ③ Disruption of CAN communication
 - ④ Over- or under-voltage conditions
- 6) No programmable server operating out of boot memory shall transmit a non-diagnostic communication message or unsolicited diagnostic message.
- 7) All servers operation out of boot memory shall be able to receive diagnostic messages.
- 8) A server shall be capable of using the same diagnostic CAN IDs for the duration of a programming event.
- 9) The Boot software shall support 'Extended Diagnostic Mode' with the 'disable normal message transmission', 'ECU reset', 'Tester Present' and 'Start Diagnostic Session' services to enable to restart reprogramming procedure when a sever exits the programming process unsuccessfully.

10.1.3 Server reprogramming requirements

10.1.3.1 Requirements for all servers to support programming

The DiagnosticSessionControl (10 hex) service, with sessionType equal to extendedDiagnosticSession followed by a CommunicationControl (28 hex) service that disables non-diagnostic messages and a ControlDTCSetting (85 hex) service that disables setting of DTCs, shall be used by the server to recognize the disabling of normal communication and to ensure that a server does not set DTCs while



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 124/211

another server is being programmed. The extendedDiagnosticSession shall be kept active by the client for the duration of the programming event.

10.1.3.2 Hardware requirements for programmable servers to support programming

All servers that are programmable shall be able to interface with the programming tools used by development, the assembly plant and by service via the appropriate pins of the vehicle diagnostic connector.

The only power required at the vehicle diagnostic connector for programming shall be vehicle battery power.

Any server that is properly installed in the vehicle and is programmable shall be able to be programmed via the vehicle diagnostic connector. Removal of the server from the vehicle in order to perform programming shall not be required.

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 125/211

10.2 Non-volatile server memory programming sequence

10.1.4 Programming Phase #1

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
ReadDataByIdentifier	22	VehicleManufacturerECU SoftwareVersionNumber DataIdentifier	F189	-	Physical	

Table10-1. Programming Phase #1 Sequence

10.1.4.1 ReadDataByIdentifier – ECUSoftwareVersion

The tester will send request message to compare the software of ECU in the vehicle with the software in the tester. If the software in the vehicle is later version than the software in the tester or the same version, the programming sequence will be suspended by the tester.

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

10.1.5 Programming Phase #2

※ The server[s] should be reprogrammed by the following service procedures.

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
DiagnosticSessionControl	10	default Session	81	-	Functional	Initialization Step
DiagnosticSessionControl	10	Extended Session	90	-	Functional	Pre-Programming Step
ControlDTCSetting	85	OFF	82	-	Functional	
CommunicationControl	28	noResponseRequired	02	-	Functional	
DiagnosticSessionControl	10	Programming Session	85	-	Physical	-
SecurityAccess	27	Request Seed	01	-	Physical	-
SecurityAccess	27	Send Key	02	-	Physical	-
RequestDownload	34	Data Format Identifier	-	-	Physical	Download the Boot Loader Software (Optional)
TransferData	36	Block Sequence Counter	-	-	Physical	
RequestTransferExit	37	-	-	-	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = checkProgrammingDependencies (#FF01)	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = eraseMemory	Physical	Download the Application Software
RequestDownload	34	-	-	Data Format Identifier	Physical	
TransferData	36	-	-	Block Sequence Counter	Physical	
RequestTransferExit	37	-	-	-	Physical	
RoutineControl	31	Start Routine	01	routineIdentifier = checkProgrammingDependencies (#FF01)	Physical	
ECUReset	11	Hard Reset	01	-	Physical	-
EnableNormalMessage Transmission	29	No Response Required	02	-	Functional	within 15ms after ECUReset Pos. Rsp.

Table10-2. Programming Phase #2 Sequence

10.1.5.1 DiagnosticSessionControl – extended diagnostic session

In order to be able to disable the normal communication between the servers and the setting of DTCs, it is required to start a non-defaultSession in each server where normal communication and DTCs shall be disabled. This is achieved via a DiagnosticSessionControl (10 hex) service with sessionType equal to extendedDiagnosticSession. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

Note) This diagnostic message can be replaced with DiagnosticSessionControl (10 hex) service and sessionType equal to ExtendedDiagnosticSession (03 hex).

10.1.5.2 ControlDTCSetting – DTC setting type = off

The client disables the setting of DTCs in each server using the ControlDTCSetting (85 hex) service with DTCSettingType equal to “off”. The request shall be transmitted functionally addressed to all servers.

Response messages are not required in this step.

10.1.5.3 CommunicationControl – disable non-diagnostic communication

The client disables the transmission and reception of non-diagnostic messages using the CommunicationControl (28 hex) service. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

Note) This diagnostic message can be replaced with CommunicationControl (28 hex) service and sub-function equal to DisableRxAndTx – suppressPosRspMsgIndicationBit = ON (83 hex).

10.1.5.4 DiagnosticSessionControl – programming session

The programming event is started in the server(s) via a physically addressed request of the DiagnosticSessionControl (10 hex) service with sessionType equal to programmingSession. When the server receives the request, it shall allocate all necessary resources required for programming. It is implementation specific whether the server starts executing out of boot memory.

Note) This diagnostic message can be replaced with DiagnosticSessionControl (10 hex) service and sessionType equal to ProgrammingSession (02 hex).

10.1.5.5 SecurityAccess – read seed / send key

A programming event should be secured. The SecurityAccess (27 hex) service shall be mandatory for all systems.

10.1.5.6 RoutineControl – erase memory

The memory of the server shall be erased in order to allow an application software/data download. This is achieved via a routine, using the RoutineControl (31 hex) service to execute the erase routine.

10.1.5.7 Request Download / TransferData / RequestTransferExit

Each download of a contiguous block of application software/data to a non-volatile server memory location (either a complete application software/data module or part of a software/data module) shall always follow the general data transfer method using the following service sequence:

- RequestDownload (34 hex)
- TransferData (36 hex)
- RequestTransferExit (37 hex)

10.1.5.8 RoutineControl – check reprogramming dependencies

Once all application software/data blocks/modules are completely downloaded, the client shall verify if the download has been performed successfully by initiating a routine in the server using the RoutineControl (31 hex) service. This routine either triggers the server to check the reprogramming dependencies and to perform all necessary action to proof that the download and programming into non-volatile memory was successful, or it requests the server to calculate the checksum and submit the checksum to the client via a RoutineControl positive response message (requestRoutineResults) for a comparison with a checksum contained in the client.



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 128/211

10.1.5.9 ECUReset – hard reset

The client transmits an ECUReset (11 hex) service request message onto the CAN network with resetType equal to hardReset. This shall be done physically addressed. The reception of the ECUReset (11 hex) request message causes the server(s) to perform a reset and to start the defaultSession.

10.2.2.10 CommunicationControl – enable non-diagnostic communication

The client enables the transmission and reception of non-diagnostic messages using the EnableNormalMessageTransmission (29 hex) service. This service must be requested within 15ms after ECU Reset positive response to prevent Time-Out Errors. When the Gateway is reprogramming by itself, the client transmits the EnableNormalMessageTransmission service in 400ms ~ 500ms after ECU Reset positive response. The request shall be transmitted functionally addressed to all servers. Response messages are not required in this step.

Note) This diagnostic message can be replaced with CommunicationControl (28 hex) service and sub-function equal to EnableRxAndTx – suppressPosRspMsgIndicationBit = ON (80 hex).

10.1.6 Programming Phase #3

Service Name	SID (hex)	Sub-function Name	Sub-function ID (hex)	Identifiers	Request Type	Remark
ReadDataByIdentifier	22	VehicleManufacturerECU SoftwareVersionNumber DataIdentifier	F189	-	Physical	

Table10-3. Programming Phase #3 Sequence

10.1.6.1 ReadDataByIdentifier – ECUSoftwareVersion

After the completion of reprogramming, the tester will send request message ReadDataByIdentifier to the ECU. If the ECU send a positive response message, it shows that the reprogramming procedure is successfully completed and the ECU has complete set of updated application software and application data.

10.3 Non-volatile server memory programming message flow examples

10.1.7 General Information

The following presents the message for a non-volatile server memory-programming event of a single server.

Assume that,

- Two ECUs (07E0/07E8 & 07E1/07E9) and Tester are in CAN network
- Transfer of four (4) modules, where each module has a length of 511 bytes (data bytes 02 hex through FE hex)
- The network layer buffer size of the server that is re-programmed is 255 bytes (reported in the RequestDownload positive response message)

10.1.8 Programming Phase #1

Assume that,

- BS = 4, STmin = 2 (in Default Session)
- Vehicle Manufacturer ECU Software Version Number = 123456789 (ASCII)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	03	22	F1	89	55	55	55	55	Single Frame
07E8	Response	8	10	0D	62	F1	89	31	32	33	First Frame
07E0	Request	8	30	04	02	55	55	55	55	55	Flow Control
07E8	Response	8	21	34	35	36	37	38	39	AA	Consecutive Frame #1

Table10-4. ReadDataByIdentifier (VehicleManufacturerECUSoftwareVersionNumber)

10.1.9 Programming Phase #2

Assume that,

- BS = 0, STmin = 0 (in ProgrammingSession)
- seed of the server (2 bytes) : 3657 hex
- key of the server (2 bytes) : C9A9 hex

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	10	90	55	55	55	55	55	Single Frame

Table10-5. DiagnosticSessionControl (extendedSession)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	85	82	55	55	55	55	55	Single Frame

Table10-6. ControlDTCSetting (off)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	28	02	55	55	55	55	55	Single Frame

Table10-7. CommunicationControl (DisableRxAndTx)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	10	85	55	55	55	55	55	Single Frame

07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
------	---------------	---	----	----	----	----	----	----	----	----	--------------

※ when receiving 10 85 requests, the server does not send positive response (SPRMIB=True), however when it responses NRC 78h , final positive response is necessary.

Table10-8. DiagnosticSessionControl (programmingSession)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	27	01	55	55	55	55	55	Single Frame
07E8	Response	8	02	67	01	36	57	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-9. SecurityAccess (requestSeed)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	02	27	02	C9	A9	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	02	67	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-10. SecurityAccess (sendKey)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	04	31	01	FF	00	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	04	71	01	FF	00	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-11. RoutineControl (eraseMemory)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-12. RequestDownload – Module #1

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-13. TransferData – Module #1 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-14. TransferData – Module #1 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-15. TransferData – Module #1 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-16. RequestTransferExit – Module #1

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-17. RequestDownload – Module #2

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-18. TransferData – Module #2 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-19. TransferData – Module #2 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-20. TransferData – Module #2 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-21. RequestTransferExit – Module #2

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-22. RequestDownload – Module #3

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 133/211

Table10-23. TransferData – Module #3 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-24. TransferData – Module #3 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-25. TransferData – Module #3 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-26. RequestTransferExit – Module #3

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	09	34	00	33	00	1B	67	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E0	Phys. Request	8	21	00	01	FF	55	55	55	55	Consecutive Frame #1
07E8	Response	8	04	74	20	00	FF	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-27. RequestDownload – Module #4

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	01	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	01	AA	AA	AA	AA	AA	Single Frame



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 134/211

07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
------	---------------	---	----	----	----	----	----	----	----	----	--------------

Table10-28. TransferData – Module #4 (block #1)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	10	FF	36	02	02	03	04	05	First Frame
07E8	Response	8	30	00	00	AA	AA	AA	AA	AA	Flow Control
07E0	Phys. Request	8	21	06	07	08	09	0A	0B	0C	Consecutive Frame #1
07E0	Phys. Request	8	22	0D	0E	0F	10	11	12	13	Consecutive Frame #2
07E0	Phys. Request	8	23	14	15	16	17	18	19	1A	Consecutive Frame #3
...	...	...	...	...	...	...	...	...	...	...	...
07E0	Phys. Request	8	23	F4	F5	F6	F7	F8	F9	FA	Consecutive Frame #35
07E0	Phys. Request	8	24	FB	FC	FD	FE	55	55	55	Consecutive Frame #36
07E8	Response	8	02	76	02	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-29. TransferData – Module #4 (block #2)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	07	36	01	02	03	04	05	06	Single Frame
07E8	Response	8	02	76	03	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-30. TransferData – Module #4 (block #3)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	01	37	55	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	01	77	AA	AA	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-31. RequestTransferExit – Module #4

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	04	31	01	FF	01	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	03	7F	31	78	AA	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07E8	Response	8	04	71	01	FF	01	AA	AA	AA	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame
07DF	Func. Request	8	02	3E	80	55	55	55	55	55	Single Frame

Table10-32. RoutineControl (checkProgrammingDependencies)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Phys. Request	8	02	11	01	55	55	55	55	55	Single Frame
07E8	Response	8	02	51	01	AA	AA	AA	AA	AA	Single Frame
07E9	Response	8	02	51	01	AA	AA	AA	AA	AA	Single Frame

Table10-33. ECUReset (hardReset)



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 135/211

CAN ID	Request / Response	DLC	Data Bytes								Comments
07DF	Func. Request	8	02	29	02	55	55	55	55	55	Single Frame

Table10-34. EnableNormalMessageTransmission (noResponseRequired)

10.1.10 Programming Phase #3

Assume that,

- BS = 4, STmin = 2 (in Default Session)
- Vehicle Manufacturer ECU Software Version Number = 12345678A (ASCII)

CAN ID	Request / Response	DLC	Data Bytes								Comments
07E0	Phys. Request	8	03	22	F1	89	55	55	55	55	Single Frame
07E8	Response	8	10	0D	62	F1	89	31	32	33	First Frame
07E0	Request	8	30	04	02	55	55	55	55	55	Flow Control
07E8	Response	8	21	34	35	36	37	38	41	AA	Consecutive Frame #1

Table10-35. ReadDataByIdentifier (VehicleManufacturerECUSoftwareVersionNumber)

10.4 Servers status in programming procedure

ECU #1 is the server actually being programmed.

ECU #2 is the server which is not actually programmed (programmable or non-programmable).

ECU #3 is the server which is participating in a hi-speed CAN network but the server itself is diagnosed over another vehicle bus (e.g. KWP2000 ...) or does not support diagnostic.

Tester			ECU #1	ECU #2	ECU #3
Service Name	SID (hex)		DiagnosticSession / CommunicationControl setting / DTCSetting		
ReadDataByIdentifier	22	Phys.	defaultSession / default (enableRxAndTx) / default (ON)		- / default (enableRxAndTx) / -
DiagnosticSessionControl - extendedSession	10	Func.	<u>extendedSession</u> / default (enableRxAndTx) / default (ON)		
ControlDTCSetting	85	Func.	extendedSession / default (enableRxAndTx) / <u>OFF</u>		
CommunicationControl - disableRxAndTx	28	Func.	extendedSession / <u>disableRxAndTx</u> / OFF		- / EnableRxAndDisableTx / -
DiagnosticSessionControl - programmingSession	10	Phys.	<u>programmingSession</u> / disableRxAndTx / OFF		
SecurityAccess	27	Phys.			
SecurityAccess	27	Phys.			
RequestDownload	34	Phys.			
TransferData	36	Phys.			
RequestTransferExit	37	Phys.			
RoutineControl	31	Phys.			
RoutineControl	31	Phys.			
RequestDownload	34	Phys.			
TransferData	36	Phys.			
RequestTransferExit	37	Phys.			
RoutineControl	31	Phys.			
ECUReset - hardReset	11	Phys.			
EnableNormalMessage Transmission - noResponseRequired	29	Func.	extendedSession / <u>enableRxAndTx</u> / OFF	- / enableRxAndTx / -	
ReadDataByIdentifier	22	Phys.			

Annex. A Negative Response Codes

When an ECU detects any error condition on communication or have internal problem, it cannot respond or react to the request normally. In this case the ECU should respond to tester with negative response code described in table A-1.

*) These codes are general negative response codes. These NRCs are in general supported by each diagnostic service and not listed in the list of applicable response codes of the diagnostic services. The occurrence depends on the system configuration (Hex value – 10, 11, 33, 78, 7F, 21, 7E (with a sub-function))

Hex value	responseCode	Mnemonic
10	generalReject *) This response code indicates that the requested action has been rejected by the server. The generalReject response code shall only be implemented in the server if none of the negative response codes defined in this document meet the needs of the implementation.	GR
11	serviceNotSupported *) This response code indicates that the requested action will not be taken because the server does not support the requested service. The server shall send this response code in case the client has sent a request message with a service identifier, which is either unknown or not supported by the server. Therefore this negative response code is not shown in the list of negative response codes to be supported for a diagnostic service, because this negative response code is not applicable for supported services.	SNS
12	subFunctionNotSupported This response code indicates that the requested action will not be taken because the server does not support the service specific parameters of the request message. The server shall send this response code in case the client has sent a request message with a known and supported service identifier but with "sub function" which is either unknown or not supported.	SFNS
13	incorrectMessageLengthOrInvalidFormat This response code indicates that the requested action will not be taken because the length of the received request message does not match the prescribed length for the specified service or the format of the paramters do not match the prescribed format for the specified service.	IMLOIF
14	responseTooLong This response code shall be reported by the server if the response to be generated exceeds the maximum number of bytes available by the underlying network layer. - EXAMPLE This problem may occur when several DIDs at a time are requested and the combination of all DIDs in the response exceeds the limit of the underlying transport protocol.	RTL
21	busyRepeatRequest This response code indicates that the server is temporarily too busy to perform the requested operation. In this circumstance the client shall perform repetition of the "identical request message" or "another request message". The repetition of the request shall be delayed by a time specified in the respective implementation documents.	BRR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 138/211

Example: In a multi-client environment the diagnostic request of one client might be blocked temporarily by a NRC \$21 while a different client finishes a diagnostic task.

NOTE) If the server is able to perform the diagnostic task but needs additional time to finish the task and prepare the response, the NRC \$78 shall be used instead of NRC \$21.
This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.

22

conditionNotCorrect

CNC

This response code indicates that the requested action will not be taken because the server prerequisite conditions are not met.

24

requestSequenceError

RSE

This response code indicates that the requested action will not be taken because the server expects a different sequence of request messages or message as sent by the client. This may occur when sequence sensitive requests are issued in the wrong order.

EXAMPLE A successful SecurityAccess service specifies a sequence of requestSeed and sendKey as sub-functions in the request messages. If the sequence is sent different by the client the server shall send a negative response message with the negative response code 24 hex - . requestSequenceError.

31

requestOutOfRange

ROOR

This response code indicates that the requested action will not be taken because the server has detected that the request message contains a parameter which attempts to substitute a value beyond its range of authority (e.g. attempting to substitute a data byte of 111 when the data is only defined to 100), or which attempts to access a dataIdentifier/routineIdentifier that is not supported or not supported in active session. This response code shall be implemented for all services, which allow the client to read data, write data or adjust functions by data in the server.

33

securityAccessDenied *)

SAD

This response code indicates that the requested action will not be taken because the server's security strategy has not been satisfied by the client. The server shall send this response code if one of the following cases occur:

- the test conditions of the server are not met,
- the required message sequence e.g. DiagnosticSessionControl, securityAccess is not met,
- the client has sent a request message which requires an unlocked server.

Beside the mandatory use of this negative response code as specified in the applicable services within this standard, this negative response code can also be used for any case where security is required and is not yet granted to perform the required service.

35	invalidKey This response code indicates that the server has not given security access because the key sent by the client did not match with the key in the server's memory. This counts as an attempt to gain security. The server shall remain locked and increment its internal securityAccessFailed counter.	IK
36	exceedNumberOfAttempts This response code indicates that the requested action will not be taken because the client has unsuccessfully attempted to gain security access more times than the server's security strategy will allow.	ENOA
37	requiredTimeDelayNotExpired This response code indicates that the requested action will not be taken because the client's latest attempt to gain security access was initiated before the server's required timeout period had elapsed.	RTDNE
70	uploadDownloadNotAccepted This response code indicates that an attempt to upload/download to a server's memory cannot be accomplished due to some fault conditions.	UDNA
71	transferDataSuspended This response code indicates that a data transfer operation was halted due to some fault. The active transferData sequence shall be aborted.	TDS
72	generalProgrammingFailure This response code indicates that the server detected an error when erasing or programming a memory location in the permanent memory device (e.g. Flash Memory).	GPF
73	wrongBlockSequenceCounter This response code indicates that the server detected an error in the sequence of blockSequenceCounter values. Note that the repetition of a TransferData request message with a blockSequenceCounter equal to the one included in the previous TransferData request message shall be accepted by the server.	WBSC
78	requestCorrectlyReceived-ResponsePending *) This response code indicates that the request message was received correctly, and that all parameters in the request message were valid, but the action to be performed is not yet completed and the server is not yet ready to receive another request. As soon as the requested service has been completed, the server shall send a positive response message or negative response message with a response code different from this. The negative response message with this response code may be repeated by the server until the requested service is completed and the final response message is sent. This response code might impact the application layer timing parameter values. The detailed specification shall be included in the data link specific implementation document. This response code shall only be used in a negative response message if the server will not be able to receive further request messages from the client while completing the requested diagnostic service. When this response code is used, the server shall always send a final response (positive or negative) independent of the suppressPosRspMsgIndicationBit	RCRRP

	value. A typical example where this response code may be used is when the client has sent a request message, which includes data to be programmed or erased in flash memory of the server. If the programming/erasing routine (usually executed out of RAM) is not able to support serial communication while writing to the flash memory the server shall send a negative response message with this response code. This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	
7E	subFunctionNotSupportedInActiveSession This response code indicates that the requested action will not be taken because the server does not support the requested sub-function in the session currently active. Within the programmingSession negative response code SFNS (subFunctionNotSupported) may optionally be reported instead of negative response code SNFSIAS (subFunctionNotSupportedInActiveSession). This response code shall only be used when the requested sub-function is known to be supported in another session, otherwise response code SFNS (subFunctionNotSupported) shall be used. This response code shall be supported by each diagnostic service with a sub-function parameter, if not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	SNFSIAS
7F	serviceNotSupportedInActiveSession *) This response code indicates that the requested action will not be taken because the server does not support the requested service in the session currently active. This response code shall only be used when the requested service is known to be supported in another session, otherwise response code SNS (serviceNotSupported) shall be used. This response code is in general supported by each diagnostic service, as not otherwise stated in the data link specific implementation document, therefore it is not listed in the list of applicable response codes of the diagnostic services.	SNSIAS
81	rpmTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for RPM is not met (current RPM is above a pre-programmed maximum threshold).	RPMTH
82	rpmTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for RPM is not met (current RPM is below a pre-programmed minimum threshold).	RPMTL
83	enginesRunning This is required for those actuator tests which cannot be actuated while the Engine is running. This is different from RPM too high negative response, and needs to be allowed.	EIR
84	enginesNotRunning This is required for those actuator tests which cannot be actuated unless the Engine is running. This is different from RPM too low negative response, and needs to be allowed.	EINR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 141/211

85	engineRunTimeTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for engine run time is not met (current engine run time is below a preprogrammed limit).	ERTTL
86	temperatureTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for temperature is not met (current temperature is above a preprogrammed maximum threshold).	TEMPTH
87	temperatureTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for temperature is not met (current temperature is below a preprogrammed minimum threshold).	TEMPTL
88	vehicleSpeedTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met (current VS is above a pre-programmed maximum threshold).	VSTH
89	vehicleSpeedTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met (current VS is below a pre-programmed minimum threshold).	VSTL
8A	throttle/PedalTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for throttle/pedal position is not met (current TP/APP is above a preprogrammed maximum threshold).	TPTH
8B	throttle/PedalTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for throttle/pedal position is not met (current TP/APP is below a preprogrammed minimum threshold).	TPTL
8C	transmissionRangeNotInNeutral This response code indicates that the requested action will not be taken because the server prerequisite condition for being in neutral is not met (current transmission range is not in neutral).	TRNIN
8D	transmissionRangeNotInGear This response code indicates that the requested action will not be taken because the server prerequisite condition for being in gear is not met (current transmission range is not in gear).	TRNIG
8F	brakeSwitch(es)NotClosed (Brake Pedal not pressed or not applied) For saftey reasons, this is required for certain tests before it begins, and must be maintained for the entire duration of the test.	BSNC
90	shiftLeverNotInPark For saftey reasons, this is required for certain tests before it begins, and must be maintained for the entire duration of the test.	SLNIP
91	torqueConverterClutchLocked This response code indicates that the requested action will not be taken because the server prerequisite condition for torque converter clutch is not met (current	TCCL



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 142/211

TCC status above a preprogrammed limit or locked).		
92	voltageTooHigh This response code indicates that the requested action will not be taken because the server prerequisite condition for voltage at the primary pin of the server (ECU) is not met (current voltage is above a pre-programmed maximum threshold).	VTH
93	voltageTooLow This response code indicates that the requested action will not be taken because the server prerequisite condition for voltage at the primary pin of the server (ECU) is not met (current voltage is below a pre-programmed maximum threshold).	VTL
F0	CRLintegrityFailed CRL integrity check (Verification of CRL Signature) is failed. The hash value of SHA-256 of Certificate authority's public key and the CRL Issuer Public Key Identifier value are not matched	CRLF
F1	CRLvalidityPeriodFailed The current time is not included between CRL effective date and CRL expiration date	CRLPD
F2	RoleandRightofCertificateDenied Role and Right of Certificate verification failed. The request service or the target from the external diagnostic tool are not matched with Certificate Holder Reference : Authorization	RRCD
F3	securityAccessDeniedfromCGW This response code indicates that the requested action will not be taken from the CGW because the CGW's security strategy (Secure Access) has not been satisfied by the client.	SADCGW
F4	LowerVersionDownloadAttempt This response code indicates that the requested action will not be taken because the downloaded software is lower version than the existing version.	LVDA
F5	ServiceNotSupportedInActiveConnection This response code indicates that the requested action will not be taken because the service is not allowed in active connection(CAN or Ethernet).	SNSIAC

Table A-1. Definition of responseCode values

Annex. B Diagnostic and Communication Management Functional Unit Data Parameter Definitions

B.1 CommnuicationType Parameter Definition

The communicationType is a 1-byte value. The bit-encoded low nibble of this byte represents the communicationTypes, which can be controlled via the CommunicationControl (28 hex) service.

For example, a communicationType with a bit combination (Bits 1-0) of "11b" is valid and disables both "normalCommunicationMessages" and "networkManagementCommunicationMessages" messages.

The high nibble of the communicationType 1-byte value defines which of the subnets connected to the receiving node shall be disabled/enabled when an appropriate CommunicationControl service is received.

Encoding of Bit	Value	Description	Cvt	Mnemonic
0 - 1	0	Reserved	M	
	1	NormalCommnuicationMessages This value references all application-related communication (inter-application signal exchange between multiple in-vehicle servers).	M	NCM
	2	NetworkManagementCommunicationMessages This value references all network management related communication.	M	NWMCM
	3	NetworkManagementCommunicationMessage and NormalCommunicationMessages This value references all network management and application-related communication.	M	
2 - 3	0~3	Reserved		
4 - 7	0	Disable/Enable specified communicationType (see encoding of bit 0-1) in the receiving node and all connected subnets. This only disables the node's communication into the subnets but not the communication of other nodes on the subnet (receiving node is not responsible to disable communication in each node of the subnet).	M	
	1~E	Disable/Enable specific subnet identified by subnet number.	U	
	F	Disable/Enable network which request is received on (Receiving node (server))	U	

Table B-1. Definition of CommunicationType and SubnetNumber Byte

B.2 EventWindowTime Parameter Definition

HEX	Description	Cvt	Mnemonic
00 - 01	Reserved	M	
02	InfiniteTimeToResponse This value specifies that the event window shall stay active for an infinite amount of time. (e.g. open window until power off)	U	ITTR
03 - 7F	Reserved (vehicle manufacture specific range)		
80 - FF	Reserved	M	

Table B-2. Definition of EventWindowTime Parameter

B.3 BaudratelIdentifier Parameter Definition

HEX	Description	Cvt	Mnemonic
00	Reserved	M	
01	PC9600Baud This value specifies the standard PC baudrate of 9.6 KBaud.	U	PC9600
02	PC19200Baud This value specifies the standard PC baudrate of 19.2 KBaud.	U	PC19200
03	PC38400Baud This value specifies the standard PC baudrate of 38.4 KBaud.	U	PC38400
04	PC57600Baud This value specifies the standard PC baudrate of 57.6 KBaud.	U	PC57600
05	PC115200Baud This value specifies the standard PC baudrate of 115.2 KBaud.	U	PC115200
06 - 0F	Reserved	M	
10	CAN125000Baud This value specifies the standard CAN baudrate of 125 KBaud.	U	CAN125000
11	CAN250000Baud This value specifies the standard CAN baudrate of 250 KBaud.	U	CAN250000
12	CAN500000Baud This value specifies the standard CAN baudrate of 500 KBaud.	U	CAN500000
13	CAN1000000Baud This value specifies the standard CAN baudrate of 1 MBaud.	U	CAN1000000
14 - FF	Reserved		

Table B-3. Definition of BaudratelIdentifier Parameter

Annex. C Data Transmission Functional Unit Data Parameter Definitions

C.1 dataIdentifier parameter definitions

The parameter dataIdentifier (DID) is to identify a server specific local data record. This parameter shall be available in the server's memory. The dataIdentifier value shall either exist in fixed memory or temporarily stored in RAM if defined dynamically by the service dynamicallyDefineDataIdentifier.

The example of DataIdentifier Parameter Table is described in following table.

Example#1]

Data Identifier (HEX)	Byte	PID	Description Text	Unit	HEX Range	Conversion
					Phy. Range	
0xZZZZ	#1 ~ #4	0x00	Supported PID : 0x01~0x20 Response : 4 Byte (A,B,C,D) Byte #1 : PID 0x01-0x08, Byte #2 : PID 0x09-0x10, Byte #3 : PID 0x11-0x18, Byte #4 : PID 0x19-0x20			
	...	...	...	...	...	...
	#	0xX1	Intake Manifold Absolute Pressure	kPa	00 ~ FF 0 ~ 255	X*1
	#	0xX2	Short Term Fuel Trim	%	80 ~ 7F -100 ~ 99.22	(singled) X*200/255
	#	0xX3	Intake Air Temperature	°C	00 ~ FF -40 ~ 215	X*1-40
	#	0xX4	Location of Oxygen Sensor	Bit 0 = 1 : Bank1 / Sensor1 Bit 1 = 1 : Bank1 / Sensor2 ... Bit 7 = 1 : Bank2 / Sensor4		
	#	0xX5	OBD requirements to which vehicle is designed	01 = OBD II 02 = OBD ... 0C~FF = Reserved		
	#	0xX6	Bit 0 : Variant Coding	0 = MT 1 = AT		
		0xX7	Bit 2 : Drive Position	0 = R 1 = D		
		0xX8	Bit 3 : Aircon Switch	0 = OFF 1 = ON		
		...	...	...	...	...
		0xXC	Bit 7 : Reserved	-	-	-
	...	...	...	...	...	...

The definition of Supported PID Byte (byte #0~byte#3) is as below.

Byte \ Bit	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Byte #0	PID 0x01	PID 0x02	PID 0x03	PID 0x04	PID 0x05	PID 0x06	PID 0x07	PID 0x08
Byte #1	PID 0x09	PID 0x0A	PID 0x0B	PID 0x0C	PID 0x0D	PID 0x0E	PID 0x0F	PID 0x10
Byte #2	PID 0x11	PID 0x12	PID 0x13	PID 0x14	PID 0x15	PID 0x16	PID 0x17	PID 0x18
Byte #3	PID 0x19	PID 0x1A	PID 0x1B	PID 0x1C	PID 0x1D	PID 0x1E	PID 0x1F	PID 0x20

*. Each bit would be set to '0' (if the assigned PID is not supported) or '1' (if the assigned PID is supported).

Table C-1. Example#1 of Record DataIdentifier Parameter Table



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 146/211

Example#2]

Data Identifier (HEX)	Byte	Description Text	Data Size	Data Format
0xZZZZ	#1~#17	VIN (Vehicle Identification Number)	17	ASCII

Table C-2. Example#2 of Identification Option DataIdentifier Parameter Table

Example#3]

Data Identifier (HEX)	Byte#	Control Enable Mask Record	Description Text	Input Output Control Parameter	Control State	Control Type	Test Condition
0xAAAA	-	-	Desired Idle Adjustment	00, 01, 02, 03	(Hex) * 10 [r/min]	Until Stop Request	IG ON & Eng. Running
0xB BBBB	-	-	MIL	00, 01, 02, 03	0x00 = OFF 0xFF = ON	Until Stop Request	IG ON & Eng. Stop
0xCCCC	-	-	Fuel Pump Relay	00, 01, 02, 03	0x00 = Stop 0xFF = Start	For 3 sec	IG ON & Eng. Stop
0xDDDD	#1	Bit 7	IAC Pintle Position	00, 01, 02, 03	X [count]	Until Stop Request	IG ON & Eng. Running
	#2	Bit 6	RPM (High Byte)		X [U/min]		
	#3	Bit 6	RPM (Low Byte)				
	#4 [bit7-4]	Bit 5	Pedal Position A		X*8 [%]		
	#4 [bit3-0]	Bit 4	Pedal Position B		X*8 [%]		
	#5	Bit 3	EGR Duty Cycle		X*100/255 [%]		
...	...	...	...	...	...	...	...

Table C-3. Example#3 of Input Output DataIdentifier Parameter Table

Hex	Description	Cvt	Mnemonic
0100-A5FF	Reserved for HMCKIA Specific – Record Data Identifiers	U	
A600-A7FF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	
A800-ACFF	Reserved for HMCKIA Specific – Record Data Identifiers	U	
AD00-AFFF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	
B000-B1FF	Reserved for HMCKIA Specific – Record Data Identifiers	U	
B200-BFFF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	
C000-C2FF	Reserved for HMCKIA Specific – Record Data Identifiers	U	
C300-CEFF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	
CF00-EFFF	Reserved for HMCKIA Specific – Record Data Identifiers	U	
C0DE	EOLCodingNumber Notation – EOL Coding Number : This value shall be used to reference the vehicle variant coding i.e)EOL Coding Record data size shall be 4 or 8bytes(extended)	U	EOLCN
DE00-DE50	EthernetPhyDiagnosticRecordData These values are available for EthernetPHYDiagnosticsData, used to provide the health information/statistics of Ethernet PHY. Further details e.g. Ethernet PHY diagnostic features, protocols, and procedures, shall be referred to the relevant specification	U	EPDRD



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 147/211

ED90	NonSleepDataCollectedEachEcuDataIdentifier This value shall be used to reference non-sleep log data collecting from each ECU. Each ECU shall response the data to CCU by RDBI(ReadDataByIdentifier) when the ECU can't enter to the sleep mode abnormally and CCU sends diagnostic request as ED90 DID. CCU shall collect the data from the ECU and Tester can get the all collected data by another diagnostic RDBI from CCU. Each ECU shall response non-sleep log data in Max 100 Bytes. Specific data information responded shall be managed in each ECU's specification. For example, following data can be included in the data. - Network Message and PNC bit info related to sleep and wakeup - HW Input / Output pin info related to wakeup. - internal specific data and flag value in ECU to cause non-sleep. - Each application session info to hold entering sleep mode	M	NSDCEEDID
ED91-ED95	Reserved for NonSleepDataCollectedEachEcuDataIdentifier ED91-ED95 (Reserved for future non sleep data requirement)	M	RNSDCEEDID
ED96-ED99	VehicleLoggerEcuLoggingDataIdentifier This value shall be used to reference ECU internal logging information to be collected for vehicle logger. Vehicle logger in CCU references data in each ECU periodically at predefined interval, and the ECUs shall respond with their own internal logging information. Each response shall contain less than 1KB data, unless the exception is agreed with CCU.	U	VLELDID
EDC0	ECUPlatformLoggingData 1 – Network Mode Log Data This value shall be used to reference ECU internal logging information from the SW platform. This feature collect information of ComM and CanNm module in BSW such as ComM mode and also the state and transition value of CanNm. This functionality is provided by SW Platform itself and does not including any security and personal data at all.	U	ECUPLD1
EDC1	ECUPlatformLoggingData 2 – Wakeup/Sleep Log Data This value shall be used to reference ECU internal logging information from the SW platform. This feature collect information during the Sleep and Wakeup processes run on SW platform such as ComM Mode and NvM running state at requesting sleep and Wakeup delay at requesting wakeup. This functionality is provided by SW Platform itself and does not including any security and personal data at all.	U	ECUPLD2
EFF0-EFDF	FaultDiagnosticRecordData Please refer to Requirements document for this DIDs	U	FDRD
F010-F0FF	Reserved for HMCKIA Specific – Input Output Data Identifiers	U	
F100-F17F	Reserved for HMCKIA Specific – Identification Options Data Identifiers	U	
F180	BootSoftwareIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU boot software identification record. The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the boot software identification(s) are reported. The format of the boot software identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	BSIDID
F181	ApplicationSoftwareIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application software number(s). The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the application software identification(s) are reported. The format of the application software identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	ASIDID
F182	ApplicationDataIdentificationDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application data identification record. The first data byte of the record data shall be the numberOfModules that are reported. Following the numberOfModules the application data identification(s) are reported. The format of the application data identification structure shall be ECU specific and defined by the vehicle manufacturer.	U	ADIDID
F183	BootSoftwareFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU boot software fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	BSFPDID

1F-SG-00002

현대 · 기아 자동차
HYUNDAI · KIA MOTOR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 148/211

F184	ApplicationSoftwareFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application software fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	ASFPDID
F185	ApplicationDataFingerprintDataIdentifier This value shall be used to reference the vehicle manufacturer specific ECU application data fingerprint identification record. Record data content and format shall be ECU specific and defined by the vehicle manufacturer.	U	ADFPDID
F186	ActiveDiagnosticSessionDataIdentifier This value shall be used to report the active diagnostic session in the server. The values are defined by the diagnosticSessionType subfunction parameter in the DiagnosticSessionControl service.	U	ADSDID
F187	VehicleManufacturerSparePartNumberDataIdentifier Notation – Vehicle Part Number : or - Vehicle Part Number(Location) : *Location : e.g) FL,FR,RL,RR This value shall be used to reference the vehicle manufacturer spare part number.. Record data format shall be ASCII and size shall be 10 ~ 15 bytes. Color code could be max. 15 bytes. (e.g. 959102T250 → 0x 39 35 39 31 30 32 54 32 35 30)	M	VMSPDID
F188	VehicleManufacturerECUSoftwareNumberDataIdentifier This value shall be used to reference the vehicle manufacturer ECU (server) software number. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMECUSNDID
F189	VehicleManufacturerECUSoftwareVersionNumberDataIdentifier This value shall be used to reference the vehicle manufacturer ECU (server) software version number. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMECUSVNDID
F18A	SystemSupplierIdentifierDataIdentifier This value shall be used to reference the system supplier name and address information. Record data content and format shall be server specific and defined by the system supplier.	U	SSIDDID
F18B	ECUManufacturingDateDataIdentifier Notation – Supplier H/W manufacturing Date : Or - Supplier H/W manufacturing Date(Location) : *Location : e.g.) FL, FR, RL,RR This value shall be used to reference the ECU (server) manufacturing date. Record data format shall be HEX and size shall be 4 bytes. The content shall be ordered as Year, Month, Day.(e.g. 2018.03.06 → 0x20180306)	M	ECUMDDID
F18C	ECUSerialNumberDataIdentifier Notation – Serial Number : Or - Serial Number(Nocation) : *Location : e.g.) FL,FR,RL,RR This value shall be used to reference the ECU (server) serial number. Data shall be defined as ASCII code.	M	ECUSNDID
F18D	SupportedFunctionalUnitsDataIdentifier This value shall be used to request the functional units implemented in a server.	U	SFUDID
F18E	VehicleManufacturerKitAssemblyPartNumberDataIdentifier This value shall be used to reference the vehicle manufacturer order number for a kit (assembled parts bought as a whole for production e.g. cockpit), when the spare part number designates only the server (e.g. for aftersales). The record data content and format shall be server specific and defined by the vehicle manufacturer.	U	VMKAPNDID
F18F	RXSWINDataIdentifier Notation – RXSWIN Data: RXSWIN is information that combines local RXSWIN information into one RXSWIN. Refer to Annex J	C2	
F190	VINDataIdentifier This value shall be used to reference the VIN number. VIN data shall be defined as ASCII code	U	VINDID



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 149/211

F191	VehicleManufacturerECUHardwareNumberDataIdentifier Notation – Vehicle H/W Version : This value is HMCKIA defined ECU hardware version number shown on the drwaing of each system. H/W version shall be defined as ASCII code.	M	VMECUHNDI D
F192	SystemSupplierECUHardwareNumberDataIdentifier This value shall be used to reference the system supplier specific ECU (server) hardware number. This value shall be defined as ASCII code.	U	SSECUHWN DID
F193	SystemSupplierECUHardwareVersionNumberDataIdentifier Notation – Supplier H/W Version : Or - Supplier H/W Version(Location) : *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the system supplier specific ECU (server) hardware version number. This version shall be defined as ASCII code.	M	SSECUHWVN DID
F194	SystemSupplierECUSoftwareNumberDataIdentifier This value shall be used to reference the system supplier specific ECU (server) software number. This value shall be defined as ASCII code.	U	SSECUSWND ID
F195	SystemSupplierECUSoftwareVersionNumberDataIdentifier Notation – Supplier S/W Version : This value shall be used to reference the system supplier specific ECU (server) software version number. This value shall be defined as ASCII code.	U	SSECUSWVN DID
F196	ExhaustRegulationOrTypeApprovalNumberDataIdentifier This value shall be used to reference the exhaust regulation or type approval number (valid for those systems which require type approval). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	EROTANDID
F197	SystemNameOrEngineTypeDataIdentifier Notation – System Name : This value shall be used to reference the system name or engine type. Record data content and format shall be server specific and defined by the vehicle manufacturer. Data content format shall be ASCII e.g.) RR_C_RDR -> 8byte ASCII (0x82 0x82 0x95 0x67 0x95 0x82 0x68 0x82)	M	SNOETDID
F198	RepairShopCodeOrTesterSerialNumberDataIdentifier This value shall be used to reference the repair shop code or tester (client) serial number (e.g., to indicate the most recent service client used re-program server memory). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	RSCOTSNDI D
F199	ProgrammingDateDataIdentifier This value shall be used to reference the date when the server was last programmed. Record data content and format shall be unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	PDDID
F19A	CalibrationRepairShopCodeOrCalibrationEquipmentSerialNumberDataIdentifier This value shall be used to reference the repair shop code or client serial number (e.g., to indicate the most recent service client used re-calibrate the server). Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	CRSCOCESN DID
F19B	CalibrationDateDataIdentifier This value shall be used to reference the date when the server was last calibrated. Record data content and format shall be unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	CDDID
F19C	CalibrationEquipmentSoftwareNumberDataIdentifier This value shall be used to reference software version within the client used to calibrate the server. Record data content and format shall be server specific and defined by the vehicle manufacturer.	U	CESWNDID
F19D	ECUInstallationDateDataIdentifier This value shall be used to reference the date when the ECU (server) was installed in the vehicle. Record data content and format shall be either unsigned numeric, ASCII or BCD, and shall be ordered as Year, Month, Day.	U	EIDDID
F19E	ODXFileDataIdentifier This value shall be used to reference the ODX (Open Diagnostic Data Exchange) file of the server to be used to interpret and scale the server data.	U	ODXFDID
F19F	EntityDataIdentifier This value shall be used to reference the entity data identifier as defined in ISO 15764 for a secured data transmission.	U	EDID

1F-SG-00002

현대 · 카이 자동차
HYUNDAI · KIA MOTOR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 150/211

F1A0	SoftwareVersionfor HMCKIA VehicleManufactureDataIdentifier Notation – Vehicle S/W Version Or - Vehicle S/W Version(Location) : *Location : e.g.) FL,FR,RL,RR This value is HMCKIA defined ECU software version number. This version is used to assist the unit version or to represent all SW units. S/W version shall be defined as ASCII code. Dataidentifier parameter. ※ In case of a ECU has sub ECUs, 4 * n bytes would be used. ('n' is the number of sub ECUs) In this case the order of ECU version shall be fixed.	U	SVHVMDID
F1A1	ECUSupplierCodeDataIdentifier Notation – System Supplier Code : Or - System Supplier Code(Location) : *Location : e.g.) FL,FR,RL,RR This value shall be used to reference the ECU (server) Supplier Code. Record data format shall be ASCII and size shall be 4 bytes. (e.g. DELPHI CORPORATION : AB9H → 0x41 42 39 48)	M	ECUSCD
F1A2	ECUApplicationVersionDataIdentifier This value shall be used to reference the Application version. following format shall be used. {App ID (ASCII 5bytes) + Length(Hex 1byte) + Version(ASCII 'n' bytes)} *'m' The number of iteration 'm' depends on the number of App installed in the ECU. The length of the Version is depends on application specific.	U	ECUAVD
F1A3	ECUApplicationIVDHashDataIdentifier This value shall be used to reference the Application IVD Hash. following format shall be used. {App ID (ASCII 5bytes) + Length(Hex 1byte) + IVD(32bytes)}	C3	ECUAIVDHD
F1A4-F1AF	Reserved for HMCKIA Specific – Identification Option		
F1B0	ECUSoftwareUNITNumberDataIdentifier Nontation – The number of S/W Unit : This Identifier shall be used to reference unique system specific SW version. This value shall be used to reference the number of system specific S/W Unit in the ECU(server). The number of S/W Unit must consist of within 1byte and data type is unit8. If there is only one S/W unit, it is expressed as 1(dec). (Increase the value by 1 according to the number of S/W units.), and It can be expressed up to 15(dec). DID starts from F1B1 and be increased by 1. e.g) The number of SW unit is 3 then only three DIDs are allowed to use. (F1B1, F1B2, F1B3)	M	ECUSUNDID
F1B1	ECUSoftwareUNIT1VersionDataIdentifier Notation –S/W Unit1 Version : Or - S/W Unit1 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 1 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	M	ECUSU1VDID
F1B2	ECUSoftwareUNIT2VersionDataIdentifier Notation –S/W Unit2 Version : Or - S/W Unit2 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 2 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU2VDID



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 151/211

F1B3	ECUSoftwareUNIT3VersionDataIdentifier Notation –S/W Unit3 Version : Or – S/W Unit3 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 3 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU3VDID
F1B4	ECUSoftwareUNIT4VersionDataIdentifier Notation –S/W Unit4 Version : Or – S/W Unit4 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 4 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU4VDID
F1B5	ECUSoftwareUNIT5VersionDataIdentifier Notation –S/W Unit5 Version : Or – S/W Unit5 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 5 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU5VDID
F1B6	ECUSoftwareUNIT6VersionDataIdentifier Notation –S/W Unit6 Version : Or – S/W Unit6 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 6 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU6VDID
F1B7	ECUSoftwareUNIT7VersionDataIdentifier Notation –S/W Unit7 Version : Or – S/W Unit7 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 7 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU7VDID
F1B8	ECUSoftwareUNIT8VersionDataIdentifier Notation –S/W Unit8 Version : Or – S/W Unit8 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 8 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU8VDID
F1B9	ECUSoftwareUNIT9VersionDataIdentifier Notation –S/W Unit9 Version : Or – S/W Unit9 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 9 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU9VDID



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 152/211

F1BA	ECUSoftwareUNIT10VersionDataIdentifier Notation –S/W Unit10 Version : Or - S/W Unit10 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 10 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU10VDI D
F1BB	ECUSoftwareUNIT11VersionDataIdentifier Notation –S/W Unit11 Version : Or - S/W Unit11 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 11 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU11VDI D
F1BC	ECUSoftwareUNIT12VersionDataIdentifier Notation –S/W Unit12 Version : Or - S/W Unit12 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 12 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU12VDI D
F1BD	ECUSoftwareUNIT13VersionDataIdentifier Notation –S/W Unit13 Version : Or - S/W Unit13 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 13 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU13VDI D
F1BE	ECUSoftwareUNIT14VersionDataIdentifier Notation –S/W Unit14 Version : Or - S/W Unit14 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 14 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU14VDI D
F1BF	ECUSoftwareUNIT15VersionDataIdentifier Notation –S/W Unit15 Version : Or - S/W Unit15 Version(Name or Location) : *Name : e.g.) MCU, AP *Location : e.g.) FL, FR, RL, RR This value shall be used to reference the ECU S/W unit 15 as a system specific version that must be unique during entire ECU lifecycle. Record data content and format should be expressed by ASCII code.	C1	ECUSU15VDI D



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 153/211

F1C0	ECUSecurityInformationDataIdentifier This value shall be used to reference the security status of the ECU, such as Secure boot, Secure Debug, and HSM configuration status.			M	ECUSSDID	
	Byte #	Description	Conversion rule			Data Size
	#0	HSM Configuration Lock state This value shall be used to check HSM configuration Lock state of ECU.	(bit 7..0)			1byte
			0x00 = Disabled			
			0x01 = Enabled			
			0x02 ~ 0xFE = Reserved			
			0xFF = N/A			
	#1	Secure Boot State This value shall be used to check Secure Boot state of ECU.	(bit 7..0)			1byte
			0x00 = Disabled			
			0x01 = Enabled			
			0x02 ~ 0xFE = Reserved			
			0xFF = N/A			
	#2	Secure Debug State This value shall be used to check Secure Debug state of ECU.	(bit 7..0)			1byte
			0x00 = Disabled			
			0x01 = Enabled			
			0x02 = DebugProtectionTempStop			
0x03 = Debug Disable						
0x04 ~ 0xFE = Reserved						
0xFF = N/A						
#3	External bootloader state This value shall be used to check External bootloader state of ECU.	(bit 7..0)	1byte			
		0x00 = Unsupported				
		0x01 = Disabled				
		0x02 = Enabled				
		0xFF = N/A				
F1C1	ECUSoftwareUNIT1IVDDataIdentifier Notation – S/W Unit1 IVD Data : This value shall be used to reference the IVD value of SW unit 1. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)			M	ECUSU1HDID	
F1C2	ECUSoftwareUNIT2IVDDataIdentifier Notation – S/W Unit2 IVD Data : This value shall be used to reference the IVD value of SW unit 2. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)			C1	ECUSU2HDID	



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 154/211

F1C3	ECUSoftwareUNIT3IVDDataIdentifier Notation – S/W Unit3 IVD Data : This value shall be used to reference the IVD value of SW unit 3. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU3HDID
F1C4	ECUSoftwareUNIT4IVDDataIdentifier Notation – S/W Unit4 IVD Data : This value shall be used to reference the IVD value of SW unit 4. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU4HDID
F1C5	ECUSoftwareUNIT5IVDDataIdentifier Notation – S/W Unit5 IVD Data : This value shall be used to reference the IVD value of SW unit 5. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU5HDID
F1C6	ECUSoftwareUNIT6IVDDataIdentifier Notation – S/W Unit6 IVD Data : This value shall be used to reference the IVD value of SW unit 6. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU6HDID
F1C7	ECUSoftwareUNIT7IVDDataIdentifier Notation – S/W Unit7 IVD Data : This value shall be used to reference the IVD value of SW unit 7. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU7HDID



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 155/211

F1C8	ECUSoftwareUNIT8IVDDataIdentifier Notation – S/W Unit8 IVD Data : This value shall be used to reference the IVD value of SW unit 8. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU8HDID
F1C9	ECUSoftwareUNIT9IVDDataIdentifier Notation – S/W Unit9 IVD Data : This value shall be used to reference the IVD value of SW unit 9. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU9HDID
F1CA	ECUSoftwareUNIT10IVDDataIdentifier Notation – S/W Unit10 IVD Data : This value shall be used to reference the IVD value of SW unit 10. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU10HDI D
F1CB	ECUSoftwareUNIT11IVDDataIdentifier Notation – S/W Unit11 IVD Data : This value shall be used to reference the IVD value of SW unit 11. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU11HDI D
F1CC	ECUSoftwareUNIT12IVDDataIdentifier Notation – S/W Unit12 IVD Data : This value shall be used to reference the IVD value of SW unit 12. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU12HDI D



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 156/211

F1CD	ECUSoftwareUNIT13IVDDataIdentifier Notation – S/W Unit13 IVD Data : This value shall be used to reference the IVD value of SW unit 13. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU13HDI D
F1CE	ECUSoftwareUNIT14IVDDataIdentifier Notation – S/W Unit14 IVD Data : This value shall be used to reference the IVD value of SW unit 14. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU14HDI D
F1CF	ECUSoftwareUNIT15IVDDataIdentifier Notation – S/W Unit15 IVD Data : This value shall be used to reference the IVD value of SW unit 15. Each unit shall have own IVD value to prove SW integrity. This value has to be same with IVD which is in the SUMS. If calculation of IVD affects hard realtime performance of ECU during the drive, ECU shall stop calculation and return NRC(22) for the DID request. If SW UNIT#n is not available(e.g. The value of the ECUSoftwareUNIT#n Version is "NULL" or SW UNIT#n is not supported), ECUSoftwareUNIT#nIVDDataIdentifier shall not be supported. That means server returns NRC31. *IVD : Integrity Validation Data Data type : HEX Data length : SHA256 (32bytes)	C1	ECUSU15HDI D
F1D0-F1ED	Reserved for HMCKIA Specific – Identification Option	U	
F1EE	RXSWINEOLResultIdentifier This value shall be used to reference the result of Routine control 0x04F0 This value shall be maintained until new result has been stored. Result shall be maintained even if driving cycle has been changed. Data Type : Hex Data Length : 1Byte Volatile : Non-volatile data R/W : Read only Data definition(text table) : 0x00 : No result (Routine 0x04F0 has never been done before) 0x01 : RXSWIN Done (Routine 0x04F0 has been done) 0x02 : RXSWIN Aborted (Routine 0x04F0 has been aborted)	C2	RXSWINEOL RES
F1EF	LocalRXSWINDataIdentifier Notation – Local RXSWIN Data : RXSWIN information which is in a ECU. Refer to Annex J	M	LRXSWINDID
F1F0-F1FF	Reserved for System Specific – Identification Option	U	
F200-F2FF	Reserved – Periodic Data Identifier	U	
F300-F3FF	Reserved – Dynamically Defined Data Identifier	U	
F400-F8FF	Reserved for OBD – IMPORTANT : Do not use other than legislative ECU	M	
F900-F9FF	TachographDataIdentifier	U	TACHODID
FA00-FA0F	Airbag Deployment Data Identifier	U	ADDID
FA10-FA18	EDR Data Identifier	U	EDRDID
FA19-FAFF	Safety System Data Identifier	U	SSDID
FB00-FCFF	Reserved for Legislative Use – IMPORTANT : Do not use other than legislative ECU	M	

1F-SG-00002

현대 · 기아 자동차

HYUNDAI · KIA MOTOR



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 157/211

FD00-FEFF	Reserved for System Specific	M	
FF02-FFFF	Reserved for ISOSAE	M	

C1 : More than 1 SW unit is available on the system.
C2 : Central Gateway ECU
C3 : If Controller supports DID F1A2, This DID shall be supported.

Table C-4. DataIdentifier Parameter Table

Generic DID shall be shown as following order to the diagnostic tester.

No	DID	Notation of Diagnostic tool
1	F187	Vehicle Part Number
2	F1B0	The number of S/W Unit
3	F1B1~F1BA	S/W Unit(*N) Version : (*N is the numbering of SW unit)
4	F197	System Name :
5	F193	Supplier H/W Version
6	F18B	Supplier H/W Manufacturing Date :
7	F1A1	System Supplier Code :
8	C0DE	EOL Coding Number :
9	F18C	Serial Number :

Table C-4.1. The order of DID in the diagnostic tool and Notation

DID	Usage
F1A0	S/W version of HMCKIA
F187	Vehicle Part Number
F1B0	The number of SWUnits
F1B1	SW Unit1 version
F1B2	SW Unit2 version
F1B3	SW Unit3 version
F1B4	SW Unit4 version
F1B5	SW Unit5 version
F1B6	SW Unit6 version
F1B7	SW Unit7 version
F1B8	SW Unit8 version
F1B9	SW Unit9 version
F1BA	SW Unit10 version

Table C-4.2. List of SUMS related DID



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 158/211

C.1.1 ControlEnableMaskRecord

Byte#	Bit Evaluation		
	Bit	PID # (HEX)	Conversion
1	Bit 7	01	0 = doesn't affected by the request, 1 = affected by the request
	Bit 6	02	
	...	...	
	Bit 1	07	
	Bit 0	08	
2	Bit 7	09	
	Bit 6	0A	
	...	...	
	Bit 1	0E	
	Bit 0	0F	
...	...	...	
N	Bit 7	$(N-1)*8+1$	
	Bit 6	$(N-1)*8+2$	
	...	...	
	Bit 1	$(N-1)*8+7$	
	Bit 0	$(N-1)*8+8$	

Table C-5. Supported PID Check Byte Definitions



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 159/211

C.1.2 Supported PID Check Byte Definitions

Not all PIDs are applicable or supported by all ECUs or systems. PID \$00 is a bit-encoded PID that indicates, for each ECU, which PIDs that ECU supports.

If there are more than 32 PIDs in each LID, 32*n-th PID (0x00, 0x20, 0x40, 0x60 ...) will be a supported PID byte. The definition of Supported PID Byte is as below.

Requested PID (HEX)	Bit Evaluation		
	Bit	PID # (HEX)	Conversion
00	1st Byte, Bit 7	01	0 = not supported 1 = supported
	1st Byte, Bit 6	02	
	...	...	
	4th Byte, Bit 1	1F	
	4th Byte, Bit 0	20	
20	1st Byte, Bit 7	21	
	1st Byte, Bit 6	22	
	...	...	
	4th Byte, Bit 1	3F	
	4th Byte, Bit 0	40	
...	...	...	
C0	1st Byte, Bit 7	C1	
	1st Byte, Bit 6	C2	
	...	...	
	4th Byte, Bit 1	DF	
	4th Byte, Bit 0	E0	
E0	1st Byte, Bit 7	E1	
	1st Byte, Bit 6	E2	
	...	...	
	4th Byte, Bit 1	FF	
	4th Byte, Bit 0	Reserved Always zero(0) = not supported	

Table C-6. Supported PID Check Byte Definitions

C.2 ScalingByte Parameter Definitions

The parameter scalingByte (SBYT) consists of one byte (high and low nibble). The scalingByte high nibble defines the data type, which is used to represent the dataIdentifier (DID). The scalingByte low nibble defines the number of bytes used to represent the parameter in a datastream.

Encoding of High Nibble (HEX)	Description of Data Type	Cvt	Mnemonic
0	unSignedNumeric (1 to 4 bytes) This encoding uses a common binary weighting scheme to represent a value by mean of discrete incremental steps. One byte affords 256 steps; two bytes yields 65536 steps, etc.	U	USN
1	signedNumeric (1 to 4 bytes) This encoding uses a two's complement binary weighting scheme to represent a value by mean of discrete incremental steps. One byte affords 256 steps; two bytes yields 65536 steps, etc.	U	SN
2	bitMappedReportedWithOutMask Bit mapped encoding uses individual bits or small groups of bits to represent status. For every bit which represents status, a corresponding mask bit is required as part of the parameter definition. The mask indicates the validity of the bit for particular applications. This type of bit mapped parameter does not contain additional bytes to report the validity mask.	U	BMRWOM
3	bitMappedReportedWithMask Bit mapped encoding uses individual bits or small groups of bits to represent status. For every bit which represents status, a corresponding mask bit is required as part of the parameter definition. The mask indicates the validity of the bit for particular applications. This type of bit mapped parameter contains one validity mask byte for each status byte representing data.	U	BMRWM
4	BinaryCodedDecimal Conventional Binary Coded Decimal encoding is used to represent two numeric digits per byte. The upper nibble is used to represent the most significant digit (0 - 9), and the lower nibble the least significant digit (0 - 9).	U	BCD
5	stateEncodedVariable (1 byte) This encoding uses a binary weighting scheme to represent up to 256 distinct states. An example is a parameter, which represents the status of the Ignition Switch. Codes "00", "01", "02" and "03" may indicate ignition off, locked, run, and start, respectively. The representation is always limited to one (1) byte.	U	SEV
6	ASCII (1 to 15 bytes for each scalingByte) Conventional ASCII encoding is used to represent up to 128 standard characters with the MSB = logic 0. An additional 128 custom characters may be represented with the MSB = logic 1.	U	ASCII
7	signedFloatingPoint Floating point encoding is used for data that needs to be represented in floating point or scientific notation. Standard IEEE formats shall be used according to ANSI/IEEE Std 754 - 1985.	U	SFP
8	packet Packets contain multiple data values, usually related, each with unique scaling. Scaling information is not included for the individual values. Refer to section C.3.1 scalingByteExtension for scalingByte high nibble of bitMappedReportedWithOutMask.	U	P



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 161/211

9	formula A formula is used to calculate a value from the raw data. Formula Identifiers are specified in the table defining the formulaIdentifier encoding. Refer to section C.3.2 scalingByteExtension for scalingByte high nibble of formula.	U	F
A	unit/format The units and formats are used to present the data in a more user-friendly format. Unit and Format Identifiers are specified in the Table defining the formulaIdentifier encoding. NOTE) If combined units and/or formats are used, e.g. mV, then one scalingByte (and scalingData) for each unit/format shall be included in the readScalingDataByIdentifier positive response. Refer to section C.3.3 scalingByteExtension for scalingByte high nibble of unit / format.	U	U
B	stateAndConnectionType (1 byte) This encoding is used especially for input and output signals. The information encoded in the data byte specifies the high level physical layout, electrical levels and functional state. It is recommended to use this option for digital input and output parameters. Refer to section C.3.4 scalingByteExtension for scalingByte high nibble of stateAndConnectionType.	U	SACT
C - F	Reserved		

Table C-7. ScalingByte (High Nibble) Parameter Definitions

Encoding of High Nibble (HEX)	Description of Data Type	Cvt	Mnemonic
0 - F	numberOfBytesOfParameter This range of values specifies the number of data bytes in a data stream referenced by a parameter identifier. The length of a parameter is defined by the scaling byte(s), which is always preceded by a parameter identifier (one or multiple bytes). If multiple scaling bytes follow a parameter identifier the length of the data referenced by the parameter identifier is the summation of the content of the low nibbles in the scaling bytes. e.g. VIN is identified by a single byte parameter identifier and followed by two scaling bytes. The length is calculated up to 17 data bytes. The content of the two low nibbles may have any combination of values that add up to 17 data bytes. Note) For the scalingByte with high nibble encoded as formula or unit/format this value is \$0.	U	NROBOP

Table C-8. ScalingByte (Low Nibble) Parameter Definitions

C.3 ScalingByteExtension Parameter Definitions

C.3.1 ScalingByteExtension for ScalingByte high nibble of BitMappedReportedWithoutMask

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithoutMask.

A scalingByte with high nibble encoded as bitMappedReportedWithoutMask shall be followed by scalingByteExtension bytes representing the validity mask for the bit mapped dataIdentifier. Each byte shall indicate which bits of the corresponding dataIdentifier byte are supported for the current application.

ScalingByteExtension Byte	Description	Cvt
#1	dataIdentifier dataRecord#1 validity mask	M
...	...	C1
#p	dataIdentifier dataRecord#p validity mask	C1

C1 : The presence of this parameter depends on the size of the dataIdentifier the information is being requested for. The validity mask shall have as many bytes as the dataIdentifier has dataRecords.

Table C-9. ScalingByteExtension Bytes for BitMappedReportedWithoutMask

C.3.2 ScalingByteExtension for ScalingByte high nibble of Formula

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithoutMask.

A scalingByte with high nibble encoded as formula shall be followed by scalingByteExtension bytes defining the formula. The scalingByteExtension consists a of one byte formulaIdentifier and constants as described in Table below.

ScalingByteExtension Byte	Description	Cvt
#1	formulaIdentifier (refer to Table defining the formulaIdentifier encoding for details)	M
#2	C0 high byte	M
#3	C0 low byte	M
#4	C1 high byte	U
#5	C1 low byte	U
...	...	U
#2n+2	Cn high byte	U
#2n+3	Cn low byte	U

Table C-10. ScalingByteExtension Bytes for Formula

FormulaIdentifier (HEX)	Description	Cvt
00	$y = C0 * x + C1$	U
01	$y = C0 * (x + C1)$	U
02	$y = C0 / (x + C1) + C2$	U
03	$y = x / C0 + C1$	U

04	$y = (x + C0) / C1$	U
05	$y = (x + C0) / C1 + C2$	U
06	$y = C0 * x$	U
07	$y = x / C0$	U
08	$y = x + C0$	U
09	$y = x * C0 / C1$	U
0A – 7F	Reserved	M
80 – FF	Reserved (vehicle manufacturer specific range)	U

Table C-11. FormulaIdentifier Encoding

Formulas are defined using variables (y, x, etc.) and constants (C0, C1, C2, etc.). The variable y is the calculated value. The other variables, in consecutive order, are part of the data stream referenced by a dataIdentifier. Each constant is expressed as a two byte real number defined in Table C.8 — Two byte real number format. The two byte real numbers ($C = M * 10^E$) contain a 12 bit signed (2's complement) mantissa (M) and a 4 bit signed (2's complement) exponent (E). The mantissa can hold values within the range -2048 to +2047, and the exponent can scale the number by 10^{-8} to 10^7 . The exponent is encoded in the high nibble of the high byte of the two byte real number. The mantissa is encoded in the low nibble of the high byte and the complete low byte of the two byte real number.

High Byte								Low Byte							
High Nibble				Low Nibble				High Nibble				Low Nibble			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Exponent				Mantissa											

Table C-12. Two Byte Real Number Format

C.3.3 ScalingByteExtension for ScalingByte high nibble of Unit / Format

The parameter scalingByteExtension (SBYE) is only supported for scalingByte parameters with the high nibble encoded as formula, unit/format, or bitMappedReportedWithoutMask.

A scalingByte with high nibble encoded as unit / format shall be followed by a single scalingByteExtension byte defining the unit / format. The one byte scalingByteExtension is defined in Table C.9 — Unit / format scalingByteExtension encoding. If combined units and/or formats are used, e.g. mV, then one scalingByte (and scalingByteExtension) shall be included for each unit / format.

ScalingByteExtension Byte #1	Name	Symbol	Description	Cvt
00	No unit, no prefix			U
01	Meter	m	Length	U
02	Foot	ft	Length	U
03	Inch	in	Length	U
04	Yard	yd	Length	U
05	mile (English)	mi	length	U
06	Gram	g	mass	U
07	ton (metric)	t	mass	U
08	Second	s	time	U



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 164/211

09	Minute	min	time	U
0A	Hour	h	time	U
0B	Day	d	time	U
0C	year	y	time	U
0D	ampere	A	current	U
0E	volt	V	voltage	U
0F	coulomb	C	electric charge	U
10	ohm	W	resistance	U
11	farad	F	capacitance	U
12	henry	H	inductance	U
13	siemens	S	electric conductance	U
14	weber	Wb	magnetic flux	U
15	tesla	T	magnetic flux density	U
16	kelvin	K	thermodynamic temperature	U
17	Celsius	°C	thermodynamic temperature	U
18	Fahrenheit	°F	thermodynamic temperature	U
19	candela	cd	luminous intensity	U
1A	radian	rad	plane angle	U
1B	degree	°	plane angle	U
1C	hertz	Hz	frequency	U
1D	joule	J	energy	U
1E	Newton	N	force	U
1F	kilopond	kp	force	U
20	pound force	lbf	force	U
21	watt	W	power	U
22	horse power (metric)	hk	power	U
23	horse power (UK and US)	hp	power	U
24	Pascal	Pa	pressure	U
25	bar	bar	pressure	U
26	atmosphere	atm	pressure	U
27	pound force per square	psi	pressure	U
28	becquerel	Bq	radioactivity	U
29	lumen	lm	light flux	U
2A	lux	lx	illuminance	U
2B	liter	l	volume	U
2C	gallon (British)	-	volume	U
2D	gallon (US liq)	-	volume	U
2E	cubic inch	cu in	volume	U

1F-SG-00002

현대 · 기아 자동차

HYUNDAI · KIA MOTOR



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 165/211

2F	meter per second	m/s	speed	U
30	kilometer per hour	km/h	speed	U
31	mile per hour	mph	speed	U
32	revolutions per second	rps	angular velocity	U
33	revolutions per minute	rpm	angular velocity	U
34	counts	-	-	U
35	percent	%	-	U
36	milligram per stroke	mg/stroke	mass per engine stroke	U
37	meter per square second	m/s ²	acceleration	U
38	Newton meter	Nm	moment (e.g. torsion moment)	U
39	liter per minute	l/min	flow	U
3A	Watt per square meter	W/m ²	Intensity	U
3B	Bar per second	bar/s	Pressure change	U
3C	Radians per second	rad/s	Angular velocity	U
3D	Radians per square	rad/s ²	Angular acceleration	U
3E	Kilogram per square	kg/m ²	-	U
3F	-	-	Reserved	M
40	exa (prefix)	E	10 ¹⁸	U
41	peta (prefix)	P	10 ¹⁵	U
42	tera (prefix)	T	10 ¹²	U
43	giga (prefix)	G	10 ⁹	U
44	mega (prefix)	M	10 ⁶	U
45	kilo (prefix)	k	10 ³	U
46	hecto (prefix)	h	10 ²	U
47	deca (prefix)	da	10	U
48	deci (prefix)	d	10 ⁻¹	U
49	centi (prefix)	c	10 ⁻²	U
4A	milli (prefix)	m	10 ⁻³	U
4B	micro (prefix)	μ	10 ⁻⁶	U
4C	nano (prefix)	n	10 ⁻⁹	U
4D	pico (prefix)	p	10 ⁻¹²	U
4E	femto (prefix)	f	10 ⁻¹⁵	U
4F	atto (prefix)	a	10 ⁻¹⁸	U
50	Date1	-	Year-Month-Day	U
51	Date2	-	Day/Month/Year	U
52	Date3	-	Month/Day/Year	U

1F-SG-00002

현대 • 기아 자동차
HYUNDAI • KIA MOTOR

53	week	W	calendar week	U
54	Time1	-	UTC Hour/Minute/Second	U
55	Time2	-	Hour/Minute/Second	U
56	DateAndTime1	-	Second/Minute/Hour/Day /Month/Year	U
57	DateAndTime2	-	Second/Minute/Hour/Day /Month/Year/Local minute offset/Local hour offset	U
58	DateAndTime3	-	Second/Minute/Hour/Mon th/Day/Year	U
59	DateAndTime4	-	Second/Minute/Hour/Mon th/Day/Year/Local minute offset/Local hour offset	U
B0	degree per second	deg/s	degree/second	U
5A-FF	-	-	Reserved	M

Table C-13. Unit / Format ScalingByteExtension Encoding

C.3.4 ScalingByteExtension for ScalingByte high nibble of StateAndConnectionType

A scalingByte with high nibble encoded as stateAndConnectionType shall be followed by a single scalingByteExtension byte defining the stateAndConnectionType. The one byte scalingByteExtension is defined in Table C.9 — Encoding of scalingByte High Nibble of stateAndConnectionType. The stateAndConnectionType encoding is used specially for input and output signals. Encoded in the scalingByteExtension data byte is information about the physical layout, electrical levels and functional state.

Encoding of Bit	Value	Used with Input Signal	Used with Output Signal
0 – 2	0	State : Not Active	State : Not Activated
	1	State : Active, Function 1	State : Active, Function 1
	2	State : Error Detected	State : Plausibility Error Detected
	3	State : Not Available	State : Not Available
	4	State : Active, Function 2 (only in combination with 3 states)	State : Active, Function 2 (only in combination with 3 states)
	5 – 7	Reserved	Reserved
3 – 4	0	Signal at Low Level (Ground)	Signal at Low Level (Ground)
	1	Signal at Middle Level (Between Ground Level and +)	Signal at Middle Level (Between Ground Level and +)
	2	Signal at High Level (+)	Signal at High Level (+)
	3	Reserved	Reserved
5	0	Input Signal	Not Defined
	1	Not Defined	Output Signal
6 – 7	0	Internal Signal or via CAN not exclusively available in ECU connector	Internal Signal or via CAN not exclusively available in ECU connector
	1	Pull-down Resistor input type (2 states)	Low side switch (2 states)
	2	Pull-up Resistor input type (2 states)	High side switch (2 states)
	3	Pull-up and Pull-down resistor input type (3 state)	Low side and High side switch (3 state)

Table C-14. Encoding of scalingByte High Nibble of stateAndConnectionType



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 167/211

HEX	Description	Cvt	Mnemonic
00	Reserved		
01	sendAtSlowRate This parameter specifies that the server shall transmit the requested dataRecord information at a slow rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter slow is vehicle manufacturer specific, and pre-defined in the server.	U	SASR
02	sendAtMediumRate This parameter specifies that the server shall transmit the requested dataRecord information at a medium rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter medium is vehicle manufacturer specific, and pre-defined in the server.	U	SAMR
03	sendAtFastRate This parameter specifies that the server shall transmit the requested dataRecord information at a fast rate in response to the request message (where the # of responses to be sent = maximumNumberOfResponsesToSend). The repetition rate specified by the transmissionMode parameter fast is vehicle manufacturer specific, and pre-defined in the server.	U	SAFR
04	stopSending The server stops transmitting positive response messages send periodically/repeatedly. Note that maximumNumberOfResponsesToSend parameter should be set to 01 hex if transmissionMode = stopSending (otherwise, server operation could be undefined).	C	SS
05 - FF	Reserved		

C : stopSending shall be supported if sendAtSlowRate, sendAtMediumRate, and/or sendAtFastRate are supported.

Table C-15. Encoding of scalingByte High Nibble of stateAndConnectionType

**Annex. D Stored Data Transmission Functional Unit Data
Parameter Definitions**

D.1 GroupOfDTC Parameter Definition

Table D-1. Definition of GroupOfDTC and range of DTC numbers provides group of DTC definitions.

HEX	Description	Cvt	Mnemonic
000000 - 0000FF	Reserved for future legislative requirements	M	RFLU
0001FF	Powertrain Group	U	PG
0001XX - 3FFFXX	Powertrain DTC's (except 0001FF)	U	PDTC_
4000FF	Chassis Group	U	CG
4001XX - 7FFFXX	Chassis DTC's	U	CDTC_
8000FF	Body Group	U	BG
8001XX - BFFFXX	Body DTC's	U	BDTC_
C000FF	Network Communication Gruop	U	NCG
C001XX - FFFEXX	Network Communication DTC's	U	NCDTC_
FFFF33	Emissions-system group	C	ESG
FFFFD0	Safety-system group	U	SSG
FFFFFF	All Groups (All DTC's)	M	AG

C : This parameter selects the emission-related systems.

Table D-1. Definition of GroupOfDTC and range of DTC numbers

D.2 DTCStatusMask and StatusOfDTC Bit Definitions

D.2.1 Convention

This section defines the mapping of the DTCStatusMask / statusOfDTC parameters used with the ReadDTCInformation service. Every server shall adhere to the convention for storing bit-packed DTC status information as defined in table below. Actual usage of the bit-fields shall be defined in the implementation standards.

The following is a list of definitions used for the description of the DTC status bit definitions.

- **Test** : A test is an on-board diagnostic software algorithm that determines the malfunction status of a component or system. Some tests run only once during an operation cycle. Other tests can run every program loop, as often as every few milliseconds. The result of a test represents a completely matured / qualified failure condition. That means a test which needs a failing condition over a specific time or evaluation of additional plausibility checks before a component is considered to be failing will return a "Failed" condition after all maturation criteria have been fulfilled. Each test is associated with a unique DTC representing the root failure and detectable fault symptom.
- **Complete** : Complete is an indication that a test was able to determine whether a malfunction exists or does not exist for the current operation cycle (complete does not imply failed).
- **Test results** : While a test runs or after it has completed it may indicate one the following results to the internal failure handler :
 - **PreFailed** : This status may be used by tests in ECUs to indicate that the test is currently maturing a failure condition. One use case for this information is in manufacturing to speed up failure detection for optimised workflow while maintaining fault tolerance in the field.
 - **Failed** : This status is available after a monitor has run to its completion and indicates a completely matured failing condition.
 - **Passed** : This status is available after a monitor has run to its completion and indicates that the system or component is not failing.
- **Failure** : A failure is the inability of a component or system to meet its intended function. A failure has occurred when fault conditions have been detected for a sufficient period of time to warrant storage of a pending (for emission related components only) or confirmed DTC implying that a test returned a "Failed" result. The terms "failure" and "malfunction" are interchangeable.
- **Monitor** : A monitor consists of one or more tests used to determine the proper functioning of a component or system.
- **Monitoring cycle** : A monitoring cycle is the time in which a monitor runs to its completeness. This is a manufacturer defined set of conditions during which the tests of a monitor can run. A monitoring cycle may be executed several times during an operation cycle.
- **Operation Cycle** : An operation cycle defines the start and end conditions for monitors to run. During an operation cycle several monitoring cycles may have passed (regardless of their test results). An ECU may support several operation cycles. For body and chassis ECU's it is up to the manufacturer to define an operation cycle (e.g. time between powering up and powering down the ECU or between ignition on and ignition off). For powertrain ECU's, there are additional criteria defining an operation cycle. Emissionsrelated powertrain ECU's use an engine-running or engine-off time period to define an operation cycle which is referred to as driving cycle.

NOTE) For emissions-related monitors the criteria for the beginning and the end of an operation cycle are defined by



기술 표준 ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 170/211

legislation.

- **Pending** : The pending status of a failure is defined as a Test Result having reported a “Failed” for this failure during the current and/or the last completed operation cycle. Once the test has reported a “Passed” condition for a complete operation cycle of this failure the pending status is reset.
- **Driving cycle** : A specific type of operation cycle used for emissions-related ECU's. Refer to “OperationCycle” for further details. In emissions-related ECU's only one operation cycle shall be supported, which is identical to the driving cycle as defined by legislation.

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

D.2.2 StatusOfDTC Bit Definition

Bit	Description	Default Value	Cvt	Mnemonic
0	TestFailed	0	M	TF
	'0' = most recent result from DTC test indicated no failure detected '1' = most recent result from DTC test indicated a matured failing result.			
1	TestFailedThisOperationCycle	0	C1	TFTOC
	'0' = testFailed: result has not been reported during the current operation cycle or after a call was made to ClearDiagnosticInformation during the current operation cycle. '1' = testFailed: result was reported at least once during the current operation cycle.			
2	PendingDTC	0	U	PDTC
	'0' = This bit shall be set to 0 after completing an operation cycle during which the test completed and a malfunction was not detected or upon a call to the ClearDiagnosticInformation service. '1' = This bit shall be set to 1 and latched if a malfunction is detected during the current operation cycle.			
3	ConfirmedDTC	0	M	CDTC
	'0' = DTC has never been confirmed since the last call to ClearDiagnosticInformation or after the aging criteria have been satisfied for the DTC (or DTC has been erased due to fault memory overflow). '1' = DTC confirmed at least once since the last call to ClearDiagnosticInformation and aging criteria have not yet been satisfied.			
4	TestNotCompletedSinceLastClear	1	C2	TNCSLC
	'0' = DTC test has returned either a passed or failed test result at least one time since the last time diagnostic information was cleared. '1' = DTC test has not run to completion since the last time diagnostic information was cleared.			
5	TestFailedSinceLastClear	0	C2	TFSLC
	'0' = DTC test has not indicated a failed result since the last time diagnostic information was cleared. '1' = DTC test returned a failed result at least once since the last time diagnostic information was cleared.			
6	TestNotCompleteThisOperationCycle	1	U	TNCTOC
	'0' = DTC test has returned either a passed or testFailedThisOperationCycle = '1' result during the current drive cycle (or since the last time diagnostic information was cleared during the current operation cycle). '1' = DTC test has not run to completion this operation cycle (or since the last time diagnostic information was cleared this operation cycle).			
7	WarningIndicatorRequested	0	M	WIR
	'0' = Server is not requesting warningIndicator to be active '1' = Server is requesting warningIndicator to be active			

C1 : Bit 1 (TestFailedThisOperationCycle) is Mandatory if Bit 2 (PendingDTC) is supported. Bit 1 is User Optional if Bit 2 is not supported.
 C2 : Bit 4 (TestNotCompletedSinceLastClear) and Bit 5 (TestFailedSinceLastClear) shall always be supported together.
 *. DTCStatusMask will be determined by setting each bit of StatusOfDTC '0 = not supported' or '1 = supported'.

Table D-2. DTC Status Bit Definition Overview



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 172/211

DTC Case	Status Of Bit (Lower 4Bit)				DTC Status Mask (request) (※ If the result is greater than or equal to 1, responds corresponding DTC)		Display of tester	Reason for not display
	Bit3	Bit2	Bit1	Bit0	09 (EOL HADS, A/S GDS)	0D (EOL U-Robot, A/S GDS)		
	Confirmed	Pending	TestFailedThisOperationCycle	TestFailed				
#1	0	0	X	0	0 (no response)	0 (no response)	-	Not DTC occur and masking
#2	0	0	X	1	1	1	-	Not occur ¹⁾
#3	0	1	X	0	0 (no response)	1	Pending DTC	
#4	0	1	X	1	1	2	Pending DTC	
#5	1	0	X	0	1	1	Historic DTC	
#6	1	0	X	1	2	2	-	Not occur ¹⁾
#7	1	1	X	0	1	2	Historic DTC	
#8	1	1	X	1	2	3	Active DTC	

¹⁾ In case of supporting Pending DTC, if Test Failed Bit is set, Pending DTC is also set.

표 D-3. DTC Status Definition and Display of tester (ECU supported Pending DTC)

DTC Case	Status Of Bit		DTC Status Mask (request) (※ If the result is greater than or equal to 1, responds corresponding DTC)	Display of tester	Reason for not display
	Bit3	Bit0			
	Confirmed	TestFailed	08 (EOL, A/S GDS)		
#1	0	0	0 (no response)	-	Not DTC occur and masking
#2	0	1	0 (no response)	-	Not Occur ²⁾ and masking
#3	1	0	1	Historic DTC	
#4	1	1	1	Active DTC	

²⁾ In case of not supporting Pending DTC, if Test Failed Bit is set, Confirmed DTC is also set.
(Confirmation threshold : 1)

표 D-4. DTC Status Definition and Display of tester (ECU not supported Pending DTC)

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
0	TestFailed	U	U	TF
This bit shall indicate the result of the most recently performed test. A logical '1' shall indicate that the last test failed meaning that the failure is completely matured. Reset to logical '0' if the result of the most recently performed test returns a “pass” result meaning that all de-mature criteria have been fulfilled. Additional reset conditions may be defined by the vehicle manufacturer / implementation				
Bit state after a successful ClearDiagnosticInformation service			Logical '0'	
Reset to logical '0' if a call has been made to ClearDiagnosticInformation.				
Bit state definition		Test Equipment Display Text		
'0' = most recent result from DTC test indicated no failure detected (or test has not completed this operation cycle).		DTC test is not failed at time of request		
'1' = most recent result from DTC test indicated a matured failing result.		DTC test failed at time of request		

Table D-4. DTC Status Bit 0 TestFailed Definitions

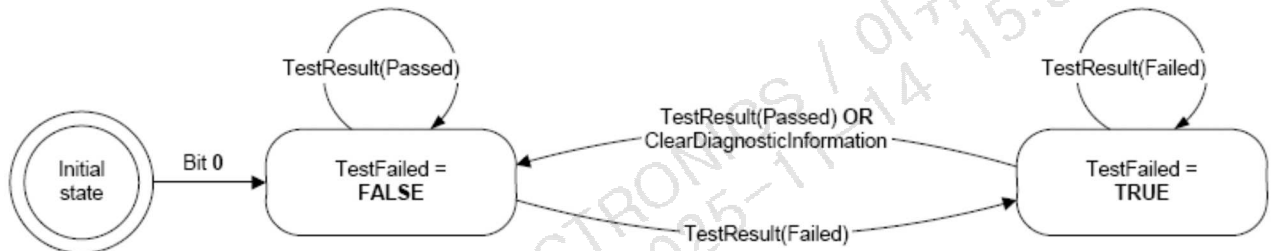


Figure D-1. DTC Status Bit 0 TestFailed Logic

Bit	Description	Cvt : Emission	Cvt : Non- Emission	Mnemonic
1	TestFailedThisOperationCycle This bit shall indicate whether or not a diagnostic test has reported a testFailed result at any time during the current operation cycle (or that a testFailed result has been reported during the current operation cycle and after the last time a call was made to ClearDiagnosticInformation). Reset to logical '0' when a new operation cycle is initiated or after a call to ClearDiagnosticInformation. If this bit is set to logical '1', it shall remain a '1' until a new operation cycle is started. Bit state after a successful ClearDiagnosticInformation service Reset to logical '0' after a call to ClearDiagnosticInformation. Bit state definition '0' = testFailed: result has not been reported during the current operation cycle or after a call was made to ClearDiagnosticInformation during the current operation cycle. '1' = testFailed: result was reported at least once during the current operation cycle. C1: Bit 1 (testFailedThisOperationCycle) is Mandatory if bit 2 (pendingDTC) is supported. Bit 1 (testFailedThisOperationCycle) is User Optional if bit 2 (pendingDTC) is not supported.	M	C1	TFTOC
Bit state after a successful ClearDiagnosticInformation service		Logical '0'		
Bit state definition		Test Equipment Display Text		
'0' = testFailed: result has not been reported during the current operation cycle or after a call was made to ClearDiagnosticInformation during the current operation cycle.		DTC test has not failed this operation cycle		
'1' = testFailed: result was reported at least once during the current operation cycle.		DTC test failed this operation cycle		

Table D-5. DTC Status Bit 1 TestFailedThisOperationCycle Definitions

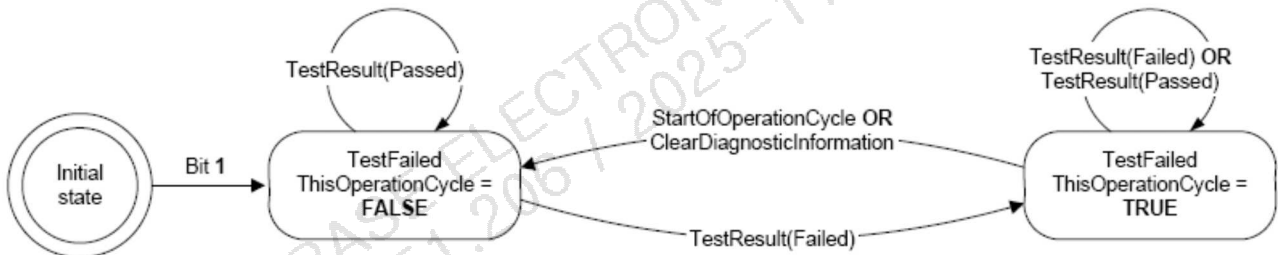


Figure D-2. DTC Status Bit 1 TestFailedThisOperationCycle Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
2	PendingDTC This bit shall indicate whether or not a diagnostic test has reported a testFailed result at any time during the current or last completed operation cycle. The status shall only be updated if the test runs and completes. The criteria to set the pendingDTC bit and the TestFailedThisOperationCycle bit are the same. The difference is that the testFailedThisOperationCycle is cleared at the end of the current operation cycle and the pendingDTC bit is not cleared until an operation cycle has completed where the test has passed at least once and never failed. If the test did not complete during the current operation cycle, the status bit shall not be changed. For example, if a monitor stops running after a confirmed DTC is set, the pendingDTC must remain set = '1'. For an OBD DTC, a pending DTC is required to be stored after a malfunction is detected during the first driving cycle. Bit state after a successful ClearDiagnosticInformation service Logical '0' Reset to logical '0' after a call to ClearDiagnosticInformation. Bit state definition '0' = This bit shall be set to 0 after completing an operation cycle during which the test completed and a malfunction was not detected or upon a call to the ClearDiagnosticInformation service. '1' = This bit shall be set to 1 and latched if a malfunction is detected during the current operation cycle.	M	U	PDTC
		Test Equipment Display Text		
		DTC test has not failed during the current or last completed operation cycle		
		DTC test failed during the current or last completed operation cycle		

Table D-6. DTC Status Bit 2 PendingDTC Definitions

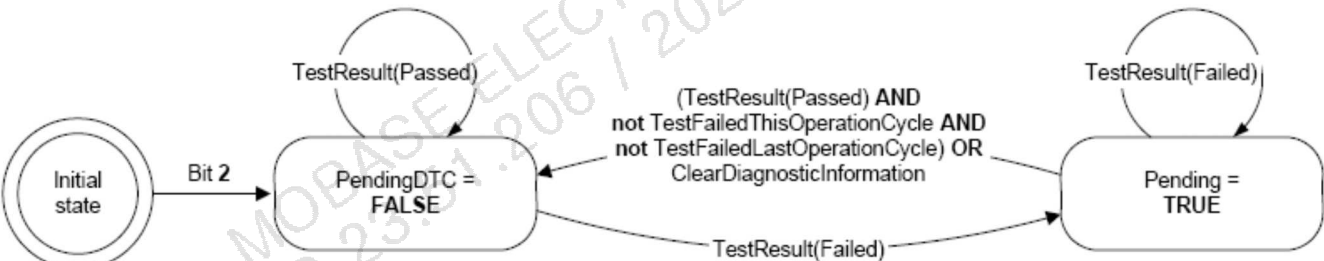


Figure D-3. DTC Status Bit 2 PendingDTC Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
3	ConfirmedDTC	M	M	CDTC
This bit shall indicate whether a malfunction was detected enough times to warrant that the DTC is stored in long-term memory (e.g. pendingDTC has been set = '1' one or more times, depending on the DTC confirmation criteria). A confirmedDTC does not always indicate that the malfunction is present at the time of the request. (testFailed can be used to determine if a malfunction is present at the time of the request). Reset to logical '0' after a call to ClearDiagnosticInformation or after aging criteria has been satisfied (e.g., 40 engine warm-ups without another detected malfunction). Furthermore this bit is reset when the fault record associated with this DTC is overwritten by a newer DTC based upon vehicle manufacturer specific fault memory overflow requirements. DTC confirmation and aging criteria are defined by the vehicle manufacturer or mandated by On Board Diagnostic regulations.				
Bit state after a successful ClearDiagnosticInformation service				Logical '0'
Reset to logical '0' after a call to ClearDiagnosticInformation.				
Bit state definition		Test Equipment Display Text		
'0' = DTC has never been confirmed since the last call to ClearDiagnosticInformation or after the aging criteria have been satisfied for the DTC (or DTC has been erased due to fault memory overflow).		A confirmed DTC is not stored in the ECU.		
'1' = DTC confirmed at least once since the last call to ClearDiagnosticInformation and aging criteria have not yet been satisfied.		A confirmed DTC is stored in the ECU.		

Table D-7. DTC Status Bit 3 ConfirmedDTC Definitions

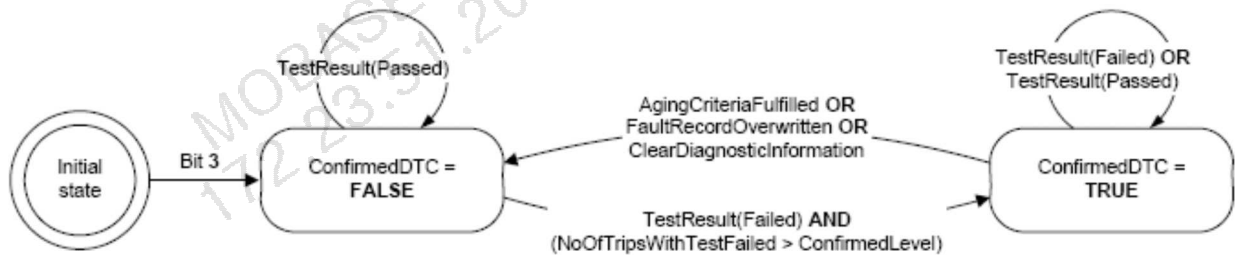


Figure D-4. DTC Status Bit 3 ConfirmedDTC Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
4	TestNotCompleteSinceLastClear	C2	C2	TNCSLC
This bit shall indicate whether a DTC test has ever run and completed since the last time a call was made to ClearDiagnosticInformation. One ('1') shall indicate that the DTC test has not run to completion. If the test runs and passes or if the test runs and fails (e.g. testFailedThisOperationCycle = '1') then the bit shall be set to a '0' (and latched).				
Bit state after a successful ClearDiagnosticInformation service			Logical '1'	
Reset to logical '1' after a call to ClearDiagnosticInformation.				
Bit state definition		Test Equipment Display Text		
'0' = DTC test has returned either a passed or failed test result at least one time since the last time diagnostic information was cleared.		DTC test completed since the last code clear		
'1' = DTC test has not run to completion since the last time diagnostic information was cleared.		DTC test not completed since the last code clear		
C2: Bit 4 (testNotCompletedSinceLastClear) and bit 5 (testFailedSinceLastClear) shall always be supported together.				

Table D-8. DTC Status Bit 4 TestNotCompleteSinceLastClear Definitions

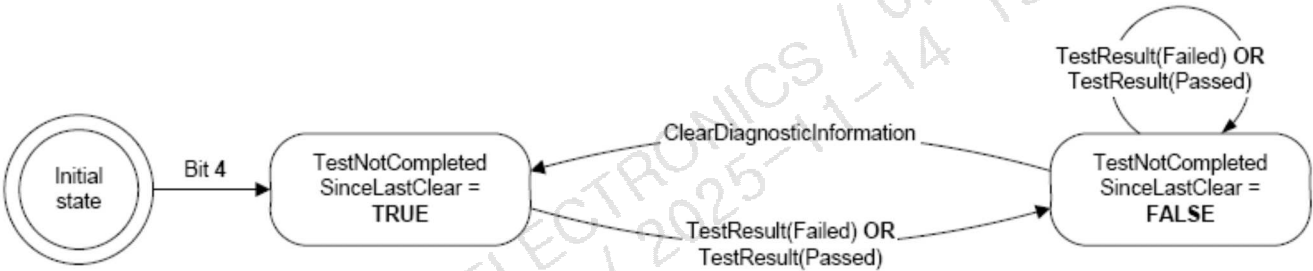


Figure D-5. DTC Status Bit 4 TestNotCompleteSinceLastClear Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
5	TestFailedSinceLastClear	C2	C2	TFSLC
This bit shall indicate whether a DTC test has ever completed with a failed result since the last time a call was made to ClearDiagnosticInformation (i.e., this is a latched testFailedThisOperationCycle = '1'). Zero ('0') shall indicate that the test has not run or that the DTC test ran and passed (but never failed). If the test runs and fails then the bit shall remain latched at a '1'. Unlike confirmedDTC, this bit is not reset by aging-criteria or due to an overflow of the fault memory.				
Bit state after a successful ClearDiagnosticInformation service			Logical '0'	
Reset to logical '0' after a call to ClearDiagnosticInformation.				
Bit state definition		Test Equipment Display Text		
'0' = DTC test has not indicated a failed result since the last time diagnostic information was cleared.		DTC test never failed since last code clear		
'1' = DTC test returned a failed result at least once since the last time diagnostic information was cleared.		DTC test failed at least once since last code clear		
C2: Bit 4 (testNotCompletedSinceLastClear) and bit 5 (testFailedSinceLastClear) shall always be supported together.				

Table D-9. DTC Status Bit 5 TestFailedSinceLastClear Definitions

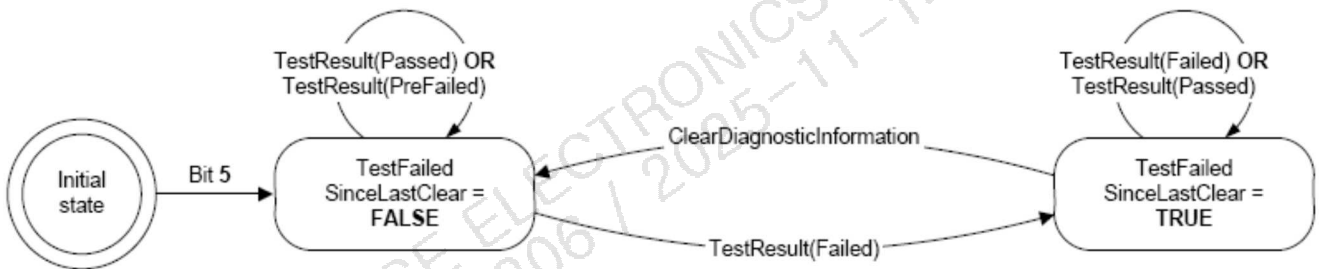


Figure D-6. DTC Status Bit 5 TestFailedSinceLastClear Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
6	TestNotCompleteThisOperationCycle	M	U	TNCTOC
This bit shall indicate whether a DTC test has ever run and completed during the current operation cycle (or completed during the current operation cycle after the last time a call was made to ClearDiagnosticInformation). A logical '1' shall indicate that the DTC test has not run to completion during the current operation cycle. If the test runs and passes or fails then the bit shall be set (and latched) to '0' until a new operation cycle is started.				
Bit state after a successful ClearDiagnosticInformation service			Logical '1'	
Reset to logical '1' after a call to ClearDiagnosticInformation.				
Bit state definition		Test Equipment Display Text		
'0' = DTC test has returned either a passed or testFailedThisOperationCycle = '1' result during the current drive cycle (or since the last time diagnostic information was cleared during the current operation cycle).		DTC test completed this operation cycle		
'1' = DTC test has not run to completion this operation cycle (or since the last time diagnostic information was cleared this operation cycle).		DTC test not completed this operation cycle		

Table D-10. DTC Status Bit 6 TestNotCompleteThisOperationCycle Definitions

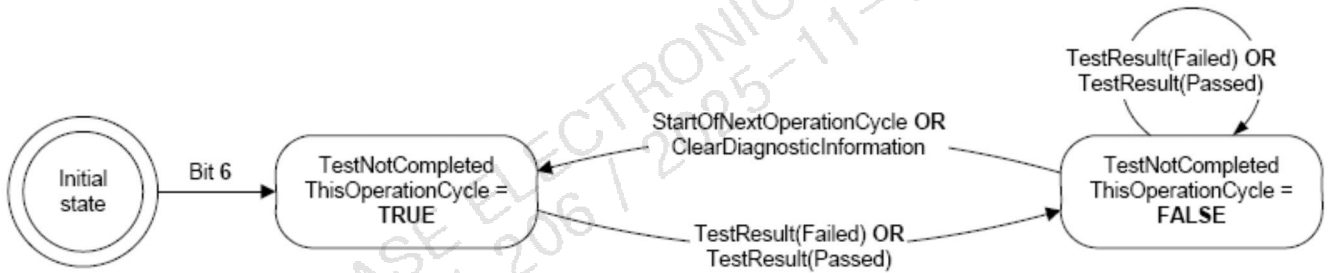
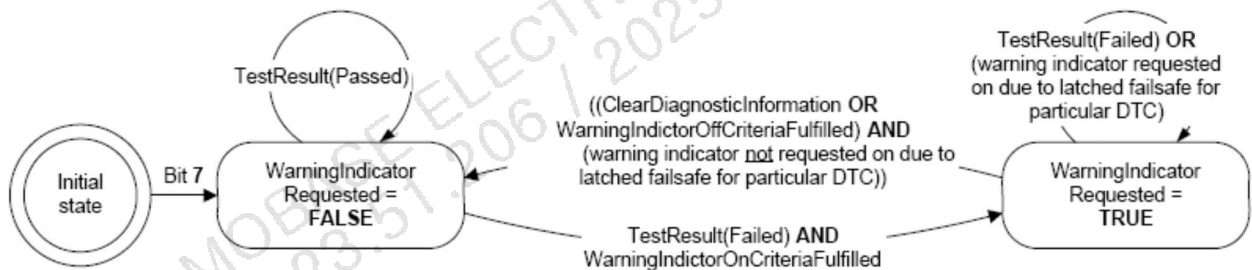


Figure D-7. DTC Status Bit 6 TestNotCompleteThisOperationCycle Logic

Bit	Description	Cvt : Emission	Cvt : Non-Emission	Mnemonic
7	WarningIndicatorRequested	M	M	WIR
This bit shall report the status of any warning indicators associated with a particular DTC. Warning outputs may consist of indicator lamp(s), displayed text information, etc. If no warning indicators exist for a given system or particular DTC, this status shall default to a logic '0' state. Conditions for activating the warning indicator shall be defined by the vehicle manufacturer / implementation, but if the warning indicator is on for a given DTC, then confirmedDTC shall also be set to '1' (with the exception of a confirmed fault with a latched failsoft strategy as described below).				
Bit state after a successful ClearDiagnosticInformation service			Logical '0'	
Reset to a logical '0' after a call to ClearDiagnosticInformation. Some ECUs may latch the failsoft strategy associated with a particular confirmed fault for the current operation cycle. If the warning indicator is still requested due to this latched failsoft following a call to ClearDiagnosticInformation, this bit shall not be cleared to a logical '0'. Rather, this bit shall remain set to logical '1' until the failsoft strategy is no longer active (e.g., test completes and passes). Additional reset conditions shall be defined by the vehicle manufacturer / implementation.				
Bit state definition		Test Equipment Display Text		
'0' = Server is not requesting warningIndicator to be active		DTC is currently not requesting warning indication		
'1' = Server is requesting warningIndicator to be active		DTC is currently requesting warning indication		

Table D-11. DTC Status Bit 7 WarningIndicatorRequested Definitions



WarningIndicatorOnCriteriaFulfilled = warning indicator exists for given system and particular DTC AND (confirmedDTC = 1 OR vehicle manufacturer or implementation-specific warning indicator enable criteria are satisfied)

WarningIndicatorOffCriteriaFulfilled = TestResult(Passed) OR vehicle manufacturer or implementation-specific warning indicator disable criteria are satisfied

Figure D-8. DTC Status Bit 7 WarningIndicatorRequested Logic

D.3 DTCSeverityMask and DTCSeverity Bit Definitions

This section defines the mapping of the DTCSeverityMask / DTCSeverity parameters used with the ReadDTCInformation service. Every server shall adhere to the convention for storing bit-packed DTC severity information as defined in Table D-11. DTCSeverity byte definition and Table D-12. DTC severity bit definitions (bit 7-5).

The severity information is reported in a 1-byte value. Only the upper 3 bits (bit 7-5) of the 1-byte value are used to represent the DTC severity information. The remaining bits (bit 4-0) have to be set to zero (0). Based on this the following bit coding applies to the 3 bit DTC severity information contained in the 1-byte DTCSeverity parameter.

DTC Severity Byte							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
3 Bit Severity Information			Reserved (To be set to zero)				

Table D-12. DTCSeverity byte definition

Bit 7-5	Description	Cvt	Mnemonic
000b	NoSeverityAvailable There is no severity information available.	M	NSA
001b	MaintenanceOnly This value indicates that the failure requests maintenance only.	M	MO
010b	CheckAtNextHalt This value indicates to the failure requires a check of the vehicle at the next halt.	M	CHKANH
011b	Reserved	M	
100b	CheckImmediately This value indicates to the failure requires an immediate check of the vehicle.	M	CHKI
101~111b	Reserved	M	

Table D-13. DTC severity bit definitions (bit 7-5)

D.4 DTC Format Structure

DTC High Byte			DTC Middle Byte		DTC Low Byte	
Bit 7-6	Bit 5-4	Bit 3-0	Bit 7-4	Bit 3-0	Bit 7-4	Bit 3-0
00 = powertrain (P) 01 = chassis (C) 10 = body (B) 11 = network (U)	00 = ISO/SAE controlled 01 = manufacturer controlled 10 = ISO/SAE controlled 11 = ISO/SAE controlled	1 st Character Of Code (Hex 0-F)	2 nd Character of Code (Hex 0-F)	3 rd Character of Code (Hex 0-F)	DTC Failure Category (refer to ISO15031-6)	DTC Failure Sub Type (refer to ISO15031-6)

Table D-14. Structure of Diagnostic Trouble Code

D.4.1 DTC Failure Category and Sub Type definition

The DTC Failure Type consists of sixteen (16) different Failure Categories, where each category is associated with sixteen (16) Sub Type Failures (also known as symptoms). The Sub Type Failures are logically grouped in a DTC Failure Type Category. This shall simplify the selection of the appropriate Sub Type Failure {Symptom} for a DTC.

The DTC Failure Category is coded in the High Nibble of the “DTCFailureTypeByte” and the Failure Sub Type is coded in the Low Nibble of the “DTCFailureTypeByte”.

High Nibble (Bit 7-4)	Low Nibble (Bit 3-0)
DTC Failure Category	DTC Failure Sub Type

Table D-15. DTC Failure Type Byte Definition

D 4.1.1 DTC Failure Category definition

The table below specifies the “DTC Failure Categories”.

DTCFailureTypeByte Category Definitions		
High Nibble (0000b - 1111b)	Category # (hex)	Category Description
0000	0	General Failure Information This category includes all other categories and is used when the fault within that failure category is unique (not amenable to standardization through assignment of a new Sub Type) or when the detected fault is best described by two or more Sub Types within that Failure Category.
0001	1	General Electrical Failures This category includes standard wiring failure modes (i.e., shorts and opens), and direct current (DC) quantities related by Ohm's Law.
0010	2	General Signal Failures This category includes quantities related to amplitude, frequency or rate of change, and wave shape.
0011	3	FM (Frequency Modulated) / PWM (Pulse Width Modulated) Failures This category includes faults related to Frequency Modulated (FM) and Pulse Width Modulated (PWM) inputs and outputs of the server. This category also includes faults where position is determined by counts.
0100	4	System Internal Failures This category includes faults related to memory, software, and internal electrical circuitry; requiring component (server, sensor, etc.) replacement.
0101	5	System Programming Failures This category includes faults related to operational software, calibrations, and options; remedied by configuring/programming a part of the system (server, sensor, etc.).
0110	6	Algorithm Based Failures This category includes faults based on comparing two or more input parameters for plausibility or comparing a single parameter to itself with respect to time.
0111	7	Mechanical Failures This category includes faults detected by inappropriate motion in response to server related input/controlled output.
1000	8	Bus Signal / Message Failures This category includes faults related to bus hardware and signal integrity. This category is also used when the physical input for a signal is located in one server and another server diagnoses the circuit or inhibits operation due to a reported failure of that circuit.
1001	9	Component Failures This category includes faults related to component failures (including parametric, performance assembly and operating environment failures).
1010 - 1110	A – E	ISO/SAE reserved This range of values is reserved by the document for future expansion.
1111	F	Vehicle manufacturer / System supplier specific This category is reserved for vehicle manufacturer/system supplier use.

Table D-16. DTC Failure Category definition

D 4.1.2 DTC Failure Subtype definition of General Failure Information

This category includes all other categories and is used when the fault within that failure category is unique (not amenable to standardization through assignment of a new Sub Type) or when the detected fault is best described by two or more Sub Types within that Failure Category.

DTCFailureTypeByte Category Definitions

Failure Type Byte (hex)	Subtype Nibble (binary)	General Failure Information Subtype Description
00	0000	No sub type information This sub type is used for failures where the base DTC text string provides the complete description of the failure itself (no Category and no Sub Type information used, e.g. emissions-related DTC (012700 hex): P0127 Intake Air Temperature Too High).
01	0001	General Electrical Failure This sub type is used for General Electrical Failures that cannot be assigned to a specific sub type (Category information and no Sub Type information, e.g. DTC (011501): P0115 Engine Coolant Temperature Circuit – General Electrical Failure).
02	0010	General Signal failure This sub type is used for General Signal Failures that cannot be assigned to a specific sub type (Category information and no Sub Type information, e.g. DTC (014802): P0148 Fuel Delivery Error – General Signal Failure).
03	0011	FM (Frequency Modulated) / PWM (Pulse Width Modulated) Failures This sub type is used for FM / PWM Failures that cannot be assigned to a specific sub type.
04	0100	System Internal Failures This sub type is used for server Internal Failures that cannot be assigned to a specific sub type.
05	0101	System Programming Failures This sub type is used for System Programming Failures that cannot be assigned to a specific sub type.
06	0110	Algorithm Based Failures This sub type is used for Algorithm Based Failures that cannot be assigned to a specific sub type.
07	0111	Mechanical Failures This sub type is used for Mechanical Failures that cannot be assigned to a specific sub type.
08	1000	Bus Signal / Message Failures This sub type is used for Bus Signal / Message Failures that cannot be assigned to a specific sub type.
09	1001	Component Failures This sub type is used for Component Failures that cannot be assigned to a specific sub type.
0A – 0F	1010 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-17. DTC Failure Subtype definition for failure category “0”



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 185/211

D 4.1.3 DTC Failure Subtype definition of General Electrical Failures

The table below specifies the standard wiring failure modes (i.e., shorts and opens), and direct current (DC) quantities related by Ohm's Law.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	General Electrical Failures Subtype Description
10	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
11	0001	Circuit short to ground This sub type is used for failures, where the server measures ground (battery negative) potential for greater than a specified time period or when some other value is expected.
12	0010	Circuit short to battery This sub type is used for failures, where the server measures vehicle system (battery positive) potential for greater than a specified time period or when some other value is expected.
13	0011	Circuit open This sub type is used for failures, where the server determines an open circuit via lack of bias voltage, low current flow, no change in the state of an input in response to an output, etc.
14	0100	Circuit short to ground or open This sub type is used for failures, where the condition detected by the server is the same for either indicated failure mode.
15	0101	Circuit short to battery or open This sub type is used for failures, where the condition detected by the server is the same for either indicated failure mode.
16	0110	Circuit voltage below threshold This sub type is used for failures, where the server measures a voltage below a specified range but not necessarily a short to ground.
17	0111	Circuit voltage above threshold This sub type is used for failures where, the server measures a voltage above a specified range but not necessarily a short to battery.
18	1000	Circuit current below threshold This sub type is used for failures, where the server measures current flow below a specified range.
19	1001	Circuit current above threshold This sub type is used for failures, where the server measures current flow above a specified range.
1A	1010	Circuit resistance below threshold This sub type is used for failures, where the server infers a circuit resistance below a specified range.
1B	1011	Circuit resistance above threshold This sub type is used for failures, where the server infers a circuit resistance above a specified range.
1C	1100	Circuit voltage out of range This sub type is used for failures, where the server measures a voltage outside the expected range but not identified as too high or too low.
1D	1101	Circuit current out of range This sub type is used for failures, where the server measures a current outside the expected range but not identified as too high or too low.
1E	1110	Circuit resistance out of range This sub type is used for failures, where the server measures a resistance outside the expected range but not identified as too high or too low.

1F	1111	Circuit intermittent This sub type is used for failures, where the server momentarily detects one of the conditions defined above, but not long enough to set a specific sub type.
----	------	--

Table D-18. DTC Failure Subtype definition for failure category “1”

D 4.1.4 DTC Failure Subtype definition of General Signal Failures

The table below specifies quantities related to amplitude, frequency or rate of change, and waveshape.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	General Signal Failures Subtype Description
20	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
21	0001	Signal amplitude < minimum This sub type is used for failures where the server measures a signal voltage below a specified range, but not necessarily a short to ground (e.g. low gain)
22	0010	Signal amplitude > maximum This sub type is used for failures where the server measures a signal voltage above a specified range but not necessarily a short to battery (e.g., gain too high).
23	0011	Signal stuck low This sub type is used for failures where the server measures a signal that remains low when transitions are expected.
24	0100	Signal stuck high This sub type is used for failures where the server measures a signal that remains high when transitions are expected.
25	0101	Signal shape / waveform failure This sub type is used for failures where the shape of the signal (plot of the amplitude with respect to time) is not correct, e.g., improper circuit impedance.
26	0110	Signal rate of change below threshold This sub type is used for failures where the signal transitions more slowly than is reasonably allowed.
27	0111	Signal rate of change above threshold This sub type is used for failures where the signal transitions more quickly than is reasonably allowed.
28	1000	Signal bias level out of range / zero adjustment failure This sub type is used for failures where the server applies a bias voltage to a circuit upon which is superimposed a signal voltage (e.g., Oxygen Sensor circuit.). This sub type is also used for failures where the server applies a zero signal level to a circuit upon which is superimposed a signal voltage (e.g., bias voltage to an Oxygen Sensor circuit, or a filtered digital m/sec2 signal while vehicle stands still for a lateral accelerator sensor module.)
29	1001	Signal signal invalid This sub type is used for failures where the value of the signal is not plausible given the operating conditions.
2A – 2E	1010 - 1110	ISO/SAE reserved This range of values is reserved by the document for future expansion.
2F	1111	Signal erratic This sub type is used for failures where the signal is momentarily implausible (not long enough for "signal invalid") or discontinuous.

Table D-19. DTC Failure Subtype definition for failure category “2”

D 4.1.5 DTC Failure Sub Type definition of FM (Frequency Modulation) / PWM (Pulse Width Modulation) failures

The table below specifies faults related to Frequency Modulated (FM) and Pulse Width Modulated (PWM) inputs and outputs of the server. This category also includes faults where position is determined by counts.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	FM (Frequency Modulation) / PWM (Pulse Width Modulation) Failures Subtype Description
30	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
31	0001	No signal This sub type is used for failures where the server does not detect a signal which ought to be present (e.g., wheel speed signals present for three of the four wheels and brakes not applied.)
32	0010	Signal low time < minimum This sub type is used for failures where the server detects the low pulse is too narrow with respect to time.
33	0011	Signal low time > maximum This sub type is used for failures where the server detects the low pulse is too wide with respect to time.
34	0100	Signal high time < minimum This sub type is used for failures where the server detects the high pulse is too narrow with respect to time.
35	0101	Signal high time > maximum This sub type is used for failures where the server detects the high pulse is too wide with respect to time.
36	0110	Signal frequency too low This sub type is used for failures where the server detects excessive duration for one cycle of the output across a specified sample size.
37	0111	Signal frequency too high This sub type is used for failures where the server detects insufficient duration for one cycle of the output across a specified sample size.
38	1000	Signal frequency incorrect This sub type is used for failures where the server measures an incorrect number of cycles in a given time period.
39	1001	Incorrect has too few pulses This sub type is used for failures where the server measures too few pulses (e.g., position is calibrated in counts from one extreme to the other).
3A	1010	Incorrect has too many pulses This sub type is used for failures where the server measures too many pulses (e.g., position is calibrated in counts from one extreme to the other).
3B - 3F	1011 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-20. DTC Failure Subtype definition for failure category "3"

D 4.1.6 DTC Failure Sub Type definition of system internal failures

The table below specifies faults related to memory, software, and internal electrical circuitry; requiring component (server, sensor, etc.) replacement.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	System Internal Failures Subtype Description
40	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
41	0001	General checksum failure This sub type is used by the server to indicate an incorrect checksum calculation where memory type is not specified.
42	0010	General memory failure This sub type is used by the server to indicate a memory failure where memory type is not specified.
43	0011	Special memory failure This sub type is used by the server to indicate a memory failure where the specific memory type is not defined in this category.
44	0100	Data memory failure This sub type is used by the server to indicate a data (or working) memory failure for embedded systems using FLASH memory. This is equivalent to RAM in RAM/ROM/EEPROM embedded systems.
45	0101	Program memory failure This sub type is used by the server to indicate a program memory failure for embedded systems using FLASH memory. This is equivalent to ROM in RAM/ROM/EEPROM embedded systems.
46	0110	Calibration / parameter memory failure This sub type is used by the server to indicate a calibration / parameter memory failure for embedded systems using FLASH memory. This is equivalent to EEPROM in RAM/ROM/EEPROM embedded systems.
47	0111	Watchdog / safety μC failure This sub type is used by the server to indicate a watchdog / safety μ C failure.
48	1000	Supervision software failure This sub type is used by the server to indicate a supervision software failure.
49	1001	Internal electronic failure This sub type is used by the server to indicate the detection of an internal circuit failure.
4A	1010	Incorrect component installed This sub type is used by the server to indicate a mismatch between the hardware connected to the server and the hardware expected by the server.
4B	1011	Over temperature This sub type is used by the server to indicate the detection of an internal temperature above the expected range.
4C - 4F	1100 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-21. DTC Failure Subtype definition for failure category "4"



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 189/211

D 4.1.7 DTC Failure Sub Type definition of system programming failures

The table below specifies faults related to operational software, calibrations, and options; remedied by configuring/programming a part of the system (control module, sensor, etc.)

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	System Internal Failures Subtype Description
50	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
51	0001	Not programmed This sub type is used by the control module to indicate that programming is required.
52	0010	Not activated This sub type is used by the control module to indicate that some portion of the program has not been enabled.
53	0011	Deactivated This sub type is used by the control module to indicate that some portion of the program has not been disabled.
54	0100	Missing calibration This sub type is used by the control module to indicate that an operational range, etc., for a sensor or actuator must be taught to the control module, e.g. by programming or learning
55	0101	Not configured This sub type is used by the control module to indicate the need to enter (program) the sub system option content or the vehicle option content.
56 - 5F	0110 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-22. DTC Failure Subtype definition for failure category "5"

D 4.1.8 DTC Failure Sub Type definition of algorithm based failures

The table below specifies faults based on comparing two or more input parameters for plausibility, comparing a single parameter to itself with respect to time, or inhibits operation due to a reported failure of that circuit.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	Algorithm Based Failures Subtype Description
60	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
61	0001	Signal calculation failure This sub type is used for algorithm based calculation failures.
62	0010	Signal compare failure This sub type is used for failures where the server compares two or more input parameters for plausibility.
63	0011	Circuit / component protection time-out This sub type is used for failures where the server detects a function is active for greater than a specified time period.
64	0100	Signal plausibility failure This sub type is used for failures where the server detects plausibility failures.
65	0101	Signal has too few transitions / events This sub type is used for failures where the server monitors a parameter over time within specified limits and detects fewer than the expected number of transitions.
66	0110	Signal has too many transitions / events This sub type is used for failures where the server monitors a parameter over time within specified limits and detects more than the expected number of transitions.
67	0111	Signal incorrect after event This sub type is used for failures where the server does not see the correct change of a parameter or group of parameters in response to a particular event.
68	1000	Event information This sub type is used by the server to indicate the detection of a system event that was not caused by the server itself but forces the server to store a DTC (e.g. missing functionality from another system/server).
69 - 6F	1001 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-23. DTC Failure Subtype definition for failure category "6"

D 4.1.9 DTC Failure Sub Type definition of mechanical failures

The table below specifies faults detected by inappropriate motion in response to server related input/controlled output.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	Mechanical Failures Subtype Description
70	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
71	0001	Actuator stuck This sub type is used for failures where the server does not detect any motion in response to energizing a motor, solenoid, relay, etc.
72	0010	Actuator stuck open This sub type is used for failures where the server does not detect any motion upon commanding the operation of a motor, solenoid, relay, etc., to close some piece of equipment.
73	0011	Actuator stuck closed This sub type is used for failures where the server does not detect any motion upon commanding the operation of a motor, solenoid, relay, etc., to open some piece of equipment.
74	0100	Actuator slipping This sub type is used for failures where the server detects excessive duration to command a motor, solenoid, relay, etc., to move a piece of equipment to a desired position.
75	0101	Emergency position not reachable This sub type is used for failures where the server is unable to command a motor, solenoid, relay, etc., to move a piece of equipment to the emergency position.
76	0110	Wrong mounting position This sub type is used for failures where the server detects incorrectly mounted components, e.g., acceleration sensor showing a position error of 90°.
77	0111	Commanded position not reachable This sub type is used for failures where the server is unable to command a motor, solenoid, relay, etc., to move a piece of equipment to the commanded position either due to a failure in the actuator or its mechanical environment.
78	1000	Alignment or adjustment incorrect This sub type is used for failures where the server detects incorrectly adjusted or aligned components
79	1001	Mechanical linkage failure This sub type is used for failures where the server detects that the actuator is operational but the driven device is not operating e.g. drive cable for power sliding door broken
7A	1010	Fluid leak or seal failure This sub type is used for failures where the server detects that a mechanical component has an unexpected gas or liquid flow in, out or through the component.
7B	1011	Low fluid level This sub type is used for failures where the server detects that a fluid level is too low for proper operation of the system.
7C - 7F	1100 - 1111	ISO/SAE reserved This range of values is reserved by the document for future expansion.

Table D-24. DTC Failure Subtype definition for failure category “7”

D 4.1.10 DTC Failure Sub Type definition of bus signal failures

The table below specifies faults related to bus hardware and signal integrity. This category is also used when the physical input for a signal is located in one server and another server diagnoses the circuit.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	Bus Signal / Message Failures Subtype Description
80	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
81	0001	Invalid serial data received This sub type is used by the server to indicate a signal was received with the corresponding validity bit equal to "invalid" or post processing of the signal determines it is invalid.
82	0010	Alive/sequence counter incorrect/not updated This sub type is used by the server to indicate, that a signal was received without the corresponding rolling count value being properly updated.
83	0011	Value of signal protection calculation incorrect This sub type is used by the server to indicate, that a message was processed with an incorrect protection (checksum) calculation.
84	0100	Signal below allowable range This sub type is used for failures where some circuit quantity, reported via serial data, is below a specified range.
85	0101	Signal above allowable range This sub type is used for failures where some circuit quantity, reported via serial data, is above a specific range.
86	0110	Signal invalid This sub type is used for failures where some circuit quantity, reported via serial data, is not plausible given the operating conditions.
87	0111	Missing message This sub type is used for failures where one (or more) expected message(s) is not received, e.g., periodic transmission where the repetition time is too high, or message not received as a result of unforeseen reset events of the concerning component (e.g. engine control unit communicating with ABS).
88	1000	Bus off This sub type is used for failures where a data bus is not available.
89 – 8E	1001 - 1110	ISO/SAE reserved This value is reserved by the document for future expansion.
8F	1111	Erratic This sub type is used for failures where the signal, reported via serial data, is momentarily implausible or discontinuous.

Table D-25. DTC Failure Subtype definition for failure category “8”



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 193/211

D 4.1.11 DTC Failure Sub Type definition of component failures

The table below specifies faults related to component failures.

DTCFailureTypeByte Category Definitions		
Failure Type Byte (hex)	Subtype Nibble (binary)	Component Failures Subtype Description
90	0000	ISO/SAE reserved This value is reserved by the document for future expansion.
91	0001	Parametric This sub type is used for failures where the server has detected that a component parameter e.g. capacitance or inductance is outside its expected range.
92	0010	Performance or incorrect operation This sub type is used for failures where the server has detected that the component performance is outside its expected range or operating in an incorrect way.
93	0011	No operation This sub type is used for failures where the server has detected that the component is not operating.
94	0100	Unexpected operation This sub type is used for failures where the server has detected that the component is operating in a way or at a time that it has not been commanded to operate.
95	0101	Incorrect assembly This sub type is used for failures where the server has detected that the component has been incorrectly installed e.g. hydraulic pipes crossed over, circuits cross wired or polarity errors.
96	0110	Component internal failure This sub type is used for failures where the server has received an indication about the component that indicates a failure e.g. an intelligent actuator or sensor is indicating an internal fault.
97	0111	Component or system operation obstructed or blocked This sub type is used for failures where the server has detected that the operation of a component is prevented by an obstruction e.g. advanced cruise system radar beam obstructed.
98	1000	Component or system over temperature This sub type is used for failures where the server has detected that the temperature is too high for the correct operation of the component or system.
99 - 9F	1001 - 1111	ISO/SAE reserved This value is reserved by the document for future expansion.

Table D-26. DTC Failure Subtype definition for failure category "9"

Annex. E Input Output Control Functional Unit Data Parameter Definitions

E.1 InputOutputControlParameter Definitions

HEX	Description	Cvt	Mnemonic
00	returnControlToECU This value shall indicate to the server that the client does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RCTECU
01	resetToDefault This value shall indicate to the server that it is requested to reset the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier to its default state. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	RTD
02	freezeCurrentState This value shall indicate to the server that it is requested to freeze the current state of the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier. - Number of controlState bytes in request: 0 - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	FCS
03	shortTermAdjustment This value shall indicate to the server that it is requested to adjust the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set pulse width of valve to a specific value/duty cycle). - Number of controlState bytes in request: depends on the dataIdentifier - Number of controlState bytes in pos. response: depends on the dataIdentifier	U	STA
04~FF	Reserved	M	

Table E-1. InputOutputControlParameter Definition



Annex. F Remote Activation of Routine Functional Unit Data Parameter Definitions

Refer to Table 8-3. Request message routineIdentifier definition

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

Annex. G Address And Length Format Identifier Parameter Values

G.1 AddressAndLengthFormatIdentifier Definitions

This parameter is a one byte value with each nibble encoded separately. See below table.

AddressAndLengthFormatIdentifier							
High Nibble				Low Nibble			
7	6	5	4	3	2	1	0
Bytes used for MemorySize Parameter				Bytes used for MemroyAddress Parameter			

Table G-1. AddressAndLengthFormatIdentifier Definition

MOBASE ELECTRONICS / 이유림
172.23.51.206 / 2025-11-14 15:32

Annex. H Diagnostic CAN Identifiers

H.1 Definition of CAN Identifier for Enhanced Diagnostic Communication

※ The latest Diagnostic CAN Identifiers refer to the iDIMS and NIDS

CAN ID (HEX)	Description
Functional Addressing	
07DF	Functional request CAN ID sent by the external test equipment (All controllers must support this CAN Identifier)
Physical Addressing	
0580~05FF	HMC specific. See also iDIMS and NIDS ※Only for Third Generation of CAN (CAN-FD)
0700~07DE	HMC specific. See also iDIMS and NIDS
07E0	Physical request CAN ID to EMS (Engine Management System), FCU (Fuelcell Control Unit)
07E8	Physical response CAN ID from EMS (Engine Management System), FCU (Fuelcell Control Unit)
07E1	Physical request CAN ID to TCU (Transmission Control Unit)
07E9	Physical response CAN ID from TCU (Transmission Control Unit)
07E2	Physical request CAN ID to HCU (Hybrid Control Unit) / VCU (Vehicle Control Unit) / HCULDC (HCU & Low DC-DC Converter)
07EA	Physical response CAN ID from HCU (Hybrid Control Unit) / VCU (Vehicle Control Unit) / HCULDC (HCU & Low DC-DC Converter)
07E3	Physical request CAN ID to MCU (Motor Control Unit) / MHSG (Mild Hybrid Starter Generator) / F_MCU (Front MCU) / R_MCU (Rear MCU)
07EB	Physical response CAN ID from MCU (Motor Control Unit) / MHSG (Mild Hybrid Starter Generator) / F_MCU (Front MCU) / R_MCU (Rear MCU)
07E4	Physical request CAN ID to BMS (Battery Management System)
07EC	Physical response CAN ID from BMS (Battery Management System)
07E5	Physical request CAN ID to OBC (On Board Charger) : From LF/JF PHEV, AE/DE EV/PHEV ICCU (Integrated Charge Control Unit) : From Osc EV LP1
07ED	Physical response CAN ID from OBC (On Board Charger) : From LF/JF PHEV, AE/DE EV/PHEV ICCU (Integrated Charge Control Unit) : From Osc EV LP1
07E6	Physical request CAN ID to SCU_FF (SBW Control Unit) : From LX2 EVSCU_FF (EV SBW Control Unit) : EV SCU
07EE	Physical request CAN ID from SCU_FF (SBW Control Unit) : From LX2 EVSCU_FF (EV SBW Control Unit) : EV SCU
07E7	Physical request CAN ID to DCU (Doesing Control Unit) : Diesel Engine Only ESC(IEB) (Electronic Stability Control Integrated Electronic Brake)
07EF	Physical response CAN ID from DCU (Doesing Control Unit) : Diesel Engine Only ESC(IEB) (Electronic Stability Control Integrated Electronic Brake)
07F0~07F3	HMC specific. See also iDIMS and NIDS
07F4~07F6	ISO15765-2 Reserved
07F7~07F9	HMC specific. See also iDIMS and NIDS
07FA~07FC	ISO15765-2 Reserved
07FD~07FF	HMC specific. See also iDIMS and NIDS
17FF0001 ~17FFFFFFE	HMC specific. See also iDIMS and NIDS

Table H-1. Definition of CAN Identifier for Enhanced Diagnostic Communication

Annex. I EOL (End Of Line) Diagnostic Mode

I.1 Dual Expression of DTC in EOL Process (HADS, U-Robot equipment)

차량제작



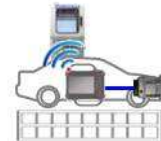
검사장비(HADS)



주행검사



U-로봇



- [HADS] Excluding pending DTC expression (DTC Case #1)
- [U-Robot] Check all DTC
- Standardization of DTC expression standard of PT Gasoline / Diesel system (Including of Bit 2 Pending Concept)
- Applying pending DTC concepts of UDS on CAN, ECUs are compliant with the following table.

DTC Case	Status Of Bit			DTC Status Mask (※ responds to corresponding DTC in case the result is more than "1")		Display of External diagnostic equipment
	Bit3	Bit2	Bit0	HADS	U-Robot	
	Confirmed	Pending	TestFailed	09	0D	
DTC Case #1	0	1	0	0 (No response)	1	Pending DTC
DTC Case #2	0	1	1	1	2	Pending DTC
DTC Case #3	1	0	0	1	1	Historic DTC
DTC Case #4	1	1	0	1	2	Historic DTC
DTC Case #5	1	1	1	2	3	Active DTC

Table I-1. Meaning of DTC according to Status Of Bit and DTC Status Mask

I.2 EOL Batch Clear DTC (Error Eraser equipment)

When conducting Batch Clear DTC of all ECUs that comply with ES95486-00, Communication processes between ECU and external test equipment shall be compliant with following details.

I.2.1 CAN ID

When conducting Clear DTC of each ECU, it communicates by using each CAN ID specified in specification. When conducting Batch Clear DTC, it communicates by using 0x7DF (Functional Request CAN ID)

I.2.2 EOL Standard Process (Error Eraser ↔ ECU)

No.	Process	Communication	Note
1	Vehicle Energize	-	Implements IG ON
2	Start of Diagnostic Communication of each ECU	Tx : 02 10 03 (02 10 90) Rx : 02 50 03 (02 50 90)	
3	...		Implements additional functions of each ECU (Coding, Calibration, Initialization etc.)

4	End of Diagnostic Communication of each ECU	Tx : 10 01 (01 20) Rx : 50 01 (01 60)	
5	Start of communication of UDS on CAN ECU	Tx : 02 10 03 Rx : 02 50 03	Tx CAN ID : 0x7DF Rx CAN ID : different depending on each ECU
6	Conducting Batch Clear DTC of UDS on CAN ECU	Tx : 04 14 FF FF FF Rx : 01 54	↑
7	Start of communication of KWP on CAN ECU	Tx : 02 10 90 Rx : 02 50 90	↑
8	Conducting Batch Clear DTC of KWP on CAN ECU	Tx : 03 14 FF 00 Rx : 01 54	↑
9	Completion of Error Erasing		Shows OK/NG results on the screen of equipment (Different depending on factories)

Table I-2 EOL Standard Process of Batch Clear DTC

- Conducting of Individual Clear DTC of each ECU and additional functions is the same as existing methods. (1~4)
- Error Eraser equipment starts communication as 0x7DF CAN ID and sends ClearDTC request.
- If each ECU receives communication protocol request which is different from its own protocol, it responds to one of negative response codes (NRC) #12, #13, #31.
- When Error Eraser equipment receives NRC #12, #13, #31, it does not show results on the screen.
- In case some ECUs respond to NRC besides #12, #13, #31, NG is displayed on the screen of equipment.
(※ The method of NG display is decided depending on the factory characteristics)

※ Negative Response Codes

1. #12 : subFunctionNotSupported
2. #13 : incorrectMessageLengthOrInvalidFormat
3. #31 : requestOutOfRange

Annex. J RXSWIN requirement

J.1 RXSWIN Read

The EDT requests RXSWIN (RDBI 22 F18F) and the Central gateway requests a local RXSWIN of each ECU (RDBI 22 F1EF), stores the result, filters (remove redundancy) and transmits the result.

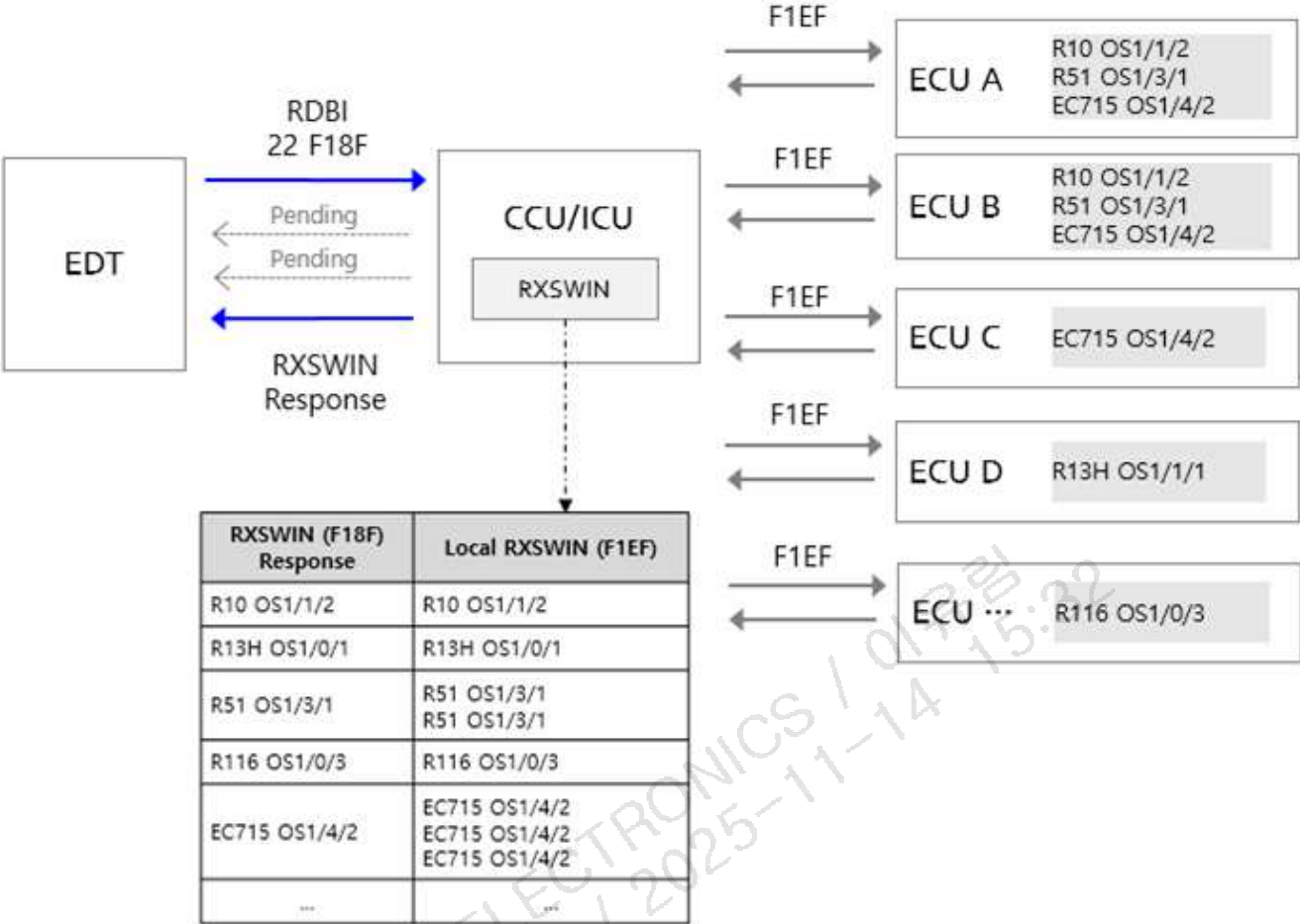


Figure J-7. RXSWIN Overview

Hex	Description	Cvt	Mnemonic
F18F	RXSWIN RXSWIN is information that combines local RXSWIN information into one RXSWIN.	C1	
F1EF	Local RXSWIN RXSWIN information which is in a ECU.	C2	

C1: Only the central gateway shall support this DID.
C2: All ECUs related to RXSWIN shall support this Identifier.

Table J-1. RXSWIN Identification

J.2 RXSWIN Request and Response sequence

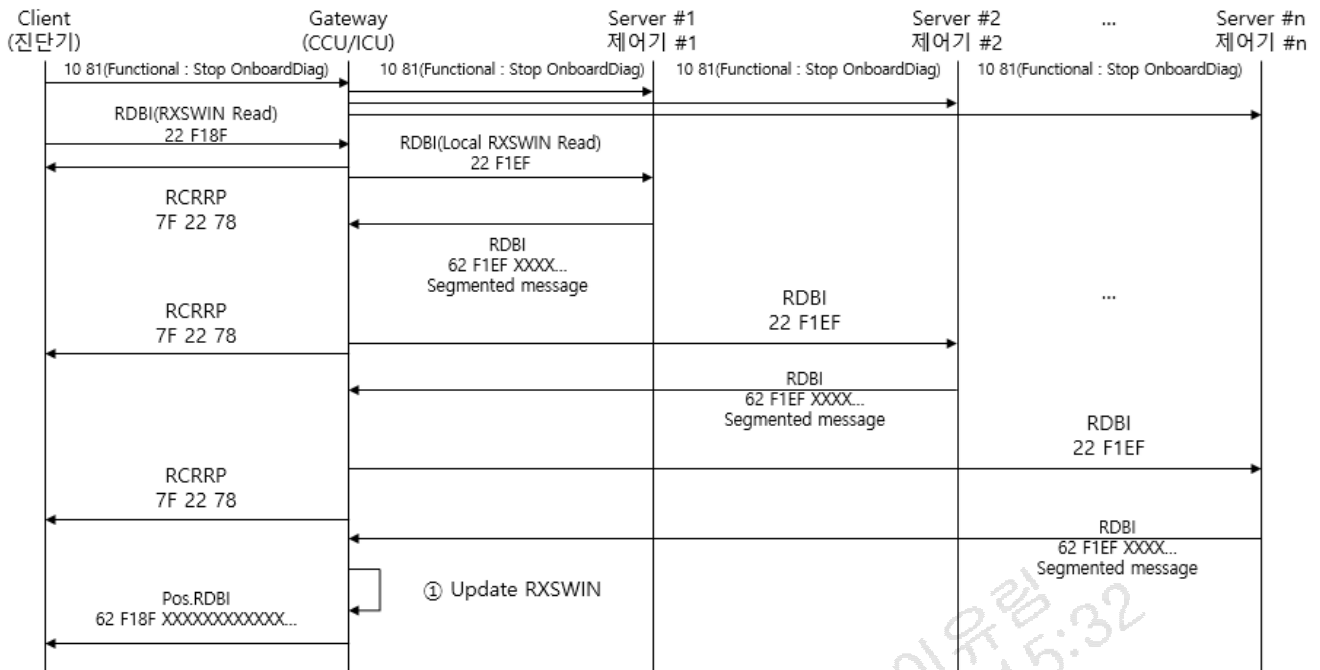


Figure J-8. RXSWIN gathering and read sequence caused by RDBI(F18F)

Exception : 10 81(Functional:Stop OnboardDiag) Although Functional addressing 10 81 causes Stop onboard diagnostics, Gateway can gather Local RXSWIN

*OnBoardDiag : Diagnostics from ECU to ECU (Client : ECU, Server : ECU)

When the diagnosis device requests RXSWIN in the IGN off state, Gateway does not request LocalRXSWIN but responds by referring to the RXSWIN table updated previously.

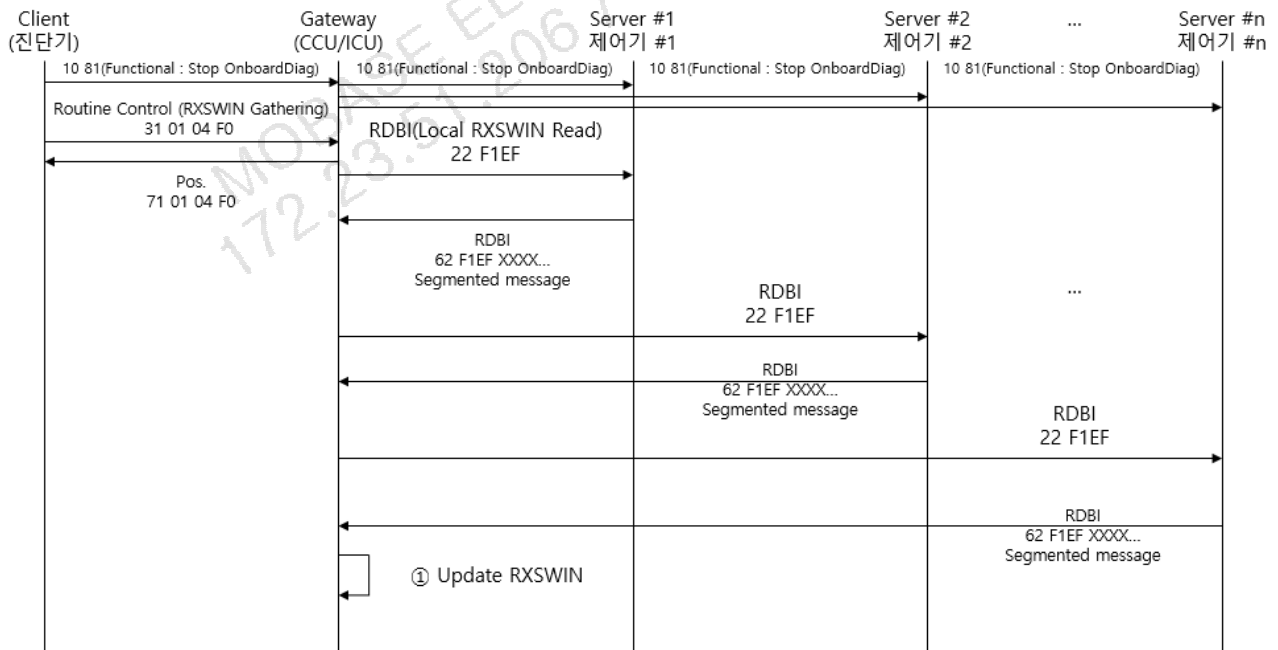


Figure J-9. Start RXSWIN gathering – routine control

The gateway can perform the gathering and reading separately. The gathering can be started by routine control(\$31). After the gateway finish gathering RXSWIN, RXSWIN shall be updated in NVM.

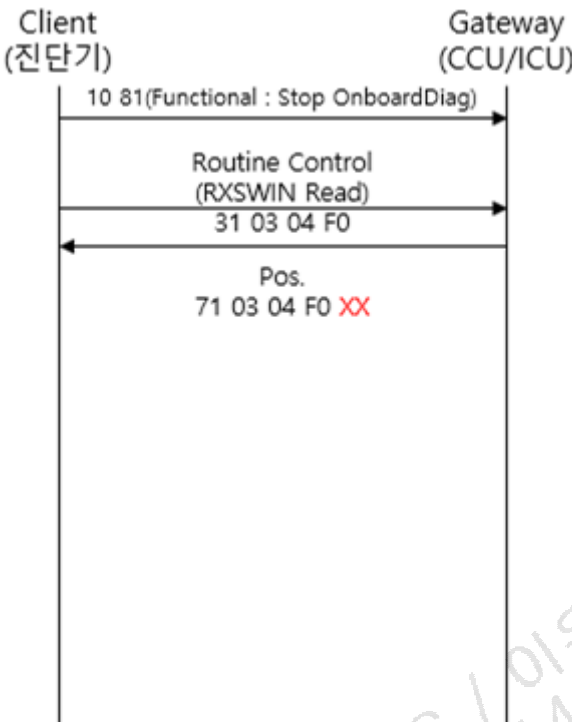


Figure J-10. Request routine result - Routine Control

Value	Routine Result	Description
0	Done	Routine has been finished correctly – RXSWIN gathering has been done
1	Collecting	Routine is running – Central Gateway is gathering RXSWIN
2	Aborted	Because of IGN off or other problems, Gathering has been aborted.

Table J-6. RXSWIN Routine result

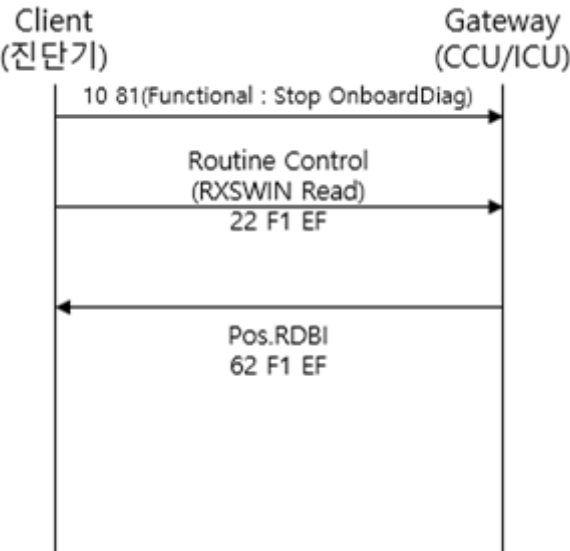


Figure J-11. Read local RXSWIN from the central gateway

If diagnostic tester requests RDBI with F1EF(local RXSWIN) to the central gateway, the central gateway doesn't gather RXSWIN and respond positive with already gathered RXSWIN which is stored in NVM.
If RXSWIN never has been gathered, the central gateway respond positive with only gateway own RXSWIN.

If the central gateway received RDBI with F1EF or F18F during the gathering RXSWIN by routine control, the central gateway shall respond positive with already gathered RXSWIN which is stored in NVM. And it doesn't affect gathering RXSWIN.

J.3 Local RXSWIN collection example for master - slave ECU

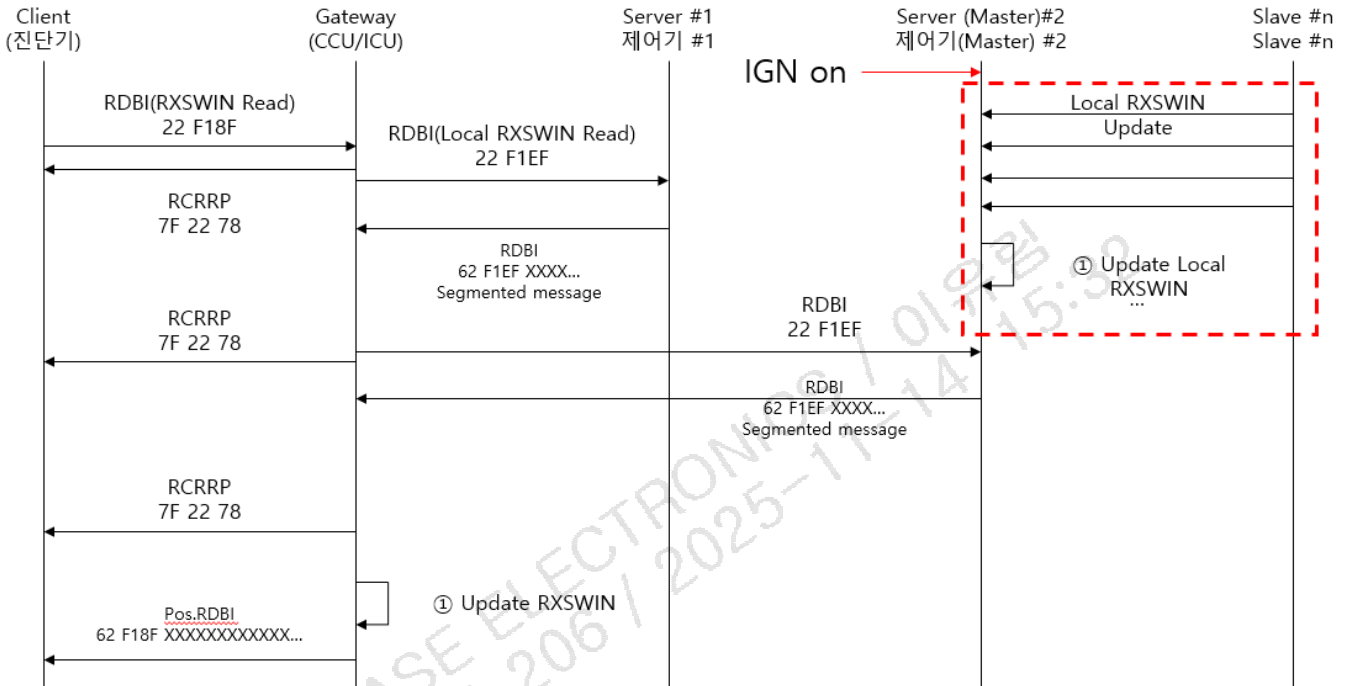


Figure J-12. Example of RXSWIN gathering – Master/Slave ECU

Master –Slave ECUs could update local rxwin after IGN on.

But This is not mandatory update sequence. Because The exchange of Rxswin between Master Slave is not scope of the Diagnostic specification.

J.4 Local RXSWIN Request Response format

Gate way Req.	HEX	22	F1	EF																														
	Description	SID	DID																															
ECU Resp.	HEX	62	F1	EF	XX	XX	0C	52	31	35	36	20	44	4E	38	2F	31	2F	31	0B	52	34	38	20	44	4E	38	2F	31	2F	30			
	Description	SID	DID	REQ CAN ID	Length	RXSWIN ASCII STRING															Length	RXSWIN ASCII STRING												
						R	1	5	6		D	N	8	/	1	/	1	R	8	3			D	N	8	/	1	/	0					

Table J-2. Example of Local RXSWIN Request Response format

IMPORTANT : XX XX is Request CAN ID to recognize where the response came from on the Gateway Application layer

REQ CAN ID Example)

Diagnostic Request CAN ID 0x700

Diagnostic Response CAN ID :0x708

REQ CAN ID value shall be 0x700



기술 표준

ENGINEERING STANDARD

규격번호 (SPEC No)
ES95486-00

페이지
(SHT/SHTS) 204/211

REQ CAN ID Example2)

Diagnostic Request CAN ID 0x17FFCA00

Diagnostic Response CAN ID : 17FF00CA

REQ CAN ID value shall be 0x50CA (See also chapter 2.4.1 addressing. For RXSWIN, 29bit to 2byte CAN ID conversion is comply with DoIP logical address conversion rule even if the server uses ethdiag protocol

Length Example1)

R156 DN8/1/1

The length of RXSWIN string including space is 12bytes

Length value shall be 0x0C (Decimal 12)

Length Example2)

R83 DN8/1/1

The length of RXSWIN string is 11bytes

Length field shall be 0x0B (Decimal 11)

RXSWIN		Length	Reg. Authority	Reg. No.	Space	Vehicle Project	/Version	/Extension	RXSWIN Result (Length)String
Ex1)	RXSWIN Initial	0x13	EC	1008/2010	“ “	CN7	/1	/0	(0x13)EC1008/2010 CN7/1/0
	RXSWIN Extension	0x13	EC	1008/2010	“ “	CN7	/1	/1	(0x13)EC1008/2010 CN7/1/1
Ex2)	Type Gasoline	0x0B	R	83	“ “	DN8	/1	/0	(0x0B)R83 DN8/1/0
	Type Diesel	0x0B	R	83	“ “	DN8	/2	/0	(0x0B)R83 DN8/2/0

Table J-3. RXSWIN structure

J.5 RXSWIN Update Example

As is (RXSWWIN table – Central Gateway)

List No.	RXSWIN STRING														
0	R	1	5	6		D	N	8	/	1	/	1			
1	R	8	3			D	N	8	/	1	/	0			
2															
...															

Example) Local RXSWIN Response 1

Gate way Req.	HEX	22	F1	EF																														
	Description	SID	DID																															
ECU Resp.	HEX	62	F1	EF	07	00	0C	52	31	35	36	20	44	4E	38	2F	31	2F	31	0B	52	34	38	20	44	4E	38	2F	31	2F	30			
	Description	SID	DID	REQ CAN ID	Length	RXSWIN ASCII STRING														Length	RXSWIN ASCII STRING													
						R	1	5	6		D	N	8	/	1	/	1	R	8		3		D	N	8	/	1	/	0					

Example) Local RXSWIN Response 2

Gate way	HEX	22	F1	EF
	Description	SID	DID	

페이지
(SHT/SHTS) 205/211

Req.																																		
ECU Resp.	HEX	62	F1	EF	07	00	0C	52	31	35	36	20	44	4E	38	2F	31	2F	32	0B	52	34	38	20	44	4E	38	2F	31	2F	30			
	Description	SID	DID	REQ CAN ID	Length	RXSWIN ASCII STRING															Length	RXSWIN ASCII STRING												
						R	1	5	6		D	N	8	/	1	/	2	R	8	3			D	N	8	/	1	/	0					

[illegible]

EDT Request	HEX	22	F1	8F																																
	Description	SID	DID																																	
ECU Resp.	HEX	62	F1	8F	0C	52	31	35	36	20	44	4E	38	2F	31	2F	32	0B	52	34	38	20	44	4E	38	2F	31	2F	30							
	Description	SID	DID	Length	RXSWIN ASCII STRING												Length	RXSWIN ASCII STRING																		
					R	1	5	6		D	N	8	/	1	/	2		R	8	3		D	N	8	/	1	/	0								

The gateway shall store the RXSWIN table in memory. The Local RXSWIN received from the ECU shall be updated to the latest value after removing redundancy.

Case 1. Update RXSWIN

No	RXSWIN
1	R156 DN8/1/0
2	R155 DN8/1/0
3	R48 DN8/1/1
4	R83 DN8/1/1
5	...

RAM

Server #n Response

R48 DN8/1/1
R83 DN8/1/2
R156 DN8/1/2



No	RXSWIN
1	R156 DN8/1/2
2	R155 DN8/1/0
3	R48 DN8/1/1
4	R83 DN8/1/2
5	...

RAM
(Gateway Response : F18F)

Case 2. Add RXSWIN

No	RXSWIN
1	R156 DN8/1/2
2	R155 DN8/1/0
3	R48 DN8/1/1
4	R83 DN8/1/1
5	
6	
7	

RAM

Server #n-1 Response

XXX DN8/1/1
R83 DN8/1/1
R156 DN8/2/1

Server #n Response



No	RXSWIN
1	R156 DN8/1/2
2	R155 DN8/1/0
3	R48 DN8/1/1
4	R83 DN8/1/1
5	XXX DN8/1/1
6	R156 DN8/2/1
7	

RAM
(Gateway Response : F18F)

Figure J-13. RXSWIN memory handling in the Central gateway-1

NOTE : Each case is not sequential.

This is a example to show how to gathering RXSWIN on the central gateway side.

The meaning of Gateway Response in the picture is that Response of RXSWIN for DID F18F is made from the RAM which is result of gathering local RXSWIN.

If one or more fields (Reg. Authority / Reg.No / Project / Version), which are the local RXSWIN of server #n, are not the same as the gathered RXSWIN fields, RXSWIN shall be added. If each field between the gathered RXSWIN and the local RXSWIN response is the same except the extension field and the local RXSWIN extension value is greater than the gathered RXSWIN, the extension shall be updated.

Under the following conditions, the central gateway responds to the RXSWIN stored on the NVM. Under the following conditions, The central gateway respond RXSWIN stored in the NVM :

- 1) IGN off
- 2) DID is F1EF

Case 3. Finalize RXSWIN gathering

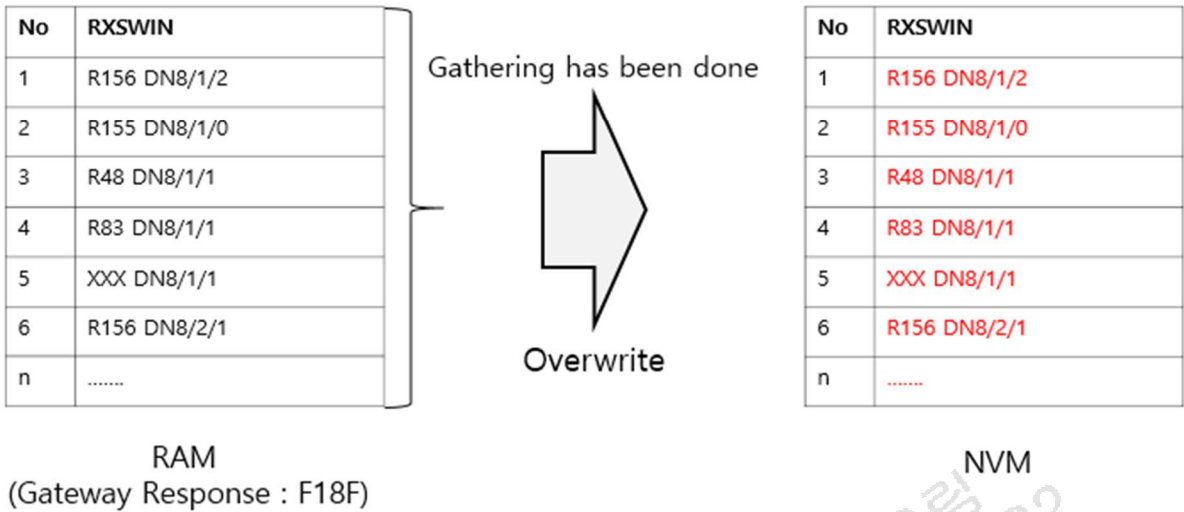


Figure J-14. RXSWIN memory handling in the Central gateway-2

After finish gathering RXSWIN to all servers, The gateway stores RXSWIN(RAM) in the NVM(overwrite).

J.6 RXSWIN Session Timer

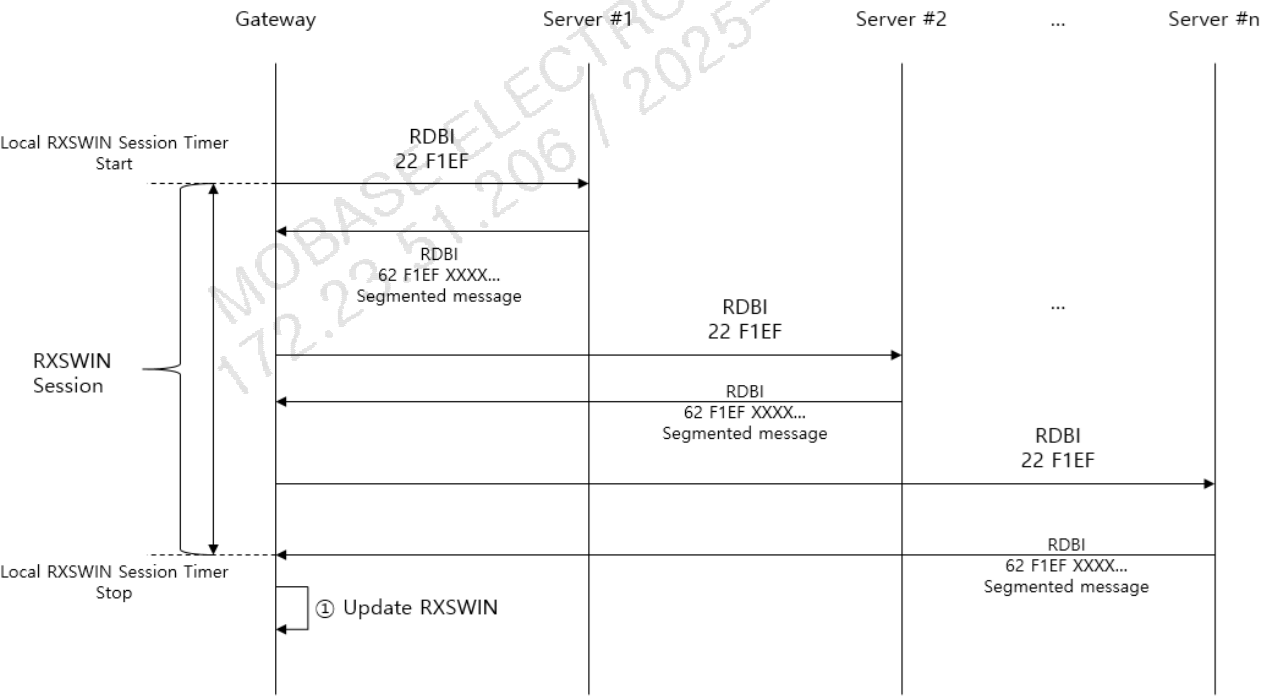


Figure J-15. RXSWIN Session timer sequence diagram

Timing parameter	Action	Condition
Local RXSWIN Session	Start	Gateway starts request of LocalRXSWIN

Local RXSWIN Session	Stop	The end of LocalRXSWIN response from All ECU which has to respond to the gateway.
		60seconds(+3sec) elapsed from Timer start. (If the 60-second timer has been terminated but Gateway has not completed the RXSWIN gathering, Gateway shall stop gathering RXSWIN.)

Table J-4 RXSWIN Session

J.7 Timing Parameter and Retry logic

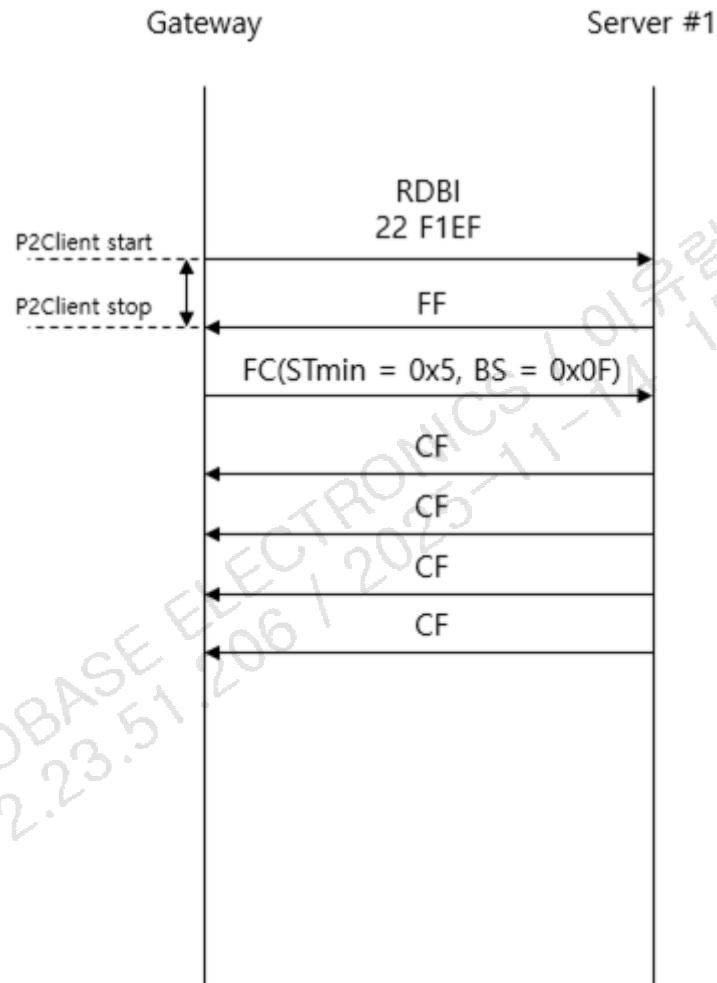


Figure J-16. RXSWIN Timing parameter diagram

Parameter	Time	Description
P2Server	50ms	ECU(Server) shall respond within 50ms
P2Client	200ms	Gateway shall await response from ECU(Server). If there is no response within P2Client, Gateway shall abort awaiting and send request to the next server(e.g. Server#2)

Table J-5 RXSWIN Timing parameter

Server shall not send RCRRP(requestCorrectlyReceivedResponsePending 0x78) for the RDBI request.

If there was P2 Client timeout or error, Gateway shall retry gathering RXSWIN after all request to the ECU has been completed.

e.g.) If there is no response from Server #3, Gateway shall proceed with RXSWIN request to the next server (Server #4). After Gateway complete gathering RXSWIN from the last server (Server#n), Gateway can retry to gather RXSWIN for failed Server#3

Gateway shall not retry gathering RXSWIN if gateway receives NRC(Except Pending) from the Server(ECU).

When the gateway retries gathering RXSWIN, P2Client is constantly 200ms.

J.8 Error handling

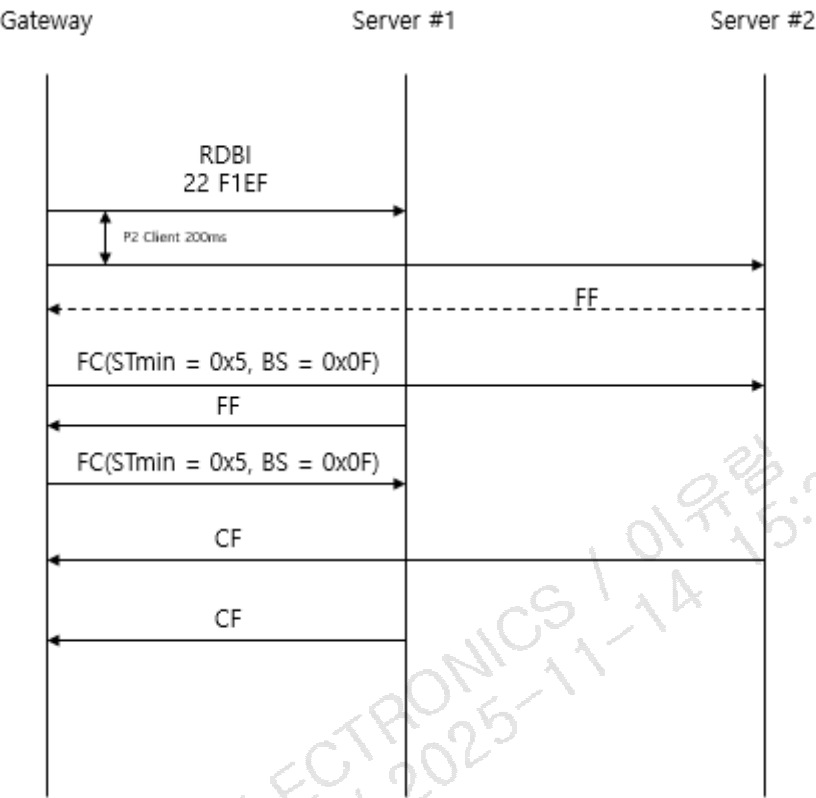


Figure J-17. RXSWIN error handling-1

In case Server #1 which is timedout responds to RDBI while Server #2 is sending RXSWIN, Gateway shall discard each response sent from Server#1 Server#2. And a retry is also required according to the J.7 Timing parameter and Retry logic.

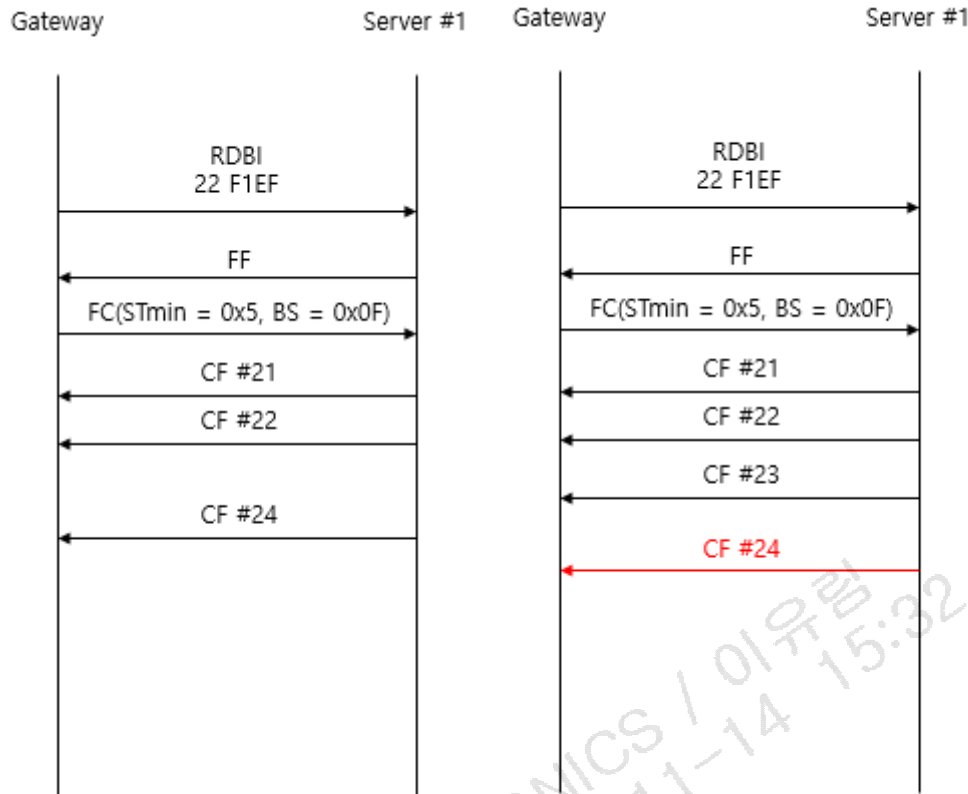


Figure J-18. RXSWIN error handling-2

In case a CF omission occurs, Gateway discard all responses. And a retry is also required according to the J.7 Timing parameter and Retry logic.

N_CR timeout is 200ms.

In case expected CF is not sent from Server(ECU) within 200ms(N_Cr), Gateway discard all responses. And a retry is also required according to the J.7 Timing parameter and Retry logic.

J.9 Addressing and DID requirement

Only a central gateway can have DID F18F.

According to the response behavior, ECU except central gateway shall respond with NRC31 in case client requests DID F18F with physical addressing.

According to the response behavior, No response is required in case client requests DID F18F with functional addressing to the ECU except central gateway.

Annex. K IVD Requirements

K.1 IVD Requirements

Controllers that use aSIMS (or FST) to create Secure Flash firmware use the hash provided when firmware is created as an IVD.

The area used to calculate the IVD shall be the entire area excluding the signature.

For controllers that do not utilize aSIMS (or FST) or have own IVD calculation algorithm, the following shall be satisfied:

- Algorithm shall be SHA256
- SHA384, SHA512, SHA3 series could be allowed but need to be announced to the HMC
- The IVD calculation area shall be the entire area. However, if it is not possible to calculate the IVD value as a whole area, it may be excluded only in areas not affected by type approval.

K.2 Hash value provision and verification method

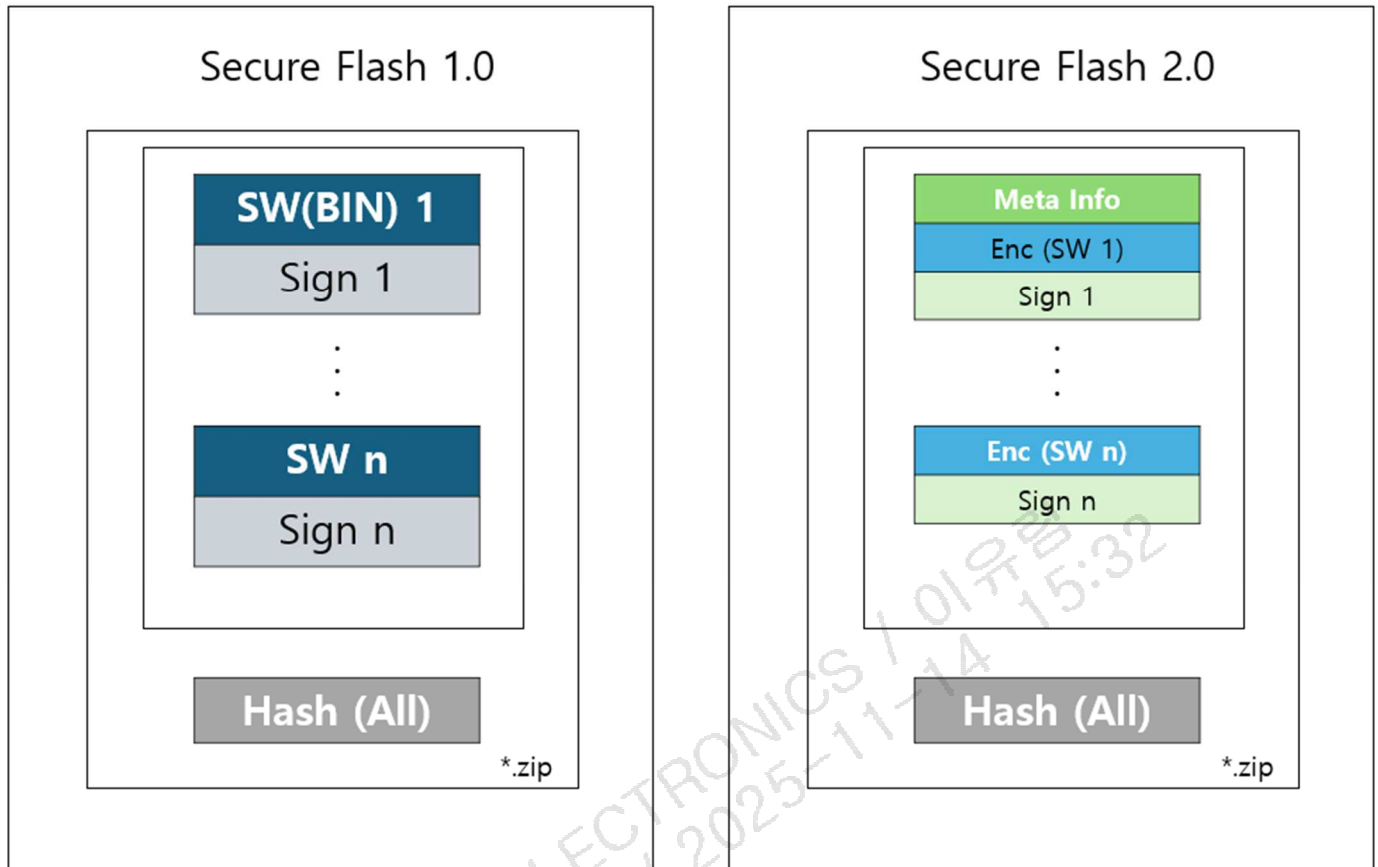


Figure K-1. Secure flash hash provision

- ※ Max n =8 (aSIMS) or 10(FST)
- ※ The signature of Secure Flash 2.0 is generated using plaintext firmware.
- ※ A hash is provided for use in the SUMS.
- ※ Except hash, Entire area shall be transmitted and reprogrammed.

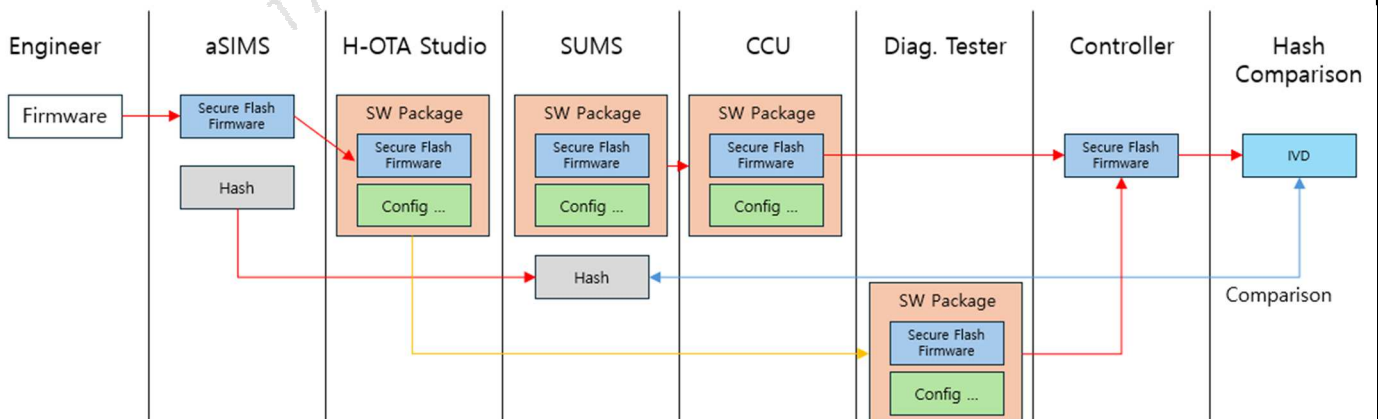


Figure K-2. Hash-IVD verification method